CONTENTS

◆ carries a drawing to build

00 · The read

01 · What these drawings are drawn to
every picker, and what feeds it
the literals

02 · The screens, in journey order

01 All bills ◆
02 Bill detail ◆
02.1 Document ◆
02.2 Charges ◆
02.3 Corrections ◆
02.4 Audit ◆
02.5 Sent ◆ **new**

03 · The bulk bill run, redrawn

the pre-bill check
How 1 and 2 are organised
17.L1–L5 the five layers ◆
16 Scope ◆
Every control, ruled ◆
the service badge
17.0 Checking ◆
17 Check ◆
17.1–17.9 the case chips ◆
the chip inventory
17.10–17.15 held · blocked · ready ◆
17.16–17.20 the bulk actions ◆
17.21–17.22 arrears · apply all ◆
18 Sign off ◆
19 Send ◆
Units or accounts · masters ◆
20 Held queue ◆
20.1–20.3 catch up · hold · release ◆
03 · carry-over

04 · Single bills

what belongs here
one rail, six kinds
21 Who and when ◆
22.1 Interim ◆

MODULE ANALYSIS · SMARTBILLHUB

Billing, as built

Fifteen screens a clerk can reach, what runs behind each one, whether anything links to it, and — the part worth the audit — the **nine places the product tells a clerk it cannot do something it can**, plus the fourteen controls it shows them and will not let them use.

ROUTE PATTERNS	SCREENS DRAWN	COMMITMENTS	NAMING NO DOOR	ENDPOINTS	DRAWINGS
7	**77**	**100**	**0**	**36**	**91**
mounted under `/:tenant/`	plus 5 layers of screen 17 and one carry-over table, which are not screens	32 minted across three passes · 2 withdrawn by ruling	every route has one; two have exactly one	across 8 url modules	70 redesign · 21 as-built — each declares which, in its own caption

ROTTING HOLDS	DEAD CONTROLS
7	**14**
proven, 5 of them load-bearing	visible and unusable

The read this document is true at

COMMIT	`a1d37a68fd641c8b73c7be46156160c0f037a33e` · Fri 11 Sep 2026 03:40 –0400 · `Merge pull request #99 from NealGudhe/claude/hi-0jmv62`
VERSION	`2.235.0` — `07-frontend/package.json`. Read off the running sidebar too, which is where a clerk sees it.
BRANCH	`claude/gracious-newton-gj0eka`, clean tree, at the tip of its remote.
IS IT CURRENT?	**Could not check.** `origin/HEAD` does not resolve in this checkout and there are two remote branches with no marked default. This is the tip of the branch I was given; whether that branch is the product is not something I can establish from here.
DESIGN SYSTEM READ FROM	`styles/tokens.css` (380 lines, read whole) · `components/AppShell.css`, `PageHeader.css`, `DataTable.css`, `KPIStrip.css`, `StateFrame.css`, `FormModal.css`, `buttonKit.ts`, `WizardFrame.tsx`. Every drawing below is only as current as those files.
THE WALK	**Partial, front end only.** Chromium 1194 via Playwright against a `vite` dev server at 1440×1100. **No backend** — Django is not installed in this environment and there is no PostgreSQL, by the user's decision. So no real row ever reached a screen, and every data-shape behaviour went unwalked.
SCREENS ACTUALLY SEEN	**4 of 15 by me, and the bill detail's four tabs by the Product Owner** — five screenshots of a real issued bill on a live tenant (`v2.234.17`), 11 September, which corrected ten details in this document. **4 of 15.** With nothing stubbed, *zero* — see §09, which is itself a finding. With three boot reads stubbed, four rendered and six threw.
REPO UNTOUCHED	**Ruled, not merely observed.** Nothing is written to the repository, and on **2026-09-13** that stopped being a working constraint and became the decision: *"We ahve to write everything in HTML artifact only Billing, as built - noting in repo"*. **So this artifact is the system of record** — it carries the rulings (§16), the judgments taken here (§16) and the one question still open (§16), because there is nowhere else for them to be. **The cost is stated rather than hidden:** 15 of the 27 rulings quoted here exist in no file, cannot be grepped, and nothing can count what depends on them.

22.2 First bill ◆
22.3 Final bill ◆
22.4 Back-bill ◆
22.5 Penalty ◆
22.6 Fee ◆
23 Confirm and send ◆
24 Periods ◆

05 · Budget bills
inside the cycle, not separate
asked from the portal
the three level columns
25 Budget bills ◆
25.1 Enrol ◆
25.2 Settle-up forecast ◆
25.3 Asks ◆
25.4 Portal · Level pay ◆

06 · Fix a bill
the thirteen reasons
what it does to the books
cancel & regenerate
four steps
26 What and why ◆
26.1 One bill ◆
26.2 Several months ◆
26.3 A batch ◆
26.4 Goodwill ◆
26.5 The page only ◆
26.6 What it changes ◆
26.7 … many ◆
26.8 Generate and send ◆

07 · Past billing runs
27 Past runs ◆

08 · Bill printing
what it costs
28 Print bills ◆
28.1 Held back ◆

09 · What happened when it was driven

10 · Absence register

11 · Rule compliance

READ THIS IF YOU ARE BUILDING FROM THIS DOCUMENT. There is one drawing per screen and it is the one to build. Nine screens in §02 open with a `BUILD THIS` band, a numbered legend and a carry-over table. Their current state is kept directly underneath, **collapsed and marked *do not build this*** — the findings in §09–§11 point at it, so it stays as evidence.

A screen with no proposed change carries a single drawing of the current state and `No commitment` — there is nothing to build from it, and that absence is the finding rather than an oversight. The contents rail marks every screen that carries work with ◆.

Every figure in every drawing is an illustration, never a value to build to (B14 rule 37). Do not assert one in a test or seed a fixture from one — appendix B lists them all with their status.

Desktop only, on purpose. Every wireframe here is one 1440-wide window scaled to fit its box as a unit, never reflowed. The customer portal is still mobile-first in the product and nothing in this document changes that — but Billing is a staff module and has no portal surface, so the question does not arise for the staff screens drawn here.

> **Two kinds of statement, kept apart.** A **grey rail like this one** describes what the code does today, including an honest statement that something is missing. It commits nobody to anything. An **accent card** is a change specification. **If a thing is not in a commitment card, nobody has undertaken to build it.** A commitment names the work, **what it waits on**, the screens it changes, and what a person could do and see once it is done. The cards are indexed in §14the nineteen minted here.
>
> **Every commitment carries one of four state chips, and only four.** Each says what the WORK waits on, never how confident anyone is:
> `Exists behind` the server already answers — only a screen or a wire is missing · `New read` a read has to be published first · `New build` new code, a table, a component or an engine change · `Blocked` it waits on a ruling, or on something this module does not own.
> **The chip is this vocabulary and nothing else.** A word describing what the code does *today* — served, stubbed, moved, healthy, a defect id — is a **description**, and it is set as `A RULE-LEFT TAG`, never a chip, so it can never be read as a plan. **A screen no specification covers carries** `No commitment` **— that absence is the finding, not an oversight.**

Reading of the flow

I read *Billing* as the `billing` module: producing bills, holding them back, putting them right, and getting them out. That takes in the seven `/:tenant/billing/…` routes and the four url modules behind them.

Deliberately out: Payments (its own module, its own screens), the ledger posting that billing hands off to, rating and plans (billing reads them), and the customer portal's view of a bill. Where a fact crosses one of those boundaries I mark it ⇄ and name both modules, and §08 collects them.

12 · Where to start — the build index
the nine that cost the most
every build component
13 · Cross-module
14 · Specification register
the one minted after the ruling
the twelve minted here
15 · What this pass changed
16 · Appendix A–K
decisions taken here
still open for the PO
every ruling, and where it lives
fields served against shown
17 · The rest of the screens, as built
03 Bill document
04 Run bills · step 1
05 step 2 What was found
06 step 3 The customers
07 step 4 The totals
08 step 5 Sign off
09 step 6 Send
10 Past billing runs ◆
11 One-time bill
12 Budget Bills
13 Fix a bill ◆
14 Pending queue ◆
15 Print bills

them.

01

What these drawings are drawn to

A violation is only a violation against a stated authority, so here is the authority, in precedence order, and the literals every drawing uses.

#	AUTHORITY	STANDING
1	`04-docs/02-decisions/14-ui-conventions.md` — **B14**	**Thirty rules, numbered up to 39.** *Measured for this pass: B14 holds rule rows 1–21 and 31–39 — **30 rules**, with 39 the highest NUMBER and 22–30 never issued. This document said "39 rules",* `CLAUDE.md` *says 38, and B14's own line 602 says thirty-one. Three sources, three numbers, none of them the count.* Rule 39 (*one spacing scale; a field is label → input → helper*) was ratified 2026-09-11. Where B14 diverges from anything below, B14 wins. `THREE SOURCES DISAGREE — MEASURED: 30`
2	`05-prototype/r10-2026-08-16/UI-RULES.md`	B14's *source*, not its authority. Twenty rules in five groups; 12–17 are the table rules this audit's §06 cites.
3	The build file — `react-tailwind-css-ui-generator`	Not a third rule set: how the two above are obeyed in this tree.

Most of B14 is ratified and unenforced, so a green suite is evidence of nothing here. No gate in this repository reads rendered output. Rule 39's own status cell says it plainly: "still **NO GATE READS COMPUTED SPACING**". Every row in §06 carries a column saying whether a machine would ever have caught it.

Every picker in this document, and what feeds it

There are **194 dropdowns** across these drawings and most are the same few repeated, so the source is named **once, here**, rather than as a note beside each one. **Three rows carry a warning: the source does not exist yet.**

PICKER	WHERE ITS OPTIONS COME FROM
TOOLBAR FILTERS, ON EVERY LIST	
Service	The tenant's service catalogue — **eleven** services, in seed order
Route	Meter Reads — `routes_list`
Cycle	Meter Reads — the cycle list, with its expected span
Plan	Plans & Rates — the tenant's rate plans
Status	The status lookup for that list's subject — account, bill or case
Year · Month · Direction	**Derived from the rows**, not a lookup — a year with no runs is not offered

+ Filter	The column set of the table it sits on
Rows per page	A fixed set. **Server-paged** — the page size is a parameter, never a client slice
DECISIONS, IN THE BULK RUN	
Hold · Estimate · Bill as is · Include	**The decision table, per case kind** — the options are the engine's, and a kind with no option for a choice refuses it by name
Ladder rung	The dunning ladder's tiers
ONE-TIME BILLS, §04	
Account	Account search — number, customer or address
Kind	The five one-time kinds. **A constant, not a table**
How it was taken	Meter Reads — `GET /reads/read-methods`, *the tenant's read-method vocabulary*
Reason · penalty or back-bill	The reason codes those two kinds accept
Charge code · fee	Your charge catalogue — live codes only, and a code with no price refuses
Grouping · periods	**Not data** — four ways to split one span
BUDGET BILLS, §05	
Re-level due · Eligibility · Asked	**Derived** — drift, the two eligibility tests, and the age of the ask
FIX A BILL, §06	
What am I fixing	**The six scope selectors. A constant**
Reason	⚠ **Meant to be tenant-extensible reference data, and that table is not built.** The thirteen seeded reasons are a map in code, and a reason it cannot class is **refused rather than defaulted**
PRINT BILLS, §08	
Scope	Three constants — a cycle, a run, or every issued bill
Manifest format	⚠ The format registry, which holds **exactly one entry** today. A second is a registration, not a change to the print set
Bill layout	⚠ The bill-template chain. **The template table does not exist**, so "current" is the caller's fact until it does
Reason · held back	The **eight** held-back reasons. A constant in code

---

A picker whose source is a constant says so, because a constant is a code change and a lookup is a settings change — and a developer reading *"the five one-time kinds"* should not go looking for a table. **Derived is the third kind:** a year with no runs in it is not offered at all.

The literals — read out of the code, not invented

Two style worlds exist in this repository and the rules name the wrong one. `UI-RULES.md` cites `.lt`, `.page-head`, `.br-steps`, `.date-input`, `.btn-danger` and `.btn-sm` — and **not one of those six is declared in the shipped app.** All six live only in the prototype. So: **tokens from the shipped app, layout from the prototype, and never a prototype class presented as though the product had it.**

FAMILY	PINNED VALUE	WHERE IT BITES
Ground / surface / ink	`#f3f2f2` · `#eae9e9` · `#201e1d`	Inputs are *darker* than the page, not lighter.
Accent	`#ec3013`	Solid fill on the active nav item — no left rail, no radius.
Radius	`--radius-sm/md/lg` = **`0px`**, all three	Every corner in this product is square on purpose. Rounded cards in a drawing mean no extraction happened.
Page title	**40px / 800** — measured on the running app	`tokens.css:203` calls `--text-h2: 32px` "the app default, every screen head". **It is wrong.** `PageHeader.tsx:272` renders an `<h1>` and `global.css:58-60` gives it `--text-h1`. One call site in the whole app passes `titleAs="h2"`. The prototype's title is 32px; the product's is 40.
Table	body **13.5px** · header **12px** uppercase, tracking `.08em` · cells `8px 12px`	Row height is declared nowhere; it computes to ≈38px.
Space	4 · 8 · 12 · 16 · **20** · 24 · 32	`--space-5: 20px` landed 2026-09-11 with rule 39. No ad-hoc 3 / 7 / 9 / 14 anywhere.
Left rail	**13 items**, 240px, ink panel; topbar 56px	Verified in the browser, not retyped: `Reports` is item 12. A horizontal nav bar would be wrong before anything else on a drawing is judged.
Font	Archivo 400/600/800, heading weight 800	`tokens.css:22` `@import`s it from Google Fonts at runtime; the file's own header records that it was never loaded for most of this product's life. It failed to load in my sandbox too, but egress here is proxied, so that is not evidence about a real deployment.

One trap that decides how a remedy gets written. A Tailwind utility only emits CSS for a file listed in `07-frontend/src/styles/route-b-migrated.txt`; written anywhere else it renders as nothing at all and every `toContain("flex")` assertion still passes. **All fifteen billing `.tsx` files are on that list** and both billing stylesheets are deleted — so utilities are the right tool in this module, and the remedies in §07 use them. That is not true of `components/AppShell.tsx`, which is not migrated.

02

The screens, in journey order

The two screens a clerk lives in — the bills list, and one bill with its five tabs. All seven entries here lead with the **drawing to build**. The bulk bill run is its own section (§03, redrawn end to

end); the other **thirteen screens** — the bill document, the six as-built run steps, past runs, one-time and budget bills, fix a bill, the pending queue and print — are all drawn in **§17, at the back**. **Eleven of the thirteen now have a redraw of their own** — one-time bills in §04, budget bills in §05, fix a bill in §06, past runs in §07 and print in §08 — and their as-built drawings stay where they are, as the evidence those sections argue from. *Moved 2026-09-12 at the PO's direction; not one word of them was cut.*

How to read a screen entry. The `BUILD THIS` band names the target and the specifications it closes. Under the drawing, a **numbered legend** gives every element four things — what it is, where its data comes from, which rule it satisfies, and whether it is new, changed, moved or unchanged. Then a **carry-over table** whose arithmetic must reconcile: if the counts do not add up, a field was lost in the drawing. Then *As built today*, collapsed. Then the spec table, the data flow and the field table, which all describe **today**, not the target.

RECONCILIATION	COUNT	NOTE
Route patterns mounted	7	App.tsx:932–953
Screens a clerk can reach	15	1 list + 4 tab screens + 6 wizard steps + 4 standalone
Drawings to build	11	Across **nine screens** — 01, 02.1, 02.2, 02.3 (two states), 02.4 (two states), 02.5 (entirely new), 10, 13, 14. Marked ◆ in the rail.
Drawings of today, kept as evidence	18	Eight collapsed under a target, ten standing alone because nothing is proposed for those screens.
Drawings in total	29	11 + 18 = 29. Nothing absorbed into a total.
Second addresses	1	`/billing/:tab` serves five of the tab screens and draws no screen of its own
Tabs **drawn** as their own entries	4	Bill detail's Document · Charges · Corrections · Audit. They share one route and one screen id (`BIL-05`), so they are numbered 02.1–02.4 rather than renumbering 03–15
Tabs still described inside an entry	6	Step 2's six sub-tabs, which share one wizard step
Screens seen in a browser	4	marked `SEEN`; the other eleven are drawn from source and say so

Group A · Finding and reading a bill

---

01 All bills

New build S3 S9 S15 S16 S44 S79 S94 S97 · New read S19 · Exists behind S1 S13 S14 S24 S25 S99

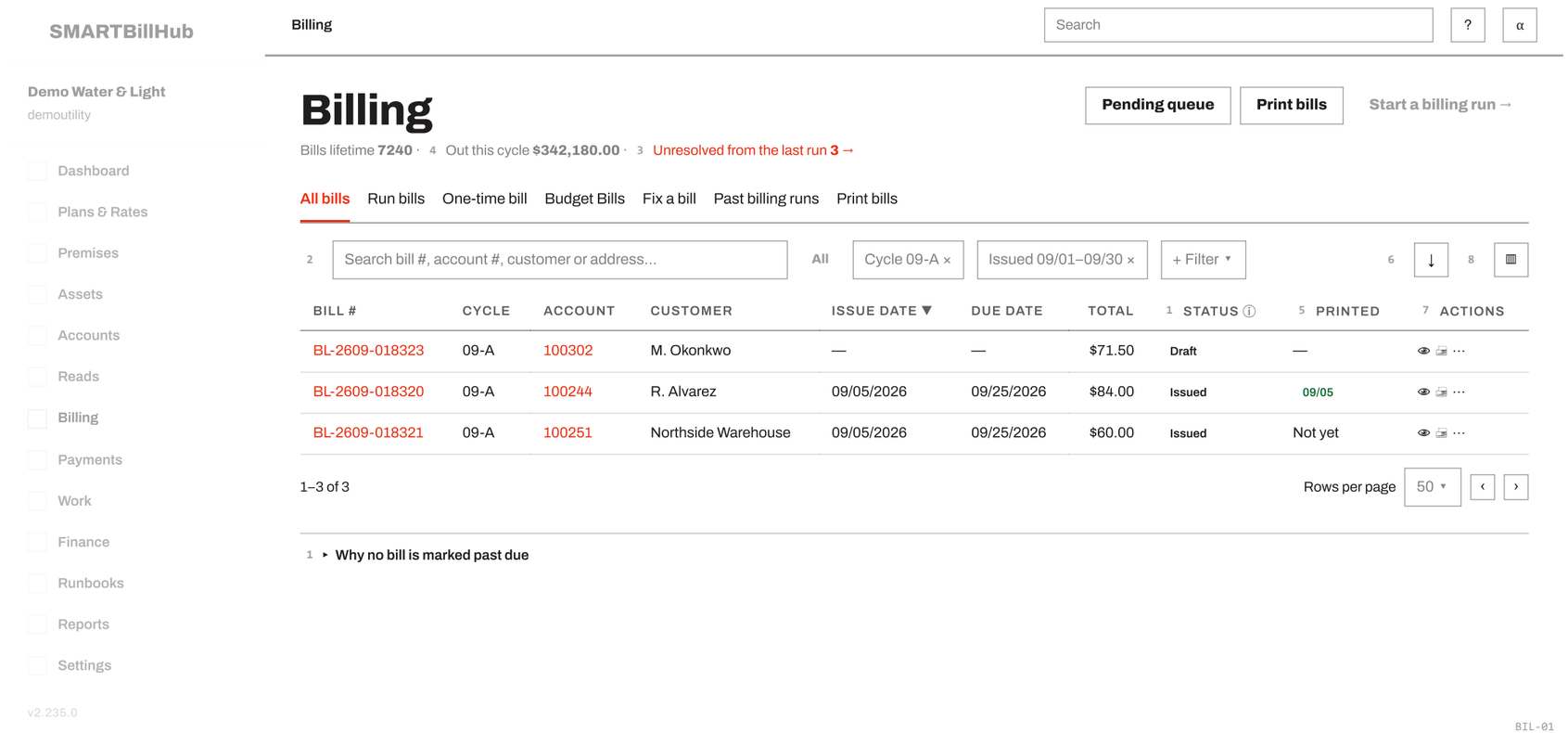
`TWO REVISIONS` This entry holds **both**: an **AS-BUILT** drawing of 2026-09-11 and a **REDESIGN** that has **never been built**. **Build the REDESIGN** — it is the first drawing below. The as-built one is kept as the evidence §09–§11 argue from; it is marked *do not build this*.

`/:tenant/billing · second address /:tenant/billing/all · S30 · BIL-01`

“A customer is on the phone about a bill. Find it.”

BUILD THIS New build **All bills — eight changes, drawn** CLOSES BIL-S13 · BIL-S14 · BIL-S15 · BIL-S16 · BIL-S3

REDESIGN drawn **2026-09-11**, still unbuilt at Version 35 (2026-09-13), drawn to B14 31 · B14 37 · the Product Owner’s 2026-09-11 ruling removing the embedded viewer. **NEVER BUILT.** Today the product ships the as-built bills list and bill detail directly below this drawing.



1 **The past-due note, collapsed and moved** · same sentence, now a `<details>` closed by default at the foot of the table, with an ⓘ on the Status header where the fact actually bites · **B14 rule 32** (a stated absence is closed by default; one sentence per section) · **MOVED**

2 **Cycle, run and date-range chips** · `f.cycle_id=`, `f.bill_run_id=`, `f.issued_from=` / `f.issued_to=` — **all four already filtered in SQL** at `selectors.py:905-920` · UI-RULES 12 · **NEW**, front end only

3 **The unresolved count becomes the door** · the figure is already computed (`bill_run_exclusions` for the newest run, unresolved) and the screen it belongs to already exists · **NEW** (a link on an unchanged field)

4 **Out this cycle, filled in** · `SUM(invoices.total)` over the current cycle, through `MoneyCell` · hard rule 2 · **NEW READ** — it is hardcoded `None` today (`selectors.py:1101-1102`). *The cheap alternative is to delete the member rather than compute it; either beats a permanent placeholder.*

5 **Printed** · the date a bill entered the print set, or *Not yet*, or — for a draft · **NEW READ**, and the largest of the eight: BR-BIL-016’s outbound queue has no table · **NEW**

6 **Download, enabled** · a `bills` resource beside `bill_run_releases` in the export registry; the CSV mirrors the list’s own `q` / `sort` / `f.*` state · BR-SHL-016 · **CHANGED** — the glyph is drawn today and does nothing

7 **A third row action** · `···` overflow carrying *Fix this bill*, so a correction starts from the row rather than from a tab that asks for a pasted id · UI-RULES 15 (two visible + overflow) · **NEW** — needs the overflow component this product does not have

8 **Column choices remembered** · `table.S30.columns` through the same preference service this screen already uses for its page size · UI-RULES 14 · **CHANGED**

What the drawing also shows, deliberately

A draft sorts to the top, and that is correct. The first row is `BL-2609-018323` with no issue date and no due date. `_apply_order` puts **nulls first on descending**, on purpose — *"a draft has no issue date and a draft run is the newest thing on the screen."* Your newest-first rule is satisfied here, and §06 now says so.

And the *Printed* column is drawn with three distinct values, not two. A dash for a draft (it cannot have been printed), *Not yet* for an issued bill that has not, and a date for one that has. Two values would collapse *"cannot"* into *"has not"*, which is the mistake this document has found five times elsewhere.

Carry-over

AS-BUILT ELEMENT	DISPOSITION	WHY
The past-due explainer	MOVED	Same sentence, same words, from above the table to a collapsed disclosure below it, plus a marker on the Status header. It is the only thing on this screen that moves.
Bills lifetime · Out this cycle · Unresolved from the last run	UNCHANGED ×3	All three stay, in the same order, in the same line. Two of them start carrying data they do not carry today — **a field whose data is broken still carries over**.
Pending queue · Print bills · Start a billing run →	UNCHANGED ×3	Same three, same order, same words. ③ adds a second route to the pending queue; it does not replace the button.
The six-tab strip	UNCHANGED	Untouched.
Search box · status segment · *+ Filter*	UNCHANGED ×3	The new chips sit beside them; the placeholder keeps its four fields and keeps not promising a cycle.
Download glyph · column chooser	UNCHANGED ×2	Both are already drawn. They start working.
Bill # · Cycle · Account · Customer · Issue date · Due date · Total · Status	UNCHANGED ×8	Same eight columns, same order, same sort.
Bill type · Template version *(hidden by default)*	UNCHANGED ×2	Still available, still hidden — and now the choice is remembered.
Actions (two icons)	UNCHANGED	Both icons stay where they are; the overflow is added after them.
Pager	UNCHANGED	Untouched.

```
As-built elements   24
  unchanged         23
  moved              1   -> the past-due explainer, collapsed below the table
  relabelled         0
  removed            0
Remedy elements     28   = 24 - 0 removed + 4 new
                           (cycle chip · date-range chip · Printed column · row overflow)
```

RECONCILES

** NOTHING IS REMOVED. ** Four of the eight changes touch no field at all -- they make a drawn control work (6, 8), turn a figure into a door (3), or collapse a sentence (1).

Order I would build them in, and it is not the order above. ② first — it is the one that changes a clerk's day, it is front-end only, and the server has been ready for it all along. Then ⑧ and ③, both trivial. Then ①, which is a rendering change. ⑥ needs one back-end resource. ④ is a decision before it is work — compute or delete. ⑦ waits on a component that does not exist anywhere in this product, so it is the one to defer rather than special-case here. ⑤ **is the most valuable and the most expensive**, and it needs a change request of its own: printing is the only channel that reaches a customer, and a clerk cannot presently tell whether it happened.

▶ **As built today** — kept because the findings below point at it. *Do not build this.*

For	A billing clerk finding one bill among a utility's whole issued history, and getting into it.
How you get here	**It is the door** — `SHELL_NAV` item 7, label *Billing*, `AppShell.tsx:232-238`, with **no submenu** (deliberate, BR-SHL-001). Also linked from the account's Bills tab (`BillsListPage.tsx:881` , "All bills →"), the Dashboard's to-do rows (`DashboardPage.tsx:990`) and global search (`GlobalSearchResults.tsx:220` , "View all Bills").
Primary action	*Start a billing run* →, a `GuardedButton` (`:1026-1031`) disabled only for a read-only clerk. It is the primary rather than *Print bills* because producing bills is the thing a clerk comes here unable to do; finding one they can already do with the search.
Tabs	Six, in the URL as a path segment: *All bills* · *Run bills* · *One-time bill* · *Budget Bills* · *Fix a bill* · *Past billing runs*. `BILLING_TABS` , `billingShared.tsx:105-113` . *Run bills* does not render a panel — the router matches `/billing/run` to the wizard first.
Table controls	**Search — server.** `?q=` over four fields (`searchFields` `:397`), resolved by `billing/selectors.py::_apply_search` , which ORs `invoice_no` | `account__account_no` with a customer-name lookup *and* an address walk through metering's own predicate. **BUT SEE DEF-BIL-4713** **Filter — server**, `f.status=` / `f.bill_type=` in BR-SHL-011's grammar. The prototype's segment strip renders as a declared filter, not a bespoke radio row. **Columns — chooser present**, and the *page size* is remembered per user (`table.S30.pageSize` , `:155`) through `GET/PUT /user-preferences/{key}` . Column *visibility* is not remembered. **Sort and pagination — SQL.** `StandardPagination` at `api.py:434-438` over a queryset; `selectors.py:935` `qs.order_by(ordered,"id")` against the `INVOICE_SORTS` whitelist. **NFR-3 COMPLIANT**
Write surfaces	**Detail** — the bill-number cell (`:715`) and a row *Open this bill* (`:815`). **Add** — **NOTHING**, correctly: a bill is produced by a run, never added to a list. **Edit** — **NOTHING**, and that is hard rule 4: an issued bill takes no UPDATE. **Update** — **NOTHING**. This screen is read-only by design and the module ships no write wrapper for it.
Walked	**SEEN** Rendered at 1440. Title measured **40px/800**; 13-item rail with *Billing* on a solid accent fill; six tabs; search, filter, download glyph, column chooser, pager all present; `scrollWidth - clientWidth = 0` . The past-due explainer above is the live string, not a paraphrase.
Cognitive load	It removes *"which of the three bill-shaped tables is this"* — one list, one grammar, and the row tells you its cycle so a clerk never has to hold the cycle in their head.
States	**Loading** — `LoadingSkeleton` of row height, never dashes. **Empty** — a titled empty state whose CTA is *Start a billing run*, so the screen says what would create content. **Failed** — A18 §4 with a plain-language reason derived from the response, *not* a hardcoded sentence.

	Domain — past-due, which this deployment cannot compute, so the explainer above stands on every visit.
Words	*Bill* throughout, never *invoice*, even though the table, the endpoint and the type are all `invoice`. The search placeholder deliberately drops r10's *"or cycles"* because the predicate cannot match a cycle (DEF-1001).
Components	`PageHeader` · `DataTable` · `FilterBar` · `MoneyCell` · `DateCell` · `StatusPill` · `RowActions` · `DownloadButton` · `GuardedButton`
Rules	B14 15 (two row icons + overflow), 16 (pagination), 19 (placeholder), 20 (13.5px body) · NFR-3 · NFR-5 · hard rule 2 (money) · hard rule 4 (immutability)
Audience	Staff. `reads_billing_runs`: System Admin Full · Billing Clerk Full · Executive View.
Health	**STUBBED** — the worst of 11 fields. 9 SERVED · 1 STUBBED · 1 NOT SERVED.

Data flow

```
invoices.invoice_no ──→ selectors.invoice_list_queryset (SQL page + sort)
                     └→ GET /api/{tenant}/invoices            (billing)
                     └→ getInvoices() → useQuery → <BillsTable/>  field ①
                                                              WHOLE

accounts.account_no ─⇄→ invoice.account (reviewed FK, B2 §2)
                      └→ same endpoint, same row            field ③  ⇄ billing→customers

premises.address_* ──⇄→ _account_ids_at_address (metering's own predicate)
                      └→ reached ONLY by ?q= — never rendered as a column
                                                  SEARCH-ONLY  ⇄ billing→metering
```

#	FIELD	COMES FROM	OWNED BY	HEALTH
①	Bill #	`invoices.invoice_no` via `GET /invoices`	billing	SERVED
②	Cycle	`invoices.cycle_label`, read through metering	billing → metering ⇄	SERVED
③	Account #	`accounts.account_no`	billing → customers ⇄	SERVED
④	Customer	the primary *contact*'s name until `customers` exists (BR-CUS-054)	billing → customers ⇄	SERVED
⑤	Issue date	`invoices.issued_at`, formatted by `DateCell` in the tenant's zone	billing	SERVED
⑥	Due date	`invoices.due_date` — derived at issue (BR-BIL-024), null on a draft	billing	SERVED
⑦	Total	`invoices.total` + `currency`, decimal strings, through `MoneyCell`	billing	SERVED
⑧	Status	`invoices.status`	billing	SERVED
⑨	Past due	*nothing — `selectors.py:961-963` serves `None` on every row*	—	STUBBED
⑩	Bill type / template version	`invoices.bill_type` · `template_version`, both hidden by default	billing	SERVED

⑪	Delivery status · run #	*no read exists — the outbound queue has no table (BR-BIL-016) and the runs list serves* `run_no` *null*	—	NOT SERVED

Two findings on this screen that a reader should not have to dig for.

① **The download glyph is a dead control.** `<DownloadButton disabled onDownload={() => undefined} />` at `:679-682`, titled *"Bills cannot be downloaded yet. No export is built for this list."* **The stated reason is still true** — I checked: `EXPORT_RESOURCES` holds twelve resources and `bills` is not among them. But billing *does* register an export now — `apps/billing/exports.py:236`, for release authorisations — so the machinery exists and only the resource is missing. The comment justifying the button still says *"there is no* `apps/billing/exports.py` *at all"*, and that half has rotted.

② **The stated blocker for the column chooser is gone.** `:44-47` says per-user column preference *"needs US-SHL-013's preference service, which is not wired."* It is wired, and **this same file uses it** for the page size (`:155`, `:190` → `GET/PUT /user-preferences/{key}`).

---

02 Bill detail

Blocked S22 · New build S9 S12 S27 S71 S86 S98 · New read S17 · Exists behind S20 S28

`/:tenant/billing/bills/:invoiceId` · tabs as `?tab=` · BIL-05 · **no A1 id**

"Is this the bill the customer received, and what is on it?"

BUILD THIS · New build

Nothing on the bill detail is lost

Across all five tabs, every element on the as-built drawings survives. Two buttons move into an overflow, one pill moves group, Delivery moves tab, two columns fold into a cell, and three sentences are promoted or merged.

AS-BUILT ELEMENT	DISPOSITION	WHY
Back link · kicker · bill number · `Issued` pill · account link	UNCHANGED ×5	Same words, same place.
`Past-due unknown` pill	MOVED	Out of the heading and into **Dates** as its single home. It already said the same thing twice on the live screen.
Service period · Due date	UNCHANGED ×2	Both stay in the identity block; the due date gains the amount beside it.
Print bill · Cancel this bill	MOVED ×2	Into the ⋯ overflow; Cancel keeps its reason. **Neither is removed.** ⚠ If you would rather *Print* stayed the primary, say so — printing is your only channel that reaches a customer, and that is a fair reading of it.
Fix this bill · View the bill as sent	UNCHANGED ×2	Both stay in the header; *View* becomes the filled primary.
Amount due — four rows and the caveat	UNCHANGED ×5	Group **Amount**. Every row and the caveat survive verbatim.

The embedded PDF and its viewer	REMOVED	**Ruled by the Product Owner, 2026-09-11:** *"we dont what full view of the document - you give a link to download but full view of document is not requires - it does not add any value to user"*. Replaced by a download and an open-in-new-tab control, plus the two facts about the saved copy. **The only REMOVED row in this document, and the only one with a ruling to quote.**
Customer's bill panel	UNCHANGED	Stays on Document.
Where this bill came from — five rows	RELABELLED	Becomes **Origin**, collapsed. All five rows kept and `RUN-0002` still linked.
How this bill was produced · Record check	MOVED	Merged into one collapsed **Record** group. *"Locked copy saved"* relabelled out of engineering language; both checks kept.
Delivery — two rows and Mark returned	MOVED	Onto the new **Sent** tab. Channel codes become words.
What each service cost	RELABELLED	**By service** on Charges, and summarised as one line under **Breakdown** on Document. The table stays.
Charges table — seven columns	MOVED ×2	*Accounting code* and *Rate version* fold under the description. Five columns, **no data lost**.
Subtotal · Tax · Total	UNCHANGED ×3	Still tie to the header.
Tax on this bill, by authority	RELABELLED	Group **Tax**. Rows and the "nothing records which service a tax belongs to" caveat kept.
When this is due — three rows and the caveat	MOVED	From Charges into **Dates** on Document, where the other dates already are.
Corrections — three sentences	MOVED ×3	One becomes a pill under **Standing**, one merges into it, one becomes the rule under **History**.
Audit — search · filter · columns · table · pager · log link · window note	UNCHANGED ×7	Only the sort direction, the timestamp format and the truncation control change.

```
As-built elements    43   (across the five tabs)
  unchanged          27
  moved              11   -> 2 buttons to the overflow · 1 pill to Dates · Delivery to Sent
                             When-this-is-due to Dates · 2 columns into a cell
                             Record merged · 3 sentences regrouped
  relabelled          4   -> Origin · By service · Tax · "Copy of what they received"
  removed             1   <- the embedded PDF viewer
                             PO 2026-09-11: "we dont what full view of the document -
                             you give a link to download but full view of document is
                             not requires - it does not add any value to user"
Remedy elements      51   = 43 - 1 removed + 9 new
                            (name · address · billed date · cycle name · copy control
                             Owed today · Usage · download control · open-in-new-tab)
                                                          RECONCILES

** ONE ELEMENT REMOVED, AND IT CARRIES ITS RULING. ** Everything else moves,
   is relabelled or stays. The two heaviest additions are reads that already
   exist: charges_by_service is served, and GET /accounts/{id}/usage is mounted.
```

```
Only "Owed today", the usage AVERAGE and the paperless election need a read
this module does not have today.
```

CORRECTED 2026-09-11 against the running product. The Product Owner sent screenshots of this screen from a live deployment (`ottumwa-water-works`, **v2.234.17** — one version behind the repo this document was read at). **Ten details in my drawings were wrong**, and every one is struck below rather than quietly fixed. The lesson is the one appendix F already names as the most expensive assumption here: *an open defect row is a claim about the past.*

~~WAS: header buttons in the order *View the bill as sent · Print bill (disabled) · Fix this bill · Cancel this bill (disabled);* one *Issued* pill; *Amount due* as a KPI band of four tiles; the Document panel showing a "cannot be opened yet" refusal; the run as a bare id; the accounting code as "Code not readable yet"; tax-by-authority as not available; one delivery channel; the printed bill unstyled Courier.~~ **NOW: all ten corrected in the drawings below.** The list is in §10.

FOR	Reading one bill exactly as it was issued — the frozen document, its charges, what corrected it, and where it went.
HOW YOU GET HERE	Seven audited doors: the bill-number cell and the row *Open this bill* on the bills table; the account's Recent-bills panel and its Credits tab; global search; the document page's back-link; and the correction-chain links on the screen itself.
PRIMARY ACTION	**There is none.** Four equal-weight outlined buttons, one of them permanently disabled. THE HEADING HAS NO ANSWER TO *WHAT DO I USUALLY DO HERE*
TABS	**Four, each drawn in full below as 02.1–02.4**, in the URL as `?tab=`: *Document* · *Charges* · *Corrections* · *Audit*. ~~WAS: Document · Lines · Chain · Arrives.~~ The ids are `document · lines · chain · audit`; the labels are not the ids, and there is no *Arrives* tab — delivery is a panel on the Document tab.
TABLE CONTROLS	The Charges tab holds three tables and the Audit tab one. **Only the Audit table has any controls** — search, a filter, a column chooser and a pager — and they are the *audit* module's, because that tab reuses S61's viewer. The three billing tables have none, which is right: a bill has the charges it has.
WRITE SURFACES	**Detail** — this is it. **Add** — **NOTHING**. **Edit** — **NOTHING, and that is hard rule 4**: no edit affordance anywhere, no write wrapper. **Update** — only *Mark returned* on a delivery row, and *Fix this bill*, which opens a *new* correction case rather than updating this row.
WALKED	SEEN, BY THE PO Five screenshots of a real issued bill on a live tenant, 11 September 2026. **This is the only screen in the document seen with real data behind it**, and it moved ten findings.
COGNITIVE LOAD	It removes *"is what I am looking at what they got?"* — the document re-renders from the pinned template, never from live joins. **It does not remove "whose bill is this?"**: the customer's name and address are in the PDF and nowhere on the screen.
STATES	**Loading** — card skeleton. **Empty** — a bill with no charge lines says so. **Failed** — *"We could not read this bill…"*. **Domain** — draft, where three header controls disable with different reasons. DEF-2901: NO DATABASE IN THIS REPO HAS A DRAFT BILL, SO THREE OF THOSE FOUR REFUSAL STATES HAVE NEVER BEEN RENDERED
WORDS	*Bill* everywhere, never *invoice* — while the table, endpoint, type and serializer all say `invoice`. *Cancel this bill*, not *void*. Two slips: *"Locked copy saved"* is engineering language (`DEF-2466`), and the delivery table prints `pdf_print` and `portal` at a clerk.
COMPONENTS	`PageHeader` · `Panel` · `PlainTable` · `GuardedButton` · `MoneyCell` · `NotRecorded` · metering's `AuditTab`
RULES	hard rule 4 · hard rule 2 · BR-BIL-030 · BR-BIL-026 · BR-CUS-040 · B14 4 (arrow on a navigating button — `DEF-1151`) · B14 32 (no internal code in clerk-facing copy — **breached three times**)

AUDIENCE	Staff.
HEALTH	**Better than I graded it.** Of the twelve fields on the four tabs, **ten are SERVED and rendering**; the two that are not are the payments detail and the meter reads, neither of which the screen asks for. Its faults are heading, grouping and vocabulary — not data.

What the three header controls actually do

Two of the three open a screen this document already draws. One opens no screen at all — it hands the browser a PDF — and that is worth writing down because it is the only control in Billing that leaves the product.

CONTROL	WHAT HAPPENS	SCREEN
Print bill	**No screen. It is an `<a href>` to a PDF.** `BillDetailPage.tsx:345-346` builds `printHref = \`/api${tenantPath(documentQuery.data.pdf_url)}\`` — so the browser opens or downloads `GET /api/{tenant}/invoices/{id}/document.pdf` (`documents_api.py:454`) and the browser's own PDF viewer takes over, with its own print dialog. **It is a `GuardedButton` with three distinct refusals**, each true for that bill: *"No saved copy was recorded for this bill, so there is nothing to print"* · *"A draft has not been issued, so there is no saved copy to print"* · *"Checking whether a printable copy exists."* The third is a real third state, not a loading fudge. ⚠ **Reaching the PDF is the closest this product comes to delivering a bill** — nothing dispatches, so a clerk pressing this button *is* the delivery mechanism.	NONE — BY DESIGN
Fix this bill	A link to `/billing/fix?bill=<invoice uuid>` (`billingOpsShared.tsx:363` → `BillDetailPage.tsx:420`) — S30's Fix-a-bill tab with this bill already chosen. The bill travels **in the URL rather than in a prop**, and the code says why: *"a prop nothing passes is dead UI… while a query parameter makes the pre-filled screen a real address."* **Disabled on a draft only**, with the right reason: *"A draft is not corrected — it is re-run. Corrections apply to bills the customer has received."* ⚠ **And this is the door that creates the rule-31 breach.** It arrives on the Fix-a-bill screen having put the invoice **UUID into a visible text box labelled *Bill***. The door is right; what it lands on is not.	→ screen 13
View the bill as sent	A link to `/billing/bills/:invoiceId/document` (`BillDetailPage.tsx:434`) — a whole screen of its own. **Rendered only when `document_url` is non-null** (`:433`), so it does not appear on a bill that has no saved copy; the control is *absent* rather than disabled, which is the opposite treatment to *Print bill* beside it. ⚠ **Two controls, two different refusal strategies, side by side.** *Print* disables with a reason; *View* vanishes. A clerk cannot learn one rule from the pair. ⚠ And the ruling of 2026-09-11 — *"full view of document is not requires"* — puts a question over this screen's whole purpose, since a full view is all it is. Raised, not assumed.	→ screen 03
Cancel this bill	Nothing. `onClick={() => undefined}`, and the reason chain has no branch that enables it. Included here for completeness: it is the fourth control in the header and it can never be pressed on any bill in any state.	NONE — PERMANENTLY DISABLED

So of four header controls: one opens a PDF outside the product, two open screens, and one does nothing. That is the argument for the single primary in R5.1 — the header currently presents four equal-weight buttons whose behaviours could not be more different.

02.1 Bill detail · Document | SEEN LIVE Blocked S22 New build S98 New read S17 S19 Exists behind S18 S20

TWO REVISIONS This entry holds **both**: an **AS-BUILT** drawing of 2026-09-11 and a **REDESIGN** that has **never been built**. **Build the REDESIGN** — it is the first drawing below. The as-built one is kept as the evidence §09–§11 argue from; it is marked *do not build this*.

`…?tab=document · the default tab · BIL-05`

“Show me the bill they actually got, and tell me it is intact.”

BUILD THIS New read **Bill detail · Document — the new heading, and the overview grouped** CLOSES BIL-S17 · BIL-S18 · BIL-S19 · BIL-S20

REDESIGN drawn **2026-09-11**, still unbuilt at Version 35 (2026-09-13), drawn to B14 31 · B14 37 · the Product Owner’s 2026-09-11 ruling removing the embedded viewer. **NEVER BUILT.** Today the product ships the as-built bills list and bill detail directly below this drawing.

SMARTBillHub

Billing

Search ? α

Demo Water & Light
demoutility

- Dashboard
- Plans & Rates
- Premises
- Assets
- Accounts
- Reads
- Billing
- Payments
- Work
- Finance
- Runbooks
- Reports
- Settings

← Billing

BILL · CYCLE BILL

BL-2608-000043 8

6 View the bill as sent | Fix this bill | ···

Issued 5

1 **Clara Castillo** · 105 Cedar Ridge Way, Unit 1, Silver Creek TX 78611 · AC-2026-00003
2 **$251.45** due **08/22/2026** · 3 billed 08/02/2026 for 07/01 – 07/31 · 4 Silver Creek — monthly

7 Document Charges Sent 2 Corrections 0 Audit

AMOUNT

Previous balance	$0.00
Payments received	$0.00
Current charges	$251.45
Total due	**$251.45**
9 Owed today on this account →	$251.45

The four figures above are the ones saved on the bill the day it was issued. **Owed today** is the live balance on the account.

10 **BREAKDOWN** why it is $251.45

Water **$132.02** · Sewer **$94.50** · Trash **$15.00** · Battery **$5.00** · Solar **$0.00** · Fire **$0.00** · Tax **$4.93** See every charge →

11 **USAGE**

SERVICE	THIS PERIOD	AVERAGE	READ
W	19 kgal	17.4 kgal	M-8841 · 08/01
S	17.1 kgal	15.7 kgal	from water volume
Ba So F	0	0	3 meters

12 **DATES**

Billed	08/02/2026
Service period	07/01 – 07/31/2026 · 31 days
Due	08/22/2026
Past due	Not checked on this deployment
Fees on this bill	None

14 **THE BILL**

⤓ Download the bill as sent (PDF) **Open it in a new tab ↗**

Saved copy	Kept — 2 pages
Template	v1

The bill a customer received is re-printed exactly as it was sent, from the copy saved when it was issued. A re-print six months later shows July's name and address, not today's.

13 ▸ **Origin**

13 ▸ **Record**

v2.235.0

BIL-05

1 **Who this is** · `customers.name` · address by walking to `premises` + the unit label (hard rule 8) · both linked · **NEW READ** — the name and address are in the PDF only today

2 **How much, and when** · `invoices.total` · `due_date`, through `MoneyCell` · hard rule 2 · **MOVED** from a panel below the fold

3 **Billed on** · `invoices.issued_at` · **NEW** — three dates matter and only two were shown

4 **The cycle, by name** · `cycles.label` — the clerk's word for *which run was this* · **NEW** (the id `CY-001` stays under Origin)

5 **One pill, not two** · `Past-due unknown` leaves the heading and lives once, under **Dates** ⑫ · B14 32 · **MOVED**

6 **One primary, and an overflow** · *View the bill as sent* filled; *Fix this bill* secondary; *Print bill* and *Cancel this bill* into ⋯, Cancel keeping its reason · UI-RULES 4–6 · **CHANGED**

7 **Five tabs, with counts** · *Sent* split out of Document; `Corrections 0` and `Sent 2` so a clerk never clicks to find nothing · **NEW**

8 **Copy the bill number** · it gets read aloud and pasted into Fix-a-bill all day · **NEW**

9 **Owed today** · the live account balance, linked · **NEW READ** — every other figure here is as-at-issue and nothing answered *what do they owe now*

10 **Breakdown** · `charges_by_service`, already served and already on the Charges tab as a table · **MOVED** (a one-line summary; the table stays)

11 **Usage** · `meter_reads` through `GET /accounts/{id}/usage`, which **is mounted** · **NEW READ** — the second question after *how much* is *why is it higher*, and nothing answered it

12 **Dates** · absorbs *When this is due* from the Charges tab and the pill from ⑤ · **MOVED**

13 **Origin and Record, collapsed** · *Where this bill came from* and *How this bill was produced* + *Record check*, merged and closed by default · B14 32 · **MOVED**

Seven groups, one word each, in the order a clerk asks them: Amount → Breakdown → Usage → Dates → the document itself → **Origin** → **Record**. The first four answer *how much, why, why that much water, and when*. The last two are reassurance and are closed.

The embedded viewer is gone, on a ruling — and it is the first thing this document is permitted to remove. Product Owner, 2026-09-11, verbatim: *"we dont what full view of the document - you give a link to download but full view of document is not requires - it does not add any value to user"*. Every carry-over table before this one reconciles with `removed = 0` because no ruling existed; this one does not.

It makes the tab better than the change suggests. The tab stops being a document reader and becomes what the groups above already made it — the bill's facts, with the document available in one click. The two things a clerk actually needed from the viewer (does a saved copy exist, and can I send it to someone) are now stated rather than inferred from a rendered page.

Two questions it raises that are yours, not mine. ① The tab is still called *Document*. With the document reduced to a download, the name no longer describes the tab — it holds Amount, Breakdown, Usage and Dates. *Overview* or *Summary* would be truer, and it is the default tab, so the name is the first word a clerk reads. **② Does the ruling extend to screen 03?** `/billing/bills/:id/document` is a whole screen whose only purpose *is* the full view. If a full view adds no value, that screen is a download link with a page around it. I have not assumed either way.

▸ **As built today** — kept because the findings below point at it. *Do not build this.*

PANELS	**Six** — *Amount due*, then four in a two-column grid paired by subject, then *Delivery* last. The grid is a fix that already landed (CR-043 finding 8, PO: *"can you manage the layout of documents - currently it is disorganizaed"*).
WHAT CORRECTED	~~WAS: the panel renders the "saved copy cannot be opened yet" refusal.~~ **NOW: on a real issued bill it does not fire** — the link is live and the PDF embeds with its own print and download. That rotting hold is *conditional* and fires only when no template is pinned. ~~WAS: the printed bill is unstyled Courier.~~ **NOW: fixed 2026-09-07** — `documents.py:1741` injects an Archivo stylesheet, and the drawing above is what it looks like.
FINDINGS THAT STAND	**Delivery is the last panel on the first tab**, below four panels of provenance — and *did the customer get it* is a top-three question. The channel names are **raw codes**. *"Locked copy saved: Yes"* is engineering language for *we kept a copy of exactly what they received*.
WALKED	SEEN LIVE

02.2 Bill detail · Charges | SEEN LIVE Blocked S22 New read S17 Exists behind S20 S23

TWO REVISIONS This entry holds **both**: an **AS-BUILT** drawing of 2026-09-11 and a **REDESIGN** that has **never been built**. **Build the REDESIGN** — it is the first drawing below. The as-built one is kept as the evidence §09–§11 argue from; it is marked *do not build this*.

…?tab=lines · the tab id is `lines`; the label is *Charges* · BIL-05

"What is the $251.45 actually made of, and which rates priced it?"

BUILD THIS Exists behind **Bill detail · Charges — *Rate version* folded under the description**

REDESIGN drawn **2026-09-11**, still unbuilt at Version 35 (2026-09-13), drawn to B14 31 · B14 37 · the Product Owner's 2026-09-11 ruling removing the embedded viewer. **NEVER BUILT.** Today the product ships the as-built bills list and bill detail directly below this drawing.

SMARTBillHub

Billing

Search ? α

Demo Water & Light
demoutility

- Dashboard
- Plans & Rates
- Premises
- Assets
- Accounts
- Reads
- Billing
- Payments
- Work
- Finance
- Runbooks
- Reports
- Settings

← Billing

BILL · CYCLE BILL

BL-2608-000043

View the bill as sent | Fix this bill | ···

Issued

Clara Castillo · 105 Cedar Ridge Way, Unit 1, Silver Creek TX 78611 · AC-2026-00003

$251.45 due **08/22/2026** · billed 08/02/2026 for 07/01 – 07/31 · Silver Creek — monthly

Document | Charges | Sent 2 | Corrections 0 | Audit

AMOUNT

Previous balance	$0.00
Payments received	$0.00
Current charges	$251.45
Total due	**$251.45**

BY SERVICE

SERVICE	LINES	AMOUNT
W	4	$132.02
S	2	$94.50
T	2	$15.00
Ba	2	$5.00
So F	4	$0.00
Tax — charged to the bill, not a service	1	$4.93

EVERY CHARGE

DESCRIPTION	USAGE	RATE	AMOUNT	TAXABLE
WATER				
water usage tier 1 (0-5 kgal) 4110 — Water usage · Residential water (inclining tiers), from 08/01/2026	5 kgal	$4.2400	$21.20	Yes
water usage tier 2 (5-15 kgal) 4110 — Water usage · Residential water (inclining tiers), from 08/01/2026	10 kgal	$6.8900	$68.90	Yes
water base charge 4115 — Water base	—	—	$8.00	Yes
SEWER				

sewer usage — 90.00% of water volume 4120 — Sewer · Sewer (% of water), from 06/01/2026	17.1 kgal	$5.0000	$85.50	Yes
Subtotal			$246.52	
Tax			$4.93	
Total			**$251.45**	

14**The bill, as a download — not an embedded viewer** · two controls and the two facts a clerk needs about the saved copy · **REMOVED (the viewer)**, ruled by the Product Owner 2026-09-11: *"we dont what full view of the document - you give a link to download but full view of document is not requires - it does not add any value to user"*

v2.235.0

TAX

PAID TO	DESCRIPTION	AMOUNT
Bexar County Tax Office	Tax Local	$4.93
Total tax		**$4.93**

A tax is charged on the bill, not on one service. Nothing records which service a tax belongs to.

BIL-05

Two changes, and neither loses a field. ① *Accounting code* and *Rate version* come out of their own columns and ride **under the description**, the way the accounting code already shares one cell today. That takes the table from seven columns to five, and stops the widest, least-used column wrapping to two lines on every row. ② The four panels get one-word group labels — **Amount · By service · Every charge · Tax** — so the tab reads as four questions rather than five headings.

Taxable is kept even though it reads *Yes* on every row of this bill: it is a dash on a tax line because B4.2 makes the question inapplicable, and that distinction is worth a column. **Nothing is removed.**

▶ **As built today — kept because the findings below point at it.** ***Do not build this.***

PANELS	Five. *Amount due* repeats here deliberately — *"Document is the bill; Charges is what the total is made of."*
WHAT CORRECTED	~~WAS: the accounting code renders as "Code not readable yet".~~ **NOW: codes are readable** — `4110 — Water usage`. My drawing showed the fallback as though it were the norm. ~~WAS: a per-authority tax subtotal is not available yet.~~ **NOW: the panel is BUILT** — Bexar County Tax Office, $4.93, with an authority subtotal that ties.
THE BEST TABLE IN BILLING	Seven columns grouped by service in accent caps, with the tier structure visible (*tier 1 · tier 2 · tier 3* at three rates), the derived sewer line stating its own rule (*90.00% of water volume*), a tax line carrying its authority, and Subtotal / Tax / Total tying to the header. **A clerk can answer "why is it $251.45" from this table alone.**
THE ONE LAYOUT FAULT	*Rate version* is the widest column, wraps to two lines on most rows, and is the least-used thing on the screen. *Taxable* reads *Yes* on every row and a dash on the tax line — correct (B4.2 makes the question inapplicable), and it is 100% redundant on this bill.
WALKED	SEEN LIVE

Is the content properly aligned? No — six things, and they are all the same kind

#	WHAT IS MISALIGNED	WHY IT MATTERS
1	**Usage is left-aligned beside two right-aligned money columns.** `0 kW`, `5 kgal`, `17.1 kgal`, `1 cart`, `0 connection` all sit left while *Rate* and *Amount* sit right.	Three numeric columns, two conventions. The shared table's own rule is `.cell-num { text-align: right; font-variant-numeric: tabular-nums }` — a quantity is a number. Right-aligned, the tiers 5 / 10 / 4 would line up and read as a total; left-aligned they do not.
2	***Rate version* wraps mid-date** — `Sample — Residential water (inclining tiers) · from 2026-` then `08-01` on the next line.	It is the widest column, the least-used value on the screen, and it breaks a date across two lines on most rows. R5.2 folds it under the description.
3	**The accounting code wraps too** — `2200-01 — Tax payable — Local · Bexar` / `County`.	Same cause: two long text columns at the right end of a seven-column table.
4	***Total* is not emphasised.** *Subtotal · Tax · Total* all render at body weight, while the header panel's *Total due* is bold.	Two totals for one bill, on one screen, styled differently. The bill's total is the figure a clerk reads aloud.
5	**The em-dash placeholders sit on opposite sides.** A base charge shows `—` left in *Usage* and `—` right in *Rate*.	Consequence of ①. Absence should look the same in every column.
6	**Row heights are ragged.** The tax row is two lines in *Description* and two in *Accounting code*; most water rows are two lines in *Rate version*; base charges are one line.	Nothing groups a tall row to its neighbours, so the eye loses the line it is reading — the same complaint that produced the Document tab's grid fix (CR-043 finding 8).

All six trace to one decision: seven columns, two of them long text, in a table type that is one size product-wide. Fold the two text columns under the description and the other four straighten out on their own.

What is missing from this tab

#	MISSING	IS THE DATA THERE?
1	**Usage per service — the biggest one.** Water is three tiers, `5 + 10 + 4 kgal`. **Nowhere does this screen say Water was 19 kgal.** A clerk has to add the tiers in their head, and *What each service cost* shows *Lines* and *Amount* but not usage.	`YES` — every line already carries its quantity and unit. A *Usage* column on the service summary is a sum of rows already on the payload.
2	**Which meter, and which read.** 19 kgal came from a meter and a pair of readings. Neither is named, and *"my bill is too high"* is answered by looking at the reading.	`A READ EXISTS` — `GET /accounts/{id}/usage` is mounted; tying a bill line to the read that priced it is new.
3	**Last period, beside this one.** Per service, so *"why is it higher"* is answerable on the screen that shows the charges.	`DERIVABLE` — the account's earlier invoices are served; nothing joins them here.
4	**Days in the period, and whether it was prorated.** The PDF says `Days 31`. The screen does not. A move-in, move-out or mid-cycle plan change produces a part period and **nothing on this tab would say so.**	`YES` — the period dates are on the payload; 31 is arithmetic.
5	**The plan the account is on.** Each line names its *rate version*; no line names the *plan*, and a clerk asked *"what are they on?"* cannot answer from here.	`CROSS-MODULE` — rating owns it; billing reads a version, not a plan name.
6	**Discounts and adjustments.** An *Adjustments on this bill* panel exists in the code and does not appear on this bill — correctly, there are none. But a clerk cannot tell *"no discount applied"* from *"discounts are not shown here"*.	`YES` — the panel exists; it needs a stated empty state.

7	*Which service each tax belongs to*	OUT OF SCOPE — the screen already says so, honestly: *"A tax is charged on the bill, not on one service. Nothing records which service a tax belongs to."* Listed so nobody re-files it.

Item 1 is the one to fix first, and it is nearly free. A *Usage* column on *What each service cost* — `Water 19 kgal · $132.02` — is a sum of quantities already on the payload, and it turns a table a clerk reads twice into one they read once. **For a water utility, usage is the reason the bill is what it is**, and this tab currently shows it only broken into tiers.

02.3 Bill detail · Corrections | SEEN LIVE Blocked S22 New read S17 Exists behind S20 S21

TWO REVISIONS This entry holds **both**: an **AS-BUILT** drawing of 2026-09-11 and a **REDESIGN** that has **never been built**. **Build the REDESIGN** — it is the first drawing below. The as-built one is kept as the evidence §09–§11 argue from; it is marked *do not build this*.

…?tab=chain · the tab id is `chain`; the label is *Corrections* · BIL-05

"Is this still the bill that counts, or has something replaced it?"

BUILD THIS Exists behind Bill detail · Corrections — two groups, and a way out

REDESIGN drawn **2026-09-11**, still unbuilt at Version 35 (2026-09-13), drawn to B14 31 · B14 37 · the Product Owner's 2026-09-11 ruling removing the embedded viewer. **NEVER BUILT.** Today the product ships the as-built bills list and bill detail directly below this drawing.

SMARTBillHub

Billing

Search ? α

Demo Water & Light
demoutility

- Dashboard
- Plans & Rates
- Premises
- Assets
- Accounts
- Reads
- Billing
- Payments
- Work
- Finance
- Runbooks
- Reports
- Settings

← Billing

BILL · CYCLE BILL

BL-2608-000043

View the bill as sent | Fix this bill | ···

Issued

Clara Castillo · 105 Cedar Ridge Way, Unit 1, Silver Creek TX 78611 · AC-2026-00003

$251.45 due **08/22/2026** · billed 08/02/2026 for 07/01 – 07/31 · Silver Creek — monthly

Document Charges Sent 2 Corrections 0 Audit

STANDING

This is the current bill

Nothing has replaced it, and no credit note or replacement bill is recorded against it.

HISTORY

Nothing to report.

Correcting a bill never changes it. The original stays exactly as the customer received it, and the correction is a credit note and, where one is needed, a replacement bill.

Fix this bill

v2.235.0

BIL-05

Three stacked sentences become two labelled groups and one action. Standing answers the question the tab exists for, as a pill rather than a sentence. **History** holds what has happened — nothing, here — and carries the rule about *why* nothing ever overwrites. And *Fix this bill* appears **on the tab about corrections**, which it currently does not: today the only route is the header button. **Every sentence on the live tab survives** — one is promoted to a pill, one is merged into the standing line.

BUILD THIS Exists behind **Bill detail · Corrections — the populated state** CLOSES BIL-S21

REDESIGN drawn **2026-09-11**, still unbuilt at Version 35 (2026-09-13), drawn to B14 31 · B14 37 · the Product Owner's 2026-09-11 ruling removing the embedded viewer. **NEVER BUILT.** Today the product ships the as-built bills list and bill detail directly below this drawing.

SMARTBillHub

Billing

Search ? α

Demo Water & Light
demoutility

- Dashboard
- Plans & Rates
- Premises
- Assets
- Accounts
- Reads
- Billing
- Payments
- Work
- Finance
- Runbooks
- Reports
- Settings

← Billing

BILL · CYCLE BILL

BL-2608-000043 8

6 View the bill as sent | Fix this bill | ···

Issued 5

1 **Clara Castillo** · 105 Cedar Ridge Way, Unit 1, Silver Creek TX 78611 · AC-2026-00003

2 **$251.45** due **08/22/2026** · 3 billed 08/02/2026 for 07/01 – 07/31 · 4 Silver Creek — monthly

7 Document | Charges | Sent 2 | Corrections 0 | Audit

1 STANDING

Replaced Open BL-2609-000051, the bill that counts now →

This bill stays exactly as the customer received it. A replacement bill was issued on 09/03/2026 and it is the one that counts.

2 WHAT HAPPENED

WHEN	WHAT	REASON	WHO	EFFECT
08/02/2026	**BL-2608-000043** issued this bill	—	Vijay Bikkad	$251.45
09/02/2026	Correction opened case CC-0041	Wrong reading used	Dana Reyes	—
09/03/2026	Approved	—	Vijay Bikkad System Admin	—
09/03/2026	CN-2609-000012 credit note cancels this bill	Wrong reading used	Dana Reyes	–$251.45
09/03/2026	BL-2609-000051 replacement bill the bill that counts now	—	Dana Reyes	$198.30

3	**Net effect on the customer**	**−$53.15**

Correcting a bill never changes it. The original stays exactly as the customer received it, and the correction is a credit note and, where one is needed, a replacement bill.

4 **CREDIT NOTES**

CREDIT NOTE	ISSUED	REASON	AMOUNT
CN-2609-000012	09/03/2026	Wrong reading used	−$251.45

5 **Fix this bill again** A further correction applies to the replacement bill, not to this one.

v2.235.0

BIL-05

1 **Standing, and the way out** · `chain.superseded` as a *Replaced* pill, with `chain.openCurrent` — *"Open BL-..., the bill that counts now"* — as a link. **Both strings already exist and are already served.** The failure this prevents is *a superseded bill quoted to a customer*, which is the reason the tab exists · **UNCHANGED DATA, NEW LAYOUT**

2 **What happened — one row per event, in order** · issued → correction opened → approved → credit note → replacement. Reads `GET /invoices/{id}/chain` plus the correction case · **NEW** — today the chain renders as prose and a credit-note table with no sequence

3 **Net effect on the customer** · `−$53.15` — the one figure a caller actually asks for, and the one nothing on the screen computes today · hard rule 2 · **NEW**

4 **Credit notes** · `detail.credit_notes` with its four served columns, kept as its own table because a credit note is a document in its own right · `DEF-2448` the *Reason* column prints the machine code today (`wrong_read_used`); drawn here as the sentence · **UNCHANGED, RELABELLED VALUE**

5 **Fix this bill again, with the truth beside it** · a further correction applies to the *replacement*, not to this bill — stated rather than discovered · **NEW**

Why this drawing is worth locking. Nobody has seen this state. `DEF-2901` records that *no database in this repo has a draft bill*, and the same is true here — this repository has no corrected bill either, so the three standings, the credit-note table and the replacement link have **never been rendered together**. Every figure above is an illustration (B14 rule 37) and none is a served value.

The one thing it must get right is the arithmetic of a correction, and that is the repo's own law. Hard rule 4: an issued bill takes no `UPDATE` — corrections are new rows. So the original stays at `$251.45` for ever, a credit note cancels it at `−$251.45`, a replacement charges `$198.30`, and the customer is `$53.15` better off. **Three documents, not one edited one** — and the row at ③ is the only place a clerk could read that without doing the sum themselves.

▸ **As built today — kept because the findings below point at it. *Do not build this.***

PANELS	One, holding **three sentences and nothing else** — and **no *Amount due***, which is right. Until CR-043 finding 9 the summary opened all four tabs and here it *"pushed the tab's actual subject below the fold."*
THREE STANDINGS	It answers one question three ways and names the replacement rather than leaving it to be found: `THIS IS THE CURRENT BILL` `REPLACED` — with *"Open BL-..., the bill that counts now"* as a link — and `WE COULD NOT CHECK`, a third state rather than a guess. *A superseded bill quoted to a customer* is the failure it exists to prevent.
	A whole tab for three sentences, and a clerk must click it to learn there is nothing. The tab label could carry the count. When a correction

THE FINDING	*does* exist this tab earns its place; empty, it costs a click to tell you so.
WALKED	SEEN LIVE

02.4 Bill detail · Audit | SEEN LIVE Blocked S22 New read S17 Exists behind S20

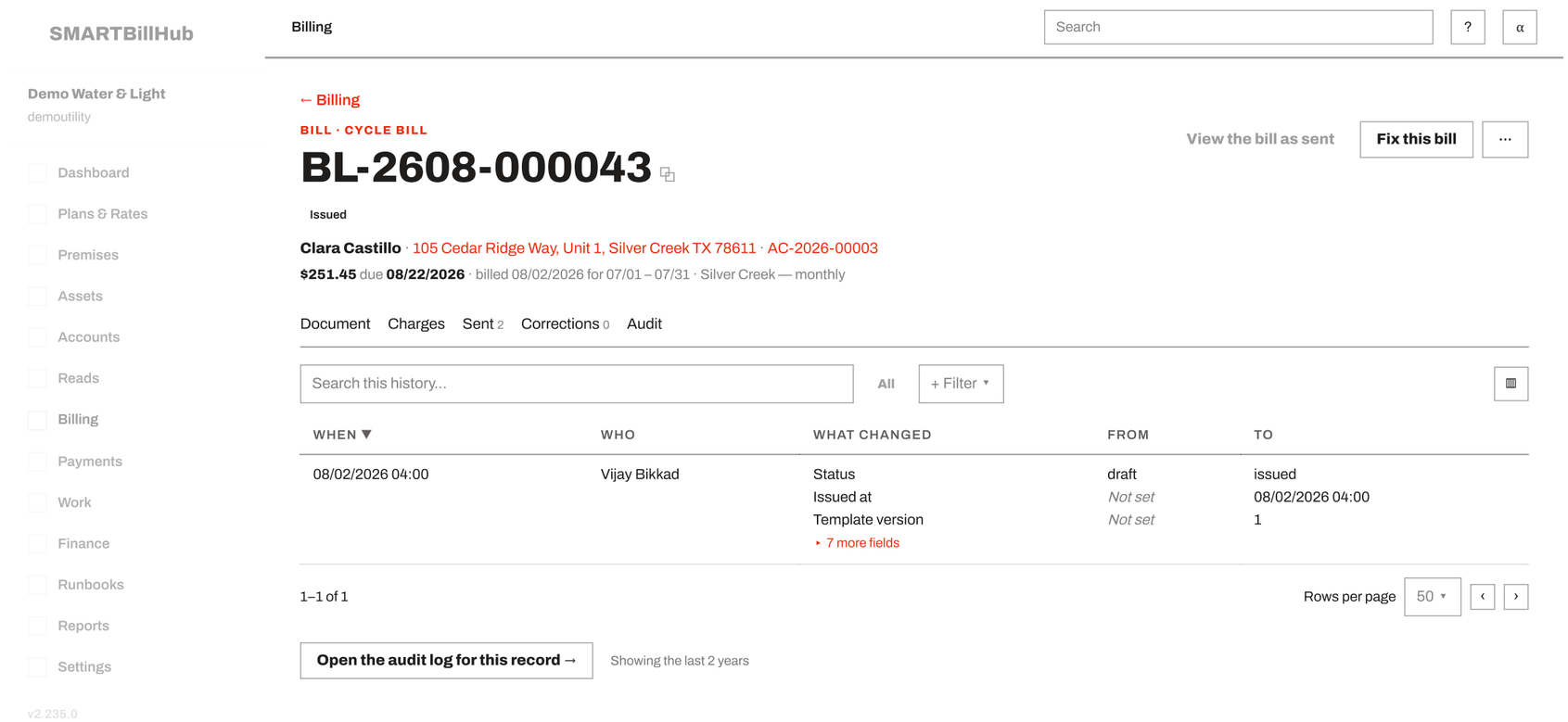
TWO REVISIONS This entry holds **both**: an **AS-BUILT** drawing of 2026-09-11 and a **REDESIGN** that has **never been built**. **Build the REDESIGN** — it is the first drawing below. The as-built one is kept as the evidence §09–§11 argue from; it is marked *do not build this*.

…?tab=audit · BIL-05

“Who touched this bill, and what did they change?”

BUILD THIS Blocked **Bill detail · Audit — newest first, dates in words, nothing hidden**

REDESIGN drawn **2026-09-11**, still unbuilt at Version 35 (2026-09-13), drawn to B14 31 · B14 37 · the Product Owner’s 2026-09-11 ruling removing the embedded viewer. **NEVER BUILT.** Today the product ships the as-built bills list and bill detail directly below this drawing.



BIL-05

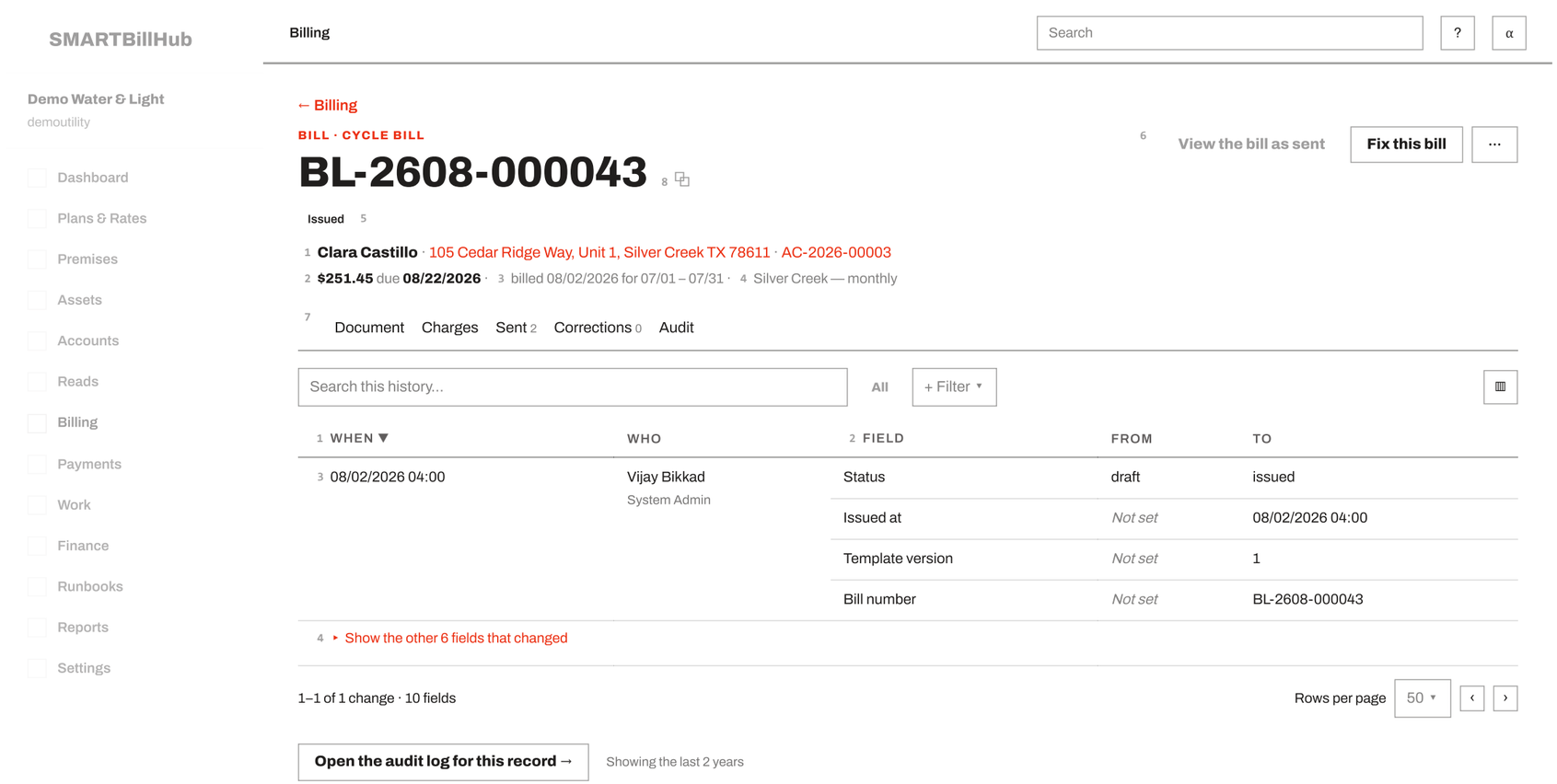
Three fixes, and none of them is a redesign — this tab is already the strongest in Billing.

① `When ▼` — **newest first**. It is sorted oldest-first today, on a history, against the standing newest-first rule. ② The raw `2026-08-02T09:00:00+00:00` becomes `08/02/2026 04:00` in the tenant's zone, through the one date formatter (BR-SHL-034). ③ `+7 more` becomes `▸ 7 more fields` that **expands in place** — today it truncates a ten-field change with no way to see the other seven without leaving the screen.

⚠ **This tab is borrowed, so these three are not billing's to fix locally.** It is metering's `AuditTab`, the S61 viewer, used by every detail screen in the product — *"one audit surface keeps one set of masking and window rules."* Changing it here changes it everywhere, which is the right outcome and the wrong blast radius to take on inside a billing change. **It needs its own change request.**

BUILD THIS **Blocked** **Bill detail · Audit — one row per field, not three parallel lists** CLOSES BIL-S22

REDESIGN drawn **2026-09-11**, still unbuilt at Version 35 (2026-09-13), drawn to B14 31 · B14 37 · the Product Owner's 2026-09-11 ruling removing the embedded viewer. **NEVER BUILT.** Today the product ships the as-built bills list and bill detail directly below this drawing.



1 **Newest first** · `When ▼`. It is sorted *oldest* first today on a history, against the standing newest-first rule · **CHANGED**

2 **One row per changed field** · *Field* · *From* · *To* as real table rows, with the change's date and author spanning them · **CHANGED — and this is the important one**

3	**The date in the tenant's own format** · `08/02/2026 04:00`, through the one date formatter (BR-SHL-034). The live screen shows a clerk `2026-08-02T09:00:00+00:00` · **CHANGED**
4	**The rest expand in place** · `▸ Show the other 6 fields that changed`, and the pager says *1 change · 10 fields* so the count is never hidden · **CHANGED**

The real fault here is not the timestamp — it is that three lists are aligned by luck. The live screen puts *Status / Issued at / Template version / +7 more* in one cell, *draft / Not set / Not set* in the next, and *issued / 2026-08-02T09:00:00+00:00 / 1 / +7 more* in the third. **Their correspondence is positional** — the second line of one cell is meant to pair with the second line of the next.

And one of those values is a 25-character ISO string, so it wraps. The moment any value wraps, the lists slip against each other and *a clerk reads the wrong old value against the wrong field.* On this bill it is nearly harmless. On a corrected bill, on a money field, it is a clerk telling a customer the wrong thing with the screen apparently agreeing. There is also no *+7 more* in the *From* column, so even the truncation does not line up.

⚠ **This tab is borrowed and the fix is not billing's to make.** `AuditTab` comes from metering's `PremisesDetailPage` — the S61 viewer every detail screen in this product uses, deliberately, so that *"one audit surface keeps one set of masking and window rules."* Fixing the alignment here fixes it on every screen, which is the right outcome and the wrong blast radius to take inside a billing change. **It needs its own change request.**

▶ **As built today** — kept because the findings below point at it. *Do not build this.*

WHAT RENDERS IT	**Not billing's own component.** `import { AuditTab } from "@/features/metering/PremisesDetailPage"` — the S61 viewer filtered to `entityType="invoice"`, so *"one audit surface keeps one set of masking and window rules."*
MECHANICALLY THE STRONGEST TAB IN BILLING	Search, a filter, a column chooser **and a pager** — the only table in the module with all four, and it has them because it is borrowed. Plus *Open the audit log for this record →* and an honest *"Showing the last 2 years"*.
THREE FAULTS, ALL VISIBLE ABOVE	① It shows a clerk a raw `2026-08-02T09:00:00+00:00` — one date formatter is BR-SHL-034 and this is not it. ② **Sorted oldest first** (*When* ▲) on a history, against the newest-first rule. ③ `+7 more` truncates a ten-field change with no way to expand in place, so the one row on this bill hides seven of the ten things that changed.
WHY IT IS LOAD-BEARING	Hard rule 4 means an issued bill takes no UPDATE, so this log is *how* a change is explained at all — and hard rule 7 requires every money action to write one.
WALKED	`SEEN LIVE`

One caveat this tab inherits. Hard rule 4's narrowing for `payments` means the value a correction replaces lives *only* in the audit log — which is pruned on a retention schedule. Whether such a row is exempt is `OQ-1330` and is **open**, so nothing here may describe this tab as a permanent record, and it does not.

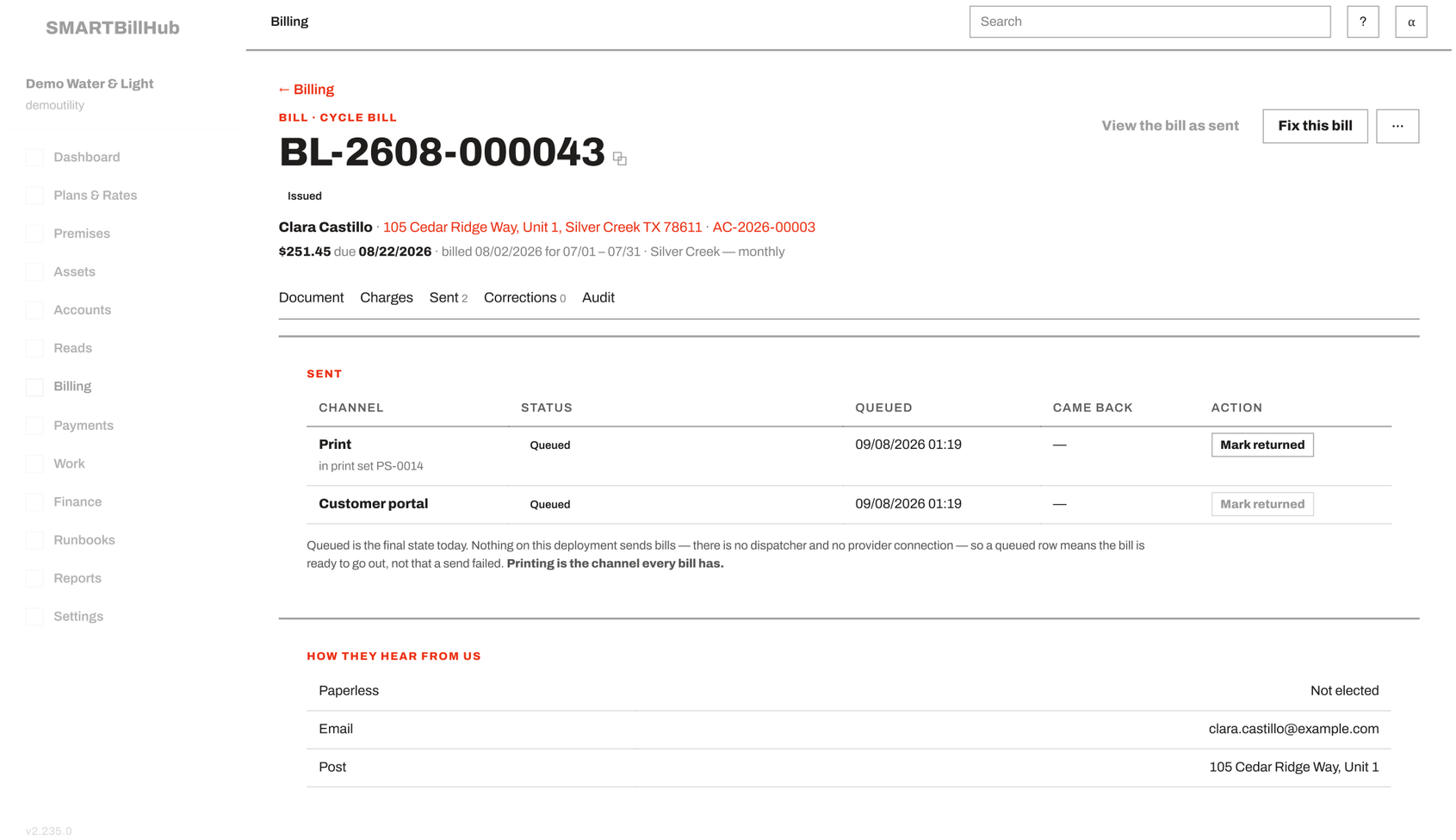
02.5 Bill detail · Sent | ENTIRELY NEW Blocked S22

`/:tenant/billing/bills/:invoiceId?tab=sent` · **does not exist today** — Deli very is currently the last panel on the Document tab

“Did the customer actually get this?”

BUILD THIS New build **Bill detail · Sent — a new tab, split out of Document** | CLOSES BIL-S19

REDESIGN drawn **2026-09-11**, still unbuilt at Version 35 (2026-09-13), drawn to B14 31 · B14 37 · the Product Owner’s 2026-09-11 ruling removing the embedded viewer. **NEVER BUILT.** Today the product ships the as-built bills list and bill detail directly below this drawing.



Why this earns a tab of its own. Delivery is currently the *last* panel on the *first* tab, below four panels of provenance — and *did they get it* is a top-three question. It is also the only place this product could ever answer it, because printing is the one channel that reaches a customer and nothing dispatches.

Two changes beyond the move. The channel names become words — `pdf print` → **Print**,

`portal` → **Customer portal** — because a machine code on a clerk's screen is B14 rule 32. And **which print set** a row went out in is added, which is the same read that would fill screen 01's *Printed* column.

One group here is a proposal, not a redraw. *How they hear from us* — the paperless election and the contact details a bill would actually go to — is **NEW** and needs a customers read. The bill-issue API mentions the paperless election; nothing shows it to a clerk. Drawn because this is the tab where a clerk asks the question.

03

The bulk bill run, redrawn

Six steps become four, pricing moves to the front, and one screen shows a clerk every account with real money on it before they sign anything.

The intent, in the Product Owner's words, 2026-09-11: *"our idea is to show all the data to billing clark before submitting the bill run - it kind of allow him to do all decision before hand, see the data correct the data and then submit it so that billing errors are reduce sbtantially"*.

The one thing standing in the way of that today is that money does not exist until step 4. A clerk makes every decision on step 3 with `amount`, `currency`, `quantity` and `unit` **null on every row** — *"phase 1 computes no money at all"* (`selectors.py:3198-3204`). They decide blind and the consequences appear one screen later.

And the fix is already built. `run_calculate` runs the prepare pass itself, takes no decisions as input, is *"idempotent by construction, not by a flag"*, and has a `background: true` mode that answers 202 with a poll address where *"the clerk may close the tab."* **That mode has existed and gone uncalled since 11 August.** Price at step 1 and the whole redraw follows.

Six steps become four

#	WAS	NOW	WHY
1	**Cycle** — two inputs	KEEP, WIDENED → 16	Five groups instead of two fields: scope, the accounts held from last time, the three recurring decisions made *once*, a label, and who already has a run open. Then pricing starts, in the background.
2	**What was found** — six sub-tabs		Three of its six tabs exist only to say their feature is unbuilt; the others show exceptions only, so a clean tab is a blank screen. It sits on eight grey bars for two minutes and re-computes on every refresh.
3	**The customers** — 16 case kinds	MERGE INTO ONE → 17	The decisions — **made without money on screen.** No search, no pagination, and seven of the fifteen kinds, six of which can never occur because the engine emits nine.
4	**The totals**		The first screen where money exists. Its *unusually large* panel is three inert words over a list that has never been built, and its three draft powers are drawn and switched off.

5	**Sign off**	KEEP, TRIMMED → 18	Five separately recorded facts and a deliberate gate — worth its own screen. It carried nine paragraphs and two absence panels; it needs three groups.
6	**Send**	KEEP, ONE DOOR ADDED → 19	The irreversible write, and the only screen that should feel like one. It tells a clerk printing is the only real channel and gives them no way to print.

```
Steps                          6  ->  4
Screens a clerk passes through      6  ->  4
Sub-tabs to open                    6  ->  0   (they become filter chips with counts)
Screens where money is visible      1  ->  3   (review, sign off, send)
Screens where a decision is made    1  ->  2   (defaults at step 1, exceptions at step 2)
Decisions a clerk makes one at a time, that a default could have settled
                                 many  ->  the genuine exceptions only
```

Where the engine and the screens disagree

Twelve places. Each is a case where the screen promises something the engine does not do, or the engine does something the screen never shows. **Every one of them is already filed** — this table is what they look like when read together rather than one row at a time.

THE ENGINE	THE SCREEN	FILED
Emits **nine** case kinds, and **never** a late fee or an extra charge	Declares **fifteen**, and gives two of them their own tab and a bulk action	DEF-2596
Computes **no money** in phase 1 — amount, currency, quantity, unit null on every row	Asks for a decision per account on step 3, with nothing to weigh it against	THE ROOT OF THIS REDRAW
`mark_for_later` and `retry_account` are complete, tested and routed as `/hold` and `/retry`	Step 4 draws all three draft powers and switches them off, saying *“none of those three has anywhere to send its request”*	DEF-2841 · DEF-2856
`flagged_drafts` raises `NotImplementedError` and has **zero callers**	Step 4 names three actions on unusually-large bills as inert `<span>`s	DEF-2855
Writes `bill_runs.status = 'draft'` once and **nothing ever advances it**	Shows a Status column, so every finished run reads *Draft* for ever	DEF-171 · DEF-602
Serves `run_no` **null** on every row	Shows a Run column	API.PY:2092 EXC(1)
Has a `background: true` mode that answers 202 with a poll address	Calls it inline, so a clerk waits two minutes at eight grey bars with no progress	DEF-1084
Refuses an account whose meter was simply **not read**	Shows a sentence blaming a **missing meter multiplier** instead	DEF-3221
Produces **no bill at all** for an account whose customer type has no base charge	Lists it **nowhere** — the account vanishes and the total is silently short	DEF-2381
Asks nine cross-module seams **once per account**; no owning module publishes a batch read	Pays **66,053 SQL statements at 1,000 accounts**, on every refresh, uncached by ruling	DEF-2704
Caps a run at **25,000 accounts**	Offers no way to bill a subset, so a 40,000-account utility cannot run a cycle	DEF-BIL-4224

	at all	
Serves `customer_name` **null** on every run-case row (a declared seam limit)	Identifies accounts by number only, on the screens where a clerk decides about a person	`SELECTORS.PY:3206-3210`

Read as a set, they have one shape: the screens were drawn from the rules and the engine was built to the rules, and nobody diffed the two. Seven of the twelve are the screen promising more than the engine delivers; four are the engine delivering more than the screen shows; one is a wrong reason reaching a clerk. **The four-step redraw fixes the shape rather than the twelve rows** — a screen that opens with priced drafts and reconciling counts cannot promise a case kind that never occurs, and cannot lose an account silently.

Every control on these screens — ruled 2026-09-11

Product Owner, verbatim: *"all dates should be date picker, all drop down should be drop down, all search should be search and multi select, with all as option available"*. It applies to **every screen in this document**, not only to the bill run.

THE RULE	WHAT IT MEANS IN PRACTICE	WHERE IT ALREADY STANDS
A date is a date picker	Never a typed string, and typing is still allowed — the picker is the control, not the only way in. Regional format throughout.	Already law and the kit component exists. Applied inconsistently — see the audit below.
A list of options is a dropdown	Past **ten** options it becomes a **searchable** dropdown. Under five it may be a segmented control.	Already law, with a threshold worth knowing: **ten**.
Multiple choice is chips	The chosen values as removable chips plus one add control. Never a grid of checkboxes. Switched-off options are named in the list, greyed, with where to switch them on.	Already law; the kit component landed 2026-09-10.
A filter multi-selects	Two values in one dimension means *either*; two dimensions means *both*.	New — and it changes a signed rule. See below.
A service is a badge	A 20px square, the service's colour, the product's own letter. Never a full-name chip in a table cell, and never colour alone.	`ALREADY LAW` and in the kit — ruled 2026-08-21, applied here 2026-09-12.
All is the absence of a filter	Never a full tick-list. A run scoped to *all services* picks up a service added later; one scoped to eleven ticks does not.	Half there — the filter bar has an *All* chip; the reading below is the new part.

The multi-select rule is not a screen change. It answers a question that has been open since 21 August and moves a signed rule. Our filters hold **one value per dimension**: tick *Water*, then tick *Gas*, and the second replaces the first. The prototype we compare against multi-selects and ORs within a dimension; we never did, and the divergence was filed rather than decided because choosing would have been resolving an open question by picking a value. **This

ruling picks it. What moves: the URL grammar `f.<name>=<value>` repeats or comma-joins, the server widens each declared filter from `=` to `IN`, and **roughly fifteen list screens share the one component** — bills, payments, plans, rates, charges, discounts, taxes, reads, accounts. A test currently pins the single-select behaviour *specifically so nobody changes it without this ruling*. **Do it once, as a shell change, ahead of these screens** — if billing multi-selects and the other fourteen lists do not, a clerk has to learn two filter behaviours in one product.

The service badge — E · W · G, on every row

Product Owner: *"we will use Utility Service Chip E, W, G"*. Every **Services** cell on every screen in this document now draws them. It is **r10's own badge and the kit already has it** — a **20px square**, the service's own colour token, the initial in the page background, 11px/800, 2px gap. The letters below are the product's map, not this document's.

BADGE	SERVICE	LETTER	COLOUR	WHOSE
E	**Electric**	`E`	`#ec3013`	r10
W	**Water**	`W`	`#1e6b8f`	r10
G	**Gas**	`G`	`#c47a1d`	r10
S	**Sewer**	`S`	`#5a5555`	r10
T	**Trash**	`T`	`#3a6b3d`	r10
B	**Broadband**	`B`	`#6b3d8a`	r10
St	**Storm water**	`St`	`#2f4a7a`	ours
So	**Solar**	`So`	`#7a9a2a`	ours
Th	**Thermal/Heat**	`Th`	`#a8256b`	ours
Ba	**Battery**	`Ba`	`#3aa39b`	ours
F	**Fire**	`F`	`#8f1f14`	CR-063

The letter cannot be derived from the name, and that is the whole reason a map exists. `name[0]` is the obvious implementation and it is wrong on **six of the eleven**: **Sewer, Solar and Storm water** all give `S`; **Trash and Thermal/Heat** both give `T`; **Broadband and Battery** both give `B`. A utility selling sewer and solar would see **two identical badges differing only in hue** — and hue alone is not a distinction a reader can rely on, nor one a deuteranope can use at all. So the four services added after r10 was drawn take a **two-character badge in the same square**, and **Fire keeps a single character because `F` was free**. Six letters are r10's verbatim; four are ours; one arrived with fire. **If you want different letters, that map is the only place that changes.**

Two things worth knowing before this is built

Electric's service colour is the brand accent (`#ec3013`) — the same red this document uses for "active" and for a primary button. It is the shipped token, so the drawings use it, but an `E` badge beside a primary control is the one place these two vocabularies touch. Worth a look on a real screen. And **the badge freed the column**: the Services cell went from a wrapping name list to a single 20px row, which is most of the width the *What is wrong* column needed.

Every input on the billing screens, audited against the four rules

SCREEN	INPUT	TODAY	BECOMES
16 · step 1	Cycle	Dropdown — but the option text carries **raw ISO dates**	Dropdown reading `Monthly · Sep 2026`; the period shows formatted, read-only, beneath
16 · step 1	Period start and end	Date picker	Unchanged
16 · step 1	Bill date, due date	Date picker	Unchanged
16 · step 1	Services	Not a control at all	**Read-only chips with unit counts.** Not a chooser — see the scope finding below
16 · step 1	Defaults, per case	Dropdown per case	Dropdown per case, and each option states **whether it bills or holds**
17 · step 2	Case filter	Six flat chips in the toolbar	**Banded chips with counts and money**, generated from what the run raised
17 · step 2	Service filter	Not offered	**Chips**, multi-select, All
17 · step 2	Decision, per row	Native select — deliberate; a table cell has ~180px	Unchanged. This one is already right
20 · Held accounts	The *as of* day	Fixed to today in browser state — no control	**Date picker.** Every age on the screen is measured against it
20 · Held accounts	Resolved rows	Checkbox, two states	Dropdown: *Still held* · *Cleared* · *All*
20 · Held accounts	Origin, stage, reason, cycle, age	No filters at all	Filter bar, all five, multi-select, All
01 · All bills	Status, type, cycle, run, dates	Filter bar, single-select	Multi-select, per the ruling

---

Where this ruling has to live

This is a UI rule and a decision, so it belongs in **our binding UI rules** and the **decision log**, with

the question it closes updated in the same commit. **This pass wrote nothing to the repository**, so it is recorded here and nowhere else — it needs a commit of its own, or the next session will not know it was ruled.

Between Scope and Check we run a *pre-bill check*, not a bill run

Product Owner, verbatim: *"between step 1 and step 2 — we are basically running pre bill checks and it is not actual bill generation"*. **That is the engine as designed, and it is not what the product does today.** The run has **seven named stages in one fixed order**, and where the line falls between them is the whole of this section.

STAGE	WHAT IT DOES	OWNER	PURE?	WHAT IT WRITES
1	resolve-subscription	billing	**pure**	nothing
2	measure-consumption	billing	**pure**	nothing
3	price-usage	rating 🔒	**pure**	nothing
4	apply-charges	rating 🔒	**pure**	nothing
5	compute-tax	rating 🔒	**pure**	nothing
6	**assemble-document**	billing	✗	**the draft bill — and its bill number**
7	post-to-ledger	billing	✗	the balanced handoff

Stages 1 to 5 are pure and they produce the money. Stage 6 is where a bill comes into existence. So a pre-bill check is not a new feature to design — **it is stages 1 to 5, stopping short of 6**, and the rule that keeps it honest is already signed: **the preview and the real run are the SAME code**. No second implementation, nothing to drift. **The precedent is already shipped:** the one-time bill's preview is documented as *"phase 1 — writes NOTHING"*, and it is a POST rather than a GET precisely so that a clerk who fixes a plan and previews again sees the fix. **And the product already uses the word** — the phase-1 screen's own name is *"Check what we found"*.

What is wrong today, and it is one verb

`calculate` runs stages 1 through **6** in a single call. So by the time a clerk first sees a number, **every ready account already has a draft bill with a bill number minted** — and a bill number, once minted and unused, is **lost** by ruling (*"if not use thats okay we will lose it; its okay; dont over complicate"*).

	TODAY	UNDER THE PRE-BILL CHECK

What runs between step 1 and step 2	stages 1–6	**stages 1–5**
What exists when Check opens	1,251 draft bills, numbered	**nothing — figures only**
Changing one decision and re-pricing	re-drafts, and **burns a block of bill numbers** each time	re-checks. No number is touched
What the clerk is honestly being shown	“here are the bills”	**“here is what we would bill”**
When bills are created	before anybody approved anything	**at sign-off — screen 18.1’s button says so**
Size of the change	—	**smaller than it sounds.** Stages 1–5 already run as a unit, a draft is already *“freely discardable”*, and the split is a line in the orchestrator

What the check costs to run — and I told you last time there was no figure

There is one. Measured on real PostgreSQL: **137 ms per account**, of which the gather and check are **88%**.

ACCOUNTS IN THE CYCLE	THE CHECK	AGAINST A REQUEST WALL OF ~60 SECONDS
1,284 — this run	**~3 minutes**	already 3× over
10,000	**~23 minutes**	23× over
25,000	**~57 minutes**	**57× over**
40,000 — the top of the ICP	**~91 minutes**	91× over

⚠ And the API still tells every caller that no such measurement exists. The note served on every `calculate` response reads: *“there is no timing figure for a bill run of any size anywhere in this product yet … the equivalent measurement for a run is owed. Choosing a threshold without it would be inventing a number.”* It was measured — **137 ms per account** — by the same change that added the resume checkpoint, and the checkpoint’s own docstring quotes the figure four files away. **A served sentence that is no longer true**, which is the most expensive shape of stale documentation this repository knows: it is why I told you last message there was no figure, and it is why the background threshold is still unset. **No defect row exists for it.**

Three things this makes non-negotiable for the screens

WHAT	WHY	SCREEN
The clerk never waits	An hour at 25,000 accounts. The check runs in the worker always, above a small bar, and **inline must refuse rather than time out** — today a deployment with no worker runs it inline and the request dies before its own *ran inline* answer can be read. The one-time import refuses in exactly this situation; billing does not.	**17.0**

Resume without re-walking	A worker killed at 9,000 of 10,000 pays **22 minutes** again. The checkpoint exists and is passed on one code path only.	17.0
Re-pricing one decision re-checks one account	At 137 ms one account is instant and the whole run is an hour. A decision that re-ran the run would make the screen unusable at any size worth having.	17 · 17.10

And the journey gets its proper name

The wizard rail now reads **Scope · Check · Sign off · Send**. It was *What to bill · Review · Sign off · Send*, and *Review* was the wrong word for the step: reviewing is what a clerk does **on** it, and what the product does **to get there** is a check. Four words, each naming an act, and the second one now tells the truth about whether anything has been billed.

How screens 1 and 2 are organised

The two screens carry almost everything a clerk decides in a month, so how the information is **grouped** matters as much as what it says. Both are built the same way: **zones separated by rules, one spacing scale, and a column order that never moves.**

Screen 16 · Scope — three zones and a commit bar

The zones, top to bottom

```
┌──────────────────────────────────────────────────────────────────┐
│ Run bills                                          Save & exit   │
│ Step 1 of 4 · nothing has been written yet                       │
├──────────────────────────────────────────────────────────────────┤ ← rule
│ A · WHAT PERIOD          the only editable scope on the screen   │
│     Cycle ▾              Period start 🗓    Period end 🗓        │
│     Bill date 🗓          Due date 🗓                            │
├──────────────────────────────────────────────────────────────────┤ ← rule
│ B · WHAT IS IN SCOPE     read-only. the consequence of zone A    │
│     1,502 units → 1,284 accounts                                 │
│     Water 1,502 · Sewer 1,480 · Fire 46          ← chips, not a chooser
│     11 masters cover 229 common areas · 1 cannot be billed today │
│     3 accounts unresolved from the last run          → Held queue ↗
├──────────────────────────────────────────────────────────────────┤ ← rule
│ C · WHAT HAPPENS WITHOUT YOU     one row per case this cycle raised
│     Missing read      Estimate from history ▾        bills       │
│     Unusually large   Investigate ▾                  holds       │
│     Estimated read    Keep the estimate ▾            bills       │
│     …                                                            │
│     ▸ With these defaults, 12 accounts will need you and 15 will be held
├──────────────────────────────────────────────────────────────────┤ ← rule
│                       Prepare — looks, writes nothing, takes ~2 min  →
└──────────────────────────────────────────────────────────────────┘
```

RULE	WHY

Zone A is the only thing that changes zone B	So a clerk learns in one interaction where scope comes from. Nothing in B or C alters the population.
Zone B holds no controls at all	It is the answer, not a question. The moment a fact in it becomes editable, a clerk has two places to set scope and no way to tell which won.
Zone C lists what this cycle raised last month	Not fifteen hypothetical case kinds. A short true list is read; a long speculative one is skipped, and skipping it is how a default nobody chose ends up deciding.
Every default states its consequence in the option text	*Estimate — bills* · *Investigate — holds*. Two of them hold and produce no bill, and a clerk cannot be expected to know which.
The consequence line is the point of the whole zone	*12 will need you and 15 will be held.* It turns a form into a forecast, and it is what lets a clerk loosen a default **before** the run rather than undo fifteen holds after it.
An open run is offered at the top, not discovered at the end	Two clerks in the same cycle is a normal Monday.
Nothing writes, and the button says so	*Prepare — looks, writes nothing.* It is true of the code and it is the sentence that makes a nervous clerk press it.

Screen 17 · Check — five layers, each answering one question

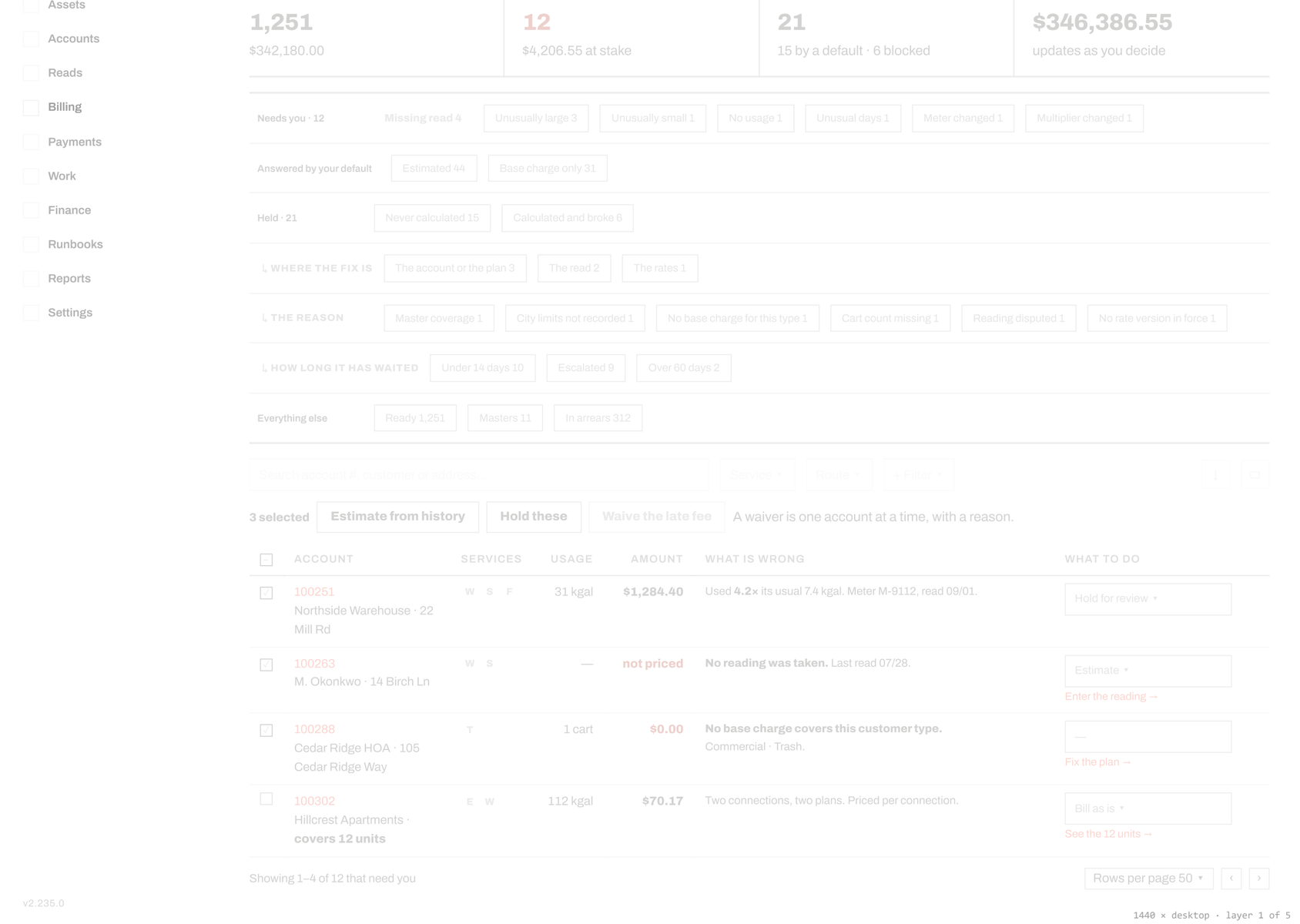
The five layers, drawn. Each frame shows **one layer at full strength and the other four dimmed** — dimmed rather than removed, because every layer's job is partly to stay out of the way of the others. The rules they all share are tabled after the drawings.

17.L1 Step rail | LAYER ONE | A LAYER OF SCREEN 17 | A LAYER OF SCREEN 17 | A LAYER OF SCREEN 17

`/:tenant/billing/run · screen 17, one layer at a time · the other four are dimmed, not absent · 4 steps`

"Where am I, how much is left, and can I go back?"

> REDESIGN drawn **2026-09-11**, still unbuilt at Version 35 (2026-09-13), drawn to the Product Owner's 2026-09-11 run-intent ruling, the 2026-09-11 *per connection* ruling and the 2026-09-12 on-screen-calculation ruling. **NEVER BUILT.** Today the product ships a **six**-step wizard, screens 04–09 in §17.



What it answers	**Where am I in the run.** Nothing else. It is the only element on the screen that is about the WIZARD rather than about the bills.
Four, not six	The as-built run has six steps and three of them were review, sign-off and breaks. Four is the redraw. A clerk counting steps is estimating how long this will take, so the count is load-bearing.
A completed step is a door	Step 1 is clickable and returns to Scope **without losing the priced run** — the run lives at its own address and its status is its phase. A stepper whose finished steps are dead makes a clerk restart to change one date.
What it must never do	Carry a count, a total or a warning. The moment it does, a clerk reads it twice — once for position and once for state — and layer 2 exists for the second reading.

17.L2 **Run state** | LAYER TWO | A LAYER OF SCREEN 17 | A LAYER OF SCREEN 17 | A LAYER OF SCREEN 17

`/:tenant/billing/run · screen 17, one layer at a time · the other four are dimmed, not absent · 4 cells · GET /billing/runs/{id}`

“What is the state of this run, right now, whatever I happen to be looking at?”

> REDESIGN drawn **2026-09-11**, still unbuilt at Version 35 (2026-09-13), drawn to the Product Owner’s 2026-09-11 run-intent ruling, the 2026-09-11 *per connection* ruling and the 2026-09-12 on-screen-calculation ruling. **NEVER BUILT.** Today the product ships a **six**-step wizard, screens 04–09 in §17.

SMARTBillHub

Billing › Run bills

Search ? α

Demo Water & Light
demoutility

- Dashboard
- Plans & Rates
- Premises
- Assets
- Accounts
- Reads
- Billing
- Payments
- Work
- Finance
- Runbooks
- Reports
- Settings

Run bills

Save & exit

Step 2 of 4 · 09-A · **September, after the water rate change** · priced 09/11/2026 07:55

1 Scope | 2 Check | 3 Sign off | 4 Send

2

READY TO BILL	NEEDS YOU	HELD	RUN TOTAL
1,251	**12**	**21**	**$346,386.55**
$342,180.00	$4,206.55 at stake	15 by a default · 6 blocked	updates as you decide

Needs you · 12 — Missing read 4 | Unusually large 3 | Unusually small 1 | No usage 1 | Unusual days 1 | Meter changed 1 | Multiplier changed 1

Answered by your default — Estimated 44 | Base charge only 31

Held · 21 — Never calculated 15 | Calculated and broke 6

↳ WHERE THE FIX IS — The account or the plan 3 | The read 2 | The rates 1

↳ THE REASON — Master coverage 1 | City limits not recorded 1 | No base charge for this type 1 | Cart count missing 1 | Reading disputed 1 | No rate version in force 1

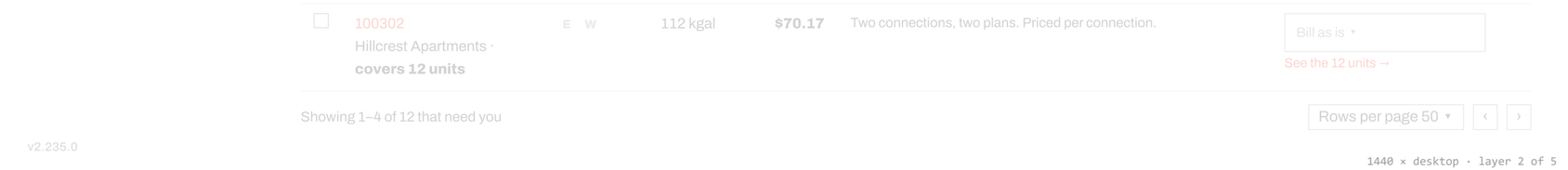
↳ HOW LONG IT HAS WAITED — Under 14 days 10 | Escalated 9 | Over 60 days 2

Everything else — Ready 1,251 | Masters 11 | In arrears 312

Search account #, customer or address… | Service ▾ | Route ▾ | + Filter ▾ | ↓ | ▭

3 selected | Estimate from history | Hold these | Waive the late fee | A waiver is one account at a time, with a reason.

	ACCOUNT	SERVICES	USAGE	AMOUNT	WHAT IS WRONG	WHAT TO DO
☑	100251 Northside Warehouse · 22 Mill Rd	W S F	31 kgal	**$1,284.40**	Used **4.2×** its usual 7.4 kgal. Meter M-9112, read 09/01.	Hold for review ▾
☑	100263 M. Okonkwo · 14 Birch Ln	W S	—	**not priced**	**No reading was taken.** Last read 07/28.	Estimate ▾ Enter the reading →
☑	100288 Cedar Ridge HOA · 105 Cedar Ridge Way	T	1 cart	**$0.00**	**No base charge covers this customer type.** Commercial · Trash.	— Fix the plan →



What it answers	**The run.** 1,251 ready, 12 needing a decision, 21 held, and what the whole thing comes to. It reconciles: **1,251 + 12 + 21 = 1,284 accounts in scope**.
The rule that makes it trustworthy	**A filter changes the list and never these four cells.** If the totals moved when a chip was pressed, no clerk could tell a filter from a decision — and on a screen whose whole purpose is deciding, that is the one confusion that cannot be allowed.
Money beside every count	*1,251 · $342,180.00* and *12 · $4,206.55 at stake*. A count says how much work; the money says whether the work matters.
Held carries its split	*15 by a default · 6 blocked*. Two very different things were both “held” on the as-built screen: one is a decision a clerk can change here, the other needs a fix somewhere else.
Run total says when it is stale	While a re-price is in flight the cell reads *updating*. It is the only figure on the screen that moves after a decision, so it is the only one that needs to say so.

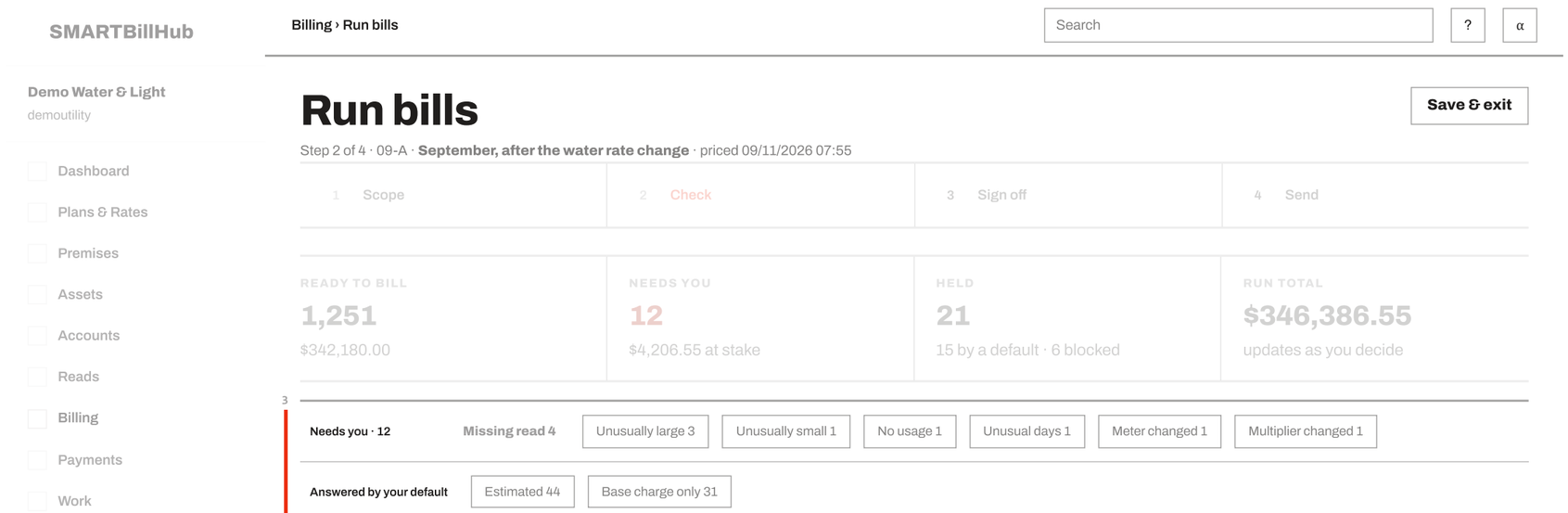
17.L3 Navigation

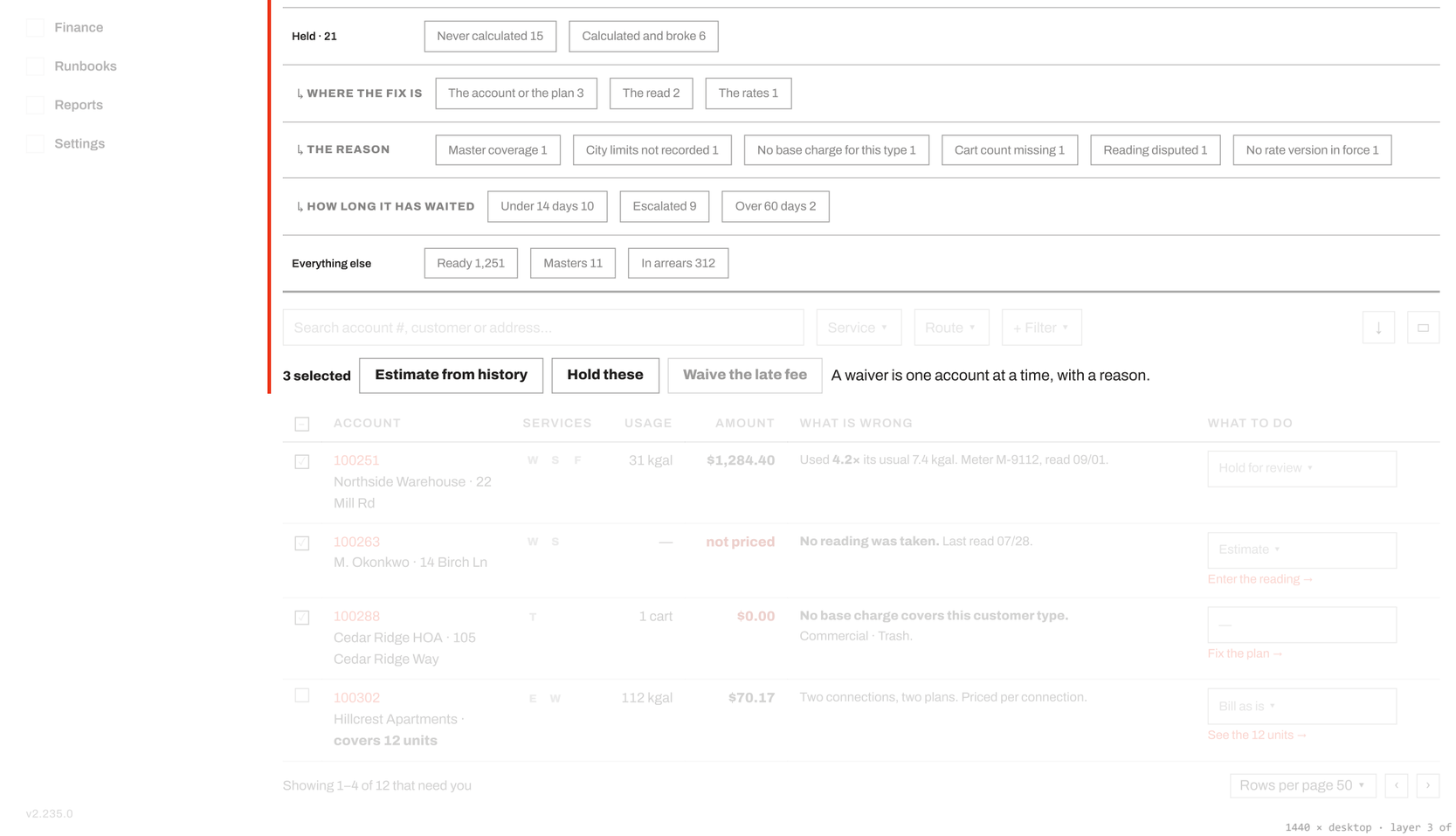
LAYER THREE | A LAYER OF SCREEN 17 | A LAYER OF SCREEN 17 | A LAYER OF SCREEN 17

/:tenant/billing/run · **screen 17, one layer at a time** · the other four are dimmed, not absent · 4 bands · 15 chips · generated, never declared

“What needs me, what has already been answered, and what is stuck?”

REDESIGN drawn **2026-09-11**, still unbuilt at Version 35 (2026-09-13), drawn to the Product Owner’s 2026-09-11 run-intent ruling, the 2026-09-11 *per connection* ruling and the 2026-09-12 on-screen-calculation ruling. **NEVER BUILT.** Today the product ships a **six**-step wizard, screens 04–09 in §17.





Held is three questions, not one	A held account raises three different questions and the band answers all three, indented under it: **why it is here** (never calculated · calculated and broke) · **where the fix is** (the account or the plan · the read · the rates) · **the reason** (six named ones) · and **how long it has waited**. **All four partition the same rows, and the arithmetic is checked:** never-calculated + broke = the band · the three stages = broke · the six reasons = broke · the three age bands = the band. On all three count sets in this document. ⚠ **Age is on step 2 because step 1 sweeps up.** It would be meaningless otherwise — everything held for the first time in run 09-A is 0 days old. But step 1 offers *"also include the 21 accounts held from RUN-0002"*, so the run contains rows that have been waiting since August and July. **An account held month after month is the thing that quietly loses revenue**, and *Over 60 days 2* is the only place on this screen that says so. **And the words now match the held queue exactly** — *the read*, not *fix the read*. One dimension had two spellings across two screens, which is the same fault as *Unusual days*. The band label carries the verb instead.
What it answers	**Where to go. Seven bands, twenty-six tags, and the same twenty-six on every screen** — bounded above and below by a rule, each band separated by a hairline, band names in a fixed label so the tags line up in a column.
Ordered by obligation	*Needs you → answered by your default → held → everything else.* A clerk works top to bottom and is **finished when the first band is empty**. That is a definition of done the screen states rather than implies.

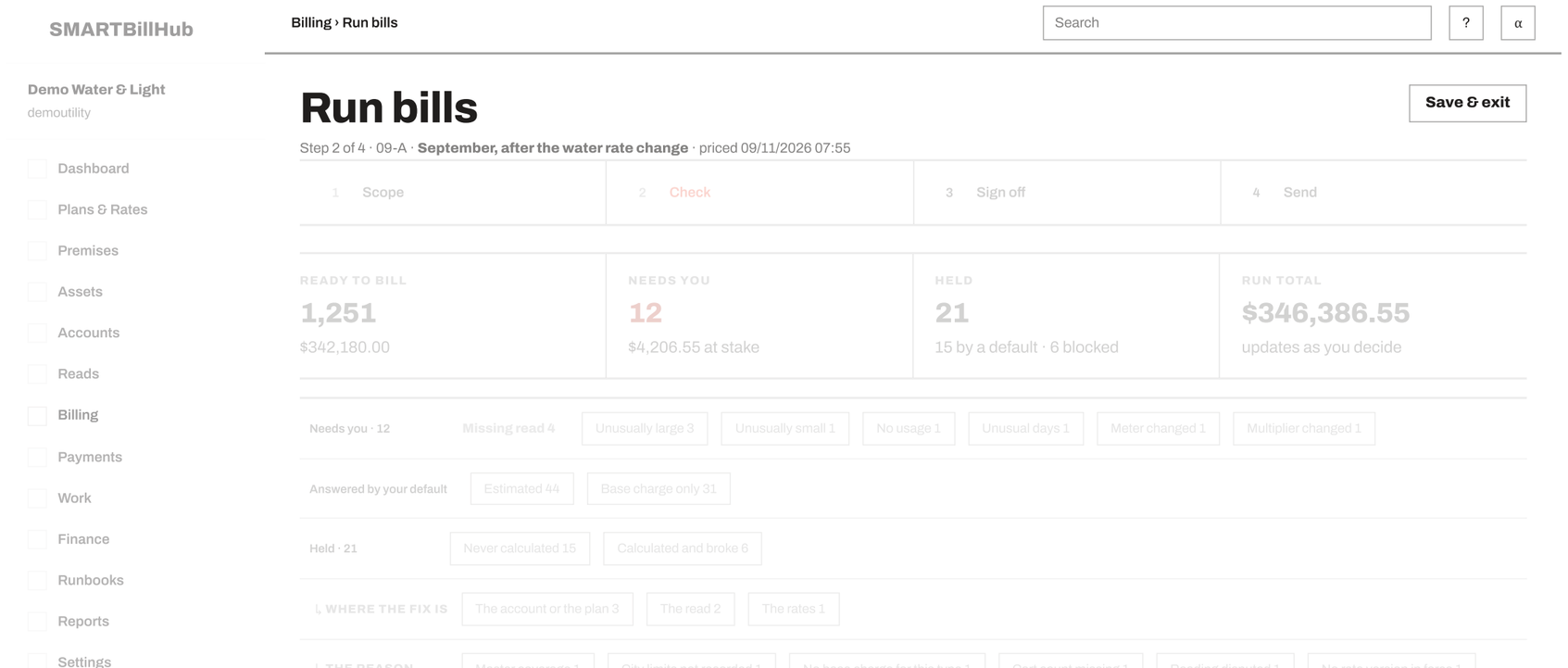
Generated from what the run raised	A case kind the engine did not raise **gets no chip**. The as-built screen declares fifteen kinds where nine are emitted, which is how three tabs came to exist whose only content is an explanation that the feature is not built.
The tag set never changes, and that is now enforced	A clerk learns one strip and it is the same on all thirty frames. **Three screens used to disagree** — the in-flight check drew 8 tags, the arrears view 10 and the chip-wide bulk action 3 — so the strip taught a different lesson depending on where you landed. It is one generated component now, and the three count sets in this document (the 1,284-account run, the in-flight check, a finished 25,000-account run) each **reconcile**: the tags in a band sum to the band, and the bands sum to the accounts in scope. **A tag counts CASES; a band counts ACCOUNTS.** Twelve accounts raise nineteen case rows, because one account can be both unusually large and short of a reading — so the tags under *Needs you · 12* sum to more than twelve, on purpose, and the labels say which is which.
Tags only — search and filters have moved out	Search, *Service*, *Route* and *+ Filter* used to sit above the bands. They are now **below** them, in layer 4, because they do a different job: a tag says **which population** you are looking at, and a filter **narrows whatever population is open**. Mixing them in one strip made a text box look like a case kind. **Layer 3 is now nothing but tags.**

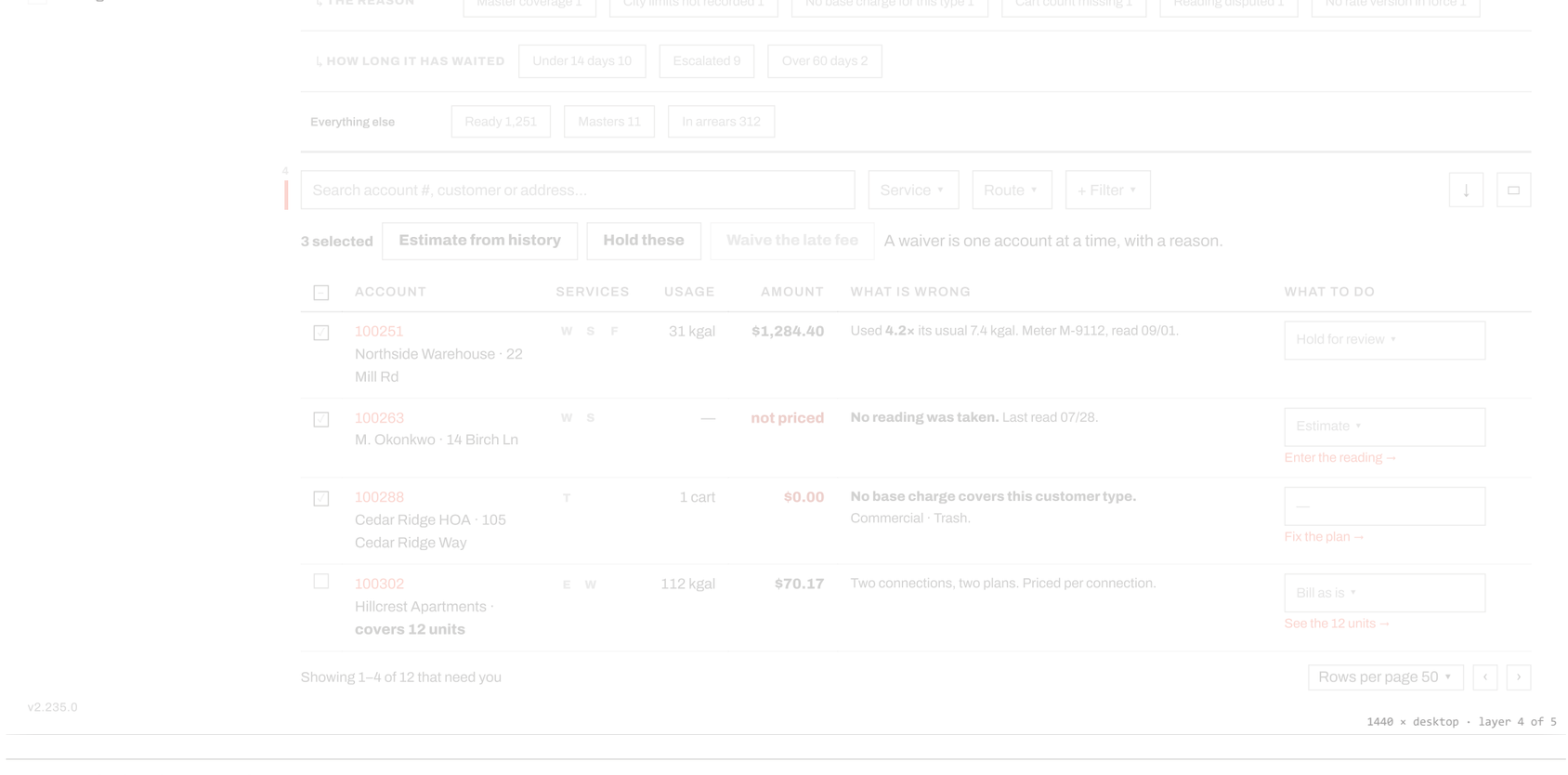
17.L4 Selection | LAYER FOUR | A LAYER OF SCREEN 17 | A LAYER OF SCREEN 17 | A LAYER OF SCREEN 17

`/:tenant/billing/run · screen 17, one layer at a time · the other four are dimmed, not absent · appears only with a selection`

“I have picked some rows. What can I do to all of them at once, and what will it cost?”

> **REDESIGN** drawn **2026-09-11**, still unbuilt at Version 35 (2026-09-13), drawn to the Product Owner’s 2026-09-11 run-intent ruling, the 2026-09-11 *per connection* ruling and the 2026-09-12 on-screen-calculation ruling. **NEVER BUILT.** Today the product ships a **six**-step wizard, screens 04–09 in §17.





How a clerk selects, and what a selection means

There were no checkboxes on any row, on any frame. The bar said *"3 selected"* and nothing on the screen could have selected them — a bar describing a state the screen cannot reach. There is a **38px selector column** now, first, pinned like every other.

· **Header checkbox is tri-state** — empty, some, all — and selects **this page**.
· **Select all in the chip is a separate, explicit control.** Never the header tick: a header that silently meant 231 instead of the 50 you can see is how a clerk comes to believe 231 accounts were handled.
· **Changing chip clears the selection**, and says so. Paging keeps it, with a running count.
· **Shift-click selects a range.** One behaviour, forty clicks saved.
· **A row already decided by hand is never overwritten** by a bulk action — it is counted separately in the confirm.
· **No selector where nothing can be actioned.** *The account or the plan* (17.11) has no retry and no choice, so it has no checkbox — twenty-eight frames of twenty-nine. *Masters* has one, with bulk drawn **refused** and its reason beside it.
· **A selection spanning bands becomes one call per case kind**, not one call, and the bar says *3 actions* rather than pretending otherwise.

If you say 231, all 231 are actioned — ruled 2026-09-12

PO: *"if clark say 231 then all 231 should be actioned"*. That settles a real question and it settles it against my own recommendation, so the consequence is worth stating.

The selection is a snapshot, pinned when the clerk makes it, and the snapshot wins. The alternative — the server re-resolving *"everything in this chip"* at commit time — would have been one small call at any size, but it can act on a set the clerk never saw. So the client holds the 231 row ids and sends them. **The served endpoint already takes exactly that**: a list of case ids, N rows, one server-minted action id, so the audit can answer *"was this considered individually, or was it one of a thousand?"*

What must still be reported back, because the world moves. A clerk selects 231 at 09:12; while the confirm is open a meter reader syncs two readings. All 231 are still actioned — but those two now have a real reading, and **a real reading beats an estimate**. So the result says so: *"231 actioned. 2 have since been read and will bill on the reading, not the estimate."* Nobody is surprised, and no bill is estimated over a number that exists.

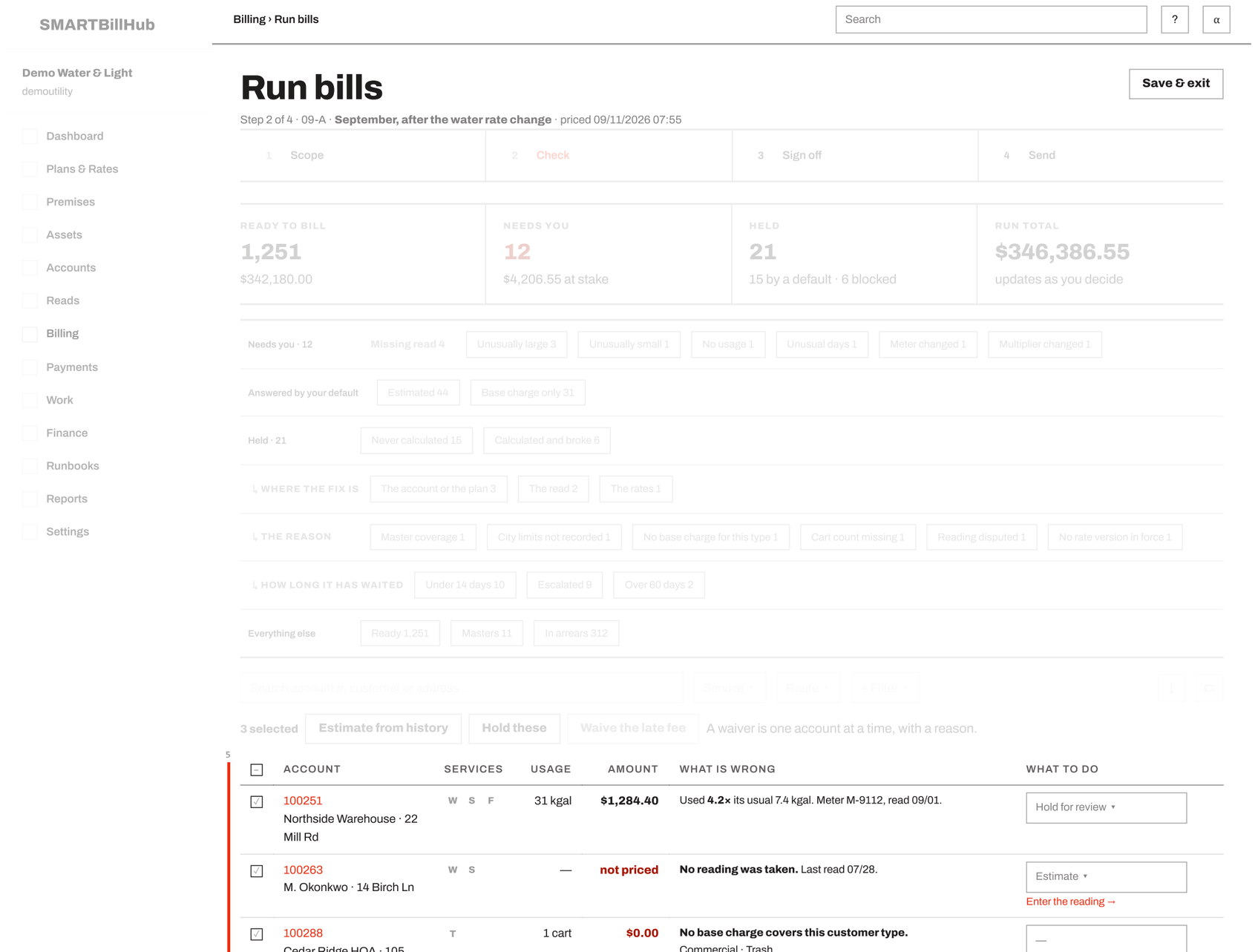
	And a floor on the size: holding ids client-side is fine at 231 and at 5,908. Above **10,000** in one chip, *select all* refuses and asks the clerk to narrow it — a single action over more than ten thousand accounts should be deliberate.
The estimate could not be submitted at all — and the fix needs a progress state	⚠ **Today, choosing *Estimate from history* refuses.** The server accepts a quantity and a unit; **no screen has ever sent them**; and our own rule says a consumption is never recorded without its unit. So **the run's most common decision cannot be recorded** — and 17.16 and 17.22 are both built on it. Reported in the endpoint's own docstring; no defect row. **PO ruling 2026-09-12:** *"you can calculate it on screen — in such case we will have to show work in progress to clark"*. So: **the screen orchestrates and displays; the server still owns the rule.** The screen asks for the estimates of the selected rows, shows *Working out 231 estimates… 140 done* while they arrive, then submits each with its own quantity and unit. **Why the rule stays on the server:** two paths must not produce two figures. The screen sends a number it *received*, never one it worked out, or *"average of the last three reads"* gets implemented twice and drifts — the same shape as fifteen declared against nine emitted. **And the progress state is honest rather than decorative:** it appears because the screen genuinely does not hold 231 histories. For three rows a clerk can see, the estimate is already on the row — sent immediately, no wait.
Every column is pinned, and only one is elastic	The toolbar and the bulk bar are the same on all twenty-nine frames — **search, Service, Route, + Filter, export, column selector** — and so is the table beneath them. **Identity, figures and the action column are pinned by a `colgroup`; only the prose column is elastic**, so it absorbs whatever is left and **no cell can take another cell's width**. **The contract:** Account `24ch` · Services `94px` (four badges and their gaps) · a quantity `10–12ch` · a date `14–16ch` · money `13ch` · **What to do** `240px`, which is the widest control in the product plus its cell padding · **What is wrong: the remainder**. ⚠ **What made this necessary, and it is the same fault twice.** `th` is `white-space:nowrap` globally — correct for a short column heading, and the reason a long header can widen its own column until the next one starves. It broke the row-label tables in this document first, and then **six pinned columns here** whose two-word labels were wider than the column. Headers wrap inside a pinned table now. **Verified across 595 cells in 29 tables: nothing overflows its column at any width from 1920 down to 400.**
What it answers	**How do I narrow this, and what am I doing to what I picked.** Two things now live here: **search, service, route and + Filter**, which narrow whatever band is open; and the **bulk bar**, which appears only once rows are selected. They belong together because both act *inside* a population the tags above have already chosen.
Filters narrow; tags choose	The distinction is worth keeping sharp. A **tag** in layer 3 answers *which population* — the accounts needing a decision, the ones held, the ones in arrears. A **filter** here answers *which of those* — this service, this route, this search term. Layer 2's figures never move for either. And the bulk bar offers **only what every selected row shares**: three rows raising three different cases offer the actions all three accept. A mixed selection offering the union would apply a choice to a case that cannot take it, which the engine refuses by name anyway.
Every bulk action confirms with its arithmetic	Estimating four accounts shows each estimate, its basis, and **+$124.70 on the run total** before committing. Holding three shows **–$3,737.00**. The screens are 17.16 to 17.20.
At 25,000 accounts this layer IS the screen	This run has 12 rows needing a decision. A 25,000-account cycle has roughly **234**. So **apply to every row in this chip** stops being a convenience and becomes the only way the screen works — and it must still show what it will do before it does it.
One refusal is drawn, deliberately	*Waive the late fee* is disabled with its reason beside it, because a waiver is one account at a time with a reason of its own. A greyed control with no stated cause reads as unfinished work and gets wired up by the next lane in good faith.

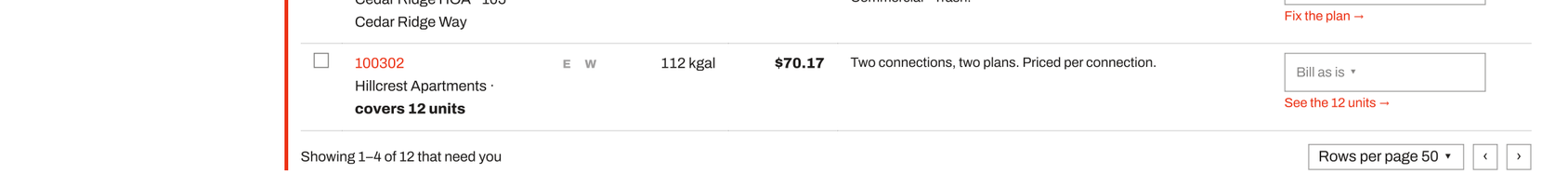
17.L5 **One row** | LAYER FIVE | A LAYER OF SCREEN 17 | A LAYER OF SCREEN 17 | A LAYER OF SCREEN 17

/:tenant/billing/run · **screen 17, one layer at a time** · the other four are dimmed, not absent · 6 columns · the order never moves

“One account. What is it, what is wrong with it, what will it be billed, and what am I going to do about it?”

REDESIGN drawn **2026-09-11**, still unbuilt at Version 35 (2026-09-13), drawn to the Product Owner’s 2026-09-11 run-intent ruling, the 2026-09-11 *per connection* ruling and the 2026-09-12 on-screen-calculation ruling. **NEVER BUILT.** Today the product ships a **six**-step wizard, screens 04–09 in §17.





v2.235.0

1440 × desktop · layer 5 of 5

The column grammar, and it is fixed	**who → what → the facts → how much → what is wrong → what to do.** Only the middle changes per chip — a comparison on *Too high*, two spans on *New meter*, a basis on *Estimated*. **The first two and last two are in the same place on all fifteen chips**, so a clerk learns the table once.
What is wrong is a SENTENCE	*Used 4.2× its usual 7.4 kgal. Meter M-9112, read 09/01.* Not a reason code, not a rule id. The as-built screen shows a clerk raw engineering reasons on this column, and it is the single most-reported thing about these screens.
Money is never blank and never a false zero	*not priced* where nothing could be computed, **$0.00** in red where the engine really did produce zero and that is the defect. A blank cell and a zero are different facts and the column keeps them apart.
One row, one decision	Where an account raises two cases it appears on two chips. Compressing them would produce one control meaning two things.
A row with a fix elsewhere carries the door	*Enter the reading*, *fix the plan*, *see the 12 units*. A decision screen with no route to the fix makes a clerk decide blind or leave and lose their place.
A master says so on the row	Row 4 reads **covers 12 units** under the name and prices **per connection** — $70.17 on two plans. Without that line a twelve-home bill reads as one household's.

RULE	WHY
Layer 2 is the run. Layer 3 is the view.	A filter changes the list and **never** the four KPI cells. If the totals moved when a chip was pressed, no clerk could tell a filter from a decision.
The bands are ordered by obligation	*Needs you → answered by your default → held → everything else*. A clerk works top to bottom and is **finished when the first band is empty** — which is a definition of done the screen states rather than implies.
Four groups, not twenty-six chips in a row	The whole region is bounded by a rule above and below, and each band is separated by a hairline, with its name in a fixed-width label so the chips line up in a column. **The old strip was one undifferentiated row**, which made *Ready 1,251* and *Missing read 4* look like the same kind of thing.
One spacing scale	12px inside a band, 16px between the region and its neighbours, and the chips themselves on one padding. Nothing is spaced by eye.
The column order never moves	**who → what → the facts → how much → what is wrong → what to do.** Only the middle changes per chip — a comparison on *Unusually large*, two spans on *Meter changed*, a basis on *Estimated*. The first two and last two are in the same place on all fifteen chips, so a clerk's eye learns the table once.
The bulk bar exists only with a selection	And it offers only what **every** selected row shares. A bar that is always there, mostly disabled, teaches a clerk to ignore it.
A decision re-prices in the background	The run total is the only figure that moves, and it says *updating* while it does. Everything else on screen is stable, so the clerk never loses their place mid-list.
Nothing lives behind a hover	A disabled control carries its reason as words beside it; a refusal is five or six words; colour is never the only signal.

One row, one decision, one reason	Where an account raises two cases it appears on two chips. Compressing them into one row produces a control that means two things, which the engine refuses by name anyway.

One thing the layers make visible that nothing else does: a chip counts *cases* and a band counts *accounts*. Twelve accounts raise nineteen case rows across seven chips, because one account can be both unusually large and short of a reading. So the band label says *Needs you · 12* and the chips beneath it sum to more than twelve — on purpose, and the screen says which is which rather than letting a clerk add the chips up and get a different answer from the KPI cell.

16 Scope

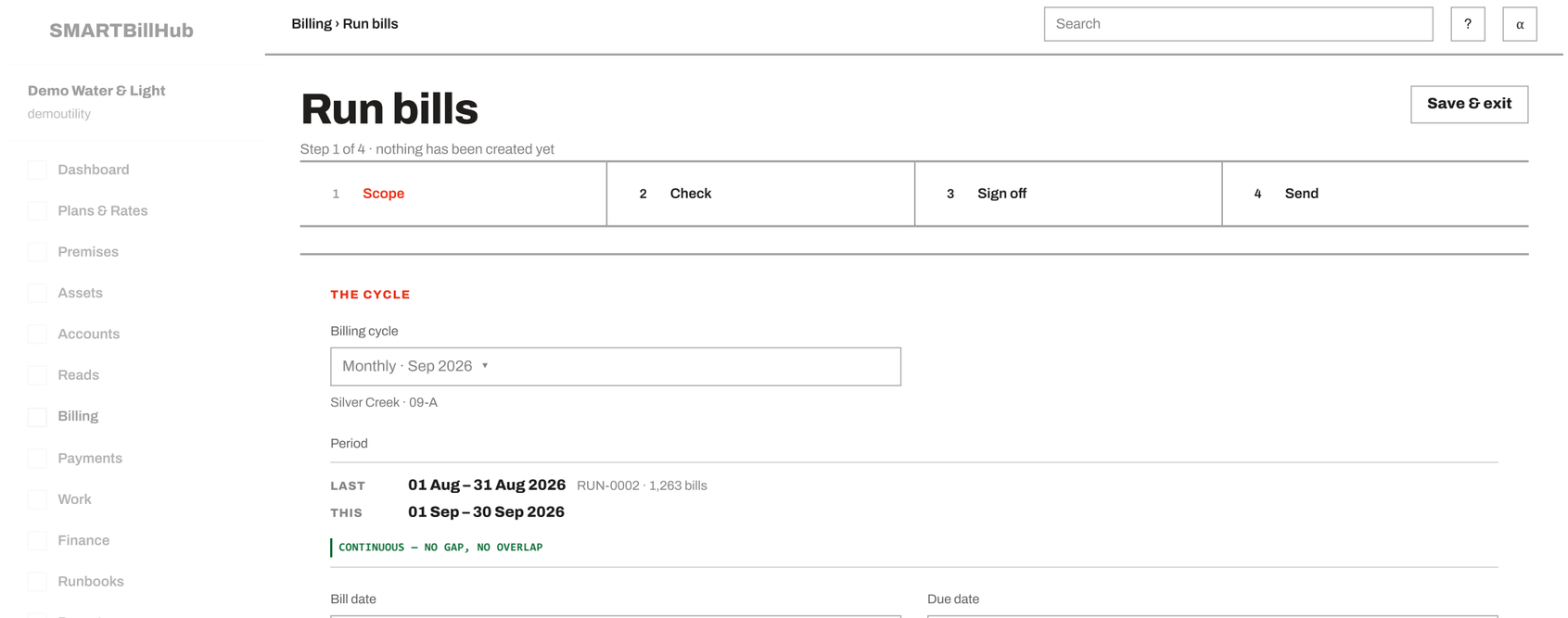
STEP ONE · Blocked S89 · New build S38 S90 S95 · New read S46 · Exists behind S24 S25 S26 S34

`step 1 of 4 — what to bill · /:tenant/billing/run · S31 · BIL-02 · w as step 1 of 6`

“It is the end of the month. Bill everybody — and let me settle the routine decisions before I start, not one account at a time afterwards.”

BUILD THIS · New build · **Two inputs become five groups, and pricing starts here** · CLOSES BIL-S24 · BIL-S25 · DEF-106 · DEF-1084

REDESIGN drawn **2026-09-11**, still unbuilt at Version 35 (2026-09-13), drawn to the Product Owner’s 2026-09-11 run-intent ruling, the 2026-09-11 *per connection* ruling and the 2026-09-12 on-screen-calculation ruling. **NEVER BUILT.** Today the product ships a **six**-step wizard, screens 04–09 in §17.



Reports

Settings

09/30/2026

Prices at this date

10/21/2026

Bill date + 21 days to pay

SCOPE new

1

Which accounts

Every account in the cycle ▾

Or a route, an area, or a named set.

2

Also include

☑ The 21 accounts held from RUN-0002

Waiting 11–27 days.

WHAT THIS COVERS

UNITS IN THE CYCLE	ACCOUNTS TO BILL	MASTER ACCOUNTS	METERS THIS CYCLE	POINTS WITH NO METER
1,502	**1,284**	**11** 229 areas · 1 blocked	**1,598** + 96 sub-meters	**1,526** billed without a reading

Services · 3 of 11 W 1,502 units S 1,480 units F 46 lines

3 unresolved from the last run see the held queue → · Last time: **1,263 bills**

DEFAULTS new

3

When there is no reading

Estimate from history ▾

When a bill looks unusually large

Hold it for review ▾

Unusual days

Pro-rate by days ▾

Holds the bill — pricing not built

4

Label this run

September, after the water rate change

ALREADY RUN FOR THIS CYCLE

RUN	LABEL	STARTED	BY	ACCOUNTS	STATUS	ACTIONS
RUN-0002	August cycle	09/02/2026 04:00	Vijay Bikkad	1,284	Issued	👁
RUN-0003	September, first attempt	09/11/2026 07:40	Dana Reyes	—	Open — not signed off	**Take it over**

5 Dana Reyes has this cycle open. Take it over, or ask them to sign off.

About three minutes, in the background. You can close this tab.

6 **Check this run →**

v2.235.0

BIL-02

1 **Scope** · route, area or a named set instead of all-or-nothing on a whole cycle · **NEW READ** — and the practical answer to the **25,000-account ceiling against a 40,000 ICP** (`DEF-BIL-4224`, high): a T5 utility bills by route

2 **Sweep up the held accounts** · the pending queue already knows exactly who was missed and how long they have waited; a run has never asked · **NEW** — today the only route is a catch-up run startable from no screen (`DEF-2843`)

3 **Run-level defaults** · the three recurring decisions, made once · **NEW — the single biggest error-reducer in this redraw.** It turns hundreds of identical per-account choices into one deliberate choice, and leaves step 2 for the cases that are genuinely unusual

4	**A label** · **NEW** — the history is unreadable without one: every finished run reads `Draft` and every run number is served null (`DEF-171` · `DEF-602`, both high). A label makes a past run identifiable *while* those are fixed
5	**Who already has a run open, and a way to take it over** · `bill_runs.run_by` is populated and served as `ran_by` · **NEW** — today a started run cannot be returned to at all and the refusal names no one and links nowhere (`DEF-106`, high)
6	**Check this run** →, in the background, with the wait stated · `POST /billing/runs/{id}/calculate` with `background: true`, which answers 202 and a poll address · **CHANGED** — **this mode has existed and gone uncalled since 11 August**, and calling it is the fix for the two grey minutes (`DEF-1084`)

Why pricing moves to step 1, and why it costs nothing to do it. `run_calculate`'s own docstring settles it: it runs *"phase 1's prepare pass (which writes nothing), then one DRAFT per ready account"* — so it does not need the prepare step to have happened separately. **It takes no decisions as input.** It is *"idempotent by construction, not by a flag"*: stage 6 replaces an account's draft rather than adding a second, stages 1–5 are pure, *"so a re-run converges on the same drafts"*. And *"nothing money-bearing exists when it returns"* — a draft has no issue date, no stamped accounting code, no journal line and no delivery row.

So pricing early is safe, repeatable and already supported. What it buys is the whole point of this redraw: **step 2 opens with money on every row.**

The last period, beside this one	The clerk's question is not *"what is this period?"* — the cycle answers that. It is **"is billing continuous?"**, and nothing can answer that except the **previous** period sitting next to this one. **A gap is days nobody billed. An overlap is days billed twice.** Both are silent, both are money, and neither is visible anywhere in this product today. **The three states, with their words:** · `CONTINUOUS` *"This period starts the day after the last one ended. No gap, no overlap."* · `GAP` *"2 days were never billed: 30–31 Aug."* Not a refusal — a mid-cycle change can make a gap legitimate — but it is stated, and it is stated **before** the run starts rather than found afterwards. · `OVERLAP` *"1 day would be billed twice: 31 Aug."* **This one should refuse**, or at the least take an explicit acknowledgement: double-billing a day is customer-visible, and corrections then undo it one bill at a time. It costs nothing. The previous run's period is on the **same indexed row** the selection card already reads for its *1,263 bills* — one query, two facts. **Say the word and the gap and overlap states get drawn as their own frames.**
The four controls that were wrong	**Bill date was a bare text box — no picker anywhere on a screen whose whole job is dates.** It is a picker now. **Due date** and **the period** were not on the screen at all: the due date is a real field (derived, and the run stores what you type), and the period is read-only text rather than raw ISO inside a dropdown option. The cycle dropdown now reads `Monthly · Sep 2026` with the site name in the helper.
What this will cover — and why it is all counts	Resolving the scope *for real* costs **137 ms per account**, so a live preview that priced anything would take **57 minutes at 25,000 accounts**. Every figure in this card is therefore a **count**, and the card says so in its own last line. **No money, and no forecast.**
Two numbers, not one	`1,502 units → 1,284 accounts`. They are different facts and **no screen in this product has ever shown both**. The gap between them is where every master-account failure lives.
Points with no meter	**1,526** — sewer and fire, billed without a reading. It is the most useful number on the card: it says how much of the run is *not* driven by a meter, and it is what the *Base charge only* chip on step 2 will be counting.
What each figure costs to fetch	**Cheap today:** units (one query through metering's published scope read) · accounts (one count) · masters (one count on the account's kind, which the run already reads) · services with their unit counts (the same scope read once per service, so eleven queries) · the last run's bill count (one indexed row). ⚠ **NOT cheap today: meters, sub-meters and unmetered points.** Every metering read billing holds is **per**

	unit — service points, reads, sizes, disputes — so counting meters across the cycle through today's seam is **25,000 calls**. It needs **one new published read from metering**, an aggregate over the cycle. One query to run, and it does not exist; billing may not reach those tables itself. That is `BIL-S46`.
Carried over, and the one caveat	**3 unresolved from the last run**, with a door to the held queue. ⚠ That count is computed **from the list**, deliberately, so the chip can never disagree with what the queue shows — but the list is unpaginated and sorts in memory. Honest at 3, and the same breach the held queue already carries at 400.
Last time, for scale — kept, but only just	**1,263 bills**, and **no money beside it**. It is in because it is one indexed row and costs nothing; it is a *count* because a money comparison is exactly what this stage was told not to show. If it reads as clutter, it is one line to remove.

17.0 Checking | IN FLIGHT · New build S96 · Exists behind S40 S41 S45

`/:tenant/billing/run · between Scope and Check · stages 1–5, pure · nothing written`

"I pressed Prepare on 25,000 accounts. Do not show me a spinner for an hour — let me start on what you have already found."

> REDESIGN drawn **2026-09-11**, still unbuilt at Version 35 (2026-09-13), drawn to the Product Owner's 2026-09-11 run-intent ruling, the 2026-09-11 *per connection* ruling and the 2026-09-12 on-screen-calculation ruling. **NEVER BUILT.** Today the product ships a **six**-step wizard, screens 04–09 in §17.

SMARTBillHub

Billing › Run bills

Search ? α

Demo Water & Light
demoutility

- Dashboard
- Plans & Rates
- Premises
- Assets
- Accounts
- Reads
- Billing
- Payments
- Work
- Finance
- Runbooks
- Reports
- Settings

Run bills

Leave it running

Step 2 of 4 · 09-A · **September, after the water rate change** · checking since 07:55

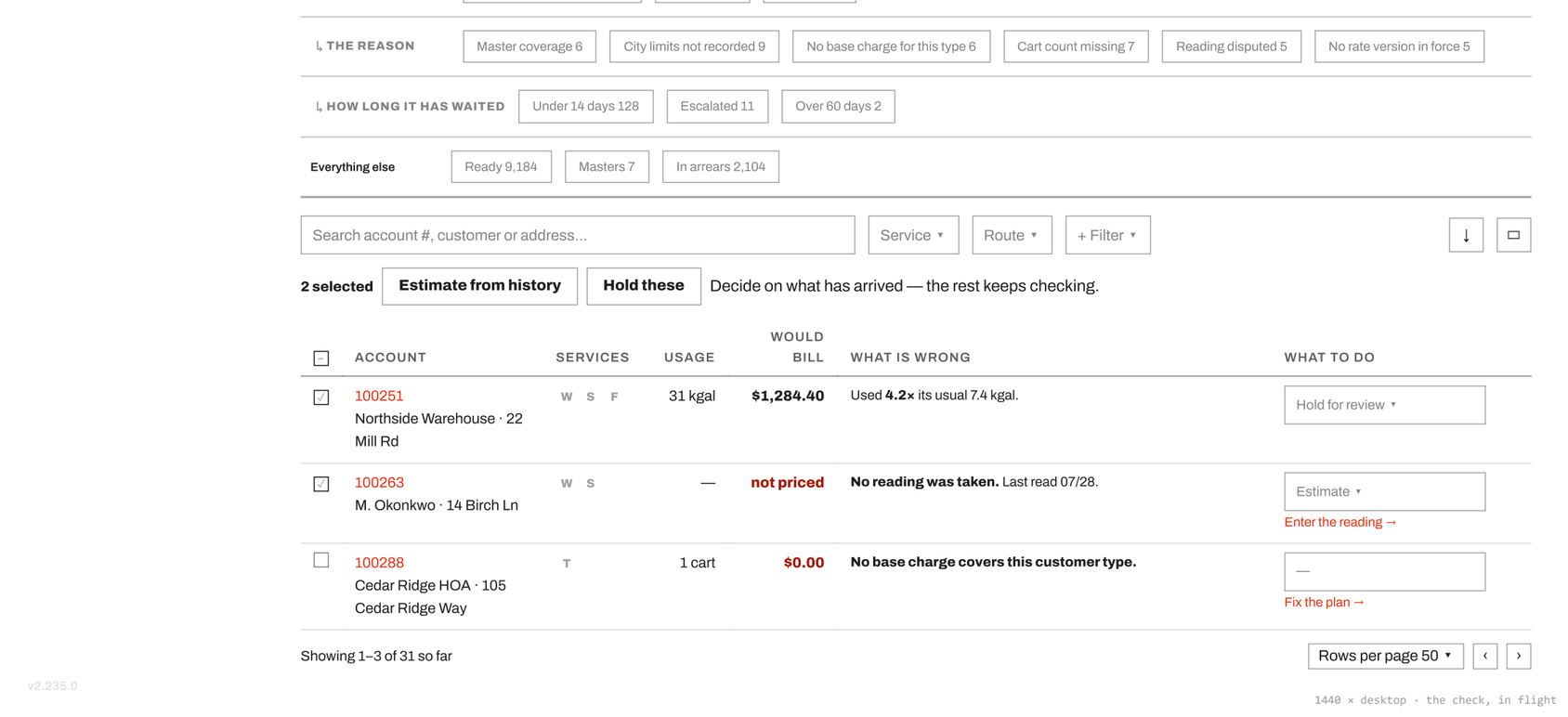
1 Scope | 2 Check | 3 Sign off | 4 Send

CHECKED SO FAR	WOULD BILL	NEEDS YOU	HELD SO FAR	BLOCKED SO FAR
9,412	$2,361,904	87	141	38
of 25,000	so far	$31,204 at stake	by your defaults	a fix elsewhere

Checking — about 35 minutes left

Nothing is being billed yet. You can close this page — it keeps going.

Needs you · 87 so far	Missing read 31	Unusually large 22	Unusually small 9	No usage 12	Unusual days 6	Meter changed 4	Multiplier changed 3
Answered by your default	Estimated 322	Base charge only 204					
Held · 141 so far	Never calculated 103	Calculated and broke 38					
↳ WHERE THE FIX IS	The account or the plan 21	The read 12	The rates 5				



Six words carry the whole screen	**“Nothing is being billed yet. You can close this page — it keeps going.”** It was four sentences and it is now two, because the long version explained *our* mechanism rather than answering the clerk’s question. The two things they need are that **no bill exists** and that **leaving is allowed** — a clerk who believes 25,000 bills are being generated behind that bar will not close the page and will not touch anything. The mechanism (stages 1–5 run, stage 6 does not, so no bill number is used until sign-off) is in §03’s pre-bill check and on `BIL-S40`, which is where a developer looks for it. **The panel header lost its count too** — *9,412 of 25,000* is already the first KPI cell, and a figure shown twice is a figure a clerk has to reconcile. What is left on the header is the one fact that appears nowhere else: **how long.**
⚠ This screen does not exist	**At 25,000 accounts it is the difference between a usable product and an unusable one.** Today the clerk presses Prepare and **waits on one request**. Measured on real PostgreSQL, a run costs **137 ms per account** — so 25,000 accounts is **about 57 minutes**, and the request wall is about **60 seconds**. There is no version of this where the clerk waits.
It opens immediately and fills in	Prepare hands the pass to the worker and **Check opens at once**, populated as the pass proceeds. The progress figures are the job’s own `done` and `total` — already served, already polled. Nothing here needs new plumbing.
The clerk works while it runs	Decisions on the accounts that have arrived are recorded immediately. **Eighty-seven decisions are waiting before the pass is half done**, and an hour spent clearing them is an hour the clerk did not spend waiting.
Every count says *so far*	A count that will grow must say so, or a clerk reads *Needs you 87* as the final answer and plans their morning around it.
What it must also do, but cannot be drawn	Refuse rather than run inline above the bar — a deployment with no worker runs the check inline today and the request dies long before its own *ran inline* answer can be read. And **resume without re-walking**: a worker killed at 9,000 of 10,000 pays **22 minutes** again unless the checkpoint is used.

17 **Check** | STEP TWO New build S27 S83 S90 S91 S92 S96 New read S73 Exists behind S26 S28 S32 S40

step 2 of 4 · /:tenant/billing/run · replaces steps 2, 3 and 4 of six · BIL-02

“Show me every account with what it will actually be billed, tell me what is wrong in words I understand, and let me fix it here — before I sign anything.”

BUILD THIS New build **One screen, priced, filterable, decisions inline** CLOSES BIL-S26 · BIL-S27 · BIL-S28

REDESIGN drawn **2026-09-11**, still unbuilt at Version 35 (2026-09-13), drawn to the Product Owner’s 2026-09-11 run-intent ruling, the 2026-09-11 *per connection* ruling and the 2026-09-12 on-screen-calculation ruling. **NEVER BUILT.** Today the product ships a **six**-step wizard, screens 04–09 in §17.

SMARTBillHub

Billing › Run bills

Search ? α

Demo Water & Light
demoutility

- Dashboard
- Plans & Rates
- Premises
- Assets
- Accounts
- Reads
- Billing
- Payments
- Work
- Finance
- Runbooks
- Reports
- Settings

Run bills

Save & exit

Step 2 of 4 · 09-A · **September, after the water rate change** · priced 09/11/2026 07:55

1 Scope | 2 Check | 3 Sign off | 4 Send

READY TO BILL	NEEDS YOU	HELD	RUN TOTAL
1,251	**12**	**21**	**$346,386.55**
$342,180.00	$4,206.55 at stake	15 by a default · 6 blocked	updates as you decide

Needs you · 12 — Missing read 4 · Unusually large 3 · Unusually small 1 · No usage 1 · Unusual days 1 · Meter changed 1 · Multiplier changed 1

Answered by your default — Estimated 44 · Base charge only 31

Held · 21 — Never calculated 15 · Calculated and broke 6

↳ **WHERE THE FIX IS** — The account or the plan 3 · The read 2 · The rates 1

↳ **THE REASON** — Master coverage 1 · City limits not recorded 1 · No base charge for this type 1 · Cart count missing 1 · Reading disputed 1 · No rate version in force 1

↳ **HOW LONG IT HAS WAITED** — Under 14 days 10 · Escalated 9 · Over 60 days 2

Everything else — Ready 1,251 · Masters 11 · In arrears 312

2 Search account #, customer or address... | Service ▾ | Route ▾ | + Filter ▾ | ↓ | ▭

3 **3 selected** | **Estimate from history** | **Hold these** | Waive the late fee | A waiver is one account at a time, with a reason.

☑	ACCOUNT	SERVICES	USAGE	4 AMOUNT	5 WHAT IS WRONG	6 WHAT TO DO
☑	100251 Northside Warehouse · 22 Mill Rd	W S F	31 kgal	**$1,284.40**	Used **4×** its usual 7.4 kgal. Meter M-9112, read 09/01.	Hold for review ▾

☐	100263 M. Okonkwo · 14 Birch Ln	W S	—	not priced	**No reading was taken.** Last read 07/28.	Estimate ▾ Enter the reading →
☐	100288 Cedar Ridge HOA · 105 Cedar Ridge Way	T	1 cart	$0.00	**No base charge covers this customer type.** Commercial · Trash.	— Fix the plan →
☐	100302 D. Whitfield · 8 Kiln St	W	12 kgal	7 *re-pricing…*	Reading corrected 2 minutes ago	waiting for the new amount

8 Showing 1–4 of 12 that need you

Rows per page 50 ▾ ‹ ›

9

Nobody is missing from this list

All 1,284 accounts in the cycle are accounted for: 1,251 ready, 12 need you, 21 held by your defaults, 0 broke. **1,251 + 12 + 21 = 1,284.**

10 Sign off on this run →

10 12 accounts still need a decision.

v2.235.0

BIL-02

1 **Five figures, and the run total moves as you decide** · `/findings` counts plus the priced drafts · **NEW READ** — today the totals are a *separate step* reached after the decisions, so a clerk decides without knowing what is at stake

2 **Search, filter chips and a column chooser** · `?q=` plus `f.*` on the run's own account set · **NEW** — steps 2, 3 and 4 have **no controls of any kind** today, over what can be thousands of rows. **The chips carry counts**, so a clerk never opens an empty tab (`DEF-1082`)

3 **Bulk actions on the selection**, with the one refusal that is a rule rather than a gap: a late-fee waiver is one account at a time, with a reason (BR-BIL-060) · **NEW**

4 **Amount, on every row — the change everything else rests on** · `invoices.total` from the draft priced at step 1 · **NEW READ**. Today `amount` , `currency` , `quantity` and `unit` are **null on every case row** and `totals` is always `{}` , because *“phase 1 computes no money at all”*

5 **What is wrong, in a clerk's words, with the evidence beside it** · *“Used 4× its usual 7.4 kgal. Meter M-9112, read 09/01”* · **CHANGED** — today a clerk is shown `premises_jurisdiction_unstated` , a rule id, a raw ISO timestamp, and a catch-all with no code at all (`DEF-518` · `DEF-2432` · `DEF-2384` · `DEF-2895`)

6 **The decision inline, and a link out where the fix is upstream** · `POST /billing/runs/{id}/exceptions/{case}/resolve` , and *Enter the reading →* / *Fix the plan →* for the cases billing may not write · **NEW**

7 **A row re-prices when you change it, or when you come back from fixing something** · a scoped re-run of `calculate` , which *replaces* that account's draft rather than adding one · **NEW** — and it is safe because the engine already guarantees it

8 **Pagination** · `/findings` , `/exceptions` , `/pricing-shadow` and `/breaks` are **all unpaginated today** — three of them by declared design · NFR-3 · **NEW**

9 **Nobody is missing, and the arithmetic proves it** · `1,251 + 12 + 21 = 1,284` · **NEW — this is the most important thing on the screen**, see below

10 **Sign-off is refused while anything is undecided**, and says how many · BR-SHL-025 · **NEW**

Callout ⑨ is the one I would not ship without, and it is there because of a live high-severity defect.

`DEF-2381` : *“A BILL RUN SAYS NOTHING WHEN AN ACCOUNT'S CUSTOMER TYPE IS COVERED BY NO BASE CHARGE — **the account produces no bill and appears in no list on the review step.**”* An account can currently fall out of a run **silently**, and the clerk signs off on a total that is short by however many of those there were. Nothing on any of the six steps would tell them.

would tell them.

A count that reconciles is the cheapest possible guard against that whole class. If ready + needs-you + held + broke does not equal the accounts in the cycle, the screen says so and refuses sign-off. It costs one subtraction and it makes silent omission impossible — which is the entire point of showing a clerk everything before they submit.

What the three merged steps each contributed, so nothing is lost. Step 2 (What was found) gave six dimensions of the account set — they become the filter chips at ②, with counts, so the three of six that exist only to explain an unbuilt feature (`DEF-1081`) simply do not appear. **Step 3 (The customers)** gave the per-account decision — it becomes the inline control at ⑥, now with the money beside it. **Step 4 (The totals)** gave the priced result and the break list — it becomes the band at ① and the *Broke* chip, live rather than one screen later.

And one thing on step 4 deliberately does not carry over: the three overcharge actions that are inert `<span>`s over a list that has never existed (`DEF-2855`, high). Here *unusually large* is a filter chip with a real count and a real decision control, which is what those three words were always meant to be.

Step 2, one chip at a time — the three bands

The twenty screens that follow are step 2 with one chip selected, and one per bulk action. They share a shell: the four KPI cells never change, because **a filter changes the list and never the run**.

BAND	WHAT IT MEANS	WHAT THE CLERK DOES	CHIPS
Needs you — 12	No default applies, or the default would hold the account.	Decides, per row or in bulk.	Seven — one per case kind the engine raised
Answered by your default	A step-1 default already chose. The rows say which.	Skims, and overturns the odd one.	Two — estimated reads, base-charge-only
Held — 21	Will not be billed in this run. 15 by a default, 6 blocked.	Changes their mind, or goes and fixes the data.	Four — by a default, and three by where the fix is
Everything else	Nothing raised, nothing to decide.	Skims by amount.	Two — Ready, and Masters as a cross-cutting view

Chips are generated from what the run raised. They are never a list in the code. The screen today declares **fifteen** case kinds and the engine emits **nine** — so three tabs exist whose only content is an explanation that the feature is not built, and four more can never populate. Generating the strip fixes that class of defect permanently: a kind the engine did not raise **gets no chip**. Every chip also carries a **count and a money figure**, because *Unusually large 3 · $4,206 at stake* tells a clerk where to spend attention and a bare count does not.

$4,206 at stake tells a clerk where to spend attention and a bare count does not.

The nine categories you named, against what the engine actually emits

YOU SAID	ENGINE CASE	REALITY	SCREEN
High consumption	`high_usage`	**Emitted** — carries meter, this period, trailing average, ratio and your threshold	17.2
Low consumption	`low_usage`	**Emitted** , same five facts	17.3
Zero consumption	`no_usage`	**Emitted** — and a zero read is not a missing one	17.4
Day threshold	`days_threshold`	**Emitted** — carries meter and days	17.5
Plan & rates missing	`no_charge_applies_to_this_customer_type` and five siblings	**It is a break, not a case** — so it needs no chip of its own: it is blocked, with the fix named, and correctly offers no retry	17.13, 17.11
Prorated bills	—	**No case kind exists.** Nothing detects a part period as such, and the two *prorate* **choices** that do exist are recorded and unpriced	17.5, 17.6
Late payment	`late_fee` , `past_due`	**Declared and never emitted.** The screen offers a tab for it; the engine raises nine kinds and neither of those is one	17.20, drawn as the refusal
Credit applied	`true_up`	**Declared, not emitted as a case.** There is a Credits view that shows exceptions only	—
Not billed in last 3 months	—	**No case kind exists** — and it is the most valuable one on your list, because it catches revenue quietly going missing. Screen 20 is the nearest thing	20

Three the engine emits that were not on your list, and they are drawn

`estimated_read` (17.8), `meter_factor` — the multiplier changed (17.7), and `replacement` — the meter changed mid-cycle (17.6). The multiplier one deserves a mention: the reading is fine and the **number it is multiplied by** moved, so a bill goes up tenfold with a read that looks entirely normal.

The chip inventory — every chip, what raises it, and what you call it

Scanning the band strip for a category by **the name you use for it** has to work. The chip carries **the product's own words**, which are not always the words of the setting that controls it — so both are in this table, beside the engine case underneath. **Fifteen chips, nine engine cases, one per emitted kind.**

CHIP ON SCREEN	WHAT YOU MAY CALL IT	ENGINE CASE	BAND	SCREEN
Missing read	No read · unread meter	`missing_read`	Needs you	17.1

Unusually large	High consumption	`high_usage`	Needs you	17.2
Unusually small	Low consumption	`low_usage`	Needs you	17.3
No usage	Zero consumption	`no_usage`	Needs you	17.4
Unusual days	**Day threshold** — the setting is in days; the product's own sentence is *"the reading covers an unusual number of days"*, and Meter Reads calls the same condition *Read interval*	`days_threshold`	Needs you	17.5
Meter changed	Meter swap · replacement mid-cycle	`replacement`	Needs you	17.6
Multiplier changed	Meter factor · CT ratio	`meter_factor`	Needs you	17.7
Estimated	Estimated read	`estimated_read`	By your default	17.8
Base charge only	Standing charge only · unmetered · vacation hold	`base_only_billing`	By your default	17.9
By a default	Held by my own default	— one exclusion row each	Held	17.10
Fix the account or the plan	Blocked · subscription problem	break stage `resolve-subscription`	Held	17.11
Fix the read	Blocked · no usable quantity	break stage `measure-consumption`	Held	17.12
Fix the rates	Blocked · pricing problem	break stage `price-usage`	Held	17.13
Ready	Clean · nothing raised	—	Everything else	17.14
Masters	Master accounts · HOA · landlord bills	a view, not a case	Everything else	17.15

Every case the engine emits has a chip. Three categories on your original list have no chip because they have no case — and that is engine work, not screen work. Prorated bills — nothing detects a part period as such, and the two *prorate* **choices** that do exist are recorded and unpriced. **Late payment** — declared in two places and **never emitted**; the screen offers a tab for it today and the engine raises nine kinds, neither of them that. **Not billed in the last three months** — nothing looks for it anywhere, and it is arguably the most valuable of the three because it catches revenue quietly going missing. **Credit applied** is a fourth, half-case: declared, not emitted, with a Credits view that shows exceptions only.

Why the chip is not called "day threshold"

Because the threshold is the **setting** and the chip is the **finding**. A clerk sets *20 days* in Settings

and then meets an account whose reading covered 13 — the chip names what happened, not what governs it, which is how every other chip here is worded. **The row closes the loop by printing all three numbers:** *read 13 days after the last one; this cycle is 31; your threshold is 20*. **If you would rather the chip said *Day threshold* to match Settings, say so and it changes in one place** — the chips are generated, so the label lives once. What it must not do is be a **third** wording, which *"short or long period"* was, and that is why it is gone.

Two options a clerk would reach for first produce no bill at all, and nothing on screen says so today. *Prorate to the cycle's days* and *prorate both segments* are both **recorded choices whose pricing does not exist** — no signed rule scales a consumption to N days, and the pricing seam prices one head, not two segments. So the account holds at a named boundary rather than being billed on invented arithmetic, which is the right call. **But a clerk picking the most reasonable-sounding option believes they have fixed the account.** Every option on these screens therefore states whether it bills or holds, and those two read *"holds the bill, pricing not built"*. It is the most important label in section 03.

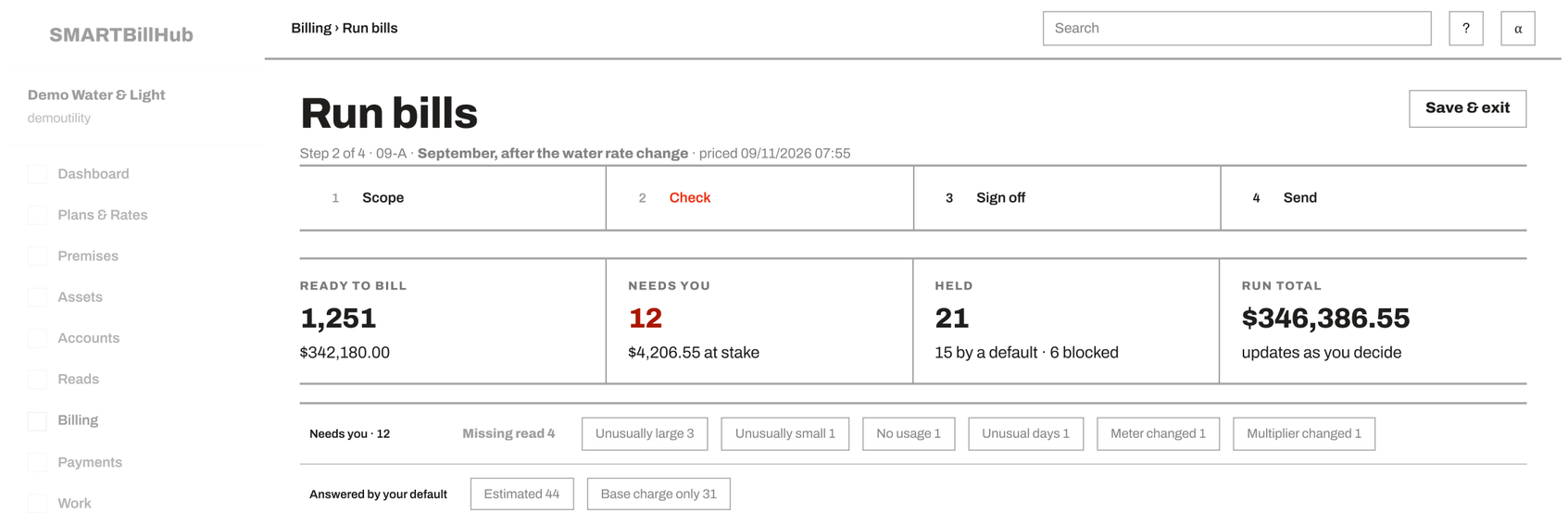
17.1 No read

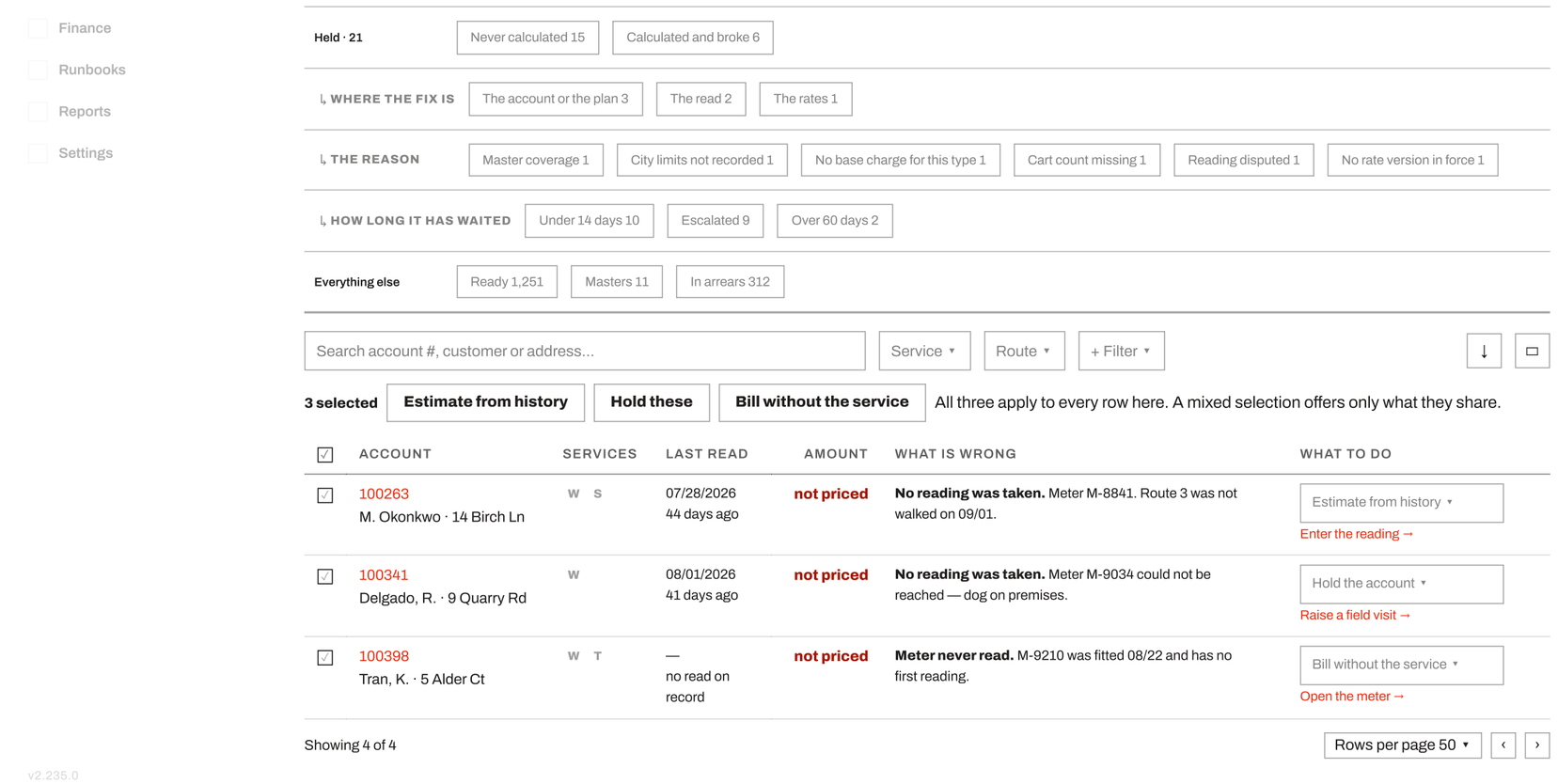
THREE WAYS · New build S91 S92 · Exists behind S32 S33 S47 S49

step 2 chip · 4 rows · missing_read · /:tenant/billing/run · step 2, one chip · BIL-02 · engine case missing_read

"Four accounts have no reading. Tell me what each one is, and let me choose per account — not one rule for all four."

> **REDESIGN** drawn **2026-09-11**, still unbuilt at Version 35 (2026-09-13), drawn to the Product Owner's 2026-09-11 run-intent ruling, the 2026-09-11 *per connection* ruling and the 2026-09-12 on-screen-calculation ruling. **NEVER BUILT.** Today the product ships a **six**-step wizard, screens 04–09 in §17.





1440 × desktop · step 2 · one chip

The three choices	The rule's own order, and the rule's own words: **estimate it** · **hold the account** · **bill without the service**. The engine publishes this list; the screen never hardcodes it, so the day a choice changes the screen changes with it.
Which ones hold	**Hold the account** holds. So does **bill without the service** on an account whose only service is the unread one — there is nothing left to bill. **Estimate** bills.
New on this screen	The **last read date and its age**, the **meter**, and **why** the read is missing in words. Today the reason reaches a clerk as a reason code.
Link out	Every row offers the screen that fixes it — enter the reading, raise a field visit, open the meter. A decision screen with no door to the fix makes the clerk decide blind.
Holds	An option marked **holds the account** produces **no bill this run**. The account goes to Held accounts and is billed by a catch-up run.

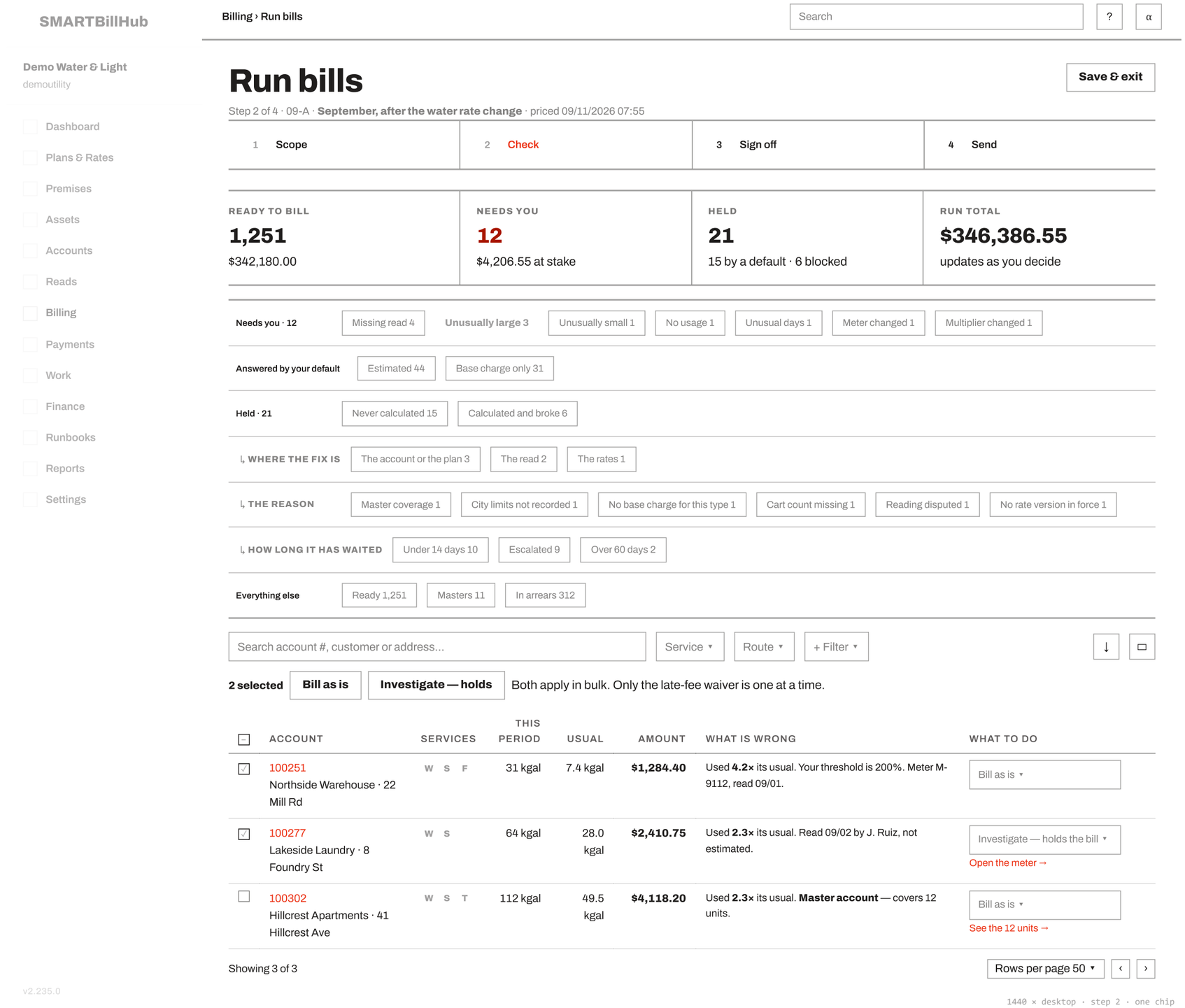
17.2 Unusually large | COMPARISON Exists behind S32 S33 S47

step 2 chip · 3 rows · high_usage · /:tenant/billing/run · **step 2, on**
e chip · BIL-02 · engine case high_usage

"Is this real, or is it a leak or a misread? Put this period beside the usual one so I can tell without opening another screen."

REDESIGN drawn **2026-09-11**, still unbuilt at Version 35 (2026-09-13), drawn to the Product Owner's 2026-09-11 run-intent ruling, the 2026-09-11 *per connection* ruling and the 2026-09-

12 on-screen-calculation ruling. **NEVER BUILT.** Today the product ships a **six**-step wizard, screens 04–09 in §17.



Why its own screen

This is a **judgement**, not a lookup. The two numbers have to be side by side, with the ratio and **your own threshold** stated — otherwise a clerk cannot tell a hot August from a burst pipe.

The two choices	**Bill as is** · **Investigate**. The engine offers exactly these two.
Which one holds	**Investigate holds the account.** A figure a human has called suspect must not reach a document while they look — so choosing it means no bill this run, and the screen says so on the option itself.
Money at stake	The band shows **$4,206.55 at stake** so a clerk knows whether to spend ten minutes here or ten seconds. A count alone does not say that.
Master rows	A master account says **covers 12 units** in the cause column, so $4,118.20 is never read as one household's water.

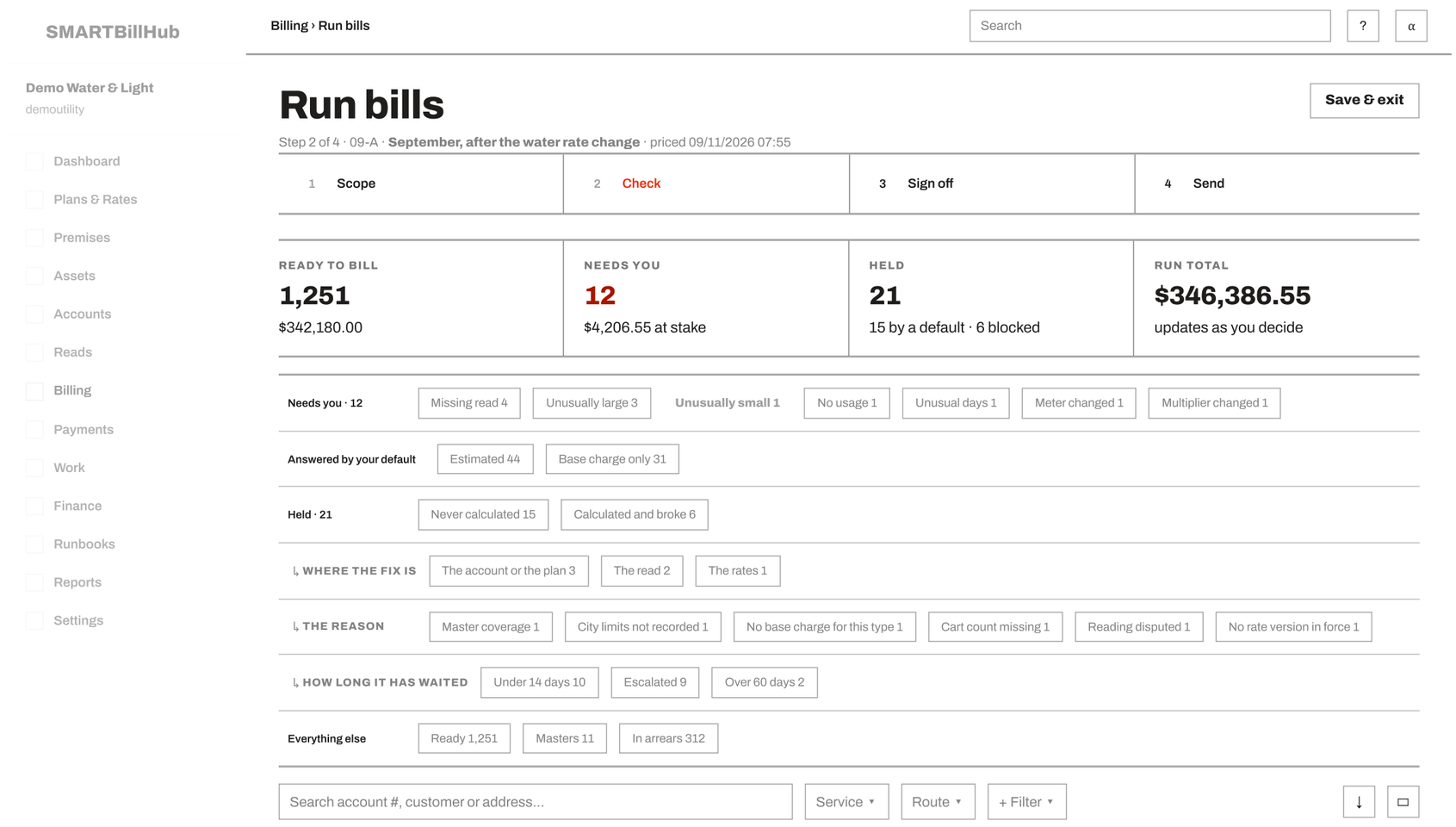
17.3 Unusually small

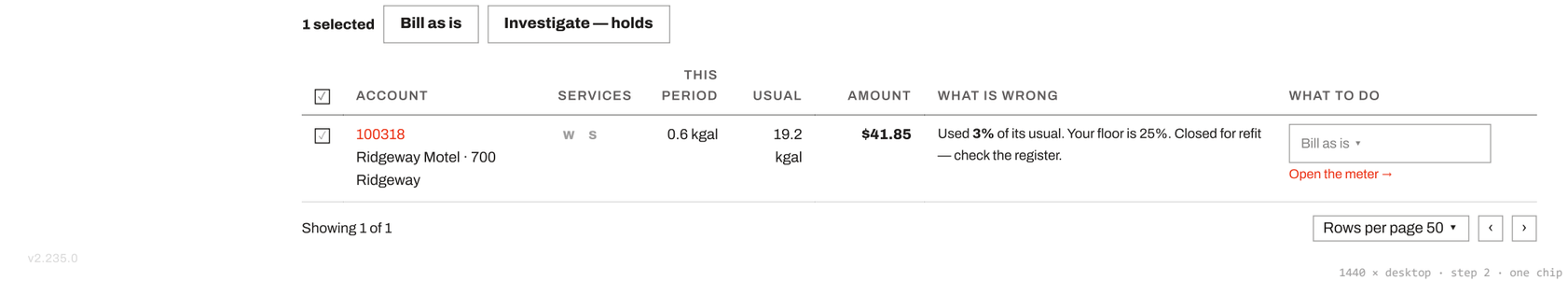
QUIET HALF Exists behind S32 S33 S47

step 2 chip · 1 row · `low_usage` · /:tenant/billing/run · **step 2, one chip** · BIL-02 · engine case `low_usage`

"A near-zero read on a business is either a closure or a stopped meter. Which?"

REDESIGN drawn **2026-09-11**, still unbuilt at Version 35 (2026-09-13), drawn to the Product Owner's 2026-09-11 run-intent ruling, the 2026-09-11 *per connection* ruling and the 2026-09-12 on-screen-calculation ruling. **NEVER BUILT.** Today the product ships a **six**-step wizard, screens 04–09 in §17.





Same shape, opposite tail	Identical columns and identical choices to **Unusually large** — deliberately, because it is the same comparison read the other way. Two chips rather than one because the **action** differs: a large read is usually a leak to chase, a small one is usually a stopped meter to replace.
Which one holds	**Investigate holds the account.**
Why it matters more than it looks	An under-read bills too little and nobody complains. It is the quiet half of the pair, which is why it gets a chip of its own rather than being folded into the large one.

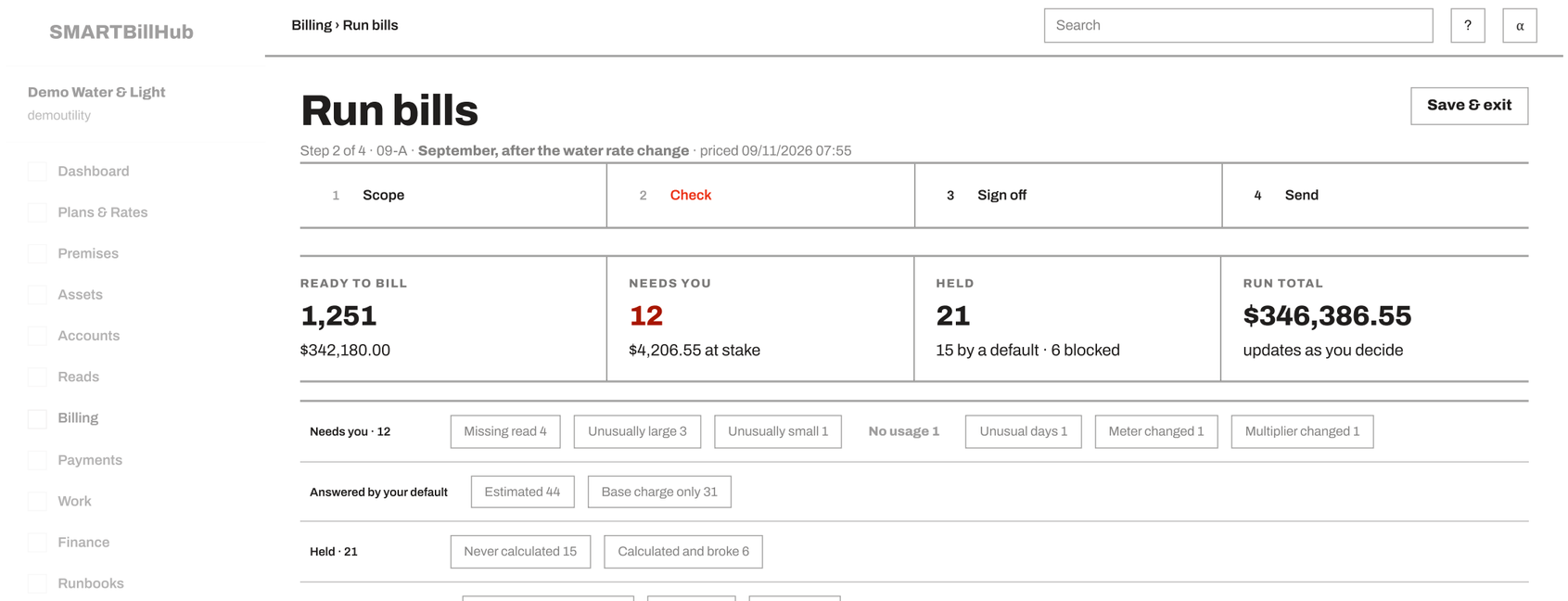
17.4 Zero use | BASE BILLS

Exists behind S32 S33 S47

`step 2 chip · 1 row · no_usage · /:tenant/billing/run · step 2, one chip · BIL-02 · engine case no_usage`

“Nothing was used. Do I still bill the standing charge?”

> **REDESIGN** drawn **2026-09-11**, still unbuilt at Version 35 (2026-09-13), drawn to the Product Owner’s 2026-09-11 run-intent ruling, the 2026-09-11 *per connection* ruling and the 2026-09-12 on-screen-calculation ruling. **NEVER BUILT.** Today the product ships a **six**-step wizard, screens 04–09 in §17.



Reports
Settings

↳ WHERE THE FIX IS | The account or the plan 3 | The read 2 | The rates 1

↳ THE REASON | Master coverage 1 | City limits not recorded 1 | No base charge for this type 1 | Cart count missing 1 | Reading disputed 1 | No rate version in force 1

↳ HOW LONG IT HAS WAITED | Under 14 days 10 | Escalated 9 | Over 60 days 2

Everything else | Ready 1,251 | Masters 11 | In arrears 312

Search account #, customer or address... | Service ▾ | Route ▾ | + Filter ▾ | ↓ | ▭

1 selected | Include | Exclude — holds

☑	ACCOUNT	SERVICES	USAGE	AMOUNT	WHAT IS WRONG	WHAT TO DO
☑	100355 Vacant — 12 Kiln Rd · owner: Bayside Holdings	W S	0 kgal	**$34.00**	**Read is zero, not missing.** Meter M-9188 read 09/01 at the same figure as 08/01. Base charges still apply.	Include — bill the base ▾ See the account →

Showing 1 of 1 | Rows per page 50 ▾ | ‹ | ›

v2.235.0

1440 × desktop · step 2 · one chip

Zero is not missing	The distinction the column makes out loud: a **zero read** was taken and says nothing was used; a **missing read** was never taken. They land on different chips because they are fixed differently.
The two choices	**Include** (bill the base charges) · **Exclude**.
Which one holds	**Exclude holds the account.**
The amount is not zero	A no-usage bill is usually **not** $0.00 — the base charges stand. The Amount column shows what Include would actually bill, so the choice is made against a figure rather than a guess.

17.5 Unusual days | UNPRICED Exists behind S32 S33 S47

`step 2 chip · 1 row · days_threshold · /:tenant/billing/run · step 2, one chip · BIL-02 · engine case days_threshold`

“This meter was only read for part of the cycle. Do I prorate it, wait, or bill it as it stands?”

REDESIGN drawn **2026-09-11**, still unbuilt at Version 35 (2026-09-13), drawn to the Product Owner’s 2026-09-11 run-intent ruling, the 2026-09-11 *per connection* ruling and the 2026-09-12 on-screen-calculation ruling. **NEVER BUILT.** Today the product ships a **six**-step wizard, screens 04–09 in §17.

SMARTBillHub

Billing › Run bills | Search | ? | α

Demo Water & Light
demoutility

Dashboard
Plans & Rates
Premises

Run bills

Save & exit

Step 2 of 4 · 09-A · **September, after the water rate change** · priced 09/11/2026 07:55

1 Scope | 2 Check | 3 Sign off | 4 Send

READY TO BILL
1,251
$342,180.00

NEEDS YOU
12
$4,206.55 at stake

HELD
21
15 by a default · 6 blocked

RUN TOTAL
$346,386.55
updates as you decide

Needs you · 12 Missing read 4 | Unusually large 3 | Unusually small 1 | No usage 1 | **Unusual days 1** | Meter changed 1 | Multiplier changed 1

Answered by your default Estimated 44 | Base charge only 31

Held · 21 Never calculated 15 | Calculated and broke 6

↳ **WHERE THE FIX IS** The account or the plan 3 | The read 2 | The rates 1

↳ **THE REASON** Master coverage 1 | City limits not recorded 1 | No base charge for this type 1 | Cart count missing 1 | Reading disputed 1 | No rate version in force 1

↳ **HOW LONG IT HAS WAITED** Under 14 days 10 | Escalated 9 | Over 60 days 2

Everything else Ready 1,251 | Masters 11 | In arrears 312

Search account #, customer or address... | Service ▾ | Route ▾ | + Filter ▾

1 selected **Bill as is** | Prorate to the cycle's days | **Wait for the next cycle — holds** | Prorate is offered per row, not in bulk — see below.

☑	ACCOUNT	SERVICES	PERIOD COVERED	DAYS	AMOUNT	WHAT IS WRONG	WHAT TO DO
☑	100362 Foster, A. · 18 Kiln Rd	W S	08/19 – 09/01/2026	13	**$28.40**	Read **13 days** after the last one; this cycle is **31**. Your threshold is 20 days. Meter fitted 08/19.	Bill as is ▾

Showing 1 of 1 | Rows per page 50 ▾ | ‹ | ›

v2.235.0

1440 × desktop · step 2 · one chip

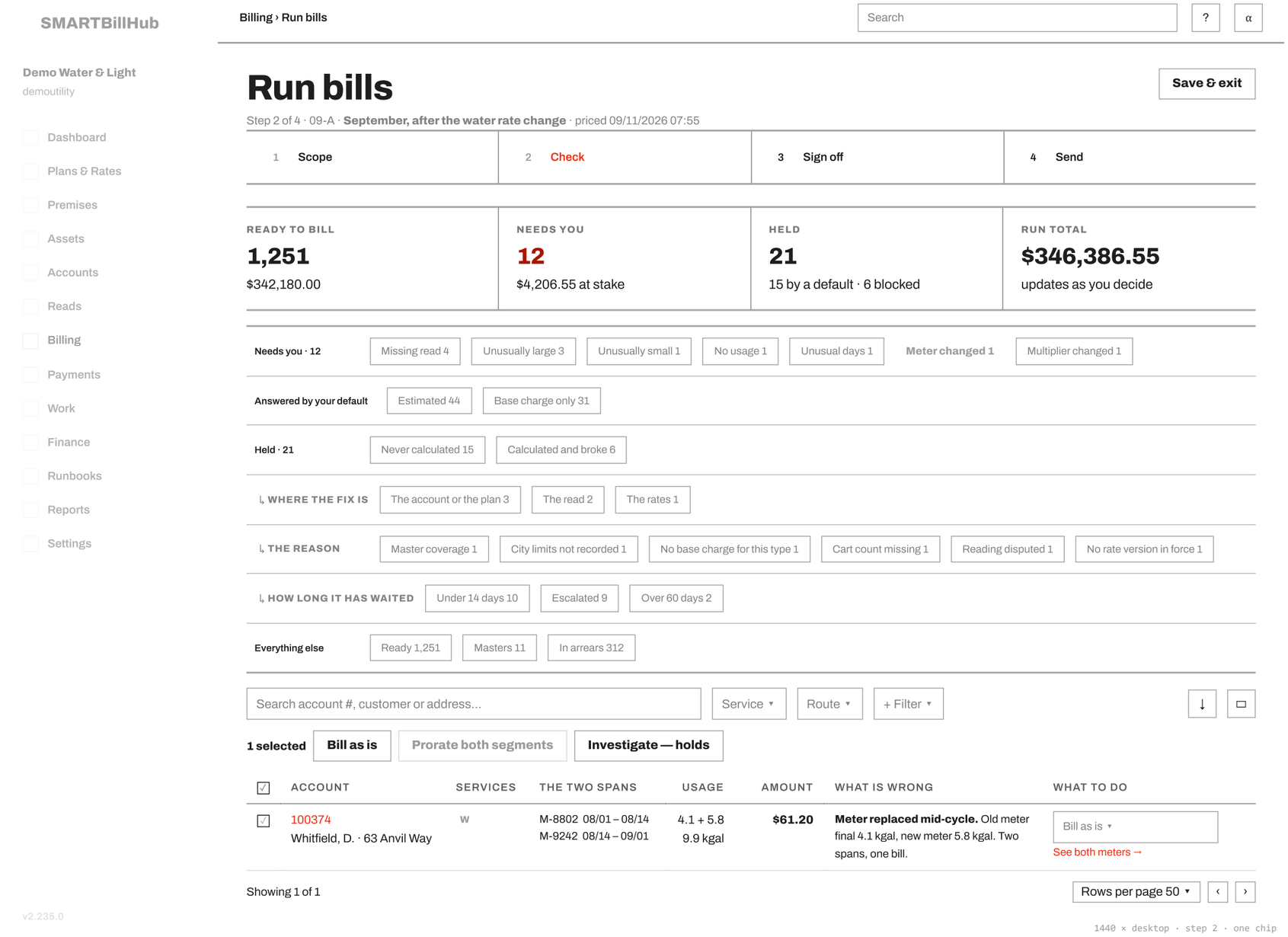
The three choices	**Prorate to the cycle's days** · **Wait for the next cycle** · **Bill as is**.
Two of the three hold, and one of those is a surprise	**Wait for the next cycle** holds, which a clerk expects. **Prorate to the cycle's days also holds** — and nobody would guess it. The choice is recorded, but **no signed rule scales a consumption to N days**, so there is nothing to price it with and the account waits at that boundary.
So the option must say so	The dropdown reads **"Prorate to the cycle's days — holds the bill, pricing not built"**. Without that sentence a clerk picks the most reasonable-sounding option, believes they have fixed the account, and it silently produces no bill. **This is the single most important label on any of these screens.**
Build note	If proration lands before ship, the sentence comes off and the option bills. Until then it is a named boundary, honestly labelled.

17.6 Meter changed | TWO SPANS Exists behind S32 S33 S47

step 2 chip · 1 row · `replacement` · /:tenant/billing/run · **step 2, one chip** · BIL-02 · engine case `replacement`

"Two meters served this address this month. Show me both spans and what each one measured."

REDESIGN drawn **2026-09-11**, still unbuilt at Version 35 (2026-09-13), drawn to the Product Owner's 2026-09-11 run-intent ruling, the 2026-09-11 *per connection* ruling and the 2026-09-12 on-screen-calculation ruling. **NEVER BUILT.** Today the product ships a **six**-step wizard, screens 04–09 in §17.



The three choices	**Prorate both segments** · **Bill as is** · **Investigate**.
Two of the three hold	**Investigate** holds, as everywhere. And **prorate both segments also holds**, for the same reason as the last screen and a second one: the pricing seam **prices one head, not two segments**. So the option is recorded and produces no bill.
The label, again	**"Prorate both segments — holds the bill, pricing not built"**. Of the three options, the only one that bills is **Bill as is**, which sums the two spans. A clerk needs to know that before choosing.

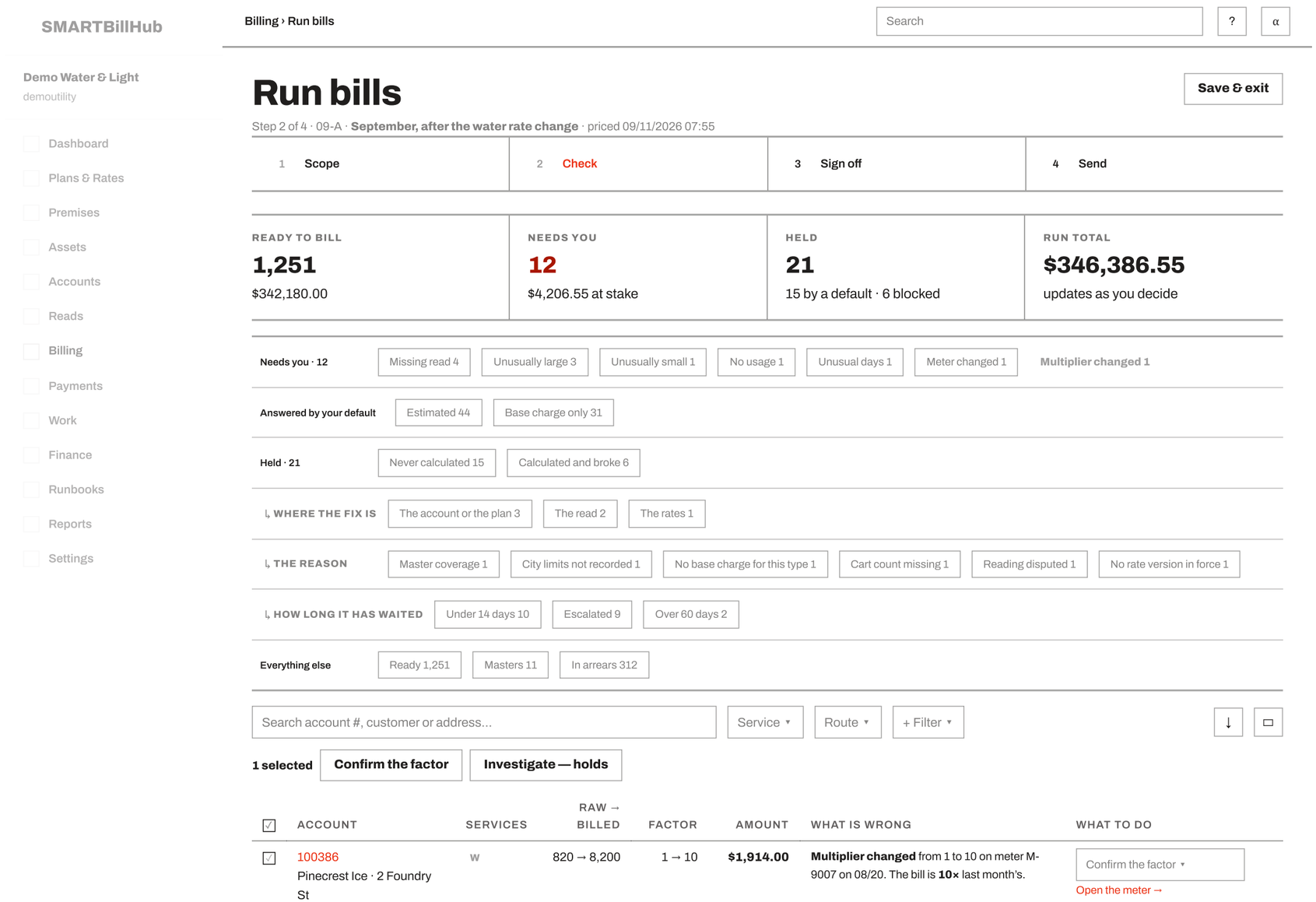
What the row must show	Both **meter numbers**, both **spans**, both **quantities**, and the sum. A single “9.9 kgal” hides the fact that two devices measured it.

17.7 Multiplier | TENFOLD Exists behind S32 S33 S47

step 2 chip · 1 row · `meter_factor` · /:tenant/billing/run · **step 2, on**
e chip · BIL-02 · engine case `meter_factor`

“Somebody changed the multiplier. Was that right? It has just multiplied this bill by ten.”

> REDESIGN drawn **2026-09-11**, still unbuilt at Version 35 (2026-09-13), drawn to the Product Owner’s 2026-09-11 run-intent ruling, the 2026-09-11 *per connection* ruling and the 2026-09-12 on-screen-calculation ruling. **NEVER BUILT.** Today the product ships a **six**-step wizard, screens 04–09 in §17.



Showing 1 of 1

Rows per page 50 ▾ ‹ ›

v2.235.0

1440 × desktop · step 2 · one chip

Why this is not a usage case	The reading is fine. What changed is the **number the reading is multiplied by** — a configuration change on the meter that moves the bill by an order of magnitude and leaves the read looking perfectly normal.
The two choices	**Confirm the factor** · **Investigate**.
Which one holds	**Investigate holds the account.**
What the row must show	The **raw reading and the billed quantity side by side**, and **the old factor and the new one**, with the date it changed. This is the one case where the arithmetic is the whole finding, so the arithmetic is on the row.

17.8 Estimated | DEFAULTED Exists behind S32 S33 S47

`step 2 chip · 44 rows · estimated_read · /:tenant/billing/run · step 2, one chip · BIL-02 · engine case estimated_read`

“Forty-four of these. I am not deciding forty-four times — show me that my default handled them and how, and let me overturn the odd one.”

> REDESIGN drawn **2026-09-11**, still unbuilt at Version 35 (2026-09-13), drawn to the Product Owner’s 2026-09-11 run-intent ruling, the 2026-09-11 *per connection* ruling and the 2026-09-12 on-screen-calculation ruling. **NEVER BUILT.** Today the product ships a **six**-step wizard, screens 04–09 in §17.

SMARTBillHub

Billing › Run bills

Search ? α

Demo Water & Light
demoutility

- Dashboard
- Plans & Rates
- Premises
- Assets
- Accounts
- Reads
- Billing
- Payments
- Work
- Finance
- Runbooks
- Reports
- Settings

Run bills

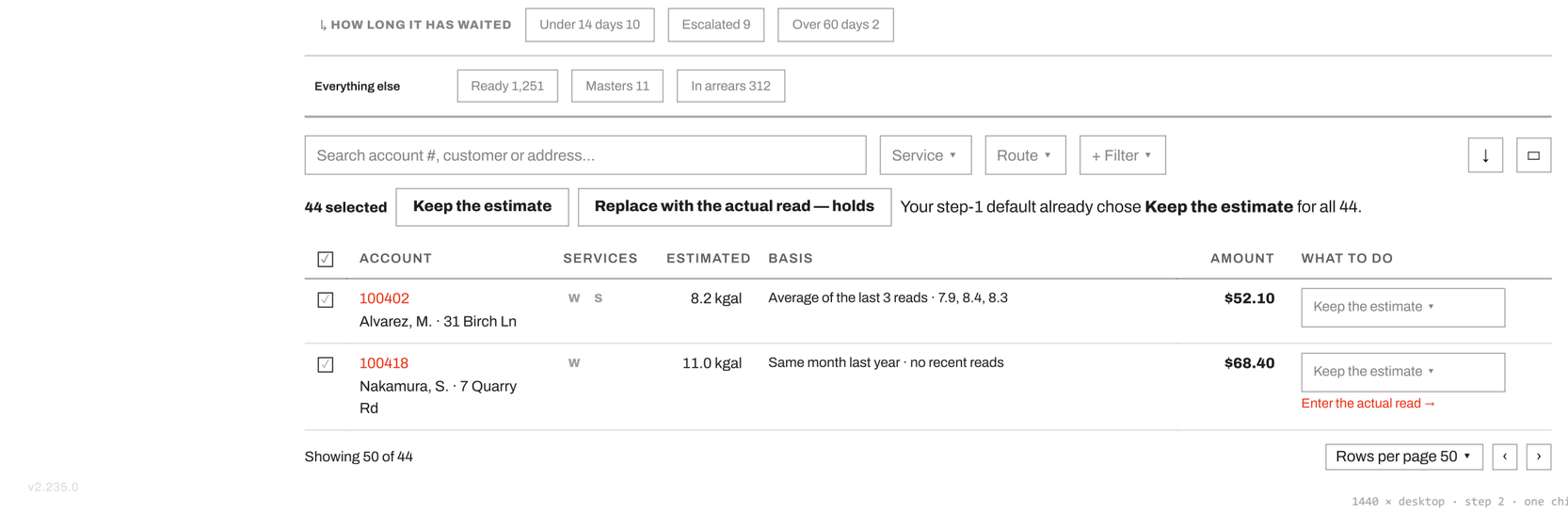
Save & exit

Step 2 of 4 · 09-A · **September, after the water rate change** · priced 09/11/2026 07:55

1 Scope | 2 Check | 3 Sign off | 4 Send

READY TO BILL	NEEDS YOU	HELD	RUN TOTAL
1,251	**12**	**21**	**$346,386.55**
$342,180.00	$4,206.55 at stake	15 by a default · 6 blocked	updates as you decide

Needs you · 12	Missing read 4	Unusually large 3	Unusually small 1	No usage 1	Unusual days 1	Meter changed 1	Multiplier changed 1
Answered by your default	**Estimated 44**	Base charge only 31					
Held · 21	Never calculated 15	Calculated and broke 6					
↳ **WHERE THE FIX IS**	The account or the plan 3	The read 2	The rates 1				
↳ **THE REASON**	Master coverage 1	City limits not recorded 1	No base charge for this type 1	Cart count missing 1	Reading disputed 1	No rate version in force 1	



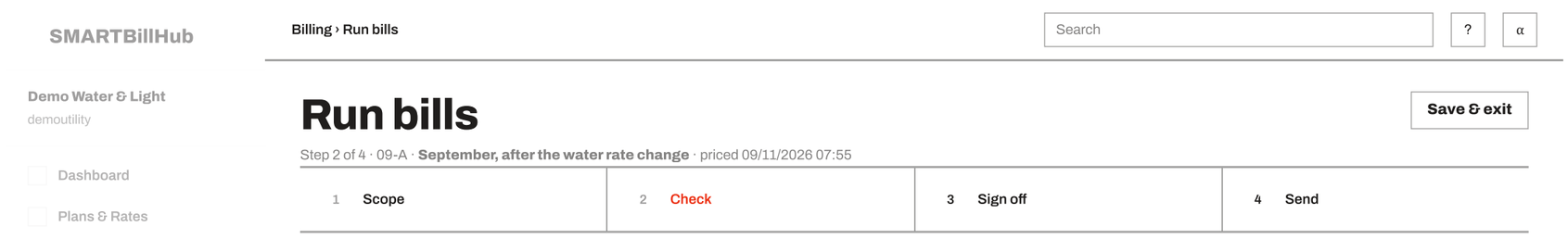
Why a band of its own	These **do not need you**. Your step-1 default already answered them, and the band label says so. Putting 44 rows in **Needs you** would bury the 12 that genuinely do.
The two choices	**Keep the estimate** · **Replace with the actual read**.
Which one holds	**Replace with the actual read holds the account.** The clerk has refused the estimate, so billing it anyway would un-make their decision — the account waits for a real reading.
What the row must show	The **basis** in words — *average of the last 3 reads*, *same month last year* — with the figures it used. An estimate with no stated basis is a number a clerk cannot defend to a customer.
Pagination	Forty-four rows are **paged on the server**, like every list in the product. No chip here loads a whole run into the browser.

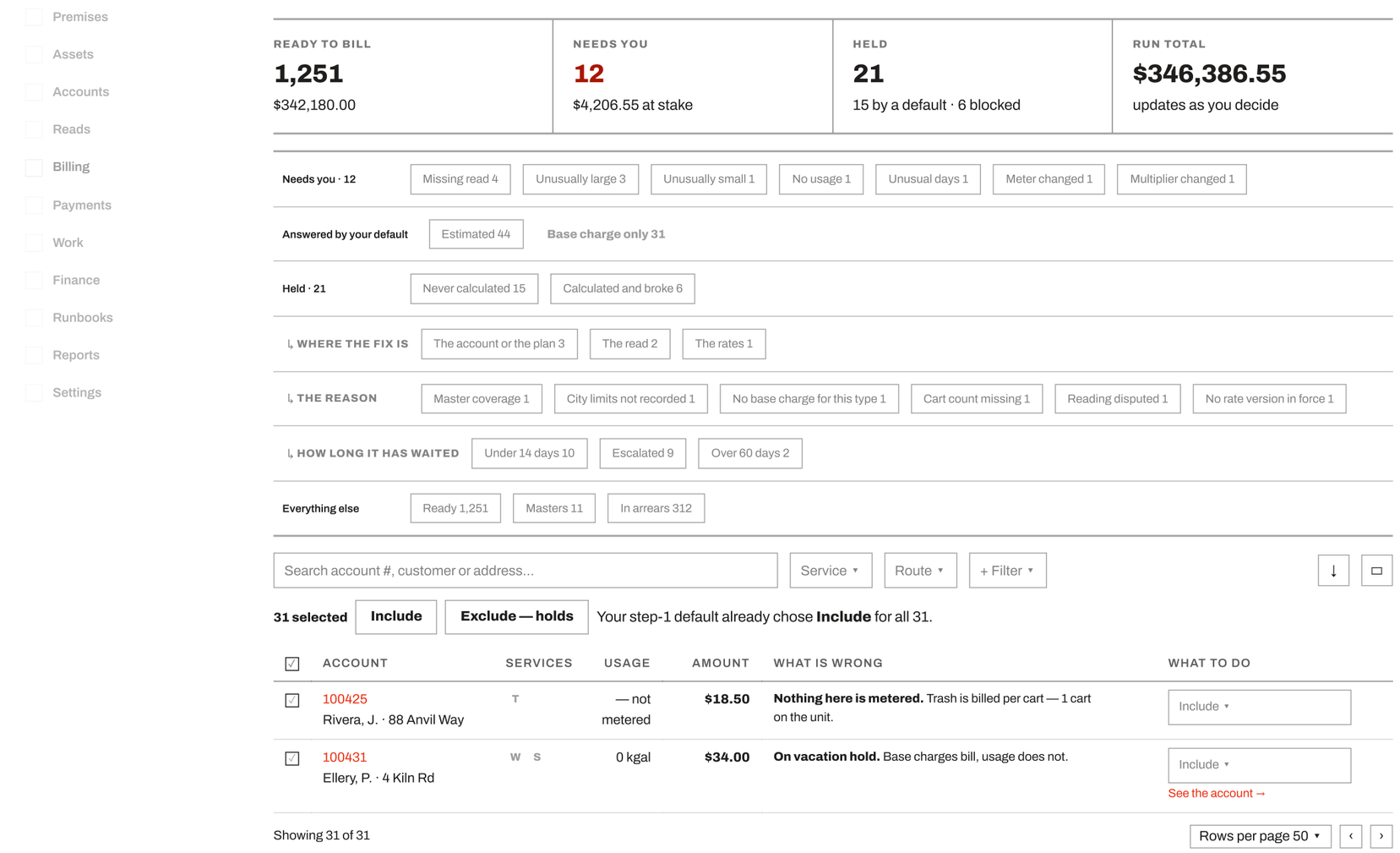
17.9 Base only | TWO CAUSES Exists behind S32 S33 S47

`step 2 chip · 31 rows · base_only_billing · /:tenant/billing/run · step 2, one chip · BIL-02 · engine case base_only_billing`

“Thirty-one accounts bill a standing charge and no usage. Tell me that is on purpose.”

> REDESIGN drawn **2026-09-11**, still unbuilt at Version 35 (2026-09-13), drawn to the Product Owner’s 2026-09-11 run-intent ruling, the 2026-09-11 *per connection* ruling and the 2026-09-12 on-screen-calculation ruling. **NEVER BUILT.** Today the product ships a **six**-step wizard, screens 04–09 in §17.





Two different reasons, one chip	A base-only bill is either a **service that is never metered** (trash by the cart, fire by the line size) or an **account deliberately not billing usage** (a vacation hold). The cause column says which, because the second is temporary and the first is not.
The two choices	**Include** · **Exclude**.
Which one holds	**Exclude holds the account.**
Not the same as blocked	This chip is **base charges that exist and will bill**. An account whose customer type has **no** base charge is a different thing entirely — it cannot be billed at all, and it is on the Blocked band, three screens down.

17.10 Held now | YOUR DEFAULTS

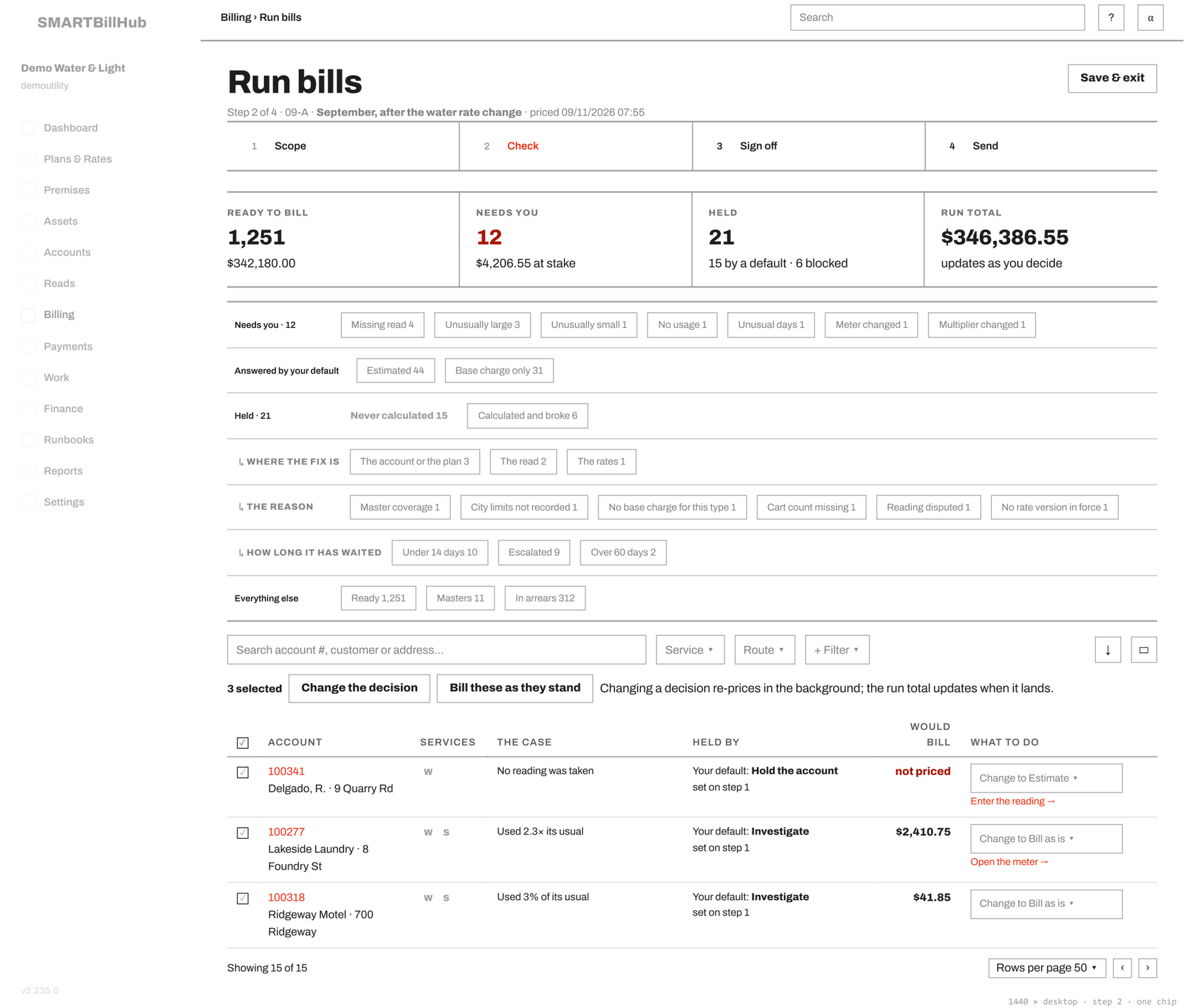
Exists behind S32 S47

`step 2 chip · 15 rows · one exclusion row each · /:tenant/billing/run · step 2, one chip · BIL-02 · one exclusion row each`

“Fifteen accounts will not be billed because of a default I set on step 1. Show me every one,

tell me which default did it, and let me change my mind here."

REDESIGN drawn **2026-09-11**, still unbuilt at Version 35 (2026-09-13), drawn to the Product Owner's 2026-09-11 run-intent ruling, the 2026-09-11 *per connection* ruling and the 2026-09-12 on-screen-calculation ruling. **NEVER BUILT.** Today the product ships a **six**-step wizard, screens 04–09 in §17.



Why this chip exists at all	A default that holds an account is a decision **nobody made on purpose for that account**. Fifteen unbilled accounts is real money not invoiced, and today the only evidence is a count. This chip is the count made accountable.
The Held by column	Names **the default**, not the case — *your default: hold the account, set on step 1*. That is the sentence that lets a clerk realise their own default was too cautious and fix it once, on step 1, for all fifteen.
Would bill	What the account **would** be billed if released. *Not priced* where there is no reading to price. That column is what turns “15 held” into a decision.
Not the same as the queue	These are held **in this run**. Accounts held by **earlier** runs are on **Held accounts** (screen 20) and are counted on step 1 as *unresolved from the last run*.

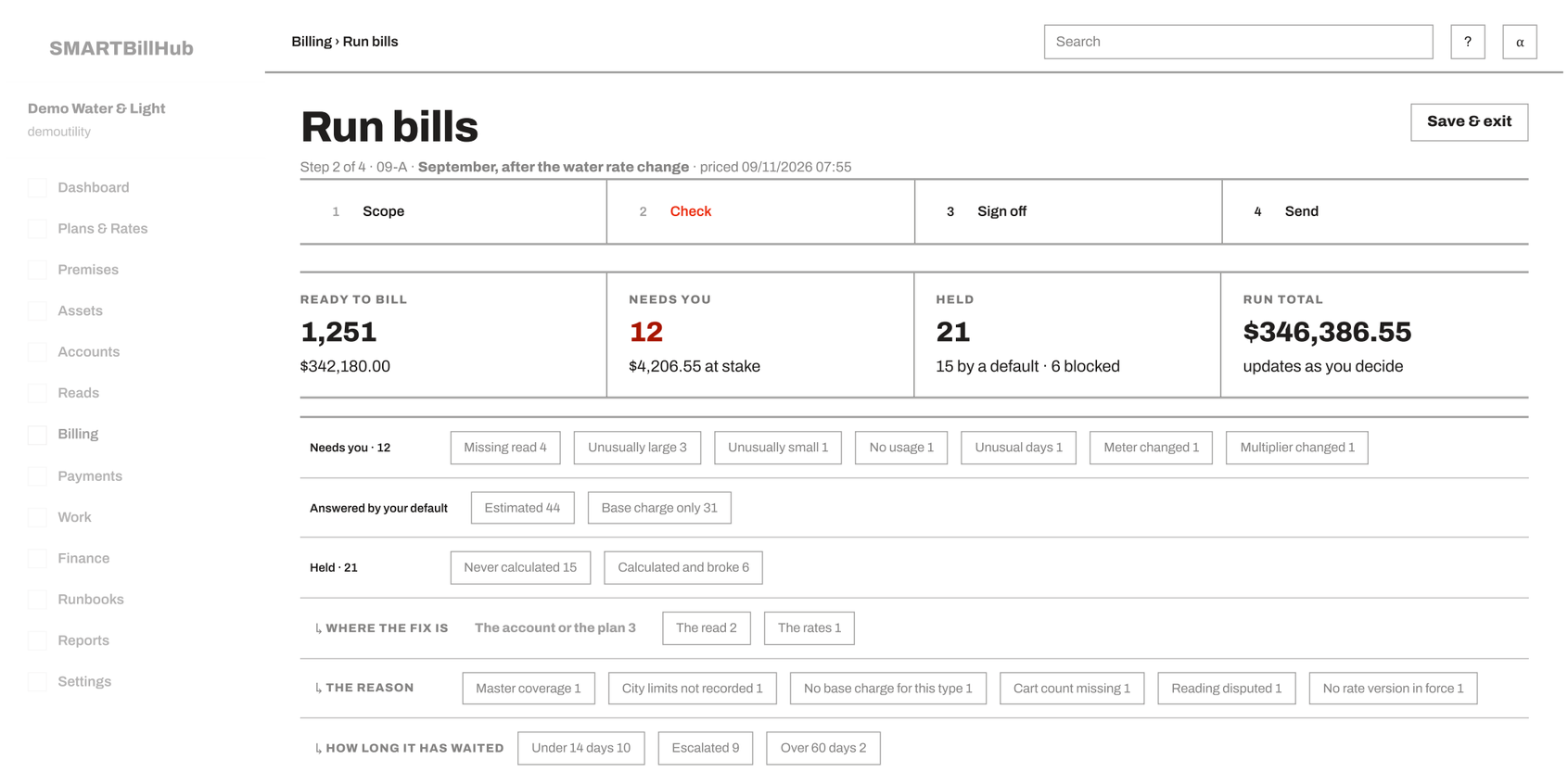
17.11 Account fix

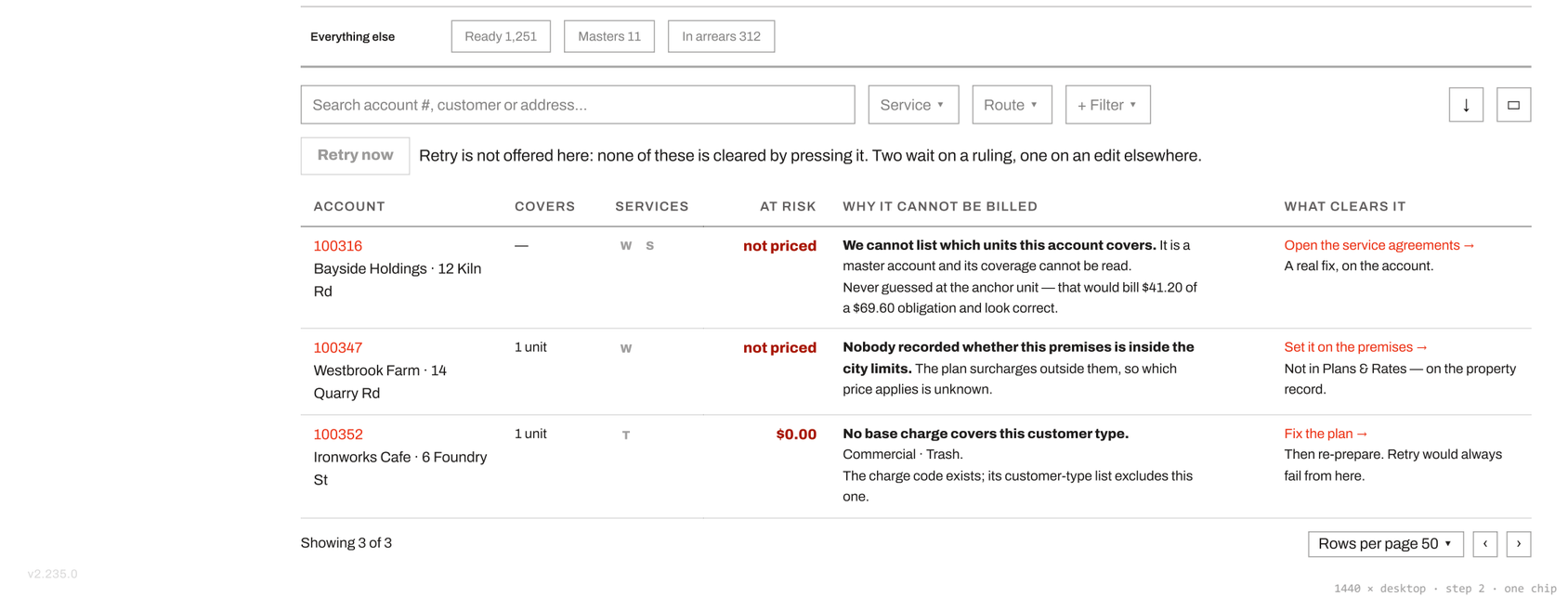
UPSTREAM · New build S39 · Exists behind S32 S34 S47

`step 2 chip · 3 rows · stage resolve-subscription · /:tenant/billing/run · step 2, one chip · BIL-02 · break stage resolve-subscription`

“Three accounts cannot be billed. Do not ask me to choose anything — tell me what is missing, where to go, and whose job it is.”

> REDESIGN drawn **2026-09-11**, still unbuilt at Version 35 (2026-09-13), drawn to the Product Owner’s 2026-09-11 run-intent ruling, the 2026-09-11 *per connection* ruling and the 2026-09-12 on-screen-calculation ruling. **NEVER BUILT.** Today the product ships a **six**-step wizard, screens 04–09 in §17.





No choice column at all	A blocked row offers **no decision**. Offering one would imply the clerk can fix it from here, and they cannot. The last column names **what clears it** instead — a link, or a ruling that is owed.
Retry is absent on purpose	The engine already knows these are **not retryable here**. A *Retry now* button on any of them is a button that always fails, and the screen draws it disabled with the reason rather than hiding the fact.
Two rows left this screen when the rulings landed	As first drawn this screen had five rows. Two were master refusals **waiting on a decision nobody had taken** — whether a repeated service bills its base once or per connection, and whose plan governs a bill spanning two. **Both were ruled on 2026-09-11: per connection.** They now price and bill, and they have left this screen for **Ready**. What remains is three rows with three real fixes, each in a different place. **A blocked row should always be a fix somebody can go and make** — if it is not, it is our decision to take, not a clerk's to stare at.
One master left, and it is a real fix	Row 1 is the only master here now, and unlike the two that left it **is** a data fix: the account's coverage cannot be read, so the bill cannot be assembled. The other two rows are an ordinary premises flag and an ordinary plan gap. **The structural gap is still the same one** — the cycle scopes units and the bill is per account — and a master is the only account where those differ, which is why it is still the path to test hardest.
At risk	A range where the ruling decides the figure ($52.00 – $64.00), a single figure where it does not, and *not priced* where nothing could be computed. Never $0.00 — a zero here would be a claim.

17.12 Read fix | PART RETRY

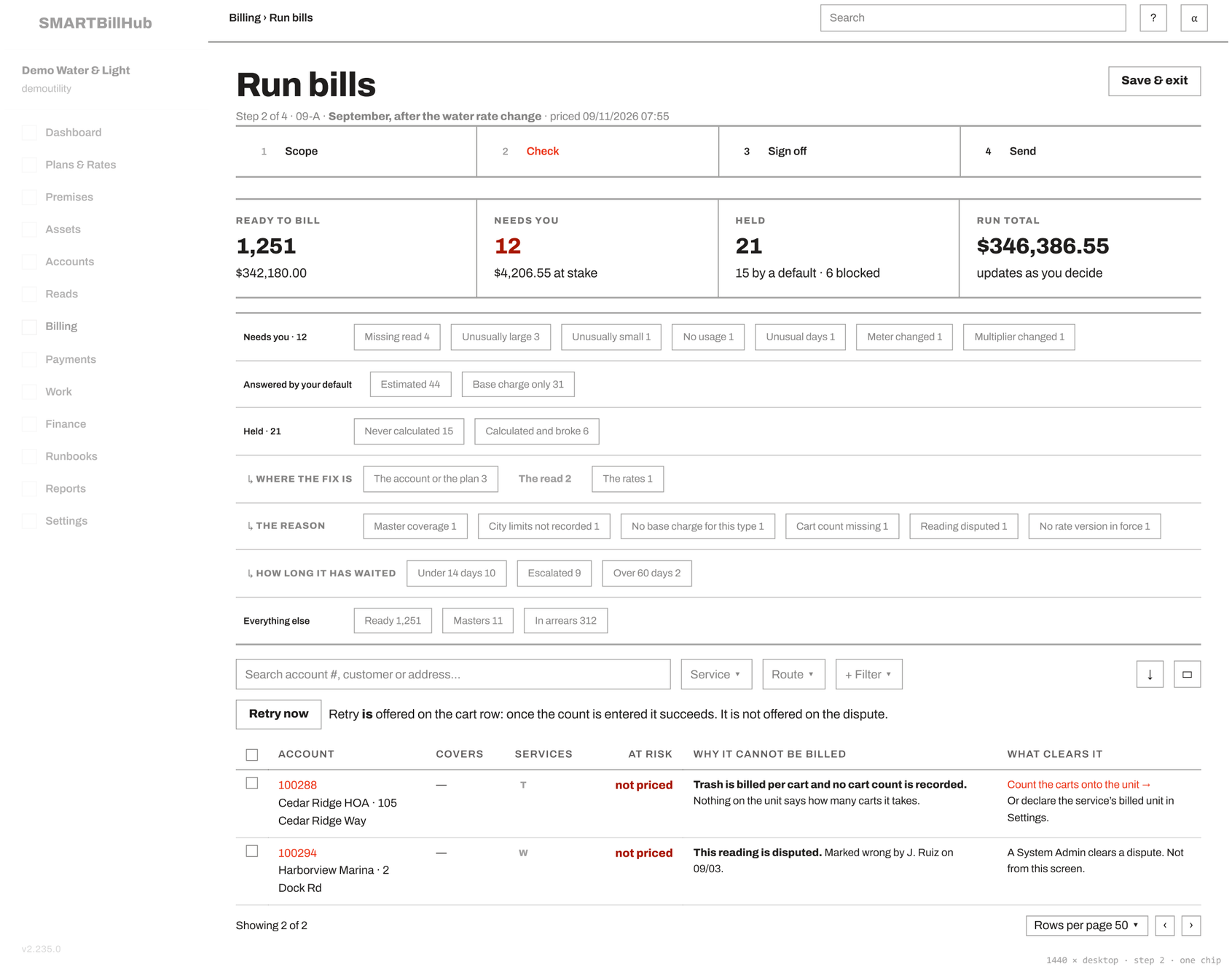
Exists behind S32 S47

`step 2 chip · 2 rows · stage measure-consumption · /:tenant/billing/run · step 2, one chip · BIL-02 · break stage measure-consumption`

"Two accounts have no usable quantity. Where do I go?"

> REDESIGN drawn **2026-09-11**, still unbuilt at Version 35 (2026-09-13), drawn to the Product Owner's 2026-09-11 run-intent ruling, the 2026-09-11 *per connection* ruling and the 2026-09-

12 on-screen-calculation ruling. **NEVER BUILT.** Today the product ships a **six**-step wizard, screens 04–09 in §17.



Retry, but only where it can work	The engine knows which breaks a retry can clear. Grouping blocked rows by **where the fix is** is what makes that legible: on this screen one row retries and one does not, and the toolbar says which.
Not a missing read	A **missing read** is a decision (estimate, hold, bill without) and is on **Needs you**. These two are different: one has no **unit of measure** at all, the other has a number a human has rejected. Neither is a choice a clerk can make here.
Permission is stated, not	The dispute row says **a System Admin clears a dispute**. A row that simply offers nothing reads as a bug.

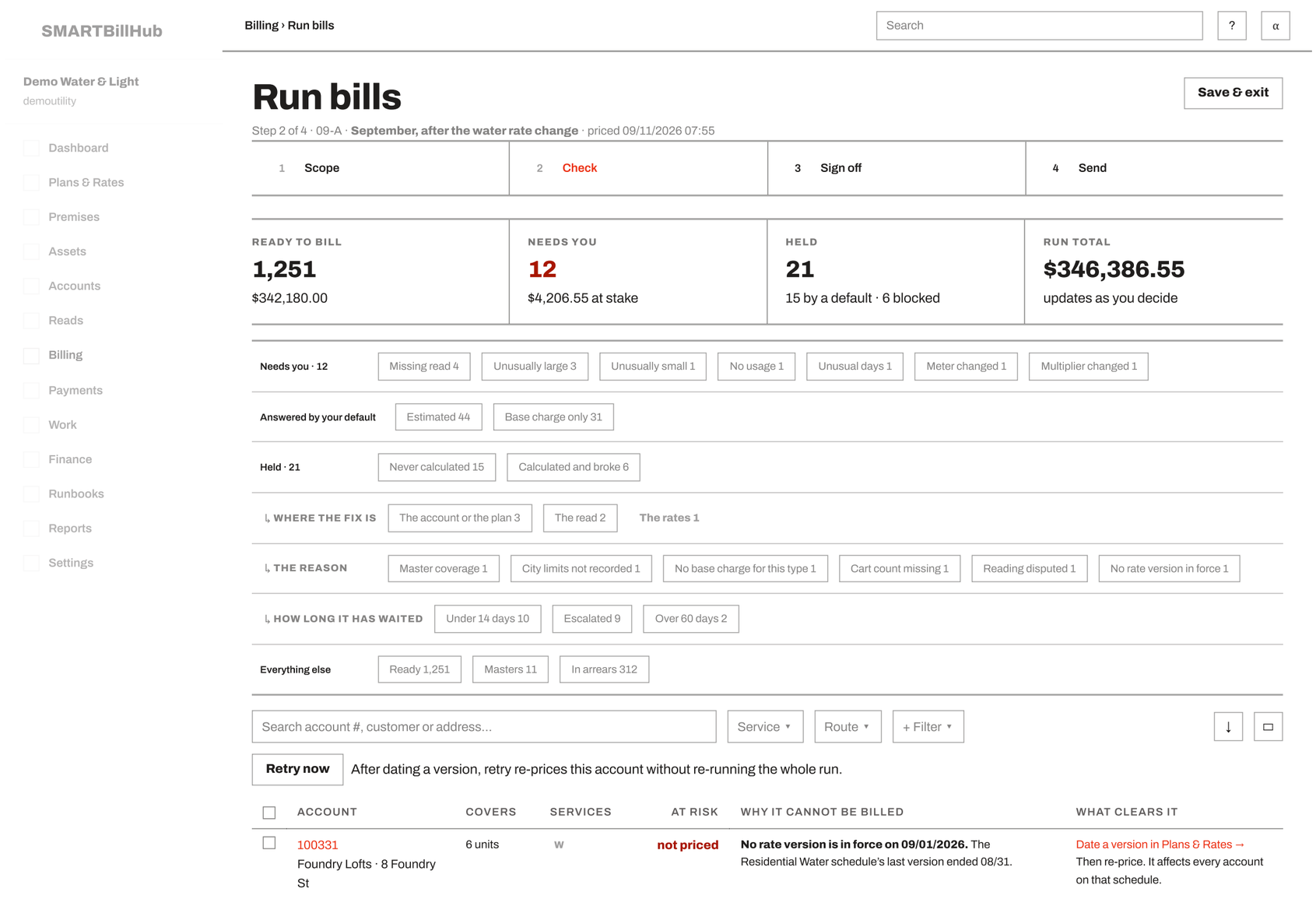
implied

17.13 Rates fix | MANY ACCOUNTS Exists behind S32 S47

`step 2 chip · 1 row · stage price-usage · /:tenant/billing/run · step 2, one chip · BIL-02 · break stage price-usage`

“One account is blocked on the rates. Is it going to be one, or is it about to be four hundred?”

> REDESIGN drawn **2026-09-11**, still unbuilt at Version 35 (2026-09-13), drawn to the Product Owner’s 2026-09-11 run-intent ruling, the 2026-09-11 *per connection* ruling and the 2026-09-12 on-screen-calculation ruling. **NEVER BUILT.** Today the product ships a **six**-step wizard, screens 04–09 in §17.



Showing 1 of 1 — Rows per page 50 — ‹ ›

1440 × desktop · step 2 · one chip

The sentence that matters	**“It affects every account on that schedule.”** A rates gap is rarely one account — it is one account *so far*. Saying so here is what stops a clerk fixing it one row at a time.
Why its own chip	The fix is in **Plans & Rates**, by a different person, with a different approval. Grouping it with the account fixes would send the clerk to the wrong screen — which is exactly what the engine’s own stage field exists to prevent.

17.14 Ready | SKIM ONLY

Exists behind S32 S34 S47

step 2 chip · 1,251 rows · nothing raised · /:tenant/billing/run · **step 2, one chip** · BIL-02 · nothing raised, nothing to decide

“Twelve hundred bills I am not going to read. Let me skim for anything that looks wrong and move on.”

> REDESIGN drawn **2026-09-11**, still unbuilt at Version 35 (2026-09-13), drawn to the Product Owner’s 2026-09-11 run-intent ruling, the 2026-09-11 *per connection* ruling and the 2026-09-12 on-screen-calculation ruling. **NEVER BUILT.** Today the product ships a **six**-step wizard, screens 04–09 in §17.

SMARTBillHub — Billing › Run bills — Search — ? — α

Demo Water & Light
demoutility

- Dashboard
- Plans & Rates
- Premises
- Assets
- Accounts
- Reads
- Billing
- Payments
- Work
- Finance
- Runbooks
- Reports
- Settings

Run bills

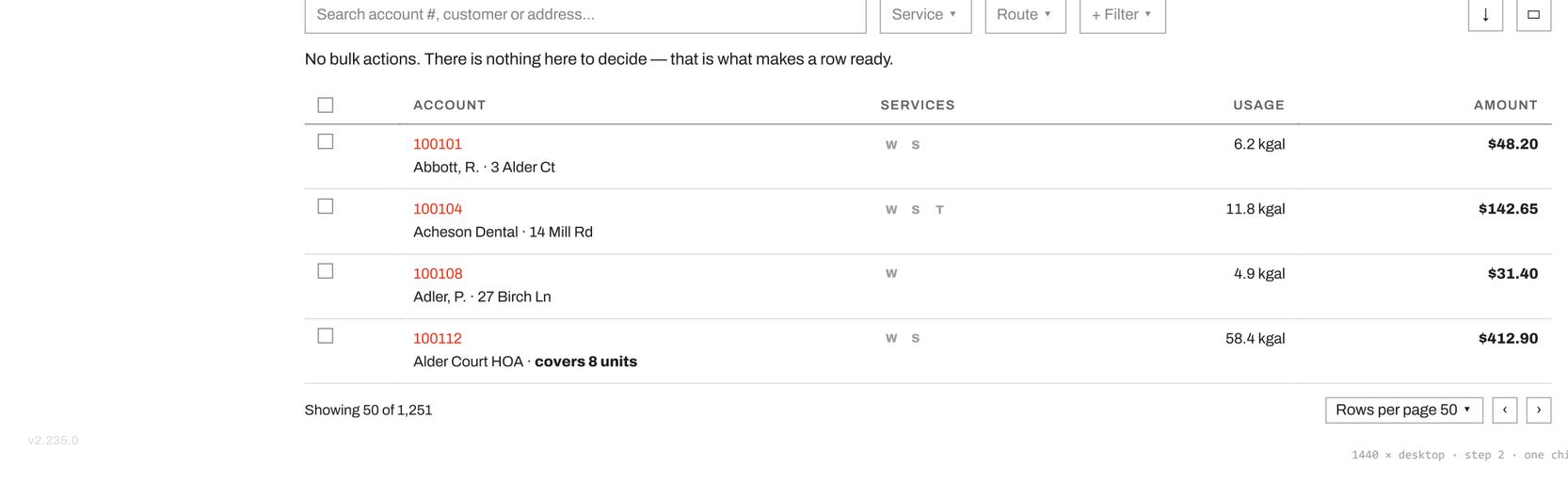
Save & exit

Step 2 of 4 · 09-A · **September, after the water rate change** · priced 09/11/2026 07:55

1 Scope | 2 Check | 3 Sign off | 4 Send

READY TO BILL	NEEDS YOU	HELD	RUN TOTAL
1,251	**12**	**21**	**$346,386.55**
$342,180.00	$4,206.55 at stake	15 by a default · 6 blocked	updates as you decide

Needs you · 12	Missing read 4	Unusually large 3	Unusually small 1	No usage 1	Unusual days 1	Meter changed 1	Multiplier changed 1
Answered by your default	Estimated 44	Base charge only 31					
Held · 21	Never calculated 15	Calculated and broke 6					
↳ WHERE THE FIX IS	The account or the plan 3	The read 2	The rates 1				
↳ THE REASON	Master coverage 1	City limits not recorded 1	No base charge for this type 1	Cart count missing 1	Reading disputed 1	No rate version in force 1	
↳ HOW LONG IT HAS WAITED	Under 14 days 10	Escalated 9	Over 60 days 2				
Everything else	**Ready 1,251**	Masters 11	In arrears 312				



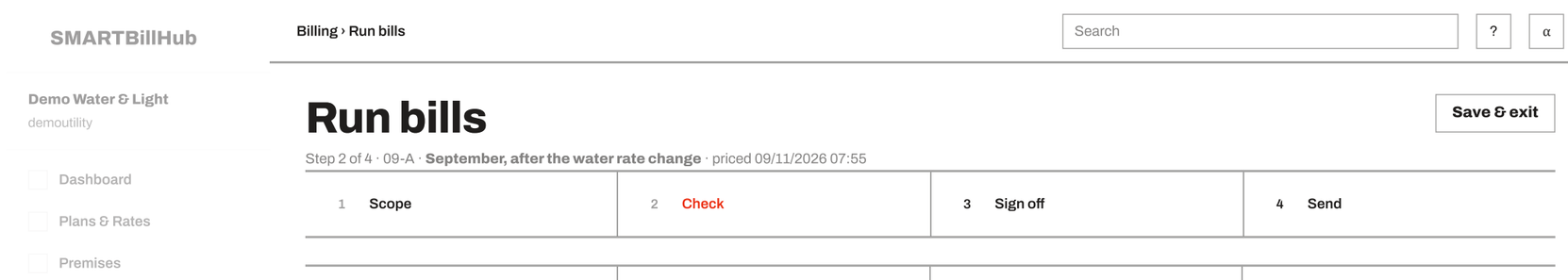
Four columns, and that is the design	Money and the quantity behind it. **No cause column, no choice column, no selection boxes** — a ready row has nothing wrong with it, and columns that are empty on every row of 1,251 are noise a clerk has to scan past.
What a clerk actually does here	Sorts by **Amount** and looks at the top and the bottom. So Amount sorts **on the server**, and sorting 1,251 rows never loads 1,251 rows.
A master still says so	Row 4 reads **covers 8 units** under the name. A $412.90 bill on a list of $30 bills is not an outlier if it covers eight homes, and a clerk skimming needs that without clicking.
Fifty a page	Paged on the server like every list in the product. No cap, no whole-run load.

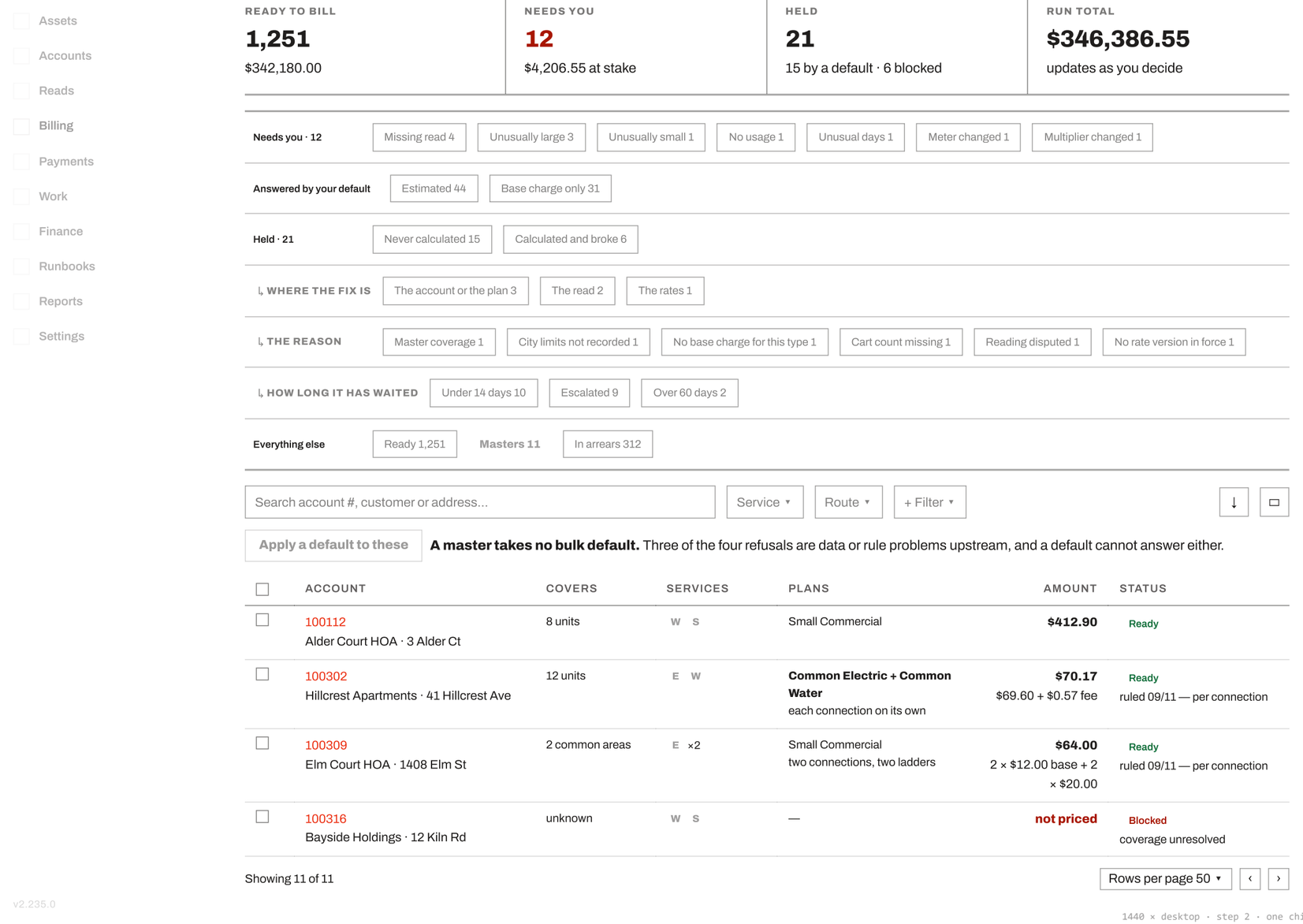
17.15 Masters | CROSS-CUTTING New build S39 Exists behind S32 S34 S47

`step 2 view · 11 rows · cuts across every band · /:tenant/billing/run · step 2, one chip · BIL-02 · cuts across every band`

“Eleven accounts bill more than one unit. Those are the ones that go wrong. Show me all eleven together, whatever band they are in — and show me the arithmetic, because two of them price two connections on one bill.”

> REDESIGN drawn **2026-09-11**, still unbuilt at Version 35 (2026-09-13), drawn to the Product Owner’s 2026-09-11 run-intent ruling, the 2026-09-11 *per connection* ruling and the 2026-09-12 on-screen-calculation ruling. **NEVER BUILT.** Today the product ships a **six**-step wizard, screens 04–09 in §17.





Why a view and not a band	A master can be ready, held or blocked — it is not a state, it is a **shape of account**. So it filters across all four bands rather than sitting in one. Eleven of 1,284 accounts, and **one of this run's six blocked rows — it was three until the 09/11 ruling**.
Covers is the first real column	Before services, before money. It is the fact that distinguishes these accounts from every other, and the fact whose absence caused the under-billing: a bill for **$41.20 of a $69.60 obligation**, correct-looking on the document and short by an entire common area.
Plans, in plain sight	Two plan names in one cell is the visible form of the refusal on the row below it. A clerk can see the problem before the engine names it.
Bulk is refused, with its reason	The control is drawn **disabled with the reason beside it**, not hidden. A missing button reads as an oversight; a refused one reads as a rule.
Test this hardest	Three refusal reasons, a fourth retired, and three open defects including one red on the branch. If one path in this run gets the most testing before ship, it is this one.

17.16 Estimate all | BEFORE AFTER New build S42 Exists behind S47 S48 S49

`step 2 bulk · 4 selected, 3 will change · /:tenant/billing/run · a bulk action, confirmed · BIL-02`

“Before I estimate four accounts at once, show me each estimate, where it came from, and what it does to the run total.”

> REDESIGN drawn **2026-09-11**, still unbuilt at Version 35 (2026-09-13), drawn to the Product Owner’s 2026-09-11 run-intent ruling, the 2026-09-11 *per connection* ruling and the 2026-09-12 on-screen-calculation ruling. **NEVER BUILT.** Today the product ships a **six**-step wizard, screens 04–09 in §17.

Estimate 4 accounts from history ×

Four accounts will be estimated and billed in this run. Each estimate is computed from that account’s own history, not from an average of the four.

ACCOUNT	ESTIMATE	BASIS	WOULD BILL
100263 M. Okonkwo · 14 Birch Ln	8.1 kgal	Average of the last 3 reads	**$51.30**
100341 Delgado, R. · 9 Quarry Rd	5.4 kgal	Average of the last 3 reads	**$34.80**
100398 Tran, K. · 5 Alder Ct	—	No history — meter fitted 08/22	**cannot estimate**
100362 Foster, A. · 18 Kiln Rd	6.0 kgal	Same month last year	**$38.60**

3 of 4 will change. The fourth has no history to estimate from, so it stays on *Needs you*.

RUN TOTAL NOW	THIS ACTION ADDS	AFTER THIS ACTION
$346,386.55	**+ $124.70**	**$346,511.25**

Cancel Estimate these 3

1440 × desktop · confirm before a bulk write

The rule this screen obeys	A bulk action shows **what it will do to the total before it commits**. Four rows in, $124.70 out, and the new run total named. A confirm that says only *“estimate 4 accounts?”* is asking a clerk to approve a number they have not been shown.
Per-account estimates, not one figure	Each row carries **its own** estimate and **its own basis**. An estimate a clerk cannot explain to the customer who rings about it is worse than no estimate.
Four selected, three will change	The row that **cannot** be estimated is named in the dialog and excluded, and the button counts **3**, not 4. A bulk action that silently skips part of its selection is how a clerk comes to believe an account was handled.
Footer order	Cancel on the left as a ghost, the commit on the right. The commit button names the act and the count — never *OK*.
While it runs	The commit shows **Working…** and cannot be pressed twice; Cancel is unavailable once the write is in flight, because it is already happening.

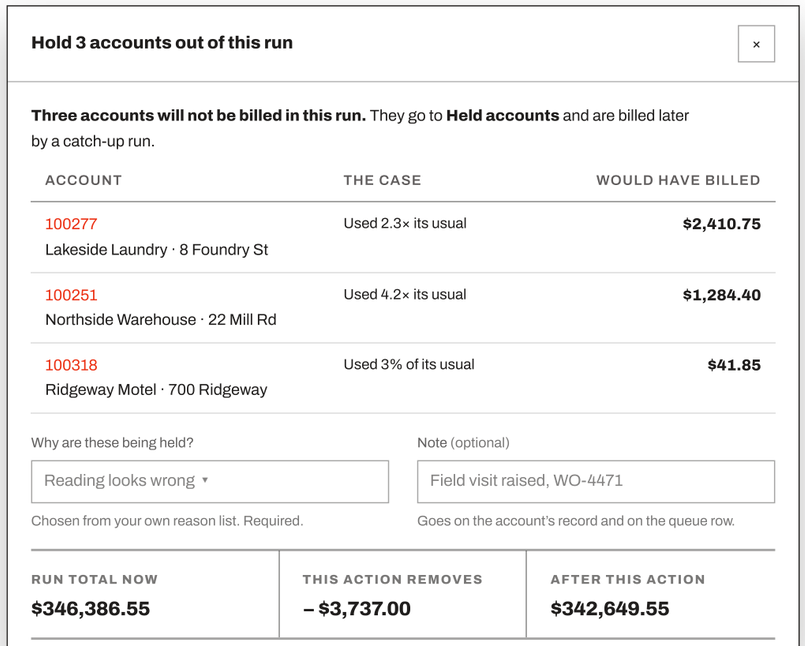
17.17 Hold these

TAKES REASON · New build S42 · Exists behind S47 S48

`step 2 bulk · 3 selected · danger tier · /:tenant/billing/run · a bulk action, confirmed · BIL-02`

“I am about to not bill $3,737. Make me say why, and make sure somebody can find these again.”

> **REDESIGN** drawn **2026-09-11**, still unbuilt at Version 35 (2026-09-13), drawn to the Product Owner’s 2026-09-11 run-intent ruling, the 2026-09-11 *per connection* ruling and the 2026-09-12 on-screen-calculation ruling. **NEVER BUILT.** Today the product ships a **six**-step wizard, screens 04–09 in §17.





1440 × desktop · a hold, with its reason

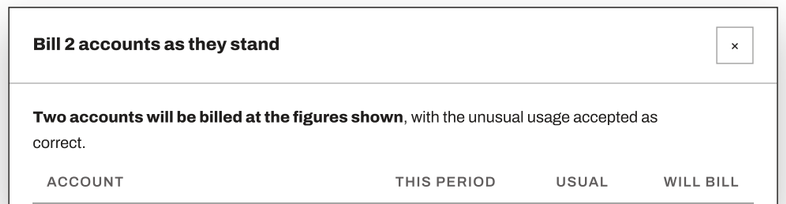
A reason is required, and it is a dropdown	From **your own reason list**, not free text — so the queue can be filtered by reason and a pattern becomes visible. The optional note is where the specifics go.
Why the danger tier	This is the only bulk action on step 2 that **removes revenue from the run**. $3,737.00 not invoiced, by one click, on three accounts. B14 puts the danger tier on a destructive commit and this is one, even though it is reversible.
The total goes DOWN, and says so	Every other confirm on this screen adds. This one subtracts, and the middle figure is signed so the direction cannot be misread.
Hold is not forget — and the dialog says where they go	The first sentence names **Held accounts** and the catch-up run. A hold with no stated destination is how an account goes unbilled for three months.
What it writes	One row per account: the account, the reason, the note, **who** held it and **when**. That row is the queue.

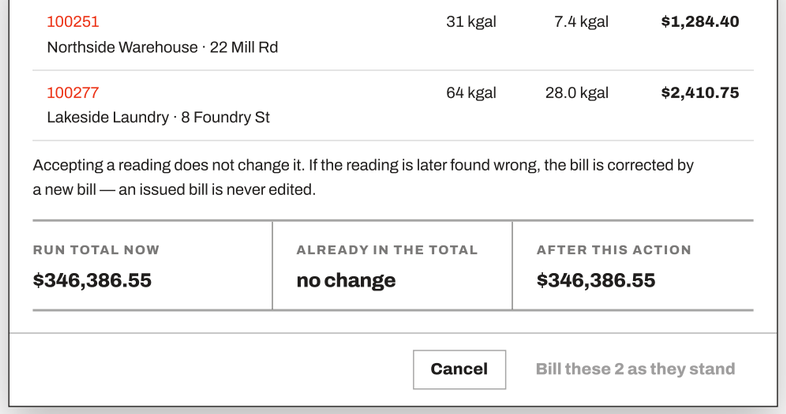
17.18 Accept | NO CHANGE New build S42 Exists behind S47 S48

`step 2 bulk · 2 selected · the total does not move · /:tenant/billing/run · a bulk action, confirmed · BIL-02`

“I am accepting two readings that look wrong. Tell me plainly that I own that.”

> REDESIGN drawn **2026-09-11**, still unbuilt at Version 35 (2026-09-13), drawn to the Product Owner’s 2026-09-11 run-intent ruling, the 2026-09-11 *per connection* ruling and the 2026-09-12 on-screen-calculation ruling. **NEVER BUILT.** Today the product ships a **six**-step wizard, screens 04–09 in §17.





1440 × desktop · accepting a figure

The honest total	**No change.** These amounts are **already** in the run total — the account was priced, the case is a flag on it. Showing a delta of zero is the point: a clerk should not be looking for money that was never going to move.
What the confirm is actually for	Not the arithmetic — the **responsibility**. This dialog exists so that accepting a 4× reading is a deliberate act with a name on it, not a default that happened.
The immutability sentence	Stated here because it is where a clerk hesitates: *if this is wrong, is it stuck?* No — a wrong bill is corrected by a new bill. Saying so is what lets them press the button.
Primary, not danger	Billing a real figure is the normal path. The danger tier is for **Hold these**, which un-bills.

17.19 Confirm factor

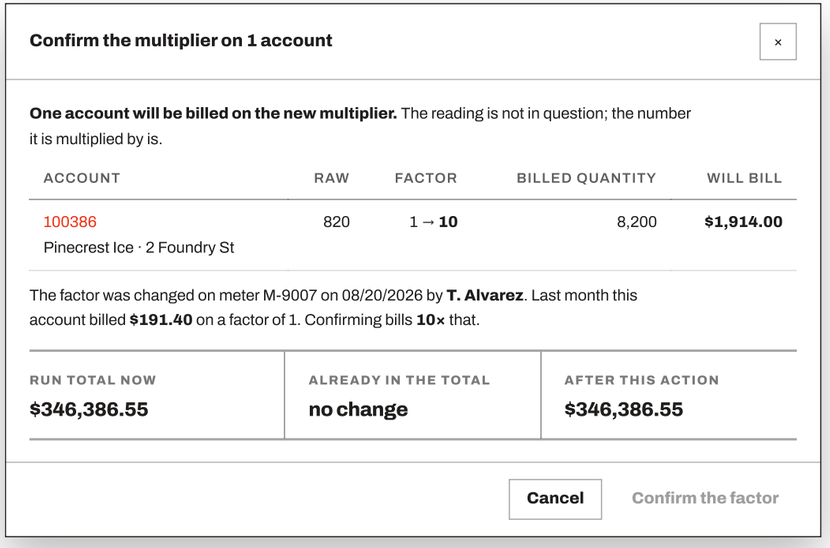
WHO WHEN New build S42 Exists behind S47 S48

`step 2 bulk · 1 selected · /:tenant/billing/run · a bulk action, confirmed · BIL-02`

“Someone changed a multiplier and the bill went up ten times. Who changed it, when, and what did this account pay last month?”

REDESIGN drawn **2026-09-11**, still unbuilt at Version 35 (2026-09-13), drawn to the Product

Owner's 2026-09-11 run-intent ruling, the 2026-09-11 *per connection* ruling and the 2026-09-12 on-screen-calculation ruling. **NEVER BUILT.** Today the product ships a **six**-step wizard, screens 04–09 in §17.



1440 × desktop · a configuration change, confirmed

Who and when, on the dialog	The factor changed **on 08/20 by T. Alvarez**. That is the fact that resolves the question — a factor corrected by the meter team is right, a factor typed by accident is not, and only the name and date distinguish them.
Last month's bill, in words	**$191.40 last month, $1,914.00 now.** The comparison a clerk would otherwise open two screens to make, stated in the sentence where the decision happens.
The arithmetic is shown, not implied	Raw 820, factor 10, billed 8,200. A clerk can check the multiplication. This is the one case where the sum **is** the finding.
Why it is still only primary	Confirming a factor the meter team set is routine. **Investigate** is the cautious option and it holds the account.

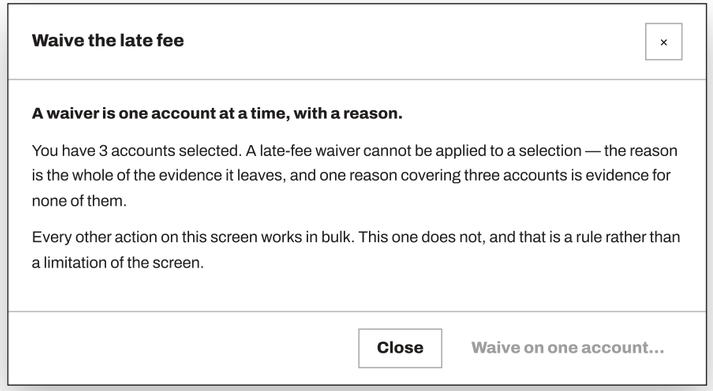
17.20 Waiver | REFUSED

New build S42 S77 S102 Exists behind S47 S48

`step 2 bulk · refused — one account at a time · /:tenant/billing/run · a bulk action, confirmed · BIL-02`

“Why is this one button greyed out when every other one works?”

> REDESIGN drawn **2026-09-11**, still unbuilt at Version 35 (2026-09-13), drawn to the Product Owner’s 2026-09-11 run-intent ruling, the 2026-09-11 *per connection* ruling and the 2026-09-12 on-screen-calculation ruling. **NEVER BUILT.** Today the product ships a **six**-step wizard, screens 04–09 in §17.



1440 × desktop · a refusal that explains itself

Why draw a refusal at all	So nobody “fixes” it later. A greyed button with no stated reason looks like unfinished work, and the next lane to pass through will wire it up in good faith and break a signed rule.
The reason is the rule’s own	A waiver is a per-case manual decision and **the reason is the whole of the evidence it leaves**. That is why bulk is refused here and nowhere else.
The refusal still has a door	The dialog offers **Waive on one account** — it does not dead-end. A refusal that leaves the clerk with nothing to do is how a rule gets worked around.
Five or six words on the control itself	The disabled button carries its reason beside it in the toolbar. Colour is never the only signal, and nobody has to hover to learn why.
⚠ The case may never appear	The engine **does not currently raise a late-fee case at all** — it is declared and never emitted. So this screen may be reachable only after that is fixed. It is drawn now because the refusal is law either way.

18 Sign off | STEP THREE New build S43 S50 Exists behind S29 S40

step 3 of 4 · /:tenant/billing/run · `GET /billing/runs/{id}/summary`

“Before I put my name on this: how many accounts, split how, how much water, how much money, how much did they already pay, and what am I NOT billing?”

REDESIGN drawn **2026-09-11**, still unbuilt at Version 35 (2026-09-13), drawn to the Product Owner’s 2026-09-11 run-intent ruling, the 2026-09-11 *per connection* ruling and the 2026-09-12 on-screen-calculation ruling. **NEVER BUILT.** Today the product ships a **six**-step wizard, screens 04–09 in §17.

SMARTBillHub

Billing › Run bills

Search ? α

Demo Water & Light
demoutility

- Dashboard
- Plans & Rates
- Premises
- Assets
- Accounts
- Reads
- Billing
- Payments
- Work
- Finance
- Runbooks
- Reports
- Settings

Run bills

Back to Check

Step 3 of 4 · 09-A · **September, after the water rate change** · nothing written yet

1 Scope	2 Check	3 Sign off	4 Send

1 WHAT IS BEING BILLED

POPULATION	ACCOUNTS	
In scope	**1,284**	1,502 units in the cycle
Regular	1,273	one unit each
Master	**11**	229 common areas · priced per connection
Bills to create	**1,260**	1,251 clean + 9 you released
Not billed	24	group E — 1,260 + 24 = 1,284

2 WHAT WAS USED

SERVICE	QUANTITY	UNIT
Water	18,402	kgal

W	Water	18,402	kgal
S	Sewer	17,118	kgal
E	Electric	122,840	kWh
T	Trash	1,204	carts
F	Fire	46	lines

3 WHAT IT IS WORTH

	AMOUNT
Charges	$345,672.15
Credit notes 23, named →	– $4,182.00
Net this run	**$341,490.15**

4 WHERE THE MONEY STANDS

	AMOUNT
Owed before this run	$128,402.55
Payments applied since the last run	– $96,318.20
Carried into this bill	**$32,084.35**
This run's charges	+ $341,490.15
Owed after this run	**$373,574.50**

5 WHAT IS NOT BEING BILLED

	ACCOUNTS	WHY
Held by a default	18	which default →
Blocked	**6**	3 the account or the plan · 2 the read · 1 the rates
Nothing to charge for	0	correct, and counted
Total unbilled	**24**	**~$6,240 not invoiced** — an estimate
In arrears	**312**	$84,204.60 owed before this bill
On the final rung	**1**	100418 · notice sent 09/09

6 SEE ONE BEFORE YOU SIGN new

Which account

100302 · Hillcrest Apartments — master, 2 plans ▾

See the specimen →

Any of the 1,260. This one shows the most.

Rendered from this run's own figures. **No bill number and no due date** — both are set when the bill is created.

☐ I have looked at the **24 not being billed** and the **1 on the final rung**.

Nothing has been written yet.

7 **Sign off and create 1,260 bills**

What a specimen is	**One real bill, rendered from this run's own priced figures, for an account you name — before a single bill exists.** Not a sample with invented data, and not an issued document, because at this moment there are none. The same renderer and the same numbers; nothing written.
Why a total cannot replace it	On 7 September the water rate changed. This run prices 1,284 accounts and this screen says **$341,490.15**. That figure is equally consistent whether water is **$4.10** or **$41.00** a kgal — it is just a bigger number, and every check on this page still foots. **A clerk cannot tell a right total from a wrong one by looking at a total.** On one bill they read *"Water 6.2 kgal at $4.10 = $25.42"* and know in a second.
Why here and not at step 4	Step 4 can already open the **issued** document — but only after 1,260 bills and 1,260 bill numbers exist. At that point a wrong rate costs 1,260 corrections. **Here the cost of being wrong is zero.** It is the same look, one step earlier, at the only moment nothing has happened yet.
What it costs to build, and it is less than it sounds	The bill's entire content is built by **one pure function** that takes stages 1–5's output and returns the payload. Stage 6 then does two separate things with it: it **creates the invoice row** and it **mints the bill number**. **A specimen is that pure function plus the renderer, and neither of stage 6's two writes.** No new arithmetic, no second implementation of anything — which also means the specimen cannot disagree with the bill, because it is the bill's own payload.
It must mint no number, and the code says what that is worth	A bill number is minted when the draft is created and **an unused one is lost** by ruling. Stage 6's own comment prices it: *"three re-runs of a 500-account run spend 1,500 numbers on 500 bills"*. A specimen that minted one would spend a number every time a clerk looked at a page.
A specimen is a bill minus exactly two facts	**Its number and its due date.** The due date is derived at issue from the bill date and the cycle's days to pay, and a draft has neither — inventing one would put a date on a document that may never exist. So the specimen **labels both absences** rather than printing a blank or a guess. Everything else on it is exactly what the customer will receive.
Which account it defaults to	The **master** — two connections, two plans, priced per connection. It is the account where the most can be wrong and the only one whose layout a total cannot describe. Any of the 1,260 can be chosen; step 4 then offers three more once they are real.

Five groups, and the order is the argument	What is billed → what was used → what it is worth → where the money stands → **what is NOT billed**. The last group is last because it is the one a clerk would otherwise never see, and the only one that costs money when it is wrong.
One title per group, and the table stops repeating it	Each group heading appears **once**, on the rule above its table. The first column header used to repeat it — *"What is being billed | Accounts | Detail"* under a heading that already said *What is being billed* — so the same words were read twice and the real column label was missing. It is **Population** now, or blank where the rows name themselves.
Alignment is a rule, not a preference	Every figure column is right-aligned on tabular figures, so the digits stack and a wrong order of magnitude is visible without reading. Labels left, figures right, the reason last. Identical in all five groups.
The explanations came off the screen	Three paragraphs explaining *why* consumption carries its unit, *why* the subtraction is shown and *why* the footing holds are gone. They were our reasoning, printed to a clerk on every run. They are in this table instead — which is where a developer looks for them.
Regular and master, split	**1,273 regular, 11 master covering 229 areas, priced per connection.** Not an axis on the summary today — but the account's kind is on the row the run already reads, so unlike the three axes that genuinely cannot be derived, this one is a small addition.
Group 4 foots by construction	**$128,402.55 – $96,318.20 = $32,084.35** carried in, plus $341,490.15 = **$373,574.50**. The seam that computes the first three guarantees the subtraction, and the same three lines print on every bill — so the screen and the paper cannot disagree.
Group 5 is the one that changes behaviour	**24 accounts, about $6,240 not invoiced**, each with why. And the two arrears rows are **inside the same table** now rather than a separate KPI strip underneath — they are the same kind of fact, and two layouts for one idea is what made this group read as an afterthought.
The tick names the unbilled half	Not a general *are you sure*. It names **the 24** and **the 1**, because those are the two things a clerk can still do something about.
18.1 is gone, and this is why	This screen and `18.1 The numbers` were the same screen twice — one summary and one granular version of it. **Two drawings of one**

screen is the confusion this document was restructured to remove, so the granular one won and the old summary is not kept alongside it. There is one screen 18.

19 Send | STEP FOUR New build S50 Exists behind S30

step 4 of 4 · /:tenant/billing/run · the bills exist; nothing has been dispatched

“You are asking me to send 1,260 bills and I have not looked at a single one. Show me a real bill first — then tell me exactly what leaves, by what route, and what happens to the people on AutoPay.”

> REDESIGN drawn **2026-09-11**, still unbuilt at Version 35 (2026-09-13), drawn to the Product Owner’s 2026-09-11 run-intent ruling, the 2026-09-11 *per connection* ruling and the 2026-09-12 on-screen-calculation ruling. **NEVER BUILT.** Today the product ships a **six**-step wizard, screens 04–09 in §17.

SMARTBillHub

Billing › Run bills

Search ? α

Demo Water & Light
demoutility

- Dashboard
- Plans & Rates
- Premises
- Assets
- Accounts
- Reads
- Billing
- Payments
- Work
- Finance
- Runbooks
- Reports
- Settings

Run bills

Print set

Step 4 of 4 · 09-A · **1,260 bills created** · nothing has left the building yet

1 Scope | 2 Check | 3 Sign off | 4 Send

1 LOOK AT ONE BEFORE IT GOES

OPEN	ACCOUNT	AMOUNT	WHY THIS ONE
Open the bill →	100302 Hillcrest Apartments	$70.17	**The biggest surprise.** A master, two connections, two plans, priced per connection
Open the bill →	100101 Abbott, R.	$48.20	**An ordinary one.** One unit, water and sewer, nothing raised
Open the bill →	100263 M. Okonkwo	$51.30	**An estimate.** The reading was missing and you estimated it

These are the real issued documents, not a sample. Open any of the 1,260 →

2 HOW THESE REACH PEOPLE

CHANNEL	BILLS	STATE
Print	**1,260**	Goes out the floor — every bill, always
Email	0	Nobody elected the read that says who chose email is not published, so no account can elect it
Text message	0	Not offered here the channel exists in notifications; billing’s own seam does not name it
In-app	0	Not offered here same — and it needs **no vendor**, so it is the cheapest of the four to turn on
Customer portal	0	Not offered here same, and also needs no vendor

3 AUTOPAY

	ACCOUNTS	VALUE	
On AutoPay	**412**	**$112,840.55**	Handed to Payments billing issues; it does not take money

Cash only — no AutoPay	31	$8,204.10	a standing privilege on the account; never charged
Not on AutoPay	817	$220,445.50	they pay as they choose

412 + 31 + 817 = 1,260.

☑ I have opened at least one bill from this run.

1,260 to print · 0 by any other channel · 412 handed to Payments for collection.

4 **Send 1,260 bills**

v2.235.0

1440 × desktop · send

⚠ You were right: a clerk should not send what they have not seen	The old step 4 asked for a dispatch decision on a **summary**. A clerk who has approved counts and totals has still never seen a page a customer will hold — and a rate set wrongly, a layout broken, a missing address all look perfectly fine in a total. **So group 1 is three real bills, and the tick above the button is *I have opened at least one*.**
Why the proof belongs HERE and not at sign-off	It is the first moment a real bill exists. Sign-off is what runs stage 6; before it there is no document, no number and nothing to open — and `GET /invoices/{id}/document.pdf` is served, so from this step on it is a link rather than new work. **A specimen at sign-off, rendered from the priced data before anything is created, would be better still** and it is genuinely new work: the renderer exists but only from a *sample* invoice, not from this run. Worth it — it would catch a wrong rate before 1,260 bills are made, not after. It is `BIL-S50`, and it is the one thing on this screen I would build next.
Three bills, chosen rather than the first three	**The biggest surprise · an ordinary one · one you estimated.** If those three look right, the run is almost certainly right; the first three alphabetically tell a clerk nothing. Any of the 1,260 can be opened.
What actually leaves the building today, and it is one channel	Notifications carries **five** channels — print, email, text, in-app, portal. Billing **queues print and email** and names no others. And the read that would say which extra channels an account elected **is not published**, so nothing elects email either. **Every bill goes out on paper, whatever the customer asked for.** The table says so per channel rather than leaving a clerk to assume email went.
The cheapest two to turn on need no vendor	**In-app and portal** wait on nobody — no gateway, no contract, no deliverability. Email and text need a vendor. So the order of work is: publish the election read, name the other three channels in billing's seam, then in-app and portal, then the two that need procurement.
AutoPay is reported here and collected somewhere else	**412 accounts, $112,840.55.** Billing issues bills; it does not take money, and it may not write payments' tables. So this group **hands off and says so** — the alternative is two modules both believing they collect. The **31 cash-only accounts are named** because a standing no-AutoPay privilege is exactly the thing a bulk collection would trample. 412 + 31 + 817 = 1,260.
Grouping, and what moved	**Look at one → how it travels → who pays automatically → send.** The old screen led with the dispatch control and buried the channel facts under it, so the only genuinely alarming number on the screen — *0 by email* — was below the button that made it permanent. The button is last now, and it carries the three counts in words.
Nothing has left the building yet	Stated in the header, because *1,260 bills created* and *1,260 bills sent* are different facts and step 3 already did the first one.

03 · carry-over

Nothing the six steps did is lost in the four

No commitment | EVIDENCE, NOT WORK | EVIDENCE, NOT WORK | EVIDENCE, NOT WORK

Merging three steps into one is the change most likely to drop something quietly, so this table is one row per element across all six original steps.

ELEMENT ON THE SIX STEPS	DISPOSITION	WHERE IT LANDS

Billing cycle select · Bill date	UNCHANGED ×2	Step 1, group **The cycle**.
Step 1's *What this step no longer asks you* panel	REMOVED	**Ruled.** It explains why a template picker and a typed due date are absent — a note about a design decision taken long ago, printed to a clerk on every run. Its content belongs in this document, and it is here. ⚠ *If you want it kept on screen, say so and it returns as a closed disclosure.*
Past runs table	MOVED	Step 1, group **Already run for this cycle** — now scoped to the cycle being run, with a label, a total, who started it and a way to take over an open one.
The *no actor column, money not served* panel under it	REMOVED	**Ruled by fact:** its claim is false — `bill_runs.run_by` exists, is populated and is served as `ran_by`. The money half is true and survives as the honest caveat at step 1's ⑤. §05 R1 records the whole finding.
Step 2's six sub-tabs	MOVED ×6	Step 2's filter chips, **each with a count**. Three of the six that exist only to explain an unbuilt feature therefore never appear — nothing is hidden, there is simply no chip for a dimension with no rows.
Step 2's KPI band — ready · blocked · decisions · silent	RELABELLED	Step 2's band, in a clerk's words — *Ready to bill · Needs you · Held · Broke · Run total* — and each figure now carries money.
Step 2's per-row facts (meter, this period, average, ratio, threshold)	UNCHANGED	Step 2's *What is wrong* cell, written as a sentence with the same figures in it.
Step 2's *base charges are not gathered* cell	REMOVED	**Ruled by fact:** false — `gather.py:699-705` gathers base charges through a three-way ladder. §05 R3 records it.
Step 3's 16 case kinds	MOVED	Step 2's chips and rows — **the nine the engine actually emits**. The seven that cannot occur are not drawn, and that is the point of the merge.
Step 3's per-row decision select	UNCHANGED	Step 2's *What to do* column, with the amount beside it.
Step 3's bulk actions, and the refused waive	UNCHANGED ×2	Step 2's selection bar. The waiver stays refused with its reason — BR-BIL-060, a rule not a gap.
Step 3's *a choice here has nowhere to be recorded* panel	MOVED	Becomes a per-row state: a row the server cannot record a choice for says so on the row, where a clerk is looking.
Step 3's *no charges are gathered yet* panel	UNCHANGED	Still true — the run produces no late fees or extra charges (`DEF-2596`). It becomes the absence of those two chips rather than a panel: no chip, no count, nothing to decide.
Step 4's KPI band — bills · total · left out · broke	MOVED	Step 2's band, live from the first paint instead of one step later.
Step 4's three overcharge action words	RELABELLED	They become a real filter chip (*Unusually large 3*) and a real decision control. They were inert `<span>`s.
Step 4's *flagged bills cannot be listed* panel	REMOVED	**Only once the list exists** — it is honest today (`DEF-2855`). If the list is still unbuilt when this ships, **the panel must stay.** Flagged so nobody drops it by following the drawing.
Step 4's three draft powers panel	RELABELLED	Two of the three become working row actions (the services are complete and routed); the third keeps its refusal.
Step 4's per-account retry and hold	UNCHANGED ×2	Step 2's row actions.
Step 5's five gate facts	UNCHANGED ×5	Step 3. BR-BIL-058 untouched.
Step 5's recomputed totals + the as-at-	UNCHANGED ×2	Step 3, in one sentence instead of nine paragraphs.

signing caveat		
Step 6's *Nothing leaves the building*	UNCHANGED	Step 4, word for word.
Step 6's Issue action	UNCHANGED	Step 4.
Save & exit	UNCHANGED	Steps 1–3.
Park, disabled on all six steps	REMOVED	**Ruled by fact:** its reason — *"nothing would list it"* — is false; the list is on step 1. Parking is genuinely unbuilt (`PARKED_HOME_PATHS` has no `bill_run` member), so **a control that cannot work and gives a false reason is worse than no control.** When parking is built it returns. §05 R5.
The six-tab Billing strip in the wizard header	UNCHANGED	Kept — it is how a clerk leaves sideways.

```
Elements across the six steps   38
  unchanged                     19
  moved                          9   -> 6 sub-tabs become chips · decisions · bands · retry/hold
  relabelled                     4   -> the KPI words · overcharge actions · draft powers
  removed                        5   <- and EVERY ONE has a reason, not a preference:
                                        · 2 panels whose claims are FALSE (actor/money · base charges)
                                        · 1 control whose REASON is false and which cannot work (Park)
                                        · 1 panel explaining a long-past design decision (step 1's note)
                                        · 1 conditional: the flagged-bills panel, ONLY once the list exists
Elements in the four steps      47   = 38 - 5 removed + 14 new
                                       (scope · sweep-up · 3 defaults · label · open-run takeover
                                       search · chips-with-counts · Amount column · plain-words cause
                                       link-out to fix · re-price · pagination · reconciling count
                                       print door)                          RECONCILES

** FOUR OF THE FIVE REMOVALS ARE THINGS THIS DOCUMENT PROVED FALSE, not things
   somebody decided to drop. The fifth is conditional and says so. **
```

17.21 In arrears | NEW CHIP New build S44 Exists behind S47

`/:tenant/billing/run` · **a new chip in Everything else** · 312 rows · m
easured against **09/11/2026**

"Three hundred accounts already owe us money. I am about to send them another bill. Who are they, how deep are they in, and is anyone about to be cut off?"

> REDESIGN drawn **2026-09-11**, still unbuilt at Version 35 (2026-09-13), drawn to the Product Owner's 2026-09-11 run-intent ruling, the 2026-09-11 *per connection* ruling and the 2026-09-12 on-screen-calculation ruling. **NEVER BUILT.** Today the product ships a **six**-step wizard, screens 04–09 in §17.

Run bills

Step 2 of 4 · 09-A · **September, after the water rate change** · arrears as at 09/11/2026

1 Scope | 2 Check | 3 Sign off | 4 Send

READY TO BILL	NEEDS YOU	HELD	IN ARREARS	RUN TOTAL
1,251	**12**	**21**	**312**	**$346,386.55**
$342,180.00	$4,206.55 at stake	15 by a default · 6 blocked	$84,204.60 owed	updates as you decide

Needs you · 12 — Missing read 4 | Unusually large 3 | Unusually small 1 | No usage 1 | Unusual days 1 | Meter changed 1 | Multiplier changed 1

Answered by your default — Estimated 44 | Base charge only 31

Held · 21 — Never calculated 15 | Calculated and broke 6

↳ **WHERE THE FIX IS** — The account or the plan 3 | The read 2 | The rates 1

↳ **THE REASON** — Master coverage 1 | City limits not recorded 1 | No base charge for this type 1 | Cart count missing 1 | Reading disputed 1 | No rate version in force 1

↳ **HOW LONG IT HAS WAITED** — Under 14 days 10 | Escalated 9 | Over 60 days 2

Everything else — Ready 1,251 | Masters 11 | **In arrears 312**

Search account #, customer or address... | Service ▾ | Route ▾ | As at **09/11/2026** | Ladder rung ▾ | + Filter ▾

1 selected | **Hold these** | Send a notice | **Billing does not send notices.** The collection ladder does, and it has four rungs of its own.

ACCOUNT	SERVICES	OWED NOW	OLDEST UNPAID	WHERE THEY ARE	THIS BILL	WHAT TO DO
100418 Harbourgate Motors · 3 Dock Rd	W S	**$4,182.60**	04/15/2026 149 days	Final 48 hours disconnect notice sent 09/09	**$284.10**	Open the ladder → Billing sends no notice.
100429 Pelham, T. · 22 Anvil Way	W	**$612.75**	06/20/2026 83 days	RUNG NOT MAPPED second 5-day — no tier is recorded	**$48.20**	Open the ladder →
100433 Sunnyside Diner · 9 Mill Rd	W S T	**$1,044.00**	07/05/2026 68 days	First 10-day	**$392.55**	Open the ladder →
100441 Okafor, N. · 41 Birch Ln	W S	**$286.40**	07/28/2026 45 days	Friendly reminder	**$62.80**	Open the ladder →

Showing 1–4 of 312 — Rows per page 50 ▾ ‹ ›

v2.235.0

1440 × desktop · arrears, at check time

Why this belongs on the check and not somewhere else

The clerk is about to add a bill to a balance that is already unpaid. **312 accounts owe $84,204.60** and today nothing on this screen says so. It is the one fact that can change a billing decision and is not currently in front of the person taking it.

The figure already exists

Billing has its own open-balance module and the position is resolved per account at issue for the bill's own arrears block. Nothing new is computed — it is aggregated and shown one screen earlier.

Past due needs a stated day, and that is why it refuses today

The *past due* filter refuses across this product for **one** reason: the tenant's clock does not reach it, and a wrong past-due *"is a claim about a customer's money"*. **Passing the business day makes it computable** — the same date picker the held queue needs. One fix, three screens.

screens.

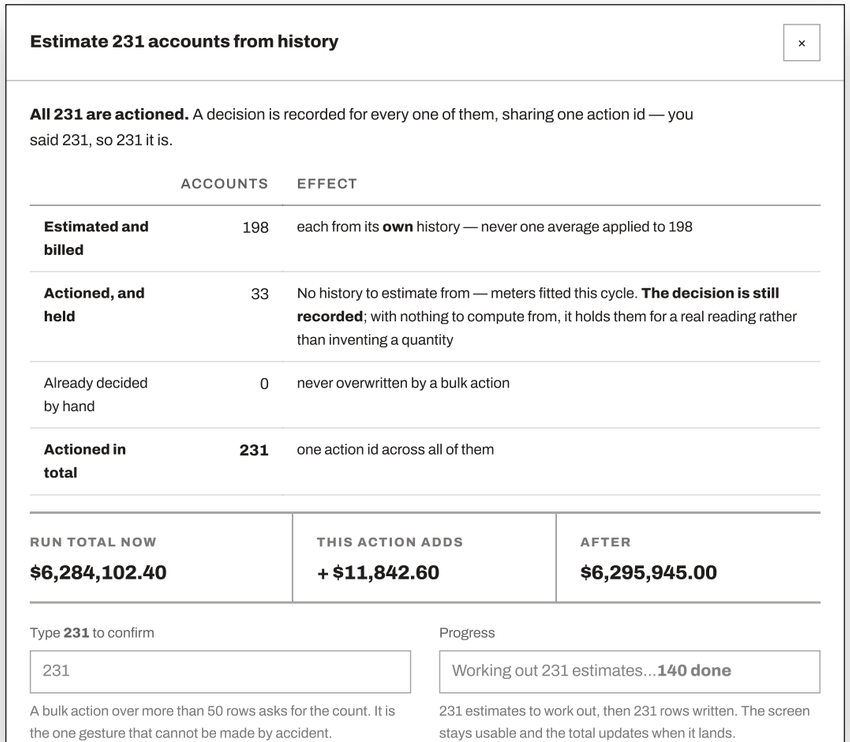
Billing shows it and does not act on it	The notice control is drawn **disabled with its reason**. Payments owns a collection ladder with four rungs and real notice machinery; a second dunning path out of billing means two systems telling one customer different things.
The row that matters is row 1	**Final 48 hours, disconnect notice sent two days ago**, and the run is about to bill them $284.10. *Marked disconnect* is a **billable** status — consuming until the cut — so this is correct behaviour that a clerk should nevertheless see before it happens.
One rung cannot be named, and the screen says so	The ladder's **second 5-day rung has no tier mapped**, deliberately: choosing one would be a statement to a regulator about what a customer was told. So that row reads **rung not mapped** rather than being rounded to the nearest rung.
What counts as arrears is a tenant setting	Seeded at **60 days past due, or twice the account's average bill** — a proposal, not a ruling. Every other mechanic in this product is tenant-configured with a seeded default and this is no different.

17.22 Apply all | AT SCALE New build S42 Exists behind S47 S48 S49

`/:tenant/billing/run · a chip-wide bulk action · the 25,000-account case`

"Two hundred and thirty-one accounts have no reading. I am not ticking them one at a time, and I am not estimating them blind either."

> REDESIGN drawn **2026-09-11**, still unbuilt at Version 35 (2026-09-13), drawn to the Product Owner's 2026-09-11 run-intent ruling, the 2026-09-11 *per connection* ruling and the 2026-09-12 on-screen-calculation ruling. **NEVER BUILT.** Today the product ships a **six**-step wizard, screens 04–09 in §17.





1440 × desktop · a chip-wide action

⚠ Why this is mandatory rather than convenient	This run has 12 accounts needing a decision. A 25,000-account cycle has **231 in one chip alone**. Ticking rows fifty to a page is not a workflow, it is a reason the cycle is late.
Selecting a chip selects the CHIP, not the page	The toolbar says so in as many words. A select-all that quietly meant *these fifty* is how a clerk comes to believe 231 accounts were handled.
It still shows what it will do first	Everything the small confirms show, at scale: **198 will change, 33 cannot, +$11,842.60**, and the new run total. A bulk action over 231 rows is not exempt from the rule — it is the reason for it.
The 33 are named, not silently skipped	Meters fitted this cycle have no history to estimate from. They stay on *Needs you* and the button counts **198**. A bulk action that reports success over rows it skipped is the most expensive kind of lie this screen can tell.
Over fifty rows it asks for the count	Typing **198** is the one gesture that cannot be made by accident, and the kit already has the control for it. Under fifty rows it is a plain confirm — the friction is proportional.
And it says how long	198 × 137 ms, so about thirty seconds, in the background. The figure is measured, not guessed.
Never overwrites a decision already taken	A row the clerk has answered is excluded and counted separately. Bulk is for the untouched.

Units or accounts — and what screens 1 and 2 must say

The cycle scopes units. The bill is per account. Those are two different numbers, the gap between them is where every master-account failure lives, and **no screen in the product shows both**.

	A REGULAR ACCOUNT	A MASTER ACCOUNT
What the run iterates	Metering publishes the **units** in the cycle; the run resolves them to accounts.	The same. A master is not a separate population.
Units one bill prices	**One** — the account's own unit.	The anchor unit **plus every common area it covers** through an open service agreement.
If coverage cannot be read	Falls back to the one unit, which is correct for it.	**Refuses, by name.**

Why the asymmetry is deliberate	—	Because the fallback *is* the defect: a bill for **$41.20 of a $69.60 obligation**, correct-looking on the document and short by an entire common area. A refusal is one exception row; a plausible short bill is a wrong bill nobody catches.
Unoccupied units on the premises	n/a	**Now priced onto the owner's bill** (changed 27 Aug — it used to refuse, footing at $69.60 while $81.60 was owed). Open the run still reports them as unbilled anyway.
Named refusal reasons	None	**Three, of which one survives.** Coverage unresolved is a data fix; *spans two plans* and *repeats a service* were **ruled on 2026-09-11 — per connection** and now price rather than refuse.
Open defects	—	**Three, one red on the branch** — a master naming a premises cannot be billed single-account at all; switching two resolution steps on by default refuses the master; and the unbilled-units report above.

What screen 1 does about it

ON THE SCREEN	WHAT IT SAYS	WHY
The scope line	`Monthly · Sep 2026 · 1,502 units → 1,284 accounts`	The two numbers side by side. A clerk who only ever sees one of them cannot reason about the difference, and the difference is where the money goes missing.
The master line	*11 master accounts cover 229 common areas. 3 of the 11 cannot be billed today.*	A master pulls in units the clerk did not pick, and three of them are already known to be stuck. Better to know at scoping than at review.
Services	**Read-only chips with unit counts** — *Water 1,502 · Sewer 1,480 · Fire 46*	**Not a chooser.** This is the one place we disagree with the obvious design, and the reason is below.
Regular vs master	A read-only split, not a filter	Same reason as services.
Route or area	**Not a filter**	Same reason as services.
Defects	*Masters take no bulk default*	Three of the four master refusals are data or rule problems upstream, and a default cannot answer either.

You said services are not shown on screen 1 at all, and they should not become a scope chooser. You are right, and there is a hard reason nobody has written down. The run is keyed on **tenant + cycle + period** and **nothing else**. A service scope is accepted, stored and honoured — it just is not in that key. So a clerk who runs *Water only* for September **consumes the key for September**, and the Sewer run is then refused as a duplicate. Those accounts are not excluded from anything, so no exclusion row exists, so they are not in the held queue, so **no catch-up will ever bill them**. That is precisely the silent lost revenue the queue was built to prevent, arriving through a different door. **There is no defect row and no open question for it — zero rows in the register mention the scope field.** Until the key carries the scope, as the catch-up key already carries its own, **screen 1 shows scope and does not choose it**. That is `BIL-S38`.

What screen 2 does about it

ON THE SCREEN	WHAT IT DOES	WHY
The row	A master row reads **covers 12 units** under the account name	So `$69.60` is never read as one household's water, and a $412.90 bill in a list of $30 bills is not mistaken for an outlier.
The band	Master refusals sit in **Held → blocked**, never in *Needs you*	One is a judgement a clerk can make; the other is missing data or an undecided rule. Putting them together is what makes a clerk stare at a row with no answer.
The choice column	**Absent** on a blocked row	Offering a choice implies the clerk can fix it from here. They cannot.
Retry	Drawn **disabled, with the reason**	The engine already knows these are not retryable. A retry button on any of them is a button that always fails.
A row with no fix	Says so, with the money that turns on it — and is then **ours to clear, not the clerk's**	No amount of data entry clears them. Saying *fix the account* would send a clerk to look for something that is not there.
Bulk	**Refused** on masters, with the reason beside the control	A hidden control reads as an oversight; a refused one reads as a rule.
The view	**Masters — 11**, cutting across all four bands	A master is a shape of account, not a state. Eleven of 1,284 accounts and **one of this run's six blocked rows — it was three until the 09/11 ruling**: the most fragile path in the run, and the one to test hardest.

Both were put to the Product Owner and both were ruled on 2026-09-11: *per connection*. Verbatim, to the first question — *"per conenction"*; to the second — *"as per connection"*. Two master refusals therefore stop being refusals, two accounts leave the blocked band, and **the connection becomes the unit of pricing inside a master bill**.

THE QUESTION	RULED	WORKED
Two covered areas take the same service. Base charge once per bill or once per connection — and do the tier thresholds pool or reset?	**Per connection.** Each connection bills its own base charge and walks its own tier ladder from the bottom. Two electric connections on one bill produce **two base lines and two usage ladders**, not one of each.	Base $12.00, $0.20/kWh, 100 kWh at each of two connections → **$64.00**. The pooled reading would have been $52.00.
Covered areas sit on different plans. Whose taxes, fees, discounts and plan snapshot govern the one bill?	**Per connection.** Each connection is governed by **its own** plan — its taxes, its fees, its discounts — and the bill carries **one plan snapshot per connection** rather than one for the bill.	Hallway on *Common Electric* (no tax) and irrigation on *Common Water* (2.00% city

fee) over $69.60 → fee **$0.57**, bill **$70.17**. Levied on the whole bill it would have been $1.39.

⚠ **The second ruling reverses a recorded ruling, and the reversal has to be written down rather than assumed.** On 2026-08-11 the same question at bill level was answered *"Tax rates are for bill not for service"*, and the pipeline was built on it: **one plan id resolves the bill's taxes, its charge codes, its discount rules and its locked-at-issue plan snapshot** — four places, one id. "Per connection" means each of those four becomes **one per connection**, and the locked snapshot — which today holds a single `plan {id, name, version}` — has to hold a set. **My reading, and what §03 is drawn to:** the reversal is **narrow — master accounts covering units on different plans, and nothing else**. An ordinary single-connection bill still resolves one plan and still takes bill-level tax, exactly as signed, because on such a bill the two readings give the same answer and there is nothing to reverse. **If you meant it generally — every bill's tax levied per service — say so, because that is a different and much larger change**, and it moves a signed figure rather than extending one.

What the rulings cost to build

Not a screen change — an engine change, and the screens above are drawn for it. The bill must carry **a line set, a tariff resolution and a plan snapshot per connection** instead of per bill, and the tier ladder must reset per connection. Both refusals can then be deleted rather than handled. It is `BIL-S39`, and the **third** master refusal — coverage unresolved — is untouched by either ruling: it is a data fix and stays exactly as drawn.

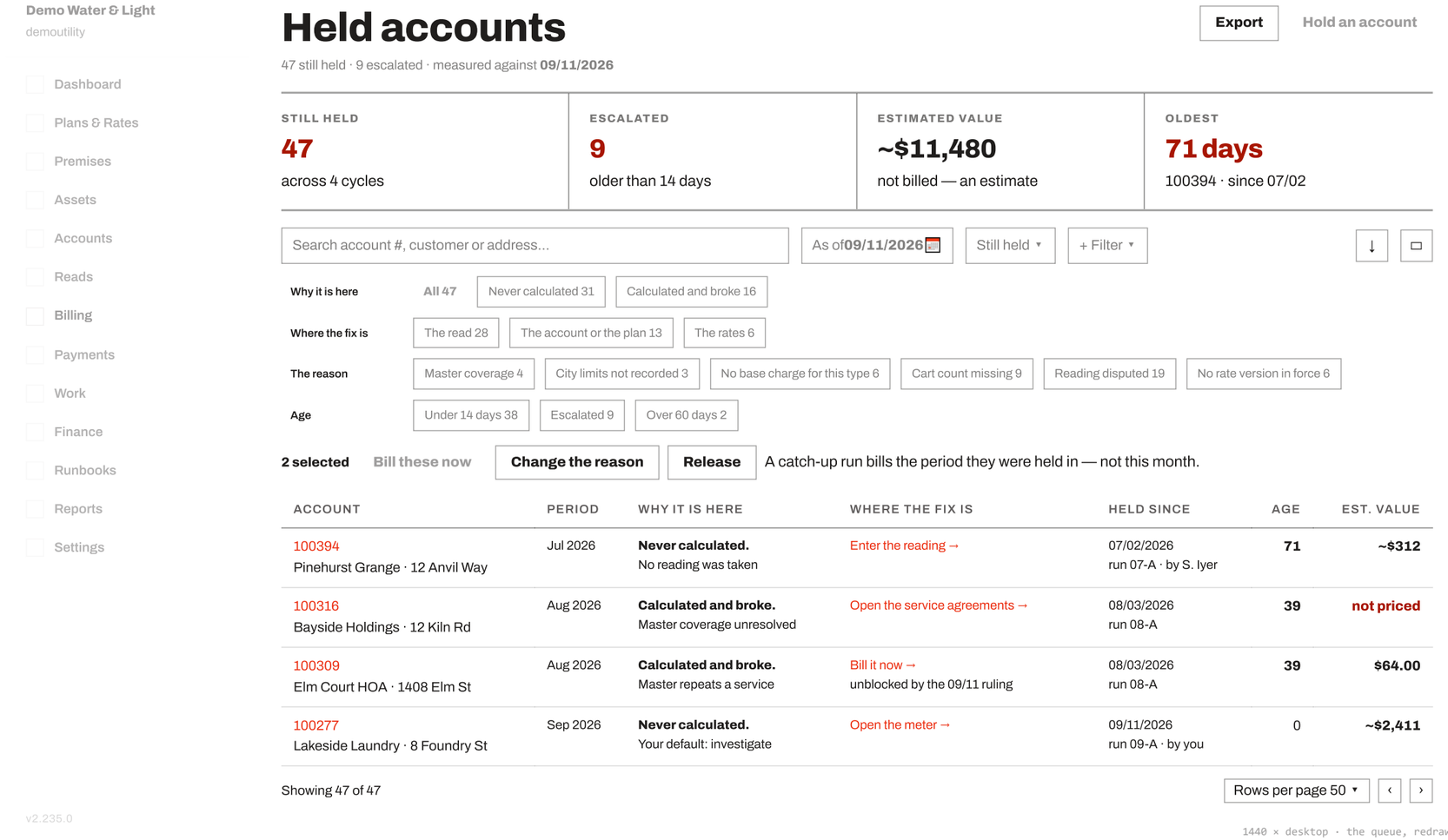
20 Held queue | NO EXIT New build S39 S44 Exists behind S35 S36 S100

47 rows · /:tenant/billing/pending · **exists, and cannot be acted on** · BIL-08

"Forty-seven accounts have not been billed. Which ones, for how long, what is it worth, and let me bill the ones I have fixed."

> REDESIGN drawn **2026-09-11**, still unbuilt at Version 35 (2026-09-13), drawn to the Product Owner's 2026-09-11 run-intent ruling, the 2026-09-11 *per connection* ruling and the 2026-09-12 on-screen-calculation ruling. **NEVER BUILT.** Today the product ships a **six**-step wizard, screens 04–09 in §17.





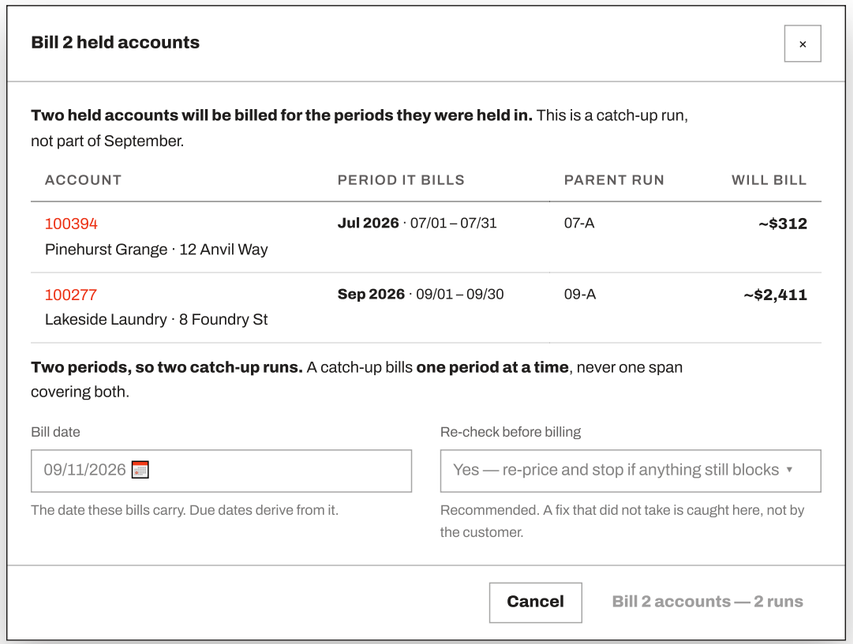
⚠ What is wrong today, in one sentence	The screen exists and **there is no way out of it.** The endpoint that bills a held account is built and served; the screen has no button for it, because the list does not choose which run to attach the catch-up to. **A clerk can hold an account and cannot release it.**
Bill these now	The missing control. Select rows, press it, and a catch-up run bills them **for the period they were held in** — keyed so it cannot collide with the run that held them. It picks the parent run from the rows; the clerk never chooses one.
As of, as a date picker	Every age on this screen is measured against a stated day, and today that day is silently “now” with no control. It becomes a **date picker** — which is also what lets a clerk ask *what was still held at month end?*
The two columns the screen exists for	**Why it is here** and **where the fix is**. *Never calculated* is fixed by entering a reading; *calculated and broke* is usually fixed by fixing configuration. A queue that said only “held” would send every one of them to the same wrong place.
Estimated value, labelled as an estimate	A held account has no bill, so there is no amount — and a zero would be a claim. But “47 held” with no money attached cannot be prioritised. So: **~$11,480**, marked as an estimate, and *not priced* where even an estimate is impossible.
Filters, and multi-select	Why it is here · where the fix is · age band · cycle · reason, each multi-select with **All** as the absence of a filter. Today the screen has **no filters at all**, is not paged, and sorts in memory — fine at 21 rows, unusable at 400 and a breach of our own pagination rule.
Escalation	Nine rows are older than the escalation threshold. That threshold is **14 days hard-coded as a default** and is owed as a tenant setting — every utility currently escalates on our number.

20.1 Catch up | OLD PERIOD Exists behind S35

the catch-up · /:tenant/billing/pending · endpoint served, control absent

“I have fixed these two. Bill them — for the month they were meant to be billed in, not this one.”

> REDESIGN drawn **2026-09-11**, still unbuilt at Version 35 (2026-09-13), drawn to the Product Owner’s 2026-09-11 run-intent ruling, the 2026-09-11 *per connection* ruling and the 2026-09-12 on-screen-calculation ruling. **NEVER BUILT.** Today the product ships a **six**-step wizard, screens 04–09 in §17.



1440 × desktop · the catch-up, confirmed

It bills the OLD period, and says so twice	A July hold produces a **July** bill, not a September one. That is the single most misunderstandable thing about a catch-up, so the period appears in the column heading and in bold on the row.
Two periods means two runs	Stated before committing, with the count on the button. A clerk who is told *“bill 2 accounts”* and gets two separate runs in their history will think something went wrong.
Why the original run cannot just be re-run	A run is keyed on its cycle and period. Re-running July would be refused as a duplicate of itself — which is exactly why held work needs a catch-up whose key carries **which accounts**. Without this screen those accounts are unbillable for that period, forever.

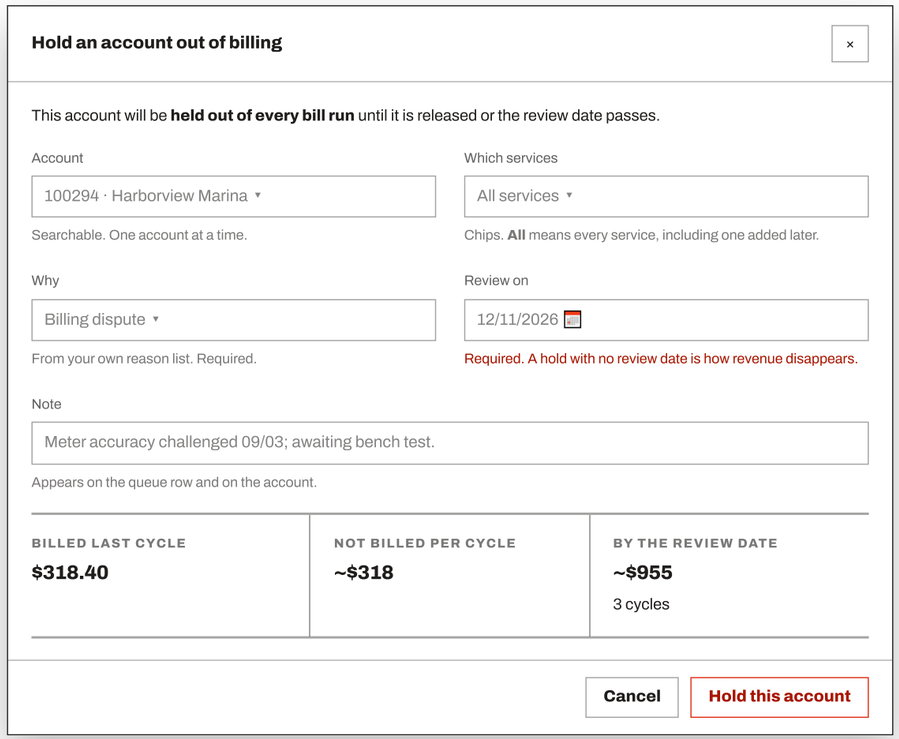
Re-check before billing	Defaults to yes. The gap between fixing the data and pressing this button can be days, and a fix that did not take should be found here.
Nothing is chosen for the clerk	The parent run comes from the rows. Today's absence of this control is precisely because the list *would* have had to choose one — so the screen shows it instead of picking silently.

20.2 Hold one | RULING OWED Blocked S37

a standing hold · /:tenant/billing/pending/hold · does not exist in any form

"This account is in dispute. Stop billing it until I say otherwise — and do not make me remember to do this every month."

> REDESIGN drawn **2026-09-11**, still unbuilt at Version 35 (2026-09-13), drawn to the Product Owner's 2026-09-11 run-intent ruling, the 2026-09-11 *per connection* ruling and the 2026-09-12 on-screen-calculation ruling. **NEVER BUILT.** Today the product ships a **six**-step wizard, screens 04–09 in §17.



1440 × desktop · a standing hold

⚠ There is no such thing today, at all	Six account statuses exist and **not one means "billable, but hold the bill"**. *Vacation hold* still bills the standing charges. *Disconnected* and *pending activation* do not bill at all — and *disconnected* is seeded and nothing assigns it yet. So a clerk with a disputed account holds it **by hand, in every run, forever**.
The review date is required, and that is the whole safety	A standing hold means an account can go unbilled for months on a decision taken once. That is exactly the silent lost revenue the queue exists to prevent. So a review date is **mandatory**, the hold appears in **every run's blocked band** so it can never go quiet, and it escalates

and that is the whole safety design	exists to prevent. So: a review date is **mandatory**, the hold appears in **every run's blocked band** so it can never go quiet, and it escalates with age like any other queue row.
The money is on the dialog before it is committed	**$318.40 last cycle · ~$955 by the review date.** A hold is a revenue decision and should be taken against a figure.
Services as chips, with All	A hold may cover every service or only the disputed one. **All** is the absence of a restriction, not eleven ticks — so a service added later is covered rather than silently left out.
What we need from you	Whether a standing hold may exist at all, and whether a review date is **required** as drawn or merely offered. Everything else on this screen follows from those two answers.

20.3 Release | AUDIT TRAIL Blocked S37

`the release · /:tenant/billing/pending/release · does not exist in any form`

"The dispute is settled. Put this account back — and show me what we missed while it was held."

> REDESIGN drawn **2026-09-11**, still unbuilt at Version 35 (2026-09-13), drawn to the Product Owner's 2026-09-11 run-intent ruling, the 2026-09-11 *per connection* ruling and the 2026-09-12 on-screen-calculation ruling. **NEVER BUILT.** Today the product ships a **six**-step wizard, screens 04–09 in §17.

Release 100294 back into billing ×

This account returns to normal billing from the **next** run. Periods it was held for are billed by a catch-up — releasing does not bill them.

Why is it being released?

Dispute resolved ▾

Required. A hold ending is as recorded as a hold starting.

Bill the held periods now?

No — leave them in the queue ▾

Choosing yes opens the catch-up confirm.

Note

Bench test 09/28 found the meter within tolerance.

WHAT HAPPENED	WHEN	WHO	REASON
Held	09/03/2026	S. Iyer	Billing dispute
Review date extended	10/12/2026	S. Iyer	Bench test queued
Released	09/11/2026	you	Dispute resolved

3 periods were not billed while this hold was in force — Jul, Aug and Sep 2026, about **$955**. They stay in the queue.

Cancel Release this account

1440 × desktop · releasing, with its history

Releasing is not billing, and the first sentence says so	The account rejoins the **next** run. The months it missed are still owed and are still in the queue. A clerk who assumes release means catch-up leaves three periods unbilled.
The history is shown, not stored silently	Held, extended, released — each with who and when. That trail is what a hold is worth: without it, an account unbilled for three months has no explanation anyone can find.
What was missed, in money	**3 periods, about $955.** Stated at the moment of release, which is the one moment somebody is looking.
A reason on the way out too	Required. A hold that can be released without a reason turns the reason on the way in into paperwork.
⚠ The audit trail has a limit worth stating	Where a correction replaces a value, the old value lives only in the audit log — which is pruned on a schedule. Whether these rows are exempt from pruning has never been decided, so **this screen must not be described as a permanent record** until it is.

One removal needs your ruling and I have not taken it. Step 1's *"What this step no longer asks you"* panel explains why there is no template picker and no typed due date. Both absences are correct and well-reasoned — but the explanation is printed to a clerk on **every single run**, and it is about a decision taken months ago. I have moved it into this document and taken it off the screen. **If you want it kept, it comes back as a closed disclosure rather than a panel** — say which.

Single bills

One account, one bill, outside the cycle. **Seven bill types exist in the schema and five are a clerk's to choose**; a sixth — the fee — earns a screen here too, and three more belong to other journeys. Every screen in this section is **new**: the tab exists and can draft, but no kind has ever had a screen of its own. **All six are raised through one three-step wizard** — the same rail a clerk already learned on the bulk run, drawn once for the two shared steps and once per kind for the middle one.

Which of these belong here, and which belong to another journey

You asked whether the other four paths belong on a single-bill tab or in move-in, move-out, new connection and disconnect. **One of the four does belong here. Three do not**, and saying so is worth more than a screen, because it stops the same bill being built twice.

PATH	BELONGS	WHY
Interim · **First** · **Final** · **Back-bill** · **Penalty**	**Here — five of the six step-2 kinds**	A clerk starts them, for one named account, and each needs a window no cycle states.
Fee	**Here — step 2 as screen 22.6**	The reconnection path is **one caller** of it. This is the only door to charging **any** live code to **one** account — a returned cheque, a tamper charge, a meter test. Without it every new fee needs a new journey.

Cut bill	**Disconnect**	Raised automatically **at the cut**, from the disconnect screen, with no clerk input: *"raise the final metered bill for the status, and leave any remainder outstanding"*. Nothing to put on a form.
Rebill	**Fix a bill** — §06	A correction to a bill that already exists. It starts from the bill, never from a blank form, and it is already drawn.
Mid-period split	**The bulk run**	A plan or rate change mid-cycle splits the period into segments, priced separately, one tax pass, one draft. **Derived, with no clerk input at all.** ⚠ And nothing calls it — built, tested, no caller anywhere.
Deposit invoice	**Nowhere — not built**	The prototype offers it; no story owns it. The code refuses by name and the tab **names it as unbuilt**, so a clerk looking for it learns why rather than concluding the product forgot. Not drawn, at your direction.

First and Final also exist inside the move-in and move-out journeys, and that is not a duplication. A cycle run marks a **first** bill automatically for accounts whose move-in falls in its period, and a move-out raises a **final** bill at step 3 of its own flow. **The one-time tab is the recovery door** — for the account the run missed, or the move-out that was recorded without billing. Same engine, same refusals; a different reason for being there.

The one rule every screen in this section obeys

There is no amount field, no description field and no line editor — and the enforcement is that there is nowhere to put one. The request object carries **no** `amount`, `description` or `lines` member: *"a caller cannot hand-type money onto a bill through this module because there is nowhere to put it"*. The prototype drew step 2 with an editable description, an editable amount and a **+ Add line** button; that is superseded in writing — *lines come only from rules and charge codes, free-typed amounts never exist on any bill*, and *computed amounts are never hand-editable*. **A rejection can be deleted by a later refactor; a missing parameter cannot be passed by accident.**

One rail, six kinds — how the three steps fall

A clerk who learned **Scope → Check → Sign off → Send** on the bulk run should not meet a different pattern for one bill. **The split is not cosmetic — it matches the engine:** step 1 is the request, step 2 is what the gather pass resolves, step 3 is what pricing produced. Steps 1 and 2 write nothing at all.

STEP	SCREEN	WHAT IT IS FOR
1	**21 · Who and when** drawn once	Account, kind, window, bill date, days to pay — and a table naming what the kind will ask for next.
2	**22.1 – 22.6** one per kind	Premises → units → assets, the readings, and the plan, rate version, taxes and discounts that will price it. **This is the step the wizard exists for.**
3	**23 · Confirm and send** drawn once	The priced lines, the impact on your books, the specimen, the send options, and the one button that writes.

Step 2 keeps **the same lettered spine on all six kinds** — **A** where, **B** what, **C** readings, **D** how it prices, then the kind's own group, and last, always, the ⚠ block of what could still refuse. A kind that does not need a group **keeps it and greys it with the reason on its face**; a step that changes shape between kinds teaches nothing, and a group that vanishes looks like a bug.

Two things every kind shares

Days to pay is required. A bill with no cycle has no days to pay of its own — your ruling was that they are captured at billing time — so an empty field means the bill **drafts and cannot be issued**. That is the rule refusing, not a guessed due date starting a late-fee clock. **And the specimen is on step 3, for every kind.** The preview writes nothing and is a POST precisely so a clerk who fixes a plan and previews again sees the fix — the same control and the same guarantee as sign-off in §03.

Step 2 may collect a reading; billing may never store one. The rule binds the **billing engine**, not the browser — *"Billing may not write meter reads and does not"*, and all nineteen of its metering calls are reads. So a reading typed on step 2 **posts to Meter Reads' own endpoint**, and the bill prices only once the reading exists there. **One fact, one owner** — a second home for a reading is the defect class this document keeps finding.

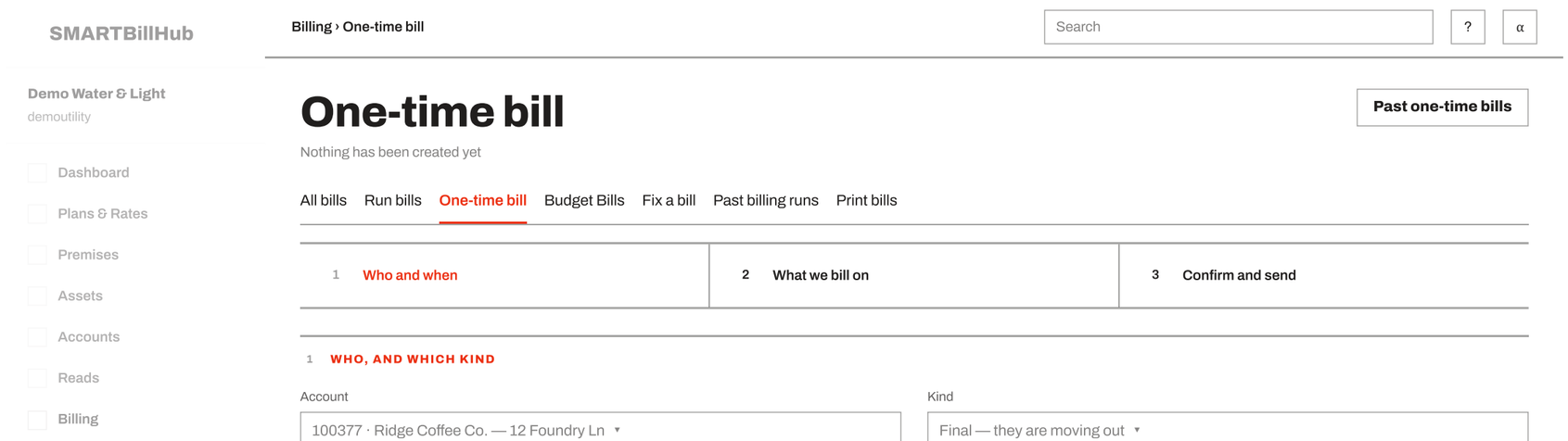
---

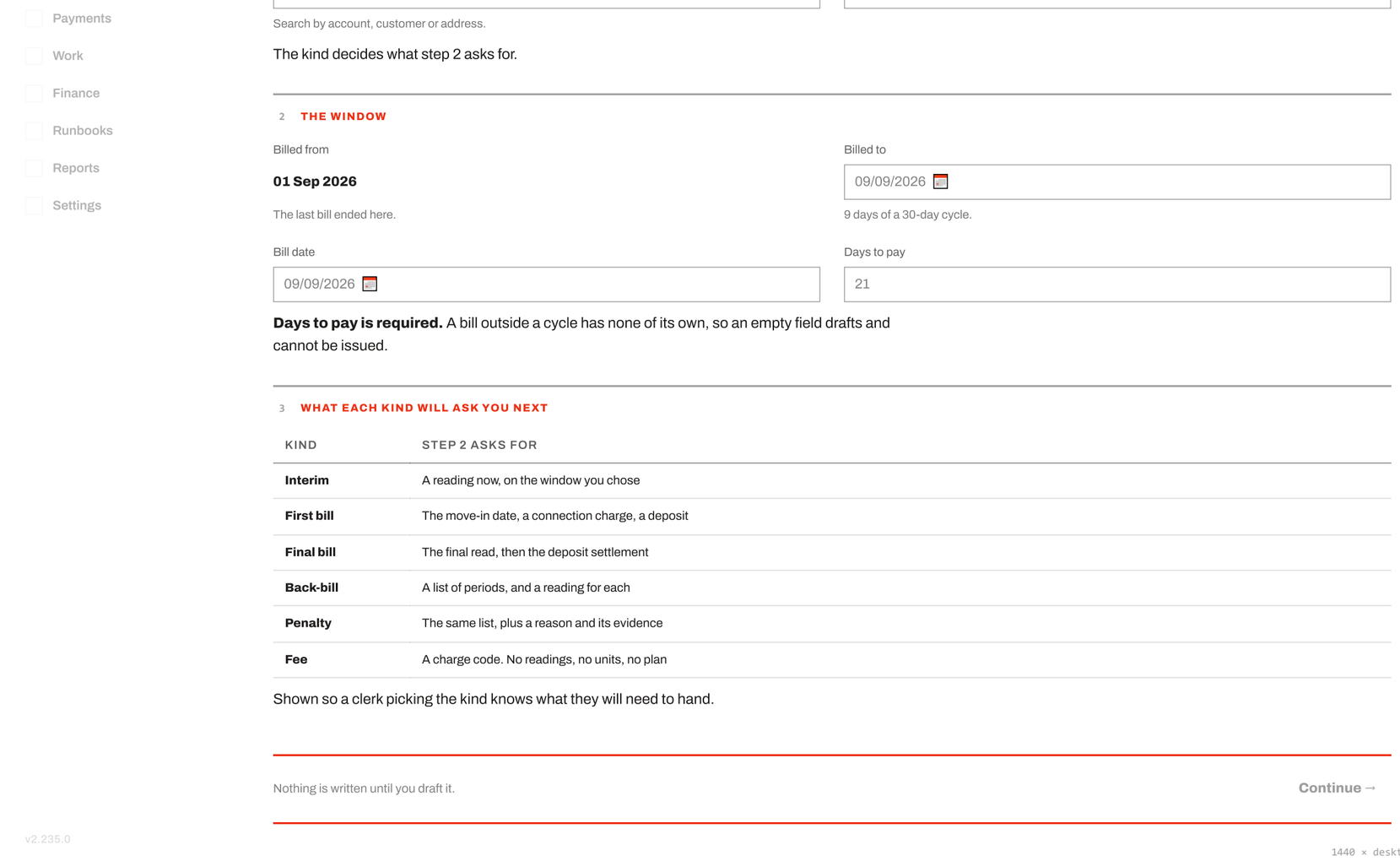
21 Who and when | STEP ONE New build S82 Exists behind S51 S54

One-time bill · step 1 of 3 · all six kinds

"Raise one bill for one account, outside the cycle. Which account, which kind, and for what window?"

REDESIGN drawn **2026-09-12**, still unbuilt at Version 35 (2026-09-13), drawn to BR-BIL-013's deposit-first order and the 2026-09-12 content ruling. **NEVER BUILT.** Today the product ships the *One-time bill* tab, screen 11 in §17.





One rail, six kinds	A clerk who learned **Scope → Check → Sign off → Send** on the bulk run should not meet a different pattern for one bill. Three steps: the request, what we bill on, then confirm.
Why the step split falls here	It matches the engine. **Step 1 is the request, step 2 is what gather resolves, step 3 is what pricing produced.** Nothing in steps 1 or 2 writes anything.
Card 3 is not decoration	The six kinds need genuinely different inputs — a fee needs no reading, a back-bill needs one per period. Naming that on step 1 stops a clerk reaching step 2 without the meter book.
Days to pay, and the refusal behind it	Your ruling was that days to pay are captured at billing time. An empty field is not defaulted: the bill **drafts and cannot be issued**, rather than a guessed due date starting a late-fee clock.

22.1 Interim

STEP TWO · MID-CYCLE New build S82 Exists behind S51 S54 S55

One-time bill · step 2 of 3 · drafts as `interim`

“Bill them now, part-way through the cycle. Show me what it prices and take the reading.”

REDESIGN drawn **2026-09-12**, still unbuilt at Version 35 (2026-09-13), drawn to BR-BIL-013's deposit-first order and the 2026-09-12 content ruling. **NEVER BUILT.** Today the product ships the *One-time bill* tab, screen 11 in §17.

SMARTBillHub

Billing › One-time bill

Search ? α

Demo Water & Light
demoutility

- Dashboard
- Plans & Rates
- Premises
- Assets
- Accounts
- Reads
- Billing
- Payments
- Work
- Finance
- Runbooks
- Reports
- Settings

One-time bill

Past one-time bills

Interim · nothing has been created yet

All bills | Run bills | One-time bill | Budget Bills | Fix a bill | Past billing runs | Print bills

1 Who and when | 2 What we bill on | 3 Confirm and send

A WHERE — PREMISES, THEN UNITS

PREMISES	UNITS IT COVERS
31 Quarry Rd, Austin TX 78704	31 Quarry Rd — one unit

The address is walked to from the premises, never stored on the account.

B WHAT — SERVICE POINTS AND THEIR ASSETS

SERVICE	SERVICE POINT	ASSET	SIZE
W	SP-4471	M-54120 · meter	3/4"
S	SP-4472	—	—
F	SP-4473	2" line	2"

One row per thing that can be priced. A service with no asset still has a base charge.

C READINGS

ASSET	AT 01 SEP	AT 18 SEP	CONSUMPTION
M-54120 · water	41,208	43,410	**2.2 kgal**
Sewer — from water	—	—	**2.2 kgal**

Reading at 18 Sep 2026: 43,410

How it was taken: Actual — visited

Saved to Meter Reads.

A reading typed here is saved to **Meter Reads**, where every other reading lives. Billing may not store one itself.

D HOW IT PRICES

WHAT	IN FORCE FOR THIS BILL
Plan	**Residential water & sewer**
Rate schedule	RES-W-2026 · tiered
Version in force	**v4, in force 01 Jul 2026**
Taxes	City sales tax 8.25%

Discounts	None on this account

The version is the one in force **at your bill date**, not today's.

⚠ **WHAT COULD STILL REFUSE**

IF THIS IS MISSING	THE BILL
Connection size on the fire line	Refuses — size picks the price
City-limits flag on the premises	Refuses — it picks the tax
A rate version at your bill date	Refuses by name

Shown here because these refuse **at pricing** — better seen now than on step 3.

Still nothing written. The reading saves when you continue.

← Back Continue →

v2.235.0

1440 × desktop

The fullest of the six	Every group applies: premises, units, assets, readings, plan, rates, taxes, discounts. The other five kinds are this screen with groups removed, and each says which and why.
Billing may not store a reading — the screen may still take one	The rule binds the **billing engine**, not the browser: *"Billing may not write meter reads and does not"*, and all nineteen of its metering calls are reads. So the field posts to **Meter Reads' own endpoint**, and the bill prices only after the reading exists there. **One fact, one owner** — a second home for a reading is the defect class this document keeps finding.
The warning block earns its place	Today an interim bill either prices or refuses and the clerk finds out at the end. **The last block is where a refusal becomes a thing you can fix** — before a number is computed, not after.
The version, not the rates	A rate schedule has versions and the bill uses the one in force **at the bill date**. Showing the schedule name alone would let a back-dated bill silently price on today's rates.

22.2 First bill

STEP TWO · DERIVED Exists behind S51 S54 S55

One-time bill · step 2 of 3 · drafts as `first`

"They moved in on the 19th. Bill the part-month and everything a new customer owes."

REDESIGN drawn **2026-09-12**, still unbuilt at Version 35 (2026-09-13), drawn to BR-BIL-013's deposit-first order and the 2026-09-12 content ruling. **NEVER BUILT.** Today the product ships the *One-time bill* tab, screen 11 in §17.

SMARTBillHub

Billing › One-time bill

Search ? α

Demo Water & Light
demoutility

Dashboard
Plans & Rates
Premises

One-time bill

Past one-time bills

First bill · nothing has been created yet

All bills Run bills One-time bill Budget Bills Fix a bill Past billing runs Print bills

1 Who and when — 2 What we bill on — 3 Confirm and send

A WHERE — PREMISES, THEN UNITS

PREMISES	UNITS IT COVERS
14 Alder Ct, Austin TX 78704	Unit B — one of four

The address is walked to from the premises, never stored on the account.

B WHAT — SERVICE POINTS AND THEIR ASSETS

SERVICE	SERVICE POINT	ASSET	SIZE
W	SP-5510	M-90441 · meter	5/8"
S	SP-5511	—	—
T	SP-5512	2 carts	96 gal

One row per thing that can be priced. A service with no asset still has a base charge.

C READINGS

ASSET	AT MOVE-IN	AT 30 SEP	CONSUMPTION
M-90441 · water	00,000	01,900	**1.9 kgal**

Reading at 30 Sep 2026: 01,900

How it was taken: Actual — visited

Saved to Meter Reads.

A reading typed here is saved to **Meter Reads**, where every other reading lives. Billing may not store one itself.

D HOW IT PRICES

WHAT	IN FORCE FOR THIS BILL
Plan	**Residential water · sewer · trash**
Rate schedule	RES-W-2026 · tiered
Version in force	**v4, in force 01 Jul 2026**
Taxes	City sales tax 8.25%
Discounts	None on this account

The version is the one in force **at your bill date**, not today's.

E WHAT A FIRST BILL ADDS

ITEM	WHERE IT COMES FROM	AMOUNT
Proration · 12 of 30 days	The move-in date on step 1	—
Connection charge	Charge code CONN · your catalogue	**$45.00**
Security deposit	The plan's rule for this category	**$150.00**

A deposit is **the customer's money held**, never revenue, and is never taxed. No rule means no line, not a zero.

⚠ WHAT COULD STILL REFUSE

IF THIS IS MISSING	THE BILL
A deposit rule for this category	No line — not a guessed figure
Whether a deposit was already paid	Refuses — it would charge twice
A rate version at your bill date	Refuses by name

Shown here because these refuse **at pricing** — better seen now than on step 3.

Still nothing written. The reading saves when you continue.

← Back Continue →

v2.235.0

1440 × desktop

Three lines a cycle bill never has	Proration, a connection charge and a deposit — group E, and all three are **derived, never typed**. The clerk supplies the move-in date; the figures come from the plan and the charge catalogue.
A deposit is not revenue and is never taxed	It goes to the **deposit-liability** designation, so the utility is holding the customer's money rather than earning it. That is structural: the deposit sits outside the tax base, not inside it at zero.
"No rule" and "no deposit" are different answers	A plan with no deposit rule for this category produces **no line**. The screen says so rather than showing $0.00, which would read as a deposit that was assessed and waived.
Already paid, and why it refuses	If the engine cannot read what deposit was already paid it **refuses** rather than assuming none. Assuming zero takes the money a second time on a bill that foots perfectly.

22.3 Final bill

STEP TWO · DEPOSIT FIRST Blocked S56 Exists behind S51 S54 S55 S57

One-time bill · step 2 of 3 · drafts as `final`

"They moved out on the 9th. Close them off, settle the deposit, and tell me whether we owe them or they owe us."

REDESIGN drawn **2026-09-12**, still unbuilt at Version 35 (2026-09-13), drawn to BR-BIL-013's deposit-first order and the 2026-09-12 content ruling. **NEVER BUILT.** Today the product ships the *One-time bill* tab, screen 11 in §17.

SMARTBillHub

Billing › One-time bill

Search ? α

Demo Water & Light
demoutility

- Dashboard
- Plans & Rates
- Premises
- Assets
- Accounts
- Reads

One-time bill

Past one-time bills

Final bill · nothing has been created yet

All bills Run bills One-time bill Budget Bills Fix a bill Past billing runs Print bills

1 Who and when 2 What we bill on 3 Confirm and send

A WHERE — PREMISES, THEN UNITS

PREMISES	UNITS IT COVERS
12 Foundry Ln, Austin TX 78704	12 Foundry Ln — one unit

The address is walked to from the premises, never stored on the account.

B WHAT — SERVICE POINTS AND THEIR ASSETS

SERVICE	SERVICE POINT	ASSET	SIZE
W	SP-7710	M-88213 · meter	3/4"
S	SP-7711	—	—

One row per thing that can be priced. A service with no asset still has a base charge.

C READINGS — THE FINAL READ

ASSET	AT 01 SEP	FINAL READ	CONSUMPTION
M-88213 · water	41,208	47,290	**6.1 kgal**

Final read at 09 Sep 2026

47,290

Saved to Meter Reads.

How it was taken

Actual — visited

A reading typed here is saved to **Meter Reads**, where every other reading lives. Billing may not store one itself.

D HOW IT PRICES

WHAT	IN FORCE FOR THIS BILL
Plan	**Commercial water & sewer**
Rate schedule	COM-W-2026 · tiered
Version in force	**v4, in force 01 Jul 2026**
Taxes	City sales tax 8.25%
Discounts	None on this account

The version is the one in force **at your bill date**, not today's.

E THE DEPOSIT, THEN ONE SETTLEMENT PATH

	BASIS	AMOUNT
Charges to the move-out day	9 days + 6.1 kgal	$289.40
Deposit held	received 12 Jul 2025	– $150.00
Interest on it	14 months @ 2%	– $3.50
Still due		**$135.90**

PICK EXACTLY ONE

Collect | Payment plan | Transfer to master — 100299 Alder Holdings

The deposit and its interest apply first. What is left picks exactly one path, and three of the five need no clerk at all.

F THIS CUSTOMER'S OTHER ACCOUNTS

ACCOUNT	POSITION	WHAT HAPPENS TO IT
A-11902 · Ridge Coffee, 8 Mill St	**Overdue $948.20**	Acted on — it seizes a refund
A-09984 · Ridge Coffee, 12 Mill St	**Credit $61.40**	Shown, **never applied**

Linked **by customer**, never by name or address. These are last night's balances, so they flag a decision rather than change the arithmetic.

⚠ **WHAT COULD STILL REFUSE**

IF THIS IS MISSING	THE BILL
The accrued deposit interest	Refuses — never principal alone
A master account for the transfer	That path is greyed, with its reason
A rate version at your bill date	Refuses by name

Shown here because these refuse **at pricing** — better seen now than on step 3.

Still nothing written. The reading saves when you continue.

← Back Continue →

v2.235.0

1440 × desktop

Deposit-first is frozen law, not a layout choice	A **$150.00** deposit held 14 months at 2% earns **$3.50**, so **$153.50** applies *before* any path is chosen — finance-signed. The order is the rule: apply the credit afterwards and a bill the deposit fully covers gets sent to collections.
Three of the five paths need no clerk	Zero → settled. Within the write-off bar **in either direction** → written off, because a $0.40 cheque costs more to post than it returns. Negative → refunded; nobody chooses whether to give back money that is not theirs. **Only a positive remainder asks a question**, which is why group E is mostly showing an outcome.
"Master account gets debt" — your own ruling, built	Ruled 2026-08-24: *"master account gets debt - if sub accoun tis unable to pay it ; even aftr deposite settlement"*. The residue moves to the master the sub-account hangs off.
The transfer target is resolved, never typed	There is **no account-id field** on this path, which makes a forged transfer *unrepresentable* rather than rejected. Precedence is the **declared owner first**, then the master that names the premises, and where both resolve they must name the same account. The option is **enabled when a master resolves and greyed with its reason otherwise** — never silently absent. ⚠ And two transfers exist: billing's own is **deliberately unpublished** because its prerequisite flag has **no column anywhere**; yours resolves; the owner rung needs `premises.owner_customer_id`, measured absent.
"Check the other accounts" — also already ruled	Ruled 2026-08-12, one word: *"offset first"*. And it is built. The read walks the **customer link and nothing else** — a shared surname, address or phone counts for nothing, because offsetting across those moves money between strangers. Worked on the repo's own fixture: **$57.13** credit on one account against **$948.20** overdue 91 days on another, and the whole $57.13 offsets with **nothing refunded** — where the naive reading posts a cheque to somebody about to be disconnected.
⚠ The sibling panel is unsigned, and is kept out of the arithmetic	A sibling **debt** seizes a refund; a sibling **credit** is **shown and never applied**. **Neither is signed** — both are prototype behaviour with an open question — and where a dispute and a sibling debt collide **the dispute wins**, because the dispute hold IS signed. The figure is the stored balance, not the ledger: enough to stop and ask, deliberately **not** enough to subtract from a refund, so it is drawn with an *as at* stamp outside the totals.
⚠ Nothing records which path was chosen	The engine computes the settlement and hands it back; **there is no settlements table**. So *"exactly one settlement path per final bill"* is enforced inside a single call and **not across calls** — a clerk can settle the same account twice, and the chosen path has nowhere to live.

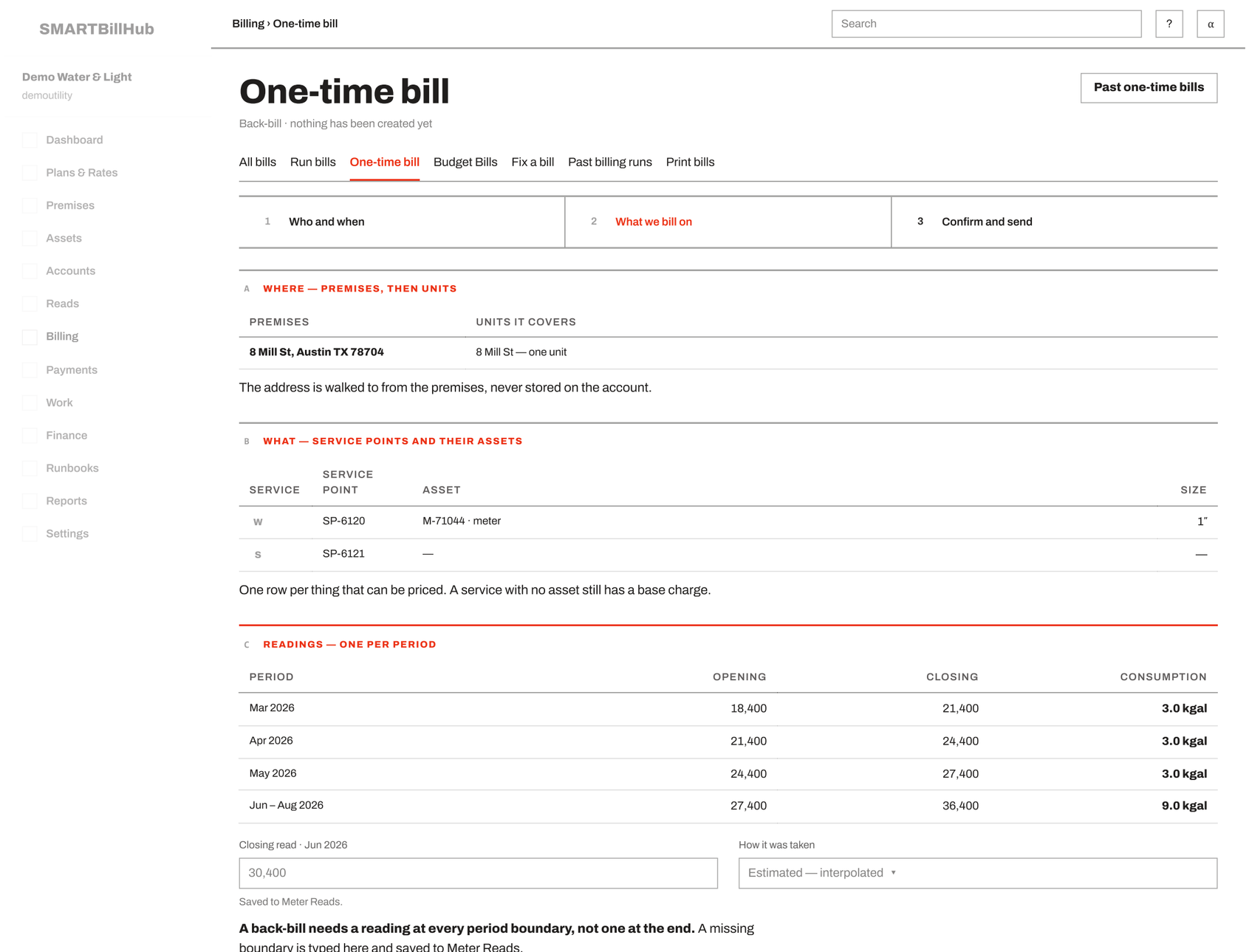
22.4 Back-bill | STEP TWO · TAKES A LIST

Exists behind S51 S52 S54 S55

One-time bill · step 2 of 3 · drafts as `backbill`

"We under-read them for six months. Recover it — priced the way it would have been billed at the time."

REDESIGN drawn **2026-09-12**, still unbuilt at Version 35 (2026-09-13), drawn to BR-BIL-013's deposit-first order and the 2026-09-12 content ruling. **NEVER BUILT.** Today the product ships the *One-time bill* tab, screen 11 in §17.



D **HOW IT PRICES**

WHAT	IN FORCE FOR THIS BILL
Plan	**Commercial water & sewer**
Rate schedule	COM-W-2026 · tiered
Version in force	**the version in force in each period**
Taxes	City sales tax, per period
Discounts	None on this account

Each period prices on **its own** version, not one version for the lot.

E **THE PERIODS, AND WHAT THE GROUPING COSTS**

	USAGE	PRICED
Six monthly periods — as drawn	18.0 kgal	**$79.50**
One period, 18 kgal together	18.0 kgal	**$104.50**
What the grouping costs the customer		**$25.00**

Edit the period list → Six periods · Mar–Aug 2026

One period puts 13 kgal in the top tier; six monthly groups put 6 there. **Both bills foot — only this panel shows the difference.**

⚠ **WHAT COULD STILL REFUSE**

IF THIS IS MISSING	THE BILL
A reading at any period boundary	That period refuses — not estimated silently
A rate version for an early period	Refuses by name, naming the period
A reason code	Required — a back-bill always says why

Shown here because these refuse **at pricing** — better seen now than on step 3.

Still nothing written. Readings save when you continue.

← Back Continue →

v2.235.0

1440 × desktop

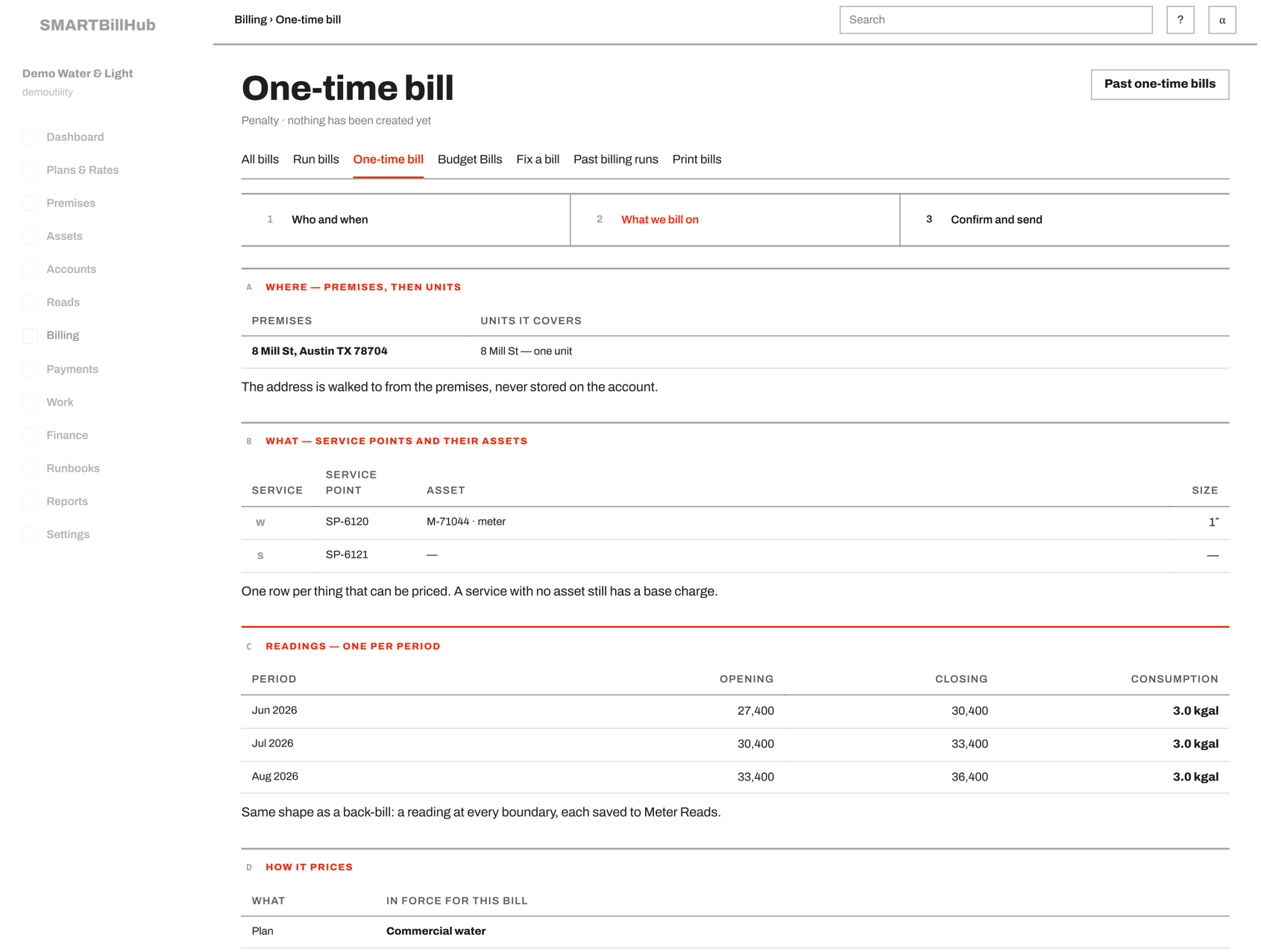
The list is the whole point, and it is worth $25.00	**18 kgal priced as one period puts 13 kgal in the top tier at $104.50; six monthly groups put 6 kgal there at $79.50.** Same usage, same rates, **$25.00** difference from the grouping alone — and both bills foot, so after the bill exists the difference is invisible.
A reading per boundary, not one at the end	This is the hardest of the six kinds for exactly that reason. Six periods need seven boundary readings, and a period missing one **refuses** rather than quietly interpolating money.
Each period prices on its own version	A rate change inside the recovered span means different periods price differently. One version for the whole span would be the cheapest thing to build and the wrong number.
Penalty is this screen plus a reason	Both kinds take a period list and both require a reason code. That is why there is no separate penalty list builder — screen 24 serves both.

22.5 Penalty | STEP TWO · A REASON New build S77 S102 Exists behind S51 S52 S54

One-time bill · step 2 of 3 · drafts as `penalty`

“The seal was broken. Recover what they actually used over those three months, and record why.”

> **REDESIGN** drawn **2026-09-12**, still unbuilt at Version 35 (2026-09-13), drawn to BR-BIL-013’s deposit-first order and the 2026-09-12 content ruling. **NEVER BUILT.** Today the product ships the *One-time bill* tab, screen 11 in §17.



Rate schedule	COM-W-2026 · tiered
Version in force	**the version in force in each period**
Taxes	City sales tax, per period
Discounts	None on this account

The version is the one in force **at your bill date**, not today's.

E **THE PERIODS, THE REASON, AND ITS EVIDENCE**

	USAGE	PRICED
Three monthly periods — as drawn	9.0 kgal	**$38.70**

Reason

Meter tampering — seal broken ▾

Evidence

WO-8841 · field report

Periods

Three · Jun–Aug 2026

Edit the list on screen 24.

A penalty recovers **what was used**, over the periods you list, with a reason recorded against it.

⚠ **WHAT COULD STILL REFUSE**

IF THIS IS MISSING	THE BILL
A reason code	Required — refuses without one
A reading at any period boundary	That period refuses
A rate version for an early period	Refuses by name

Shown here because these refuse **at pricing** — better seen now than on step 3.

Still nothing written.

← Back Continue →

v2.235.0

1440 × desktop

No amount to type	This bill re-prices what was used. A punitive charge is a **charge code your utility defines**, added like any other line.
Why that is structural rather than a validation	The request object carries **no** `amount`, `description` or `lines` member — *"a caller cannot hand-type money onto a bill through this module because there is nowhere to put it"*. A rejection can be deleted by a later refactor; a missing parameter cannot be passed by accident. The prototype drew an editable amount and a **+ Add line** button; superseded in writing.
It is back-bill's step 2 with a reason added	Same period list, same reading-per-boundary rule, same per-period rate versions. The reason and its evidence are the only additions, which is why this is a variant rather than a second builder.

22.6 Fee | STEP TWO · A CODE Exists behind S51 S54

One-time bill · step 2 of 3 · drafts as `fee`

"Charge them the reconnection fee. Nothing was metered — just the charge."

REDESIGN drawn **2026-09-12**, still unbuilt at Version 35 (2026-09-13), drawn to BR-BIL-013's deposit-first order and the 2026-09-12 content ruling. **NEVER BUILT.** Today the product ships the *One-time bill* tab, screen 11 in §17.

SMARTBillHub

Billing › One-time bill

Search ? α

Demo Water & Light
demoutility

- Dashboard
- Plans & Rates
- Premises
- Assets
- Accounts
- Reads
- Billing
- Payments
- Work
- Finance
- Runbooks
- Reports
- Settings

One-time bill

Past one-time bills

Fee · nothing has been created yet

All bills | Run bills | One-time bill | Budget Bills | Fix a bill | Past billing runs | Print bills

1 Who and when | 2 What we bill on | 3 Confirm and send

A WHERE — PREMISES, THEN UNITS

PREMISES	UNITS IT COVERS
31 Quarry Rd, Austin TX 78704	31 Quarry Rd — one unit

Shown so the clerk can confirm the account before charging it.

B WHAT — SERVICE POINTS AND THEIR ASSETS

Not asked, and that is not an omission. A fee has no usage, so there is nothing to read and no plan to price it on.

C READINGS

Not asked, and that is not an omission. A fee has no usage, so there is nothing to read and no plan to price it on.

D HOW IT PRICES

Not asked, and that is not an omission. A fee has no usage, so there is nothing to read and no plan to price it on.

E THE CHARGE

Charge code: RECON — Reconnection after cut-off ▾

Reason: Reconnected 11 Sep · WO-8907

LINE	FROM	AMOUNT
Reconnection after cut-off	Your charge catalogue · RECON	**$35.00**
City sales tax	A fee is taxed only if the code says so	**$0.00**
Total		**$35.00**

The amount is the code's. **Pick a different code to charge a different amount.**

⚠ WHAT COULD STILL REFUSE

IF THIS IS MISSING	THE BILL
The code is not in your catalogue	Refuses by name
The code has no live price	Refuses — never a zero
The code has no GL account	Refuses — the books must balance

Shown here because these refuse **at pricing** — better seen now than on step 3.

Still nothing written.

← Back Continue →

v2.235.0

1440 × desktop

The step collapses, and says so	Three of the five groups are **greyed with their reason on the face of them**. A fee has no usage, so there is nothing to read and no plan to price it on.
Why not just skip step 2 for a fee	Because the clerk still has to confirm **which account** and **which code**, and a wizard that changes shape between kinds teaches nothing. Greyed-with-a-reason is honest; a step that vanishes looks like a bug.
This is the only door to charging any code to one account	The reconnection path is **one caller** of it. Without this screen every new fee — a returned cheque, a tamper charge, a meter test — needs a new journey.
Four refusals, no defaults	An unknown code, no live price, no GL account, no tax treatment. Each refuses by name rather than charging zero, because a fee that quietly prices at nothing is a fee nobody notices was never charged.

23 Confirm and send | STEP THREE Blocked S56 Exists behind S51 S57

One-time bill · step 3 of 3 · all six kinds

“Show me the money, show me what it does to my books, let me see the bill, then create it.”

REDESIGN drawn **2026-09-12**, still unbuilt at Version 35 (2026-09-13), drawn to BR-BIL-013’s deposit-first order and the 2026-09-12 content ruling. **NEVER BUILT.** Today the product ships the *One-time bill* tab, screen 11 in §17.

SMARTBillHub

Billing › One-time bill

Search ? α

Demo Water & Light
demoutility

- Dashboard
- Plans & Rates
- Premises
- Assets
- Accounts
- Reads
- Billing
- Payments
- Work
- Finance
- Runbooks

One-time bill

Past one-time bills

Final bill · nothing has been created yet

All bills Run bills One-time bill Budget Bills Fix a bill Past billing runs Print bills

1 Who and when 2 What we bill on 3 Confirm and send

1 WHAT IT WILL BILL

LINE	BASIS	AMOUNT
Water — base charge, prorated	9 / 30 days	$18.60
Water — usage to the final read	6.1 kgal, tiered	$198.44
Sewer — base charge, prorated	9 / 30 days	$12.90
Sewer — volume from water	6.1 kgal	$37.40

City sales tax	8.25%	$22.06
Charges		**$289.40**
Deposit and its interest, applied first	150.00 + 3.50	– $153.50
Still due · to collect		**$135.90**

The lines are the kind's. **There is no amount field and no parameter to pass one to.**

2 **IMPACT ON YOUR BOOKS**

ENTRY	DEBIT	CREDIT	AMOUNT
Deposit interest accrued	6950 Deposit interest expense	2300 Customer deposit liability	**$3.50**
Deposit applied	2300 Customer deposit liability	1200 Accounts receivable	**$153.50**
The bill, issued	1200 Accounts receivable	Revenue by service, and tax payable	**$289.40**

WHEN THE REMAINDER IS	THIS PANEL
Owed by them — as drawn	**Posts.** Three entries, three keys
Zero either way	Nothing to post
Owed to them	**Refuses** — and names both figures

Three entries, three keys, so a re-run posts none of them twice.

3 **THE SPECIMEN**

See the specimen → The exact bill, rendered. It writes nothing.

Same code and same guarantee as sign-off in §03 — a preview that writes nothing can be run again after a fix.

4 **SEND**

CHANNEL	NOW	WHY
Print	Will go	The floor. Every bill prints.
Email	Not available	Needs a channel election this product does not publish.
Text · In-app · Portal	Not available	Billing's delivery seam names none of them.

☑ Print it now ☐ Leave it as a draft — I will send it later

Only what can actually leave the building is offered. The rest is named, not hidden.

This is the first thing that writes. It creates the bill, mints its number and posts to your books.

← Back **Create this bill & print**

v2.235.0

1440 × desktop

One screen for all six kinds	The lines table and the books panel are the kind's; everything around them is identical. A clerk learns this screen once.
Impact on books, and why it belongs here	This is the only step that writes. Showing the three entries **before** the button means the clerk is not the last person to find out what the bill did to the ledger.
The panel's most valuable	Take the **refund** case — the automatic, no-choice, give-it-back case. A $153.50 credit against $142.30 of charges would put **–$11.20 into**

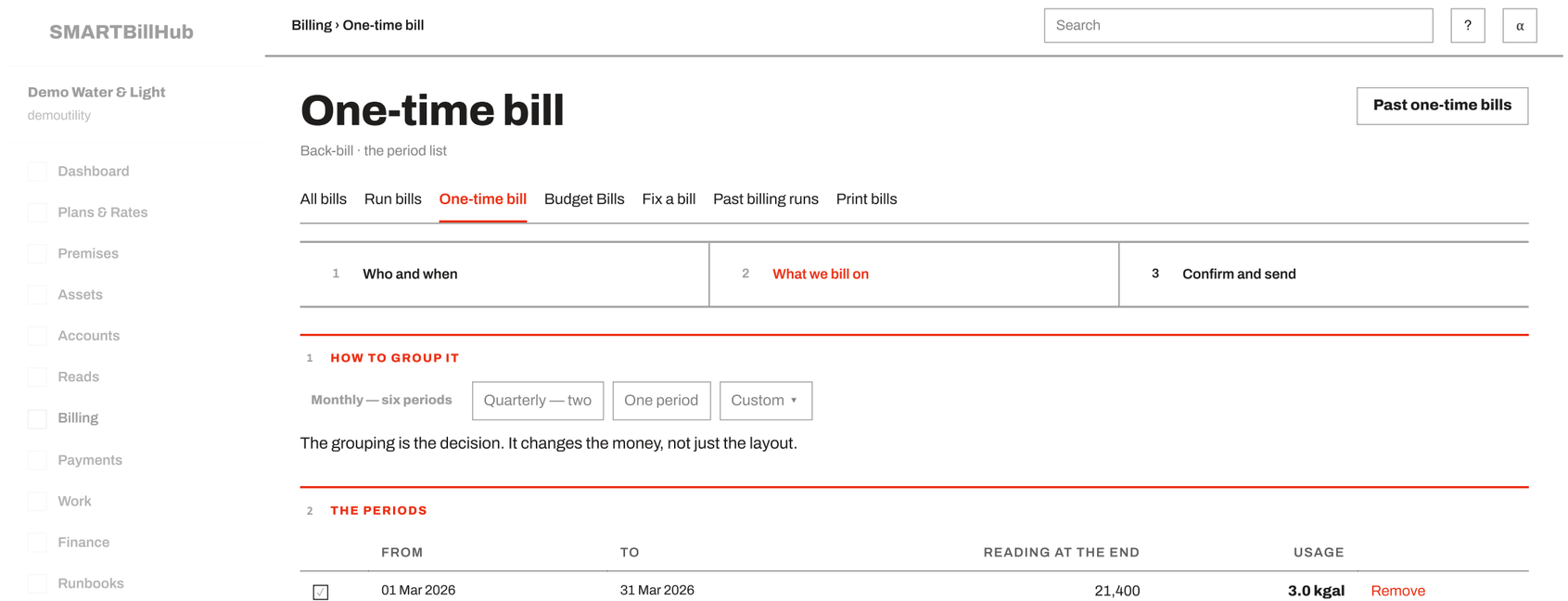
state is a refusal	**accounts receivable**: the books stating the utility is owed minus eleven dollars. And nothing could then pay it out — the signed refund row pays from *customer credit balances*, where this customer holds **$0.00**. **No signed statement moves the $11.20 across**, so the posting refuses by name rather than choosing where the excess lives.
A wrong entry is worse than a missing one	A missing one is **visible**: the deposit tie-out shows the gap in red with no dismiss control. A –$11.20 receivable reads as an ordinary credit balance on every report the product prints.
Three keys on one document	The bill's own issue entry, the interest accrual and the deposit application are **three distinct events on one invoice**, so all three lawfully exist and a resumed run replays all three and writes none of them twice.
Send offers only what leaves the building	**Print is the floor and always goes.** Email needs a channel election the product does not publish; text, in-app and portal are not named in billing's delivery seam at all. Drawing five cheerful channels would be drawing a product we do not have.
Your question: will a clerk confirm a send without seeing the bill	No — which is why the specimen is group 3 and not a link in a footer. The preview writes nothing, so it can be opened, a plan fixed, and opened again.
The one button that writes	Everything before this point is pure. This creates the bill, mints its number and posts to the ledger — and the footer says so in those words, rather than *Submit*.

24 Periods | THE LIST BUILDER Exists behind S52 S55

One-time bill · from step 2 of a back-bill or a penalty

"Six months of under-reading. How do I split it — and what does the split do to what they owe?"

REDESIGN drawn **2026-09-12**, still unbuilt at Version 35 (2026-09-13), drawn to BR-BIL-013's deposit-first order and the 2026-09-12 content ruling. **NEVER BUILT.** Today the product ships the *One-time bill* tab, screen 11 in §17.



Reports

Settings

☑	01 Apr 2026	30 Apr 2026	24,400	**3.0 kgal**	Remove
☑	01 May 2026	31 May 2026	27,400	**3.0 kgal**	Remove
☑	01 Jun 2026	30 Jun 2026	30,400	**3.0 kgal**	Remove
☑	01 Jul 2026	31 Jul 2026	33,400	**3.0 kgal**	Remove
☑	01 Aug 2026	31 Aug 2026	36,400	**3.0 kgal**	Remove

+ Add a period Six periods · 18.0 kgal · no gaps, no overlaps

A reading at every boundary. A period missing one refuses rather than being estimated into money.

3 **WHAT THIS GROUPING COSTS THE CUSTOMER**

GROUPING	USAGE	TOP TIER TAKES	PRICED
Six monthly periods — selected	18.0 kgal	6 kgal	**$79.50**
One period, all 18 kgal together	18.0 kgal	13 kgal	**$104.50**
The difference			**$25.00**

Both figures come from the same preview pass, which writes nothing.

⚠ **WHAT COULD STILL REFUSE**

IF THIS IS MISSING	THE BILL
A reading at a boundary	That period refuses, and is named
A rate version for a period	Refuses, naming the period
Gaps or overlaps in the list	Refuses — the span must be continuous

Shown here because these refuse **at pricing** — better seen now than on step 3.

Nothing is written. This list is an argument, not a bill.

← Back Use these six periods →

v2.235.0

1440 × desktop

Why this is its own screen	The grouping changes the money. Putting a decision worth **$25.00** behind a dropdown on a busy step-2 screen is how it gets made by accident.
The figure, worked	**18 kgal as one period puts 13 kgal in the top tier and prices at $104.50. Six monthly groups put 6 kgal there and price at $79.50.** Same usage, same rates, **$25.00** from the grouping alone.
After the bill exists, the difference is invisible	Both versions foot. Nothing on the issued bill, in the ledger or on any report says which grouping was used — so the comparison has to be **on this screen or nowhere**.
Serves the penalty too	A penalty takes the same period list and the same reading-per-boundary rule. One builder, two kinds — reached from screen 22.4 and screen 22.5.
Continuous, or it refuses	No gaps and no overlaps. A gap silently under-recovers and an overlap bills the same water twice, and neither is visible on the finished bill.

05

Budget bills

A flat monthly amount instead of what the meter says, trued up once a year. **The arithmetic is 60KB of finance-signed code with an append-only event ledger, and `enroll` had no caller outside tests** — a utility could not put one customer on a level. The tab exists; these five screens are what it needs — four for staff and one for the customer.

Budget bills run inside the regular cycle, and must not run separately

Confirmed by the Product Owner, 2026-09-12. **There is no separate budget run**, and adding one would be a second writer for the same money. The mechanism is unusually clean, and the sentence that carries it is the engine's own: *"the bill above is FULL ACTUALS and stays that way — every line, every GL stamp, the tax, and the total… the level is a COLLECTION SCHEDULE, never a price, so nothing here touches a line or a total."*

Revenue and tax are recognised on the **actuals every month**. What the run adds is a **second fact about the same bill** — the actual, the level, the variance movement, the running balance, and the amount the customer actually owes. Your ruling of 2026-08-11, in two words: *"Level is due."*

MONTH	ACTUAL	MOVEMENT	BALANCE	AMOUNT DUE
Jul	$292.87	−$50.87	−$50.87	**$242.00**
Aug	$231.40	+$10.60	−$40.27	**$242.00**
Sep	$198.75	+$43.25	**+$2.98**	**$242.00**

Three things make it safe to run for every account in one pass, and each is worth knowing:

PROPERTY	WHY IT MATTERS
Asked, not passed in	The call returns nothing for an account that is not enrolled, *"so there is no optional argument for a caller to forget: hard-won law 2 says a prop nothing passes is dead code, and a budget block that only appears when someone remembers to ask for it is exactly that."*
Idempotent per invoice	A database constraint, one per bill. A run that re-drafts a bill **returns the same facts rather than moving the variance twice**.
The settle-up rides the same bill	In the account's own settle-up month: one settle-up event, one audit row, the balance reset to zero, and **the level still due alongside it**.

⚠ **Which is why screen 25.2 no longer has a button.** An earlier version of this section drew a bulk *"settle up the year"* action. The run already posts it, on the bill, once per invoice — so that button was a **second writer for money that already has one**. And the settle-up month is **copied onto each account at enrolment**, so 37 accounts settle across nine different months;

a year-end batch was never the shape of this. **The screen is now a forecast.**

The three money columns, and why they are never collapsed into one

Computed level is what the trailing twelve months say. **Override** is the staff figure. **What they pay** is the one that reaches the customer. Three columns, three cells, on the table and on the form.

Arrears are not levelled

The amount due on a budget bill **is the level** — nothing folds an existing balance in. So a customer enrolled at **$85.00** a month who already owes **$312.40** owes both: the level covers what they use from now on, and the arrears stay due alongside it and keep being chased. **Screen 25.1 shows the balance before the level for exactly this reason** — a clerk who assumes the level absorbs it has told the customer something false.

The ±30% bound is measured against the COMPUTED level, never against the current one. A screen showing only *what they pay* leaves a clerk typing an override with no way to see the bound — and chaining overrides against each other is how **three audited 30% steps walk a level to twice what the trailing twelve months say**. That is why the computed figure is on screen even when nobody is overriding anything.

Nothing here drops anybody

Your ruling of 2026-08-11, verbatim: *"nodrop we chase for payments until write off"*. The two-missed-payments automatic drop-off is **superseded**. **Remove from budget is a staff act with a name on it**, and the variance balance is carried, never dropped.

When a customer asks for it from the portal

You asked what happens when somebody opts in from the customer portal, and whether that workflow was covered. **It was not — and what is there today is worse than uncovered.**

Two customer-facing reads exist and both are good: **the offer** (*what would I pay if I joined*, with twelve months of billed totals) and **the standing** (*what am I paying and where do I stand*). Neither recomputes anything — the figure is the same frozen function the staff screen uses. A customer who is not enrolled is **routed, not refused**: the body answers `standing: null` and **names where to join**.

⚠ **And where it names is a button that does not work.** The code says so in its own words: *"the join control is **held**. Enrolling is a write to `account_budget_status` and the portal may write rows of exactly six kinds; budget enrolment is none of them."* Across **64 portal routes**, both level-pay endpoints are **GET**. So a customer can see exactly what they would pay, press *Join level pay*, and nothing happens. That is the dead-control failure, on a money screen, with an open

level pay, and nothing happens. That is the dead-control failure, on a money screen, with an open question against it.

The answer is already in the product, one flow over. The hardship screen carries a `level_pay` route whose clerk-facing sentence is *"the customer would like to level out next year's payments"*, and it raises an **ordinary service request**. The rule it obeys: *a portal ask raises an ordinary service request, **never a request of a new kind***, and *the portal may never grant a protection itself — every one needs a person*.

So the customer asks, and a clerk enrols — for three reasons, not one

REASON	WHAT IT MEANS HERE
The portal writes six kinds of row	Budget enrolment is not one of them, and widening that list is a ruling rather than a screen.
Enrolment fixes a year	A level chosen by a customer who never saw the ±30% band is a year of under-collection nobody looked at.
Eligibility is unreadable from there	It rides a **rate-plan flag no endpoint publishes** — the same gap that greys bulk enrol on screen 25. The portal cannot prove a customer eligible, so it must not act as though it had.

The workflow, end to end

STEP	WHERE	WHAT HAPPENS	SCREEN
1	Portal	*Ask to join level pay* raises a **service request** carrying the level the customer was shown. The confirmation says **nothing about your bills has changed yet** and **we will come back to you**.	**25.4**
2	Staff	The ask lands on a queue, with **the level they were shown beside the level computed now**.	**25.3**
3	Staff	The clerk opens the enrol form with the account filled in, sees eligibility and the band, and enrols. The request closes and the customer's standing read starts answering.	**25.1**

Why step 2 shows two levels

A month can pass between the ask and the enrolment, and a bill lands in between — so **the figure the customer was quoted stops being the correct one**. Showing only the current level breaks a promise silently; showing only the quoted one enrols at a stale figure. On screen 25.3, **SR-4468 was shown $112.00 and the correct figure today is $119.00**.

Two names for one thing, kept on purpose

The portal says **level pay**; staff say **budget billing**. Both are already in shipped code and **both are being kept** — a customer-facing name beside a staff name is a legitimate split, unlike two words for one fact inside the product. Written down here so nobody "fixes" it later.

Two prototype controls that are named here instead of drawn

THE CONTROL	WHY NOT
“All eligible residential (312)”	Eligibility rides the rate **plan’s** budget flag, and **no endpoint publishes it**. A bulk action over 312 accounts whose eligibility nothing can read would fix what 312 people pay for a year at a level nobody looked at. **One account at a time, with the number shown first.**
“Import list from CSV”	The same gap, with a file in front of it.
“Cap increases at 15% year-over-year” · “opt-in only”	Both are tenant **settings**, and the surface that writes them is Settings, not this tab. Drawing the checkboxes here would offer a control that saves nowhere.

25 Budget bills | THE LEVEL, THREE WAYS

Blocked S75 · New build S80 S81 S82 · New read S74 · Exists behind S58 S59 S67

`Billing › Budget Bills · the tab exists; this is the table it needs`

“Who is on a level, what are they paying, and who has drifted far enough that I should look?”

REDESIGN drawn **2026-09-12**, still unbuilt at Version 35 (2026-09-13), drawn to the Product Owner’s 2026-09-12 ruling that budget bills run inside the regular cycle. **NEVER BUILT.**
Today the product ships the *Budget Bills* tab, screen 12 in §17.

SMARTBillHub

Billing › Budget Bills

Search ? α

Demo Water & Light
demoutility

- Dashboard
- Plans & Rates
- Premises
- Assets
- Accounts
- Reads
- Billing
- Payments
- Work
- Finance
- Runbooks
- Reports
- Settings

Budget Bills

218 accounts on a level · 37 carrying a shortfall

Settle up the year → + Enrol an account

All bills · Run bills · One-time bill · **Budget Bills** · Fix a bill · Past billing runs · Print bills

ENROLLED	CARRYING A SHORTFALL	DUE TO RE-LEVEL	ASKS WAITING
218	**37**	**11**	**6**
of 1,284 accounts	$4,912.60 to settle	drifted past 10%	from the portal · screen 25.3

Search account #, customer or address... · Status ▾ · Plan ▾ · Re-level due ▾ · + Filter ▾ · ↓ · ▭

3 selected · **Export these** · Enrol these · Re-level these · Enrolling in bulk is not offered — eligibility rides a plan flag nothing publishes.

	ACCOUNT	STATUS	COMPUTED	OVERRIDE	THEY PAY	ACTUAL TO DATE	VARIANCE	RE-LEVEL
☑	100377 Ridge Coffee Co.	Enrolled	$142.00	—	**$142.00**	$1,704.00	**–$61.40**	Jan 2027
☑	100412 Navarro, P.	Enrolled	$88.00	$96.00	**$96.00**	$1,056.00	**+$212.80**	Jan 2027
☑	100455 Okafor, D.	Enrolled	$74.00	—	**$74.00**	$888.00	**$0.00**	Jan 2027
☐	100498 Hale, J.	Enrolled	$103.00	—	**$103.00**	$1,236.00	**+$318.20**	**Drifted 14%**
☐	100510 Alder Holdings	Off budget	$260.00	—	**—**	—	**$0.00**	—

Three money columns, not one	**Computed · Override · They pay.** The ±30% bound is measured against the computed figure, so hiding it is how an override gets typed blind.
The variance column is the one a clerk works from	A level is a promise about a year. **+$318.20** with a **14% drift** is the row that needs a re-level now rather than in January, and it is the only thing on this table that changes by itself.
Bulk enrol is greyed with its reason, not hidden	Eligibility rides the rate plan's budget flag and **no endpoint publishes it**. Greyed-with-a-reason tells a clerk to enrol one at a time; a missing control tells them the product forgot.
Off budget is a state, not an absence	Account 100510 shows **Off budget** with a computed level still visible, because — per your ruling — leaving the level is a staff act and the utility goes on chasing the balance. A blank row would read as never enrolled.
Whole dollars, and that is the rule	A level rounds to whole dollars. Drawing cents here would invite a clerk to type them into the override field, where they would be refused.

25.1 Enrol

ONE AT A TIME · Blocked S75 · New build S80 S82 · New read S74 · Exists behind S58 S59 S67

Billing › Budget Bills · from *+ Enrol an account*, or from an ask on screen 25.3

"Put this customer on a flat monthly amount. Show me who they are, the figure, and how far I am allowed to move it."

> REDESIGN drawn **2026-09-12**, still unbuilt at Version 35 (2026-09-13), drawn to the Product Owner's 2026-09-12 ruling that budget bills run inside the regular cycle. **NEVER BUILT.** Today the product ships the *Budget Bills* tab, screen 12 in §17.

SMARTBillHub

Billing › Budget Bills

Search ? α

Demo Water & Light
demoutility

- Dashboard
- Plans & Rates
- Premises
- Assets
- Accounts
- Reads
- Billing
- Payments
- Work
- Finance
- Runbooks

Enrol an account

Cancel

Nothing is written until you enrol

All bills | Run bills | One-time bill | Budget Bills | Fix a bill | Past billing runs | Print bills

1 WHO

Account

100612 · Whitlock, R. — 44 Pecan Row ▾

Search by account, customer or address.

WHO, AND WHERE	
Account	**100612** · Whitlock, R. · customer C-2871
Where	**44 Pecan Row, Austin TX 78704** — one unit
Services	W S T Water · Sewer · Trash

CAN IT BE LEVELLED	
Plan	**Residential water & sewer** — offers budget billing
Status	Active · billing normally
Already on a level	**No** — never enrolled
Open since	**Oct 2025** · 13 months
WHAT CHANGES FOR THEM	
Balance now	**$312.40 past due** 34 days
Autopay	On · card ending 4218
Next bill	**30 Nov 2026** · cycle 09-A — the level starts here

Nine facts, and each one can change the answer. **Everything here is read; nothing on this screen edits an account.**

⚠ TWO OF THOSE NINE CHANGE WHAT YOU TELL THE CUSTOMER

$312.40 is past due	**Arrears are not levelled.** The level covers what they use from now on; the $312.40 stays due alongside it
Autopay is on	From 30 Nov the amount taken becomes **the level**, not the bill. Tell them before it changes

2 IS THIS ACCOUNT ELIGIBLE

TEST	THIS ACCOUNT
Twelve months of billing history	**13 months** — passes
The plan offers budget billing	**Residential water & sewer** — passes

Two tests, and both refuse by name: **not enough history**, or **the plan does not offer it**.

3 THE LEVEL

	BASIS	AMOUNT
Trailing twelve months, billed	12 bills	$936.00
Computed level · whole dollars	÷ 12	**$78.00**
The band you may override within	±30%	**$54.60 – $101.40**

Override the level

$85.00

Inside the band. Leave it empty to use $78.00.

What they will pay

$85.00

The band is measured against the **computed** figure, so it does not move when you override.

4 WHAT HAPPENS NEXT

Every bill from 30 Nov	Charges **$85.00**, and shows the variance against what the meter said
Once a year	The difference is **settled up** — collected or credited, on a bill
If they stop paying	**Nothing drops them.** The balance is chased and the variance is carried

This writes an enrolment and an event row. Nothing bills yet.

← Back Enrol at $85.00

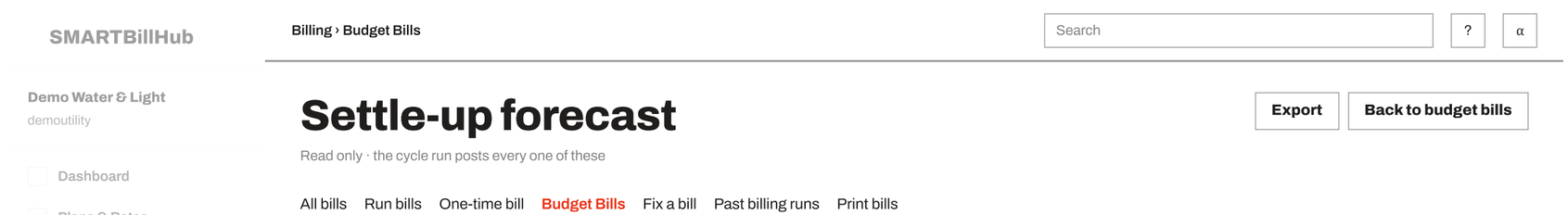
Nine facts, in three bands, and each can change the answer	**Who and where** confirms the right account. **Can it be levelled** is the decision. **What changes for them** is the phone call. A payload dump would be longer and worth less — every row here is one somebody would act on.
⚠ Arrears are not levelled, and nothing else on the screen says so	The amount due on a budget bill **is the level** — nothing folds an existing balance in. So **$312.40 past due stays due alongside $85.00 a month**, and a clerk who assumes the level absorbs it has told the customer something false.
⚠ Autopay is the one a customer notices first	From the next bill the amount taken becomes **the level** rather than the bill. A customer on autopay who was not told will see a different figure leave their account and ring about it.
Next bill, not today	**30 Nov, cycle 09-A.** A clerk enrolling on the 3rd needs to know nothing changes for four weeks — otherwise the first question they cannot answer is *when does this start?*
Already on a level is a row, because the answer must be no	One live status per account. Drawing it as a fact rather than discovering it on submit is the difference between a form and a refusal.
Eligibility is a step, not a validation message	Two tests, both shown passing or failing **before** the level is computed. A clerk who fails at the last field has already chosen a figure.
The band is drawn as a range, not a percentage	**$54.60 – $101.40** is usable; *±30%* is arithmetic homework. And the range is computed from **$78.00**, so it stays put when the override moves.
Group 4 is the promise, in three lines	What they pay, what happens at year end, and what happens if they stop paying. The third is your ruling — **nothing drops them** — and a clerk enrolling somebody should know it before they do it.
⚠ This is the caller that was missing	`enroll` is written, tested and had **no caller outside tests** — hard-won law 2. This form is the whole gap.
It reads nine facts and writes none of them	No field here edits an account. An address moves on the premises, autopay moves on the account, and arrears are worked where arrears are worked.

25.2 Settle-up forecast | READ ONLY New build S80 Exists behind S60

Billing › Budget Bills · a forecast; the cycle run does the posting

“Who is going to be trued up, when, and for how much — so I can answer them before they ring?”

REDESIGN drawn **2026-09-12**, still unbuilt at Version 35 (2026-09-13), drawn to the Product Owner’s 2026-09-12 ruling that budget bills run inside the regular cycle. **NEVER BUILT.**
Today the product ships the *Budget Bills* tab, screen 12 in §17.



SETTLING THIS MONTH	TO COLLECT	TO CREDIT	LATER IN THE YEAR
9	**$1,204.60**	**$318.20**	**28**
November	6 accounts	3 accounts	8 other months

1 **NOTHING ON THIS SCREEN WRITES ANYTHING**

	WHERE IT HAPPENS
The settle-up line	On the account's **next cycle bill**, in its own settle-up month
Who posts it	The **bill run**, inside the same write that creates the bill
If the run is re-drafted	Posted **once** — a database constraint per bill, not a check a caller can skip
What this screen is for	Seeing it coming, and answering the customer who rings to ask

There is no button here on purpose. A second way to settle the same money is a second answer to it.

Search account #, customer or address... | Month ▾ | Direction ▾ | + Filter ▾ | ↓ | ▭

ACCOUNT	SETTLES IN	PAID ON THE LEVEL	ACTUAL SO FAR	BALANCE	WILL
100412 Navarro, P.	**Nov 2026**	$1,152.00	$1,364.80	**+$212.80**	**Collect**
100498 Hale, J.	**Nov 2026**	$1,236.00	$1,554.20	**+$318.20**	**Collect**
100377 Ridge Coffee Co.	**Nov 2026**	$1,704.00	$1,642.60	**−$61.40**	**Credit**
100455 Okafor, D.	Jan 2027	$888.00	$886.40	**−$1.60**	**Credit**
100571 Bayou Laundry	Mar 2027	$2,940.00	$3,102.70	**+$162.70**	**Collect**

Showing 1–37 of 37 | Rows per page 50 ▾ | ‹ | ›

Accounts do not all settle together. Each one carries the settle-up month it was enrolled with, so this list spans nine months.

v2.235.0

1440 × desktop

⚠ This screen was drawn with a button, and the button was wrong	An earlier version of this drawing offered *"Settle these 2"*. **The cycle run already posts the settle-up**, on the account's own bill, inside the same write that creates it, **idempotently per invoice**. A bulk button would have been a second writer for the same money — corrected here rather than quietly removed, because the reasoning is the thing worth keeping.
Nine months, not one	The settle-up month is **copied onto the account at enrolment**, so 37 accounts settle across nine different months. A single year-end batch was never the shape of this.
Two directions, and neither carries tax	**Collect** or **credit**, both through the budget-variance account. The tax was charged on actual usage every month, so taxing the settle-up would tax the same water twice.
Both figures, side by side	**What they paid on the level** beside **what the year has actually cost**. One number with a sign makes every customer call start from scratch.
It is a worklist for the phone,	The value of seeing a **+$318.20** collection three weeks out is the conversation you can have before it lands on a bill.

It's a worklist for the phone, not for the ledger

25.3 Asks | THE CUSTOMER ASKS, A CLERK ENROLS

Exists behind S66 S67

Billing › Budget Bills · service requests raised from the portal

"Six people asked online to level their payments. Which can I do, and what were they told?"

REDESIGN drawn **2026-09-12**, still unbuilt at Version 35 (2026-09-13), drawn to the Product Owner's 2026-09-12 ruling that budget bills run inside the regular cycle. **NEVER BUILT.** Today the product ships the *Budget Bills* tab, screen 12 in §17.

SMARTBillHub

Billing › Budget Bills

Search ? α

Demo Water & Light
demoutility

- Dashboard
- Plans & Rates
- Premises
- Assets
- Accounts
- Reads
- Billing
- Payments
- Work
- Finance
- Runbooks
- Reports
- Settings

Asks from the portal

Export

6 customers have asked to level their payments

All bills | Run bills | One-time bill | **Budget Bills** | Fix a bill | Past billing runs | Print bills

WAITING	ELIGIBLE NOW	NOT ELIGIBLE	ENROLLED THIS MONTH
6	**4**	**2**	**11**
oldest 4 days	ready to enrol	with a reason	3 from the portal

1 WHY AN ASK AND NOT AN ENROLMENT

The portal may write	Rows of exactly **six kinds**. Budget enrolment is not one of them
What is at stake	Enrolment **fixes what somebody pays for a year**
What the portal cannot see	Eligibility rides a **rate-plan flag no endpoint publishes**
So the ask is	An **ordinary service request** — never a request of a new kind

The customer asks and **a person decides**. The same shape the hardship ask already uses.

Search account #, customer or reference… | Eligibility ▾ | Asked ▾ | + Filter ▾ | ↓ | ▭

REFERENCE	ACCOUNT	ASKED	LEVEL THEY WERE SHOWN	LEVEL NOW	ELIGIBLE	
SR-4471	100612 Whitlock, R.	08 Nov	$78.00	**$78.00**	Yes	Enrol →
SR-4468	100588 Nguyen, T.	07 Nov	$112.00	**$119.00**	Yes	Enrol →
SR-4463	100640 Amari, L.	06 Nov	$94.00	**$94.00**	Yes	Enrol →
SR-4459	100655 Duarte, S.	05 Nov	—	—	7 months only	Tell them why →
SR-4452	100671 Okpara, C.	04 Nov	—	—	Plan does not offer it	Tell them why →



Why this queue exists at all	The portal may write rows of **exactly six kinds** and budget enrolment is none of them — so a customer's ask has to land somewhere. It lands as an **ordinary service request**, which is the rule the product already states: *a portal ask raises an ordinary service request, never a request of a new kind*.
⚠ The two level columns are the whole reason this is a screen	**Level they were shown** beside **level now**. A month passes, a bill lands, and the figure the customer was quoted stops being the correct one. Showing only the current level breaks a promise silently; showing only the quoted one enrols at a stale figure. **SR-4468: shown $112.00, correct today $119.00.**
Not eligible is a row, not a rejection	**7 months only** and **plan does not offer it** are the two tests, and each row offers *Tell them why*. A customer who asked online and heard nothing asks again by telephone.
It reuses the enrol screen rather than repeating it	*Enrol* → opens **25.1** with the account filled in, so the band, the twelve-month basis and the override live in one place. Two enrolment forms would drift within a release.
Nothing here is automatic, and that is the ruling	*The portal may never grant a protection itself — every one needs a person.* The same reading covers this: a person judges, and the screen exists so judging is quick rather than optional.

25.4 Portal · Level pay | THE CUSTOMER'S SIDE Exists behind S66

Customer portal · `/level-pay` and `/level-pay/standing`

"Can I just pay the same amount every month? What would it be, and how do I start?"

REDESIGN drawn **2026-09-12**, still unbuilt at Version 35 (2026-09-13), drawn to the Product Owner's 2026-09-12 ruling that budget bills run inside the regular cycle. **NEVER BUILT.** Today the product ships the *Budget Bills* tab, screen 12 in §17.



Your bill still shows what you actually used. **Level pay changes when you pay, not what you owe.**

2 **WHAT HAPPENS AT THE END OF THE YEAR**

If you used less than you paid	We **credit** the difference on a bill
If you used more	We **collect** the difference on a bill
Either way	It is **one line on one bill**, and we tell you before it arrives

3 **JOIN**

Ask to join level pay We will look at your account and come back to you.

Nothing about your bills changes yet. Someone here checks your account and confirms the amount before it starts.

Already on level pay? Your monthly amount and how far ahead or behind you are appear here instead.

1440 × customer portal

Both reads already exist, and they are good	**The offer** — *what would I pay if I joined* — and **the standing** — *what am I paying and where do I stand*. Two routes, one subject asked two ways, and **neither recomputes anything**: the figure comes from the same frozen function the staff screen uses, with the tenant's own rounding and averaging window.
⚠ And the Join button is HELD today	The code says why, in its own words: *"enrolling is a write to* `account_budget_status` *and the portal may write rows of exactly six kinds; budget enrolment is none of them."* Across **64 portal routes**, both level-pay endpoints are **GET**. So a customer can see exactly what they would pay, press the button, and nothing happens — and the standing read even **names where to join**. That is the dead-control failure, on a money screen.
What the button does here instead	It **asks**. The ask raises an **ordinary service request** onto a clerk's worklist — screen 25.3 — carrying the level the customer was shown. That is not caution; it is the rule the product already states for every portal ask of this shape, and the hardship screen's `level_pay` route already does exactly this.
Three sentences it must say, and does	**"Nothing about your bills changes yet"** · **"we will come back to you"** · **"someone here confirms the amount"**. The hardship ask states the equivalent **on the wire** — `protection_created: false`, `granted_by: "clerk"` — *so no screen can imply otherwise*. The same fields belong on this response.
Group 1's second line is doing real work	**"Level pay changes when you pay, not what you owe."** The bill stays full actuals — every line, the tax, the total — and the level is a **collection schedule**. A customer who thinks they have bought a fixed price will dispute their first settle-up.
The highest and lowest month are on screen deliberately	**$164.20** and **$61.40** against a level of **$103.00** is the argument for joining, made with the customer's own figures rather than a leaflet's.
One landlord, four answers	Enrolment is **per account**, so somebody holding four accounts gets four levels. Adding their bills together would produce a level for a *customer* rather than for a property, **which is not a thing this product can enrol**.
Two names for one thing, kept on purpose	The portal says **level pay**; staff say **budget billing**. Both are already in shipped code and **we are keeping both** — a customer-facing name beside a staff name is a legitimate split, unlike two words for one fact inside the product. Said here so nobody "fixes" it later.

06

Fix a bill

Four steps, thirteen reasons, three classes, and one rule above all of them: **the operator picks the REASON and the system DERIVES the mechanism**. There is no mechanism dropdown anywhere in this workflow, and that is deliberate — *"a name fix that issues a credit note moves the ledger for a spelling, and a rate fix that only reprints is a silent underbilling nobody ever sees."*

Four steps, and step 2 changes with the correction

Product Owner, 2026-09-12: *"we will ahve to make it wizard - 4 steps."* Same rail as the bulk run and the one-time bill, so a clerk meets one pattern everywhere.

STEP	SCREEN	WHAT IT IS FOR
1	**26 · What and why**	The scope — one bill, several months, a run, a cycle, a plan or a service — then **the bills it resolves to, with what each asked and what has been paid**, then the reason.
2	**26.1 – 26.5** five variants	The correction itself. **One bill · several months · a batch · goodwill · the page only.**
3	**26.6** one bill **26.7** many	Four panels: **their money · your books · the tax · the loose ends.**
4	**26.8 · Generate and send**	The documents, the send options, and the one button that writes.

What each case shows, and what it deliberately does not

CASE	THEIR MONEY	TAX	DISCOUNTS	WHAT GOES OUT
26.1 One bill	The full trail, plus the answer sentence	Per authority	*"On both bills"* + which version	Credit note **+** replacement
26.2 Several months	**One row per month**, then a total	Summed, still split by authority	Each month on its own enrolment	2 documents × months
26.3 A batch	**Per-bill rows**, collapsed, searchable	By authority across the batch	Accounts affected	2 × bills, counted on the button
26.4 Goodwill	**Absent, and says why** — the bill is untouched	**Absent** — nothing was reversed	Nothing is re-priced	**Credit note only**
26.5 The page only	**Absent** — $0.00 moves	**Absent**	Unchanged	**A reissued document only**

An absent panel is drawn, never omitted

A clerk who sees no payments panel on a goodwill credit should read **why there isn't one**, not

A clerk who sees no payments panel on a goodwill credit should read **why there isn't one**, not wonder whether the screen forgot. Every absence above renders greyed with its reason on the face of it — and the reason is always the same shape: **nothing on the bill moved**.

The thirteen, and what each one does

CLASS	THE REASONS	WHAT HAPPENS	BALANCE MOVES BY
R 7 reasons	Wrong read · Wrong rate or plan · Wrong tax · A charge was missed · A charge was billed in error · Wrong period or service · Theft, restated	Credits the original **in full** and issues a rebill for the right amount	**The difference**
C 1 reason	Goodwill	A goodwill credit **on the account, tax excluded**. The bill is not touched; the credit comes off a future bill	The credit, later
D 5 reasons	Customer name · Service or mailing address · Premises or unit label · Meter number or read dates · Template or language	Reprints the document with the details corrected	**Exactly $0.00**

Three rules sit above the table

A multi-reason case takes the strongest class present, R > C > D — a document reason costs nothing when a restatement is already happening, because the rebill re-snapshots from current master data anyway. **A D-class case with a non-zero money delta is refused.** And a correction applies only to the **latest bill in its chain**, walked to a depth of twenty.

Five things people bring that are not corrections

Each is named **with its destination**, because the alternative is the same fix being built twice in two places.

WHAT THEY REACH FOR	WHAT IT ACTUALLY IS
An estimate was replaced by an actual	Trues up on the **next** bill by register arithmetic. Nothing about July was wrong — the register caught up
This bill should never have existed	A **void**. A correction says what they owe; a void says they were never billed
A period was never billed at all	A **back-bill** — §04 screen 22.4. There is nothing to restate
The payment went on the wrong bill	A payments **re-allocation**. The bills themselves are right
Master data is wrong and nothing was sent	The customers-module edit. This workflow decides what an **already-sent** document says

What a correction does to the books — confirmed

Product Owner, 2026-09-12: *"so im confirming books wil be updated"*. **Two acts, and the distinction is worth keeping: approve** records the signature, **execute** posts. The posting is not a later batch job — it happens inside the execute transaction, and if the books refuse, **nothing is written** and the bill stays correctable.

CLASS	WHAT POSTS
R	The credit note **reverses the original's own stamped GL buckets** — `DR 4100 88.60 · 4105 12.00 · 4110 39.50 / CR 1200 AR 247.87` — and then the rebill posts its own stamps.
C	`DR write-off / CR customer credit balance`, both resolved by designation. Your ruling of 2026-08-12, *"a separate write-off account"*. **The original service revenue is not touched**, so a month's revenue keeps reading as the revenue that was billed.
D	**Nothing at all.**

It is never one net entry. *"The credit note reverses the original in full and the rebill posts its own stamps. Netting them into a single entry would lose the fact that a bill was issued and reversed, which is what an auditor follows."* And **the atomicity unit is ONE BILL, not the batch**: a bill that fails leaves **zero artifacts** and stays correctable, while a bill that succeeded is never reversed because a later one failed — *"a posted pair cannot be un-posted and a delivered bill cannot be un-sent, so a batch rollback would be a lie in the UI."* The batch driver catches the failure and carries on.

Billing writes no journal lines of its own: it emits a request through the seam and **the ledger is the sole writer**.

Cancelling bills and regenerating them — your question, answered from the code

You asked for a scenario that cancels a bill for one account over several months and regenerates it, and then the same thing for a batch of accounts. **Both already exist, and neither is a new mechanism.** A class-R restatement *is* cancel-and-regenerate: the credit note is the cancellation and the rebill is the regeneration, and the two happen together on one bill so the account is never left with only half of it.

WHAT YOU WANT	HOW IT IS DONE	SCREEN
One account, several months	One class-R case, scoped to a **date range** on that account. Every bill in the window is credited in full and rebilled	**26.4**
Many accounts at once	The same case with a wider scope — **a run, a cycle, a plan, a service, or a date range**	**26.5**
A bill that should not exist	A **void**, which is not a correction — and ⚠ has no endpoint in this build	named, not drawn

There is no undo, and there never will be. The module says so in its own words, in the sentence that goes where the button would have been: *"the corrections that already executed

posted balanced journal entries, and a posted entry cannot be un-posted; their corrected bills are already queued for delivery, and a delivered bill cannot be un-sent. A button offering to undo them would be drawing something the books cannot do." **The remedy for a wrong batch is a NEW case over exactly those bills**, reviewed and approved like any other.

⚠ And the 12-month window applies to a correction that INCREASES what the customer owes. Your ruling of 2026-08-11, closing the question with option A: *"the correction workflow is NOT a route around the consumer-protection cap."* So a regenerate that adds money to a period which ended more than **twelve months** ago is **refused at preview**, per bill, naming the age. **A downward correction of the same bill is unaffected at any age**, and a theft case is exempt. This is the constraint most likely to bite the multi-month case you asked about, which is why it is on screens 26.4 and 26.5 rather than in this note alone.

---

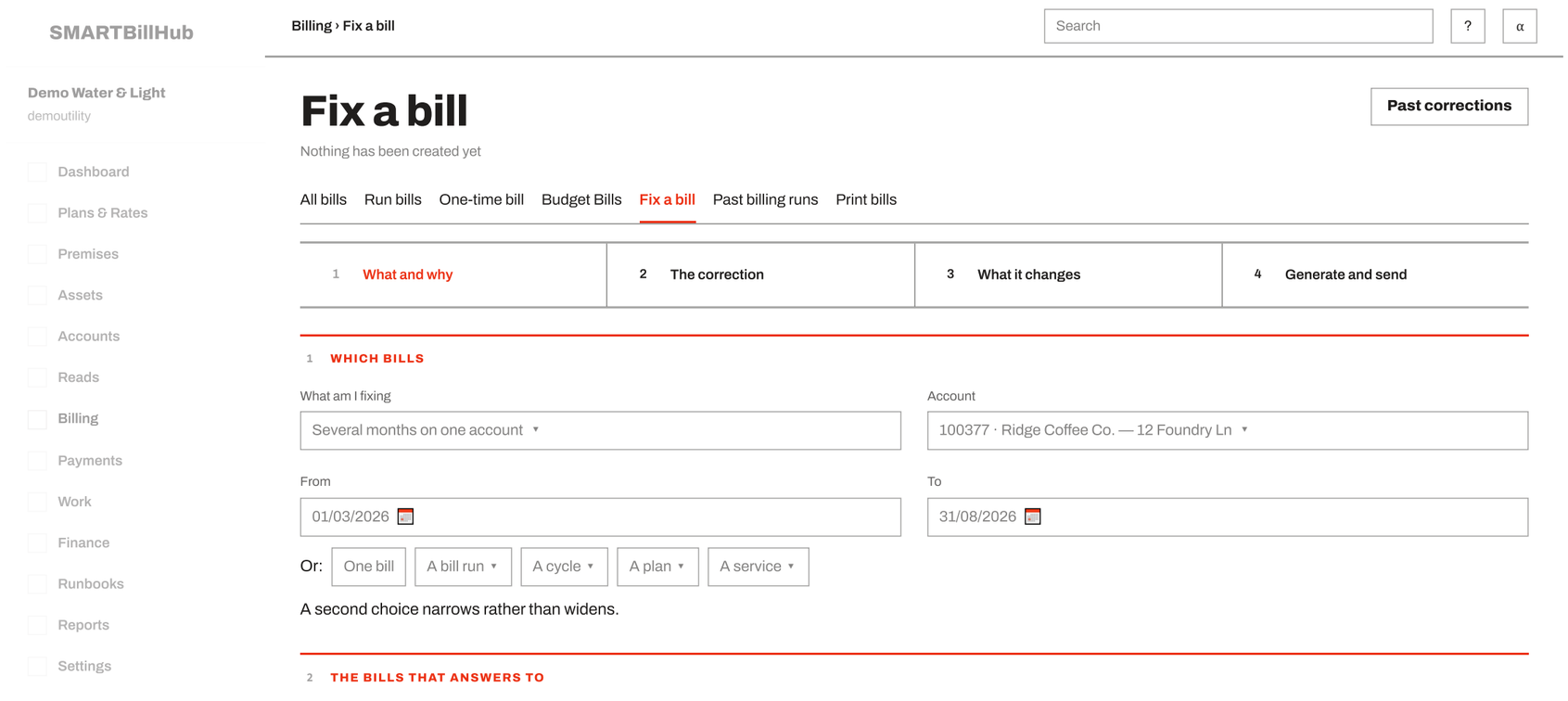
26 What and why | STEP ONE

Blocked S61 New build S80 S81 S83 Exists behind S62

Billing › Fix a bill · step 1 of 4

"These bills are wrong. Which ones, and what do I tell the system?"

REDESIGN drawn **2026-09-12**, still unbuilt at Version 35 (2026-09-13), drawn to the Product Owner's 2026-09-12 four-step wizard ruling and the same day's confirmation that approval moves the books. **NEVER BUILT.** Today the product ships the *Fix a bill* tab, screen 13 in §17.



BILL	PERIOD	AMOUNT	PAID	STATE
BIL-2603-0104	Mar 2026	$241.80	$241.80	Paid
BIL-2604-0119	Apr 2026	$238.10	$238.10	Paid
BIL-2605-0131	May 2026	$244.60	$244.60	Paid
BIL-2606-0148	Jun 2026	$239.90	$180.00	Part paid
BIL-2607-0166	Jul 2026	$247.20	$0.00	Unpaid
BIL-2608-0177	Aug 2026	$242.50	$0.00	Unpaid
Six bills		**$1,454.10**	**$904.50**	

Only an issued or paid bill can be corrected.

3 WHY — PICK ONE OR MORE

REASON	CLASS
CLASS R · THE MONEY AND THE DOCUMENT BOTH MOVE — 7	
☐ **Wrong read**	R
☐ **Wrong rate or plan**	R
☐ **Wrong tax**	R
☐ **A charge was missed**	R
☐ **A charge was billed in error**	R
☐ **Wrong period or service**	R
☐ **Theft, restated**	R
CLASS C · THE BILL IS RIGHT; THE UTILITY CHARGES LESS — 1	
☑ **Goodwill**	C
CLASS D · THE MONEY IS RIGHT, THE PAGE IS WRONG — 5	
☐ **Customer name**	D
☐ **Service or mailing address**	D
☐ **Premises or unit label**	D
☐ **Meter number or read dates**	D
☐ **Template or language**	D
⚠ NOT A CORRECTION — THESE REFUSE, AND SAY WHERE TO GO — 5	
☐ An estimate was replaced by an actual	Trues up on the **next** bill
☐ This bill should never have existed	A **void** — they were never billed
☐ A period was never billed at all	A **back-bill** — §04 screen 22.4
☐ The payment went on the wrong bill	A payments **re-allocation**
☐ Master data is wrong, nothing was sent	The customers-module edit

4 SO THIS IS WHAT WILL HAPPEN

Your reason is **Goodwill**, which **issues a goodwill credit on the account, excluding tax: the bill is not touched and nothing about it changes — the credit comes off a future bill**.

Derived from the reason, never chosen.

Nothing is written. Continue →

v2.235.0

1440 × desktop

Six ways in, one case out	**One bill · several months · a run · a cycle · a plan · a service.** A second selector is another condition on the same case, not a second case to reconcile.
Group 2 is the half that was missing	You asked to see the existing details once an account is chosen. **The paid column is the one doing work** — it is why step 3 has a payments panel, and where a clerk first sees that four of six are settled.
Eighteen rows, five of which refuse	Thirteen reasons in three banded classes, then the five things that are **not** corrections, each disabled and naming its destination. Two reasons of different classes give **one** case at the stronger class.
Group 4 is the design of this screen	The operator picks a reason; the system says what it will do. **There is no mechanism dropdown anywhere in this workflow** — that dropdown is how a spelling fix issues a credit note.
⚠ The reason map is proposed, not ratified	It waits on one sentence: *"a misspelt name changes $0.00 and issues no credit note; a wrong rate changes $19.92 and always issues a credit note plus a corrected bill."* And a **tenant's own reason is refused rather than defaulted**, because the lookup that would class it is not built.

26.1 One bill | STEP TWO · CLASS R Blocked S61 New build S80 S83

Billing › Fix a bill · step 2 of 4

"The read was wrong. Credit what we billed, bill what we should have."

REDESIGN drawn **2026-09-12**, still unbuilt at Version 35 (2026-09-13), drawn to the Product Owner's 2026-09-12 four-step wizard ruling and the same day's confirmation that approval moves the books. **NEVER BUILT.** Today the product ships the *Fix a bill* tab, screen 13 in §17.

SMARTBillHub

Billing › Fix a bill

Search ? α

Demo Water & Light
demoutility

- Dashboard
- Plans & Rates
- Premises
- Assets
- Accounts
- Reads
- Billing
- Payments
- Work
- Finance

The correction

Change the reason

One bill · wrong read · class R

All bills | Run bills | One-time bill | Budget Bills | Fix a bill | Past billing runs | Print bills

1 What and why | 2 The correction | 3 What it changes | 4 Generate and send

This bill drops by $99.24. They have already paid it, so they will be $99.24 in credit.

1 LINE BY LINE

	WAS	BECOMES	CHANGE
Water — usage	$198.44	$121.30	**–$77.14**
Sewer — volume from water	$37.40	$22.86	**–$14.54**
Base charges, unchanged	$31.50	$31.50	**$0.00**

Runbooks

Reports

Settings

Low-income discount	–$8.40	–$8.40	**$0.00**
City sales tax 8.25%	$22.06	$14.50	**–$7.56**
The bill	$289.40	$190.16	**–$99.24**

The old bill is credited in full and a replacement issued — nothing is edited.

2 **THEIR DISCOUNT IS SAFE**

Their **$8.40 low-income discount is on both bills** — priced on the discount they held **in that period**, not today's.

Nothing is written.

← Back

See what it changes →

v2.235.0

1440 × desktop

The answer is the first thing on the screen	A clerk can pick up the phone from that one line. The arithmetic underneath is for the question that follows.
Was, becomes, change — per line, not net	A single net figure is unarguable with and unexplainable to a customer.
Nothing is edited, and that is structural	An issued bill takes **no update and no delete** — the database refuses it, so a correction is always two new documents. Which is also why the prototype's *edit specific line items* control cannot be ported.
Group 2 answers the question a clerk is actually asked	*"Am I losing my discount?"* And the second half is load-bearing: the replacement prices the discount held **in that period**. That argument was once missing from the re-pricing, and it cost the customer exactly the discount.

26.2 Several months

STEP TWO · CLASS R Blocked S61 New build S80 Exists behind S69

Billing › Fix a bill · step 2 of 4 · one account, a window

"Wrong rate since March. Cancel those bills and generate them again."

REDESIGN drawn **2026-09-12**, still unbuilt at Version 35 (2026-09-13), drawn to the Product Owner's 2026-09-12 four-step wizard ruling and the same day's confirmation that approval moves the books. **NEVER BUILT.** Today the product ships the *Fix a bill* tab, screen 13 in §17.

SMARTBillHub

Billing › Fix a bill

Search

?

α

Demo Water & Light

demoutility

Dashboard

Plans & Rates

Premises

Assets

Accounts

The correction

Change the reason

Six months · one account · wrong rate · class R

All bills | Run bills | One-time bill | Budget Bills | Fix a bill | Past billing runs | Print bills

1 What and why | 2 The correction | 3 What it changes | 4 Generate and send

Six of these seven bills go up, by $159.80 in total. The seventh is too old to increase.

1 **BILL BY BILL**

	BILL	PERIOD	WAS	BECOMES	CHANGE	
☑	**BIL-2603-0104**	Mar 2026	$241.80	$268.40	**+$26.60**	Inside the 12-month window
☑	**BIL-2604-0119**	Apr 2026	$238.10	$264.30	**+$26.20**	Inside the 12-month window
☑	**BIL-2605-0131**	May 2026	$244.60	$271.50	**+$26.90**	Inside the 12-month window
☑	**BIL-2606-0148**	Jun 2026	$239.90	$266.20	**+$26.30**	Inside the 12-month window
☑	**BIL-2607-0166**	Jul 2026	$247.20	$274.40	**+$27.20**	Inside the 12-month window
☑	**BIL-2608-0177**	Aug 2026	$242.50	$269.10	**+$26.60**	Inside the 12-month window
☐	**BIL-2508-0912**	Aug 2025	$228.40	$253.90	**+$25.50**	**Cannot be increased — 13 months old**

Each bill is judged on its own age, so a window can be partly allowed.

⚠ **WHY THE SEVENTH CANNOT BE DONE**

A correction may not **increase** a bill more than **twelve months** old. **Reducing that same bill would be allowed at any age.**

2 **EACH MONTH ON ITS OWN TERMS**

Rates	The version in force **in that month**
Discounts	The discount they held **in that month**
Taxes	The rate and authority in force **in that month**

Not one version for the whole span.

The seventh bill is left exactly as it is.

← Back See what it changes →

v2.235.0

1440 × desktop

Cancel-and-regenerate is not a new mechanism	**A class-R restatement already is one:** the credit note is the cancellation, the replacement is the regeneration, and they happen together on each bill.
One case, not six	Six bills, one reason, one preview, **one approval**. Six cases would give an approver six figures and no total, which is how a month gets missed.
⚠ The cap bites per bill, and the screen shows it biting	Your ruling of 2026-08-11: *"the correction workflow is NOT a route around the consumer-protection cap."* The Aug 2025 row is **disabled with its age printed**, because a clerk who asked for seven months and got six needs to know which one. A theft case is exempt.
Only the latest bill in each chain is eligible	Redo this window twice and the second case acts on the **replacements**. A chain is walked to twenty, and twenty is a process problem somebody needs to be told about.

26.3 A batch | STEP TWO · CLASS R Blocked S61

Billing › Fix a bill · step 2 of 4 · many accounts, one case

"The whole August run priced water on the old rate. Do them again."

REDESIGN drawn **2026-09-12**, still unbuilt at Version 35 (2026-09-13), drawn to the Product Owner's 2026-09-12 four-step wizard ruling and the same day's confirmation that approval moves the books. **NEVER BUILT.** Today the product ships the *Fix a bill* tab, screen 13 in §17.

Demo Water & Light
demoutility

Dashboard
Plans & Rates
Premises
Assets
Accounts
Reads
Billing
Payments
Work
Finance
Runbooks
Reports
Settings

The correction

Change the scope

Run 08-A · Water · 412 bills · class R

All bills | Run bills | One-time bill | Budget Bills | Fix a bill | Past billing runs | Print bills

1 What and why — 2 The correction — 3 What it changes — 4 Generate and send

412 bills are affected. 389 go up and 23 go down, for a net of +$4,593.80.

1 WHICH BILLS, RESOLVED ONCE

Scope	**Bill run 08-A** · 01–31 Aug 2026
Narrowed by	Service **Water**
Bills resolved	**412** of 1,263 in the run
Frozen	**At preview** — a list of bills, not a query

A query re-run later could pick up a bill nobody approved.

2 WHICH WAY THEY MOVE

	BILLS	TOTAL
Credited in full	412	$99,812.40
Replacements issued	412	$104,406.20
↑ Owing more	389	**+$4,812.60**
↓ Owing less	23	**−$218.80**
Net across the batch		**+$4,593.80**

The two directions are never collapsed into the net.

3 ROWS THAT NEED A HUMAN EYE

	BILLS	
Already paid in full	24	List them
Posting period is closed	7	List them
Beyond the 12-month cap	0	Removed from the set
Nets to zero	0	

A warning keeps the bill in; a refusal takes it out.

Nothing is written.

← Back — See what it changes →

v2.235.0

1440 × desktop

The approved set is a list, not a query	Resolved **once, at preview**, and frozen. Re-running the query at execute time could pick up a bill issued in between, which nobody approved.
↑ and ↓ are not optional, and a test proves it	No code path emits the net without them, and the refusal was proved **red against a build that showed only the net**. **389 owing more and 23 owing less** is a different conversation from *net +$4,593.80* — and the net is what gets signed.
The totals tie out, and that is asserted	Σ credit − Σ replacement must equal Σ net, and Σ net the sum of the per-bill deltas. Drift between a total and its parts is otherwise invisible.
A warning and a refusal are drawn differently on purpose	**24 already paid** is a warning and stays in. **Beyond the cap** is a refusal and comes out. Listing both as *issues* would leave a clerk unsure what they are approving.

26.4 Goodwill | STEP TWO · CLASS C Exists behind S62

Billing › Fix a bill · step 2 of 4 · the bill is right

“The bill is correct and I want to give them something back anyway.”

> REDESIGN drawn **2026-09-12**, still unbuilt at Version 35 (2026-09-13), drawn to the Product Owner’s 2026-09-12 four-step wizard ruling and the same day’s confirmation that approval moves the books. **NEVER BUILT.** Today the product ships the *Fix a bill* tab, screen 13 in §17.

SMARTBillHub

Billing › Fix a bill

Search ? α

Demo Water & Light
demoutility

- Dashboard
- Plans & Rates
- Premises
- Assets
- Accounts
- Reads
- Billing
- Payments
- Work
- Finance
- Runbooks
- Reports
- Settings

The correction

Change the reason

Goodwill · class C

All bills | Run bills | One-time bill | Budget Bills | Fix a bill | Past billing runs | Print bills

1 What and why | 2 The correction | 3 What it changes | 4 Generate and send

The bill does not change. A $40.00 credit goes on the account and comes off their next bill.

1 THE CREDIT

	AMOUNT
Goodwill credit, agreed with the customer	**$40.00**
Tax on it	**$0.00**
Credit on the account	**$40.00**

A goodwill credit excludes tax — the tax is owed whatever the utility forgoes.

2 WHAT DOES NOT HAPPEN

Bill BIL-2609-0188	**Unchanged**

Its revenue and its tax	**Untouched**
A credit note against it	**None**
Their next bill	Carries the **$40.00** as a credit line

Your ruling: *“you dont touch the bill... everything remain as it is.”*

— NO PAYMENTS TO MOVE

The bill is untouched, so the money that met it stays where it is.

— NO TAX TO GIVE BACK

Nothing on the bill is reversed, so no remittance basis moved.

Nothing is written until an approver signs.

← Back See what it changes →

v2.235.0

1440 × desktop

The two absent panels are drawn, not omitted	A clerk who sees no payments panel should read **why there isn’t one**, not wonder whether the screen forgot. Both reasons are the same shape: nothing on the bill moved.
Group 2 exists because the obvious reading is wrong	An earlier draft called class C *a partial credit note referencing the original*. **Your ruling superseded that the same day** — *“it will exclude the tax; you dont touch the bill”* — so the screen spells out what does not happen.
A credit is not a refund	**It comes off their next bill**, not as a cheque. A clerk who promises one creates a complaint; the refund path is a different door.
Tax excluded, and a tax-inclusive figure is refused	Not split by a guess — *only the caller knows which authorities and which rates went into it.*

26.5 The page only

STEP TWO · CLASS D Exists behind S63

Billing › Fix a bill · step 2 of 4 · the page is wrong

“We spelt their name wrong. Send it again, correctly.”

REDESIGN drawn **2026-09-12**, still unbuilt at Version 35 (2026-09-13), drawn to the Product Owner’s 2026-09-12 four-step wizard ruling and the same day’s confirmation that approval moves the books. **NEVER BUILT.** Today the product ships the *Fix a bill* tab, screen 13 in §17.

SMARTBillHub

Billing › Fix a bill

Search ? α

Demo Water & Light
demoutility

- Dashboard
- Plans & Rates
- Premises
- Assets
- Accounts

The correction

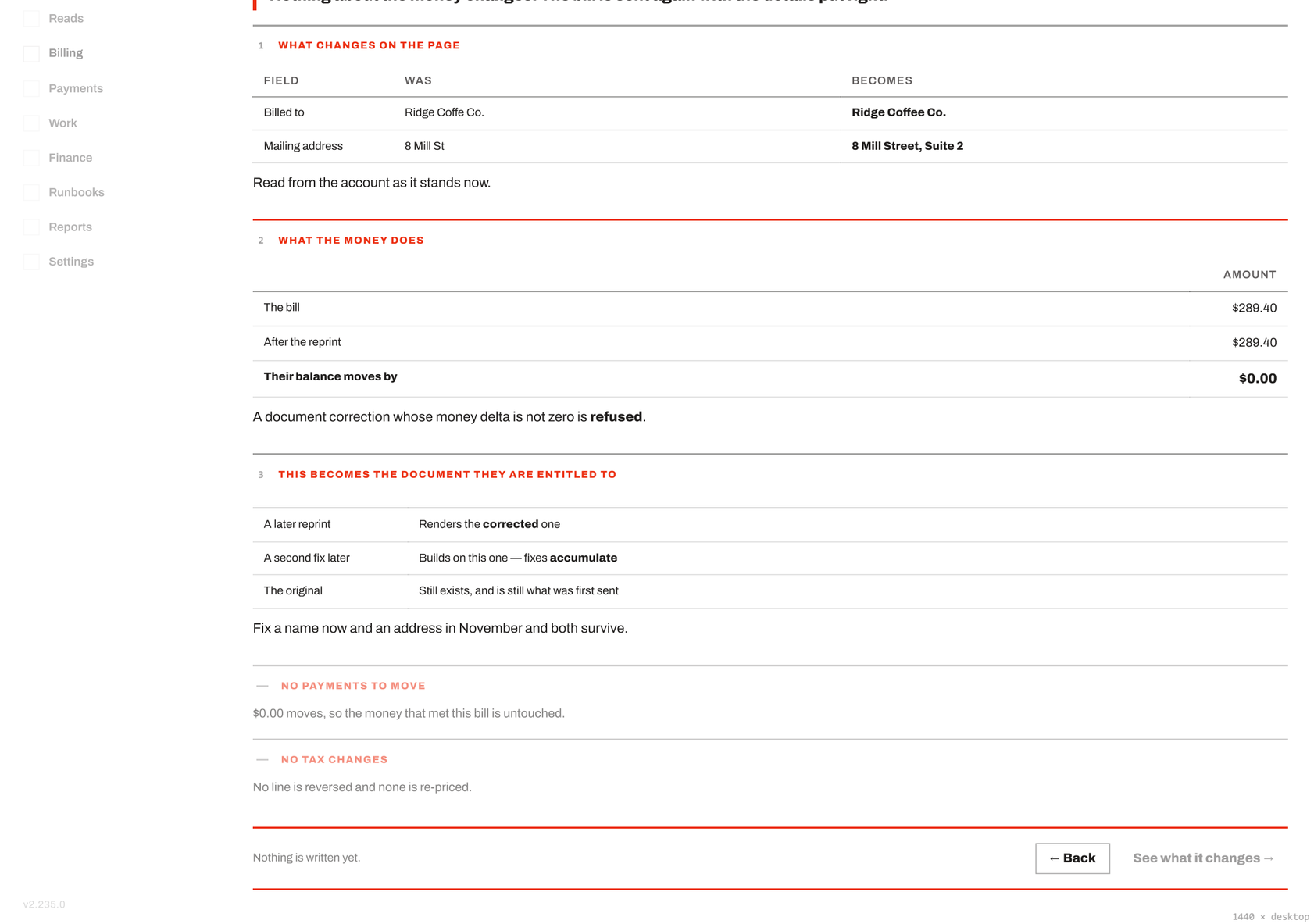
Change the reason

Customer name · class D

All bills Run bills One-time bill Budget Bills Fix a bill Past billing runs Print bills

1 What and why 2 The correction 3 What it changes 4 Generate and send

Nothing about the money changes. The bill is sent again with the details put right.



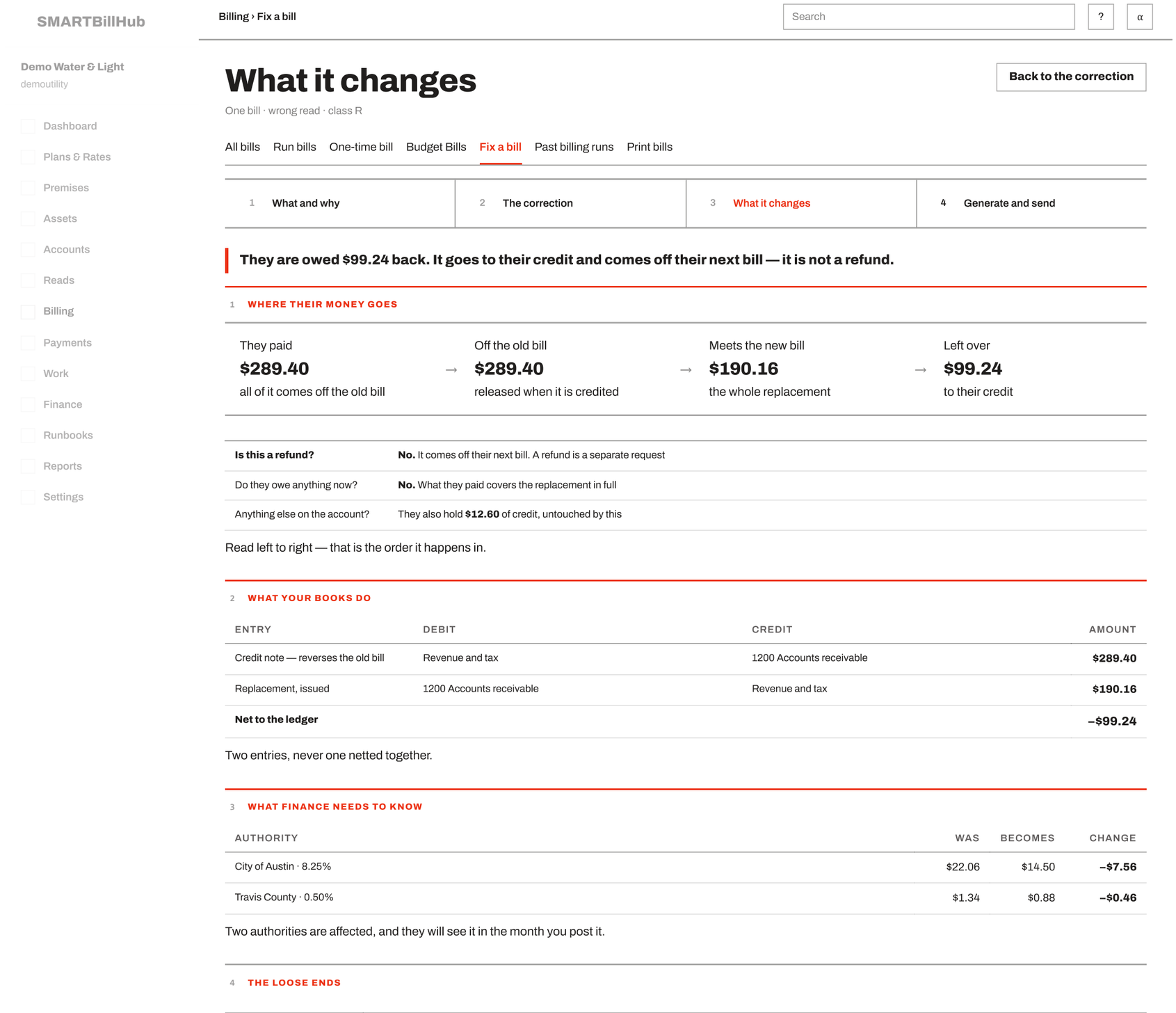
The $0.00 is drawn, not implied	It is the whole claim this class makes, and a case whose delta is **not** zero **refuses** — which is what stops a rate error being discharged with a reprint.
Group 3 is a fact worth a clerk knowing	A reissue becomes **the document the customer is entitled to**, and reissues **accumulate** — each is composed from the one before rather than the original, which *"would put a misspelt name back"*. That was a real defect.
It reads master data; it does not write it	The name was already fixed on the account. This decides what an **already-sent document** says — which is why *master data only* is a routed-away row.

26.6 What it changes | STEP THREE Exists behind S68

Billing › Fix a bill · step 3 of 4 · one bill

“What does the customer get, what happens to what they paid, and what do my books do?”

`REDESIGN` drawn **2026-09-12**, still unbuilt at Version 35 (2026-09-13), drawn to the Product Owner’s 2026-09-12 four-step wizard ruling and the same day’s confirmation that approval moves the books. **NEVER BUILT.** Today the product ships the *Fix a bill* tab, screen 13 in §17.



Late fee on this bill	**None.** ⚠ Not charged yet — and a correction will not put one back
Their discount	**$8.40 on both bills**
Due date	**15 Oct** — the original bill's, not a new 21 days

A changed preview invalidates the signature.

← Back | Send for approval →

v2.235.0

1440 × desktop

Four panels, and the first one is the phone call	**Their money · your books · the tax · the loose ends.** Each answers a different person's question, and the clerk's comes first.
The money is drawn moving, not tabulated	The engine credits the bill, releases the payment from it, then lets the replacement draw it down — and *"this call's position is the rule"*. **The panel reads left to right in that same order**, so what a clerk sees is what happens. The first box carries one number under two labels, which is how the engine serves it.
⚠ "It is not a refund" is the most important line here	Money returning to credit **comes off the next bill**. A clerk who says *we are refunding you* creates a complaint when no cheque arrives — and there IS a refund path, through a different door.
The other credit is signed, and a debt settles nothing	The engine floors it at zero for the arithmetic but serves it **signed**, precisely so the screen can state a debt. A debt cannot pay for the replacement and stays recorded where it is. If payments cannot be read the panel is **absent, never zeroed**.
⚠ The late-fee row is a stated absence	Nothing assesses a late fee today — the function **has no production caller at all**. When it lands, a credit note will reverse a fee line **in full** and the replacement will not put it back, because the re-pricing covers tariff, taxes, plan fees and discounts and **not** a late fee. Latent now, live the day assessment ships.

26.7 What it changes · many | STEP THREE Exists behind S68 S69

Billing › Fix a bill · step 3 of 4 · several months, or a batch

"Six months redone. What is the one number I give them?"

REDESIGN drawn **2026-09-12**, still unbuilt at Version 35 (2026-09-13), drawn to the Product Owner's 2026-09-12 four-step wizard ruling and the same day's confirmation that approval moves the books. **NEVER BUILT.** Today the product ships the *Fix a bill* tab, screen 13 in §17.

SMARTBillHub

Billing › Fix a bill

Search ? α

Demo Water & Light
demoutility

- Dashboard
- Plans & Rates
- Premises
- Assets
- Accounts
- Reads

What it changes

Back to the correction

Six months · 100377 Ridge Coffee Co.

All bills | Run bills | One-time bill | Budget Bills | Fix a bill | Past billing runs | Print bills

1 What and why | 2 The correction | 3 What it changes | 4 Generate and send

They will owe $159.80 more. But $709.40 is still to pay, and $549.60 of that was already unpaid.

1 **MONTH BY MONTH**

BILL	THEY PAID	NEW BILL	MOVES ACROSS	TO CREDIT	STILL TO PAY
Mar	$241.80	$268.40	$241.80	$0.00	**$26.60**
Apr	$238.10	$264.30	$238.10	$0.00	**$26.20**
May	$244.60	$271.50	$244.60	$0.00	**$26.90**
Jun · part paid	$180.00	$266.20	$180.00	$0.00	**$86.20**
Jul · unpaid	$0.00	$274.40	$0.00	$0.00	**$274.40**
Aug · unpaid	$0.00	$269.10	$0.00	$0.00	**$269.10**
Six months	**$904.50**	**$1,613.90**	**$904.50**	**$0.00**	**$709.40**

Each month settles against its own payment — money never moves between months.

⚠ **TWO BIG NUMBERS, AND ONLY ONE IS THE CORRECTION**

$159.80	What the six months **went up by**. This is the correction
$709.40	**$159.80** of increase plus **$549.60** already unpaid. Mostly **not** this correction

A customer will hear the larger one.

2 **WHAT YOUR BOOKS DO**

ENTRY	BILLS	AMOUNT
Credit notes — the six old bills reversed	6	**$1,454.10**
Replacements issued	6	**$1,613.90**
Net to the ledger		**+$159.80**

Twelve entries, and each bill commits on its own.

3 **WHAT FINANCE NEEDS TO KNOW**

AUTHORITY	WAS	BECOMES	CHANGE
City of Austin · 8.25%	$110.82	$122.99	**+$12.17**
Travis County · 0.50%	$6.72	$7.45	**+$0.73**

Summed across six months, still split by authority.

4 **THE LOOSE ENDS**

Late fees	**None on any of the six.** ⚠ Not charged yet
Discounts	**On every month**, each on that month's enrolment
Due dates	**Each keeps its original** — no fresh clock

One approval for all six.

← Back Send 6 for approval →

This is the screen the bigger-period question asked for	The release-and-re-apply runs **per bill**, so March's payment is released and re-met by March's replacement. A single batch figure cannot answer the call that comes in, which is about one month.
⚠ Two large numbers that mean different things	**$159.80** is what the correction did. **$709.40** is what is still to pay, and **$549.60** of it is July and August — bills never paid, which this correction did not cause. A screen showing only the larger figure has the clerk apologising for something they did not do.
Part-paid June is the row worth reading twice	$180.00 of $239.90 was paid. All of it moves across and **$86.20** is still to pay — the unpaid $59.90 plus the $26.30 increase. One row, two causes, both visible.
Twelve commits, not one transaction	**The atomicity unit is one bill.** A month that fails leaves **zero artifacts** and stays correctable; one that succeeded is never reversed because a later one failed — a posted pair cannot be un-posted and a delivered bill cannot be un-sent.
The batch version is the same four panels	For a run or a cycle the month rows become **bill rows, collapsed and searchable**. Nothing about the shape changes with the scale.

26.8 Generate and send | STEP FOUR New build S85

Billing › Fix a bill · step 4 of 4 · all three classes

"It is approved. Create the documents and send them."

> REDESIGN drawn **2026-09-12**, still unbuilt at Version 35 (2026-09-13), drawn to the Product Owner's 2026-09-12 four-step wizard ruling and the same day's confirmation that approval moves the books. **NEVER BUILT.** Today the product ships the *Fix a bill* tab, screen 13 in §17.

SMARTBillHub

Billing › Fix a bill

Search ? α

Demo Water & Light
demoutility

- Dashboard
- Plans & Rates
- Premises
- Assets
- Accounts
- Reads
- Billing
- Payments
- Work
- Finance
- Runbooks
- Reports
- Settings

Generate and send

Back

Six months · approved by M. Okonkwo, 12 Sep

All bills | Run bills | One-time bill | Budget Bills | Fix a bill | Past billing runs | Print bills

1 What and why | 2 The correction | 3 What it changes | 4 Generate and send

This is the first thing that writes. Twelve documents, and twelve entries in your books.

1 WHAT WILL BE CREATED

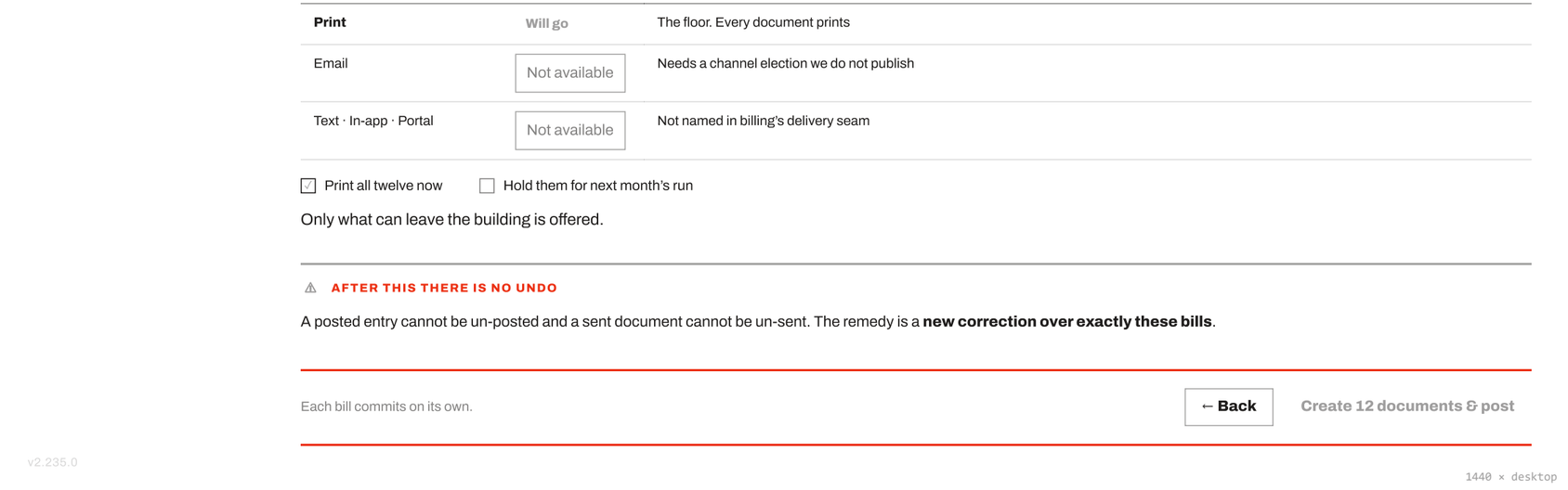
	COUNT	THE CUSTOMER GETS
Credit notes	**6**	One per old bill, cancelling it
Replacement bills	**6**	At the corrected amount, original due date
Ledger entries	**12**	Nothing — these are yours

See a credit note → **See a replacement →** Neither writes anything.

Both documents go — a replacement alone leaves two bills for one month.

2 HOW THEY GO

CHANNEL NOW WHY



Approval signs; this button writes	Confirmed by the Product Owner, 2026-09-12: **the books are updated here**, inside the execute transaction, not by a later batch job. If the books refuse, **nothing is written** and the bill stays correctable.
What each class produces here is different	**R** — a credit note *and* a replacement, per bill. **C** — a credit note only. **D** — a reissued document only. Only group 1's counts change with the class.
Both documents go, and that is not optional	A credit note cancels; a replacement charges. Sending only the replacement leaves a customer holding **two bills for one month** with nothing explaining the first.
Send offers only what leaves the building	**Print is the floor and always goes.** Email needs a channel election we do not publish; text, in-app and portal are not named in billing's delivery seam at all.
⚠ No undo, and the footer says how it commits	There is no rollback and none may be added. And **the atomicity unit is one bill** — a clerk who read *all or nothing* and saw eleven of twelve succeed would think the product broken.

07

Past billing runs

One table, and it is nearly free. `GET /billing/runs` is served, `PastRunsPanel` is written and wired into the wizard, and the actor column is populated and on the wire — so the blocker this screen names on itself (*"nothing would list it"* / *"no actor column"*) is **false in both halves that matter**. This is commitment `BIL-S1` drawn as a table with filters.

What a clerk actually comes here for

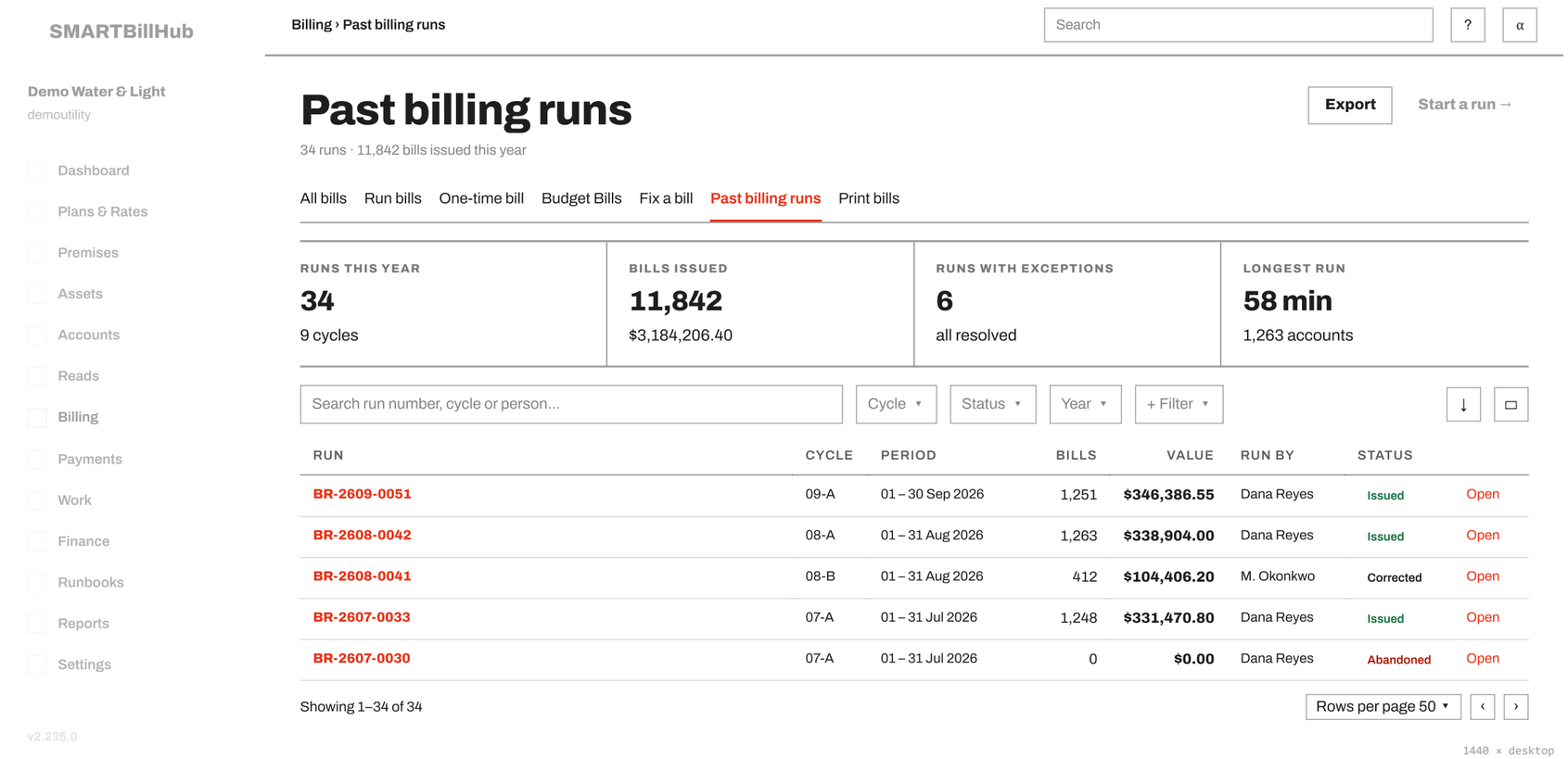
Three questions, and the table answers them in one row each: **what did we bill last month**, **who signed it off**, and **is this month's run continuous with it**. The third is why the period columns sit beside each other rather than being collapsed into one span.

27 **Past runs** | ALREADY SERVED New build S76 Exists behind S1

Billing › Past billing runs · `GET /billing/runs`

“What did we bill last month, who signed it off, and does it join up with this month?”

> REDESIGN drawn **2026-09-12**, still unbuilt at Version 35 (2026-09-13), drawn to BR-BIL-065 and the run-history read that already exists. **NEVER BUILT.** Today the product ships a grey panel reading *“It arrives with the billing run work”* — screen 10 in §17.



Run by is a real column, not a wish	`bill_runs.run_by` is populated and on the wire. The screen’s own note saying otherwise is **false**, and that is the finding rather than the table.
Two runs in one period is not an error	**08-A** and **08-B** both cover August because a utility bills in cycles, not months. Collapsing the period into the cycle name would hide exactly that, and it is the thing a clerk checks continuity against.
Abandoned is drawn, and at zero	**BR-2607-0030** issued nothing and is still a row. A run that vanished would leave a clerk unable to explain a gap in the numbering to an auditor.
Corrected is a status the correction workflow sets	Run **08-B** reads *Corrected* because §06 screen 26.5 restated its 412 water bills. The two sections meet here, and this is the only screen where a clerk can see that a whole run was redone.
The link out is the point of the row	Every row opens the run it names — its exceptions, its totals and its sign-off. A table you cannot drill into is a report, and this is a workspace.

08

Bill printing

Hand one period's bills to a printer as one archive. **The engine here is the best-built thing in this audit** — and it was proved in a real browser, not only in tests: the archive was downloaded, opened, its PDFs counted, its manifest rows counted, and one row checked against the bill it names. Two screens: **the tab** — a form over a searchable table of every bill in the period — and the list of what will not print. **Print bills is the seventh Billing tab**, ruled by the Product Owner on 2026-09-12: *"We will need Print bills as tab at Billing level."* The code reaches it from the Billing header instead, and its reasoning was about the bills-list toolbar rather than about tabs — so the ruling stands and the strip is seven wide on every drawing in §02 and §04–§08. ⚠ **The as-built drawings in §17 keep six**, because that is what is there today.

What it costs, measured rather than estimated

FIGURE	VALUE	WHAT IT IS FOR
Per bill	**0.012 s**	So the screen can say *"412 bills · about 5 seconds"* **before** anything is pressed. A screen without that half has to say *press it and find out*, and finding out costs a worker several minutes and produces nothing.
Pages per bill	**6**	The page estimate a print shop quotes from.
Ceiling	**25,000 bills**	Above it the screen refuses with the number, rather than starting something that will not finish.

What comes out

IN THE ARCHIVE	WHAT IT IS
`bills/`	One PDF per bill, named from its bill number
`manifest.csv`	One row per bill, for the print shop to reconcile against
`held-back.csv`	**Every bill that did not go, with its reason** — screen 28.1
`README.txt`	What this set is, when it was made, and what it does not claim

The archive claims nothing about the post it cannot keep. The screen says in words that **no address was CASS-certified, NCOA-processed or barcoded**, because a mail house's default assumption is otherwise and **a silence here is a claim**. The screen adds nothing of its own to that list and edits none of it.

And it changes no address

No form, no field, no save. **An address moves when the account holder says so, on the account** — this screen is a reader. A clerk who spots a bad address on 28.1 follows the link out and comes back.

---

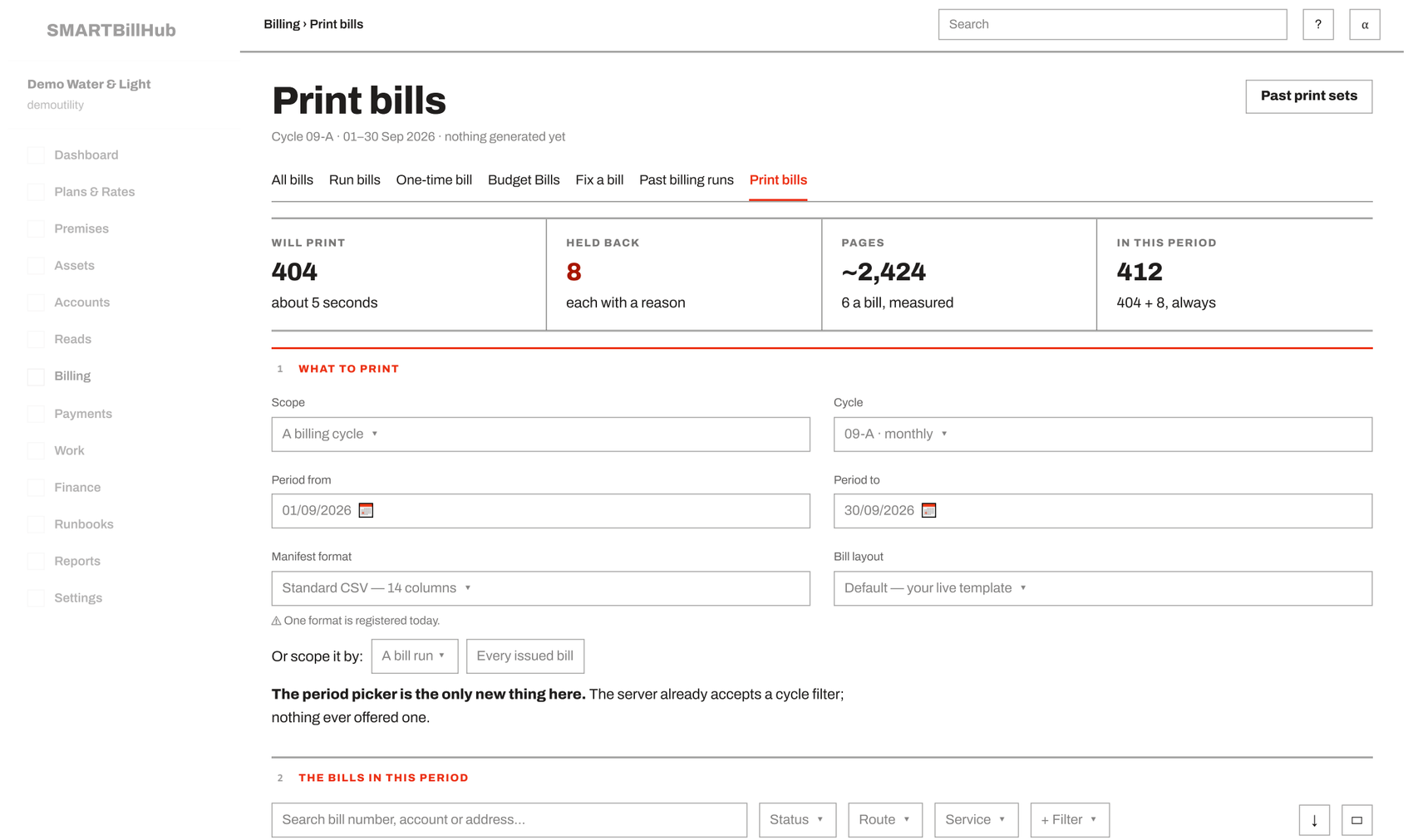
28 Print bills | A TAB, A TABLE, A BUTTON

New build S93 Exists behind S64 S65 S70

Billing › Print bills · the seventh tab

“Give me September’s bills as one archive for the print shop — and let me check a bill is in it before I send it.”

> REDESIGN drawn **2026-09-12**, still unbuilt at Version 35 (2026-09-13), drawn to the Product Owner’s 2026-09-12 ruling making Print bills a Billing tab. **NEVER BUILT.** Today the product ships the *Print bills* screen, screen 15 in §17.



The period picker is the only new thing here. The server already accepts a cycle filter; nothing ever offered one.

BILL	ACCOUNT	PERIOD	AMOUNT	PAGES	WILL IT PRINT
BIL-2609-0181	100377 · Ridge Coffee Co.	01–30 Sep	**$289.40**	6	Will print
BIL-2609-0184	100412 · Navarro, P.	01–30 Sep	**$96.00**	6	Will print
BIL-2609-0188	100377 · Ridge Coffee Co.	01–30 Sep	**$142.30**	—	Held · address incomplete
BIL-2609-0191	100455 · Okafor, D.	01–30 Sep	**$74.00**	6	Will print
BIL-2609-0244	100510 · Alder Holdings	01–30 Sep	**$1,204.60**	—	Held · document refused
BIL-2609-0270	100588 · Nguyen, T.	01–30 Sep	**$118.20**	?	Will print

Showing 1–50 of 412 Rows per page 50 ▾ ‹ ›

See the 8 held back → The same list ships in the archive as `held-back.csv`.

Search it before you generate — the question is usually about one customer's bill.

⚠ **WHAT THIS ARCHIVE DOES NOT CLAIM**

Addresses	**Not CASS-certified, not NCOA-processed, not barcoded**
This screen	Changes **no** address — no form, no field, no save

Said out loud because a mail house assumes otherwise, and a silence is a claim.

Runs in the background and produces one archive to download. **Generate the print set**

v2.235.0

1440 × desktop

⚠ It is a TAB now, and that is a change from what the code does	Product Owner, 2026-09-12: *"We will need Print bills as tab at Billing level."* Today it is a route reached from the Billing header, the posture the pending queue takes, and the code's own reasoning for that was the bills-list toolbar: its download glyph is already the bills CSV, and *"squeezing a print hand-over into that slot would give a bookkeeper asking for a spreadsheet a zip of PDFs."* **A tab is not that slot** — it is a seventh destination beside Run bills and Fix a bill, so the objection does not apply and the ruling stands. **The strip is seven wide on every drawing in §02 and §04–§07.**
The table was missing, and it is the half a clerk uses	An earlier version of this screen had the form, the counts and the button and **no list**. A clerk's commonest question here is not *how many* — it is **is this customer's bill in the set**, usually because that customer has rung. Without a searchable table the answer is *generate it and look in the zip*.
One table for the whole population, not two	**412 rows** — the 404 that will print and the 8 that will not, with the reason in the row. Two tables would make a clerk searching for one bill look twice, and the counts would stop reconciling the moment a filter was applied.
Search, filters and a column selector, like every other table here	The toolbar is the same set as §03's selection layer: one search, filters, export and the column selector. **Column widths are pinned**, so a long account name cannot push the status column off the row.
The counts always add up, and the band says so	**404 + 8 = 412.** A set that printed 404 of 412 without saying so sends a print shop a short run and tells nobody.
The time and page estimates come from a measurement	**0.012 s and 6 pages per bill**, measured rather than guessed — which is what lets the band say *about 5 seconds* and *~2,424 pages* instead of *this may take a while*. The ceiling is **25,000 bills** and above it the screen refuses with the number.
The `?` in the pages column is honest	**BIL-2609-0270** will print but its pages could not be counted, so it is excluded from the estimate and not from the set. A zero there would understate the job by six pages; a blank would look like a defect.

⚠ One manifest format exists	The registry is built and holds **exactly one entry**. The field is drawn because a second format is a *registration* rather than a change to the print set — but today it has one option and the helper says so.
The unavailable notes are drawn, not swallowed	The server sends them and the screen prints them verbatim. It adds nothing of its own and edits none of it, which is the only way that list stays true as the product changes.

28.1 Held back

EIGHT NAMED REASONS Exists behind S65 S70

Billing › Print bills · from *See the 8 held back* · also `held-back.csv`

“Eight bills are not printing. Which, why, and whose job is each one?”

REDESIGN drawn **2026-09-12**, still unbuilt at Version 35 (2026-09-13), drawn to the Product Owner’s 2026-09-12 ruling making Print bills a Billing tab. **NEVER BUILT.** Today the product ships the *Print bills* screen, screen 15 in §17.

SMARTBillHub

Billing › Print bills

Search ? α

Demo Water & Light
demoutility

- Dashboard
- Plans & Rates
- Premises
- Assets
- Accounts
- Reads
- Billing
- Payments
- Work
- Finance
- Runbooks
- Reports
- Settings

Held back

Download held-back.csv Generate the other 404

8 of 412 bills will not print

All bills | Run bills | One-time bill | Budget Bills | Fix a bill | Past billing runs | Print bills

1 WHY EACH ONE IS HELD, AND WHOSE JOB IT IS

REASON	WHAT IT MEANS	WHAT TO DO	BILLS
Address incomplete	A required line is missing	Fix the premises, then re-run	3
Address unreadable	Present but not resolvable to a deliverable form	Fix the premises	1
No bill number	The bill has no number to file under	A billing defect — raise it	0
Unsafe bill number	Characters a filename may not carry	A billing defect	0
Duplicate filename	Two bills would write the same file	A billing defect	0
Document refused	The renderer refused this bill	Open the bill and see why	2
Original unreadable	A kept original could not be read back	Re-issue the document	1
Page count unknown	The PDF parsed but its pages could not be counted	Prints anyway — left out of the page estimate	1

Eight named reasons, and every held bill carries exactly one. Three of them are ours to fix, not a clerk’s.

Search bill number or account… Reason ▾ + Filter ▾ ↓ ▭

BILL	ACCOUNT	REASON	
BIL-2609-0188	100377 · Ridge Coffee Co.	Address incomplete	Open the premises →
BIL-2609-0204	100412 · Navarro, P.	Address incomplete	Open the premises →
BIL-2609-0219	100455 · Okafor, D.	Address incomplete	Open the premises →
BIL-2609-0231	100498 · Hale, J.	Address unreadable	Open the premises →

BIL-2609-0244	100510 · Alder Holdings	Document refused	Open the bill →
BIL-2609-0251	100533 · Whitlock, R.	Document refused	Open the bill →
BIL-2609-0266	100571 · Bayou Laundry	Original unreadable	Re-issue →
BIL-2609-0270	100588 · Nguyen, T.	Page count unknown	Prints anyway

⚠ **THIS SCREEN FIXES NOTHING, AND SAYS SO**

Every row links **out**. An address moves on the premises and a document is re-issued from the bill — **neither is editable here**, because billing may not write another module's record.

The eight stay unprinted until their cause is fixed upstream.

← Back Generate the other 404

v2.235.0

1440 × desktop

Eight reasons, and they are the code's own	Address incomplete · address unreadable · no bill number · unsafe bill number · duplicate filename · document refused · original unreadable · page count unknown. **Every held bill carries exactly one**, so the reason column can never be blank.
Three are billing defects, not clerk work	**No bill number**, **unsafe bill number** and **duplicate filename** are ours to fix, and the *what to do* column says so rather than sending a clerk to look at a premises that is perfectly fine. All three read **0** today, which is the answer you want.
The last reason still prints	**Page count unknown** excludes a bill from the page *estimate*, not from the set. Listing it under the same heading as a bill that will not go out would be the kind of small lie a print shop discovers first.
It links out; it does not edit	An address moves on the premises and a document is re-issued from the bill. **Billing may not write another module's record** — the same boundary the one-time wizard obeys when it posts a reading to Meter Reads.
The same list ships in the archive	`held-back.csv` carries these rows, so the print shop and the clerk read one list rather than two.

09

What happened when it was driven

The only section whose findings cannot be reconstructed from source — which is exactly why an invented one would be undetectable. So: the numbers first.

Read it as the smoke test a build has to pass. With nothing stubbed, **not one billing screen could be reached**; with three boot reads stubbed, four rendered and six threw. So **"the screen renders at all" is the first acceptance test**, before anything in §12 is judged done.

SCREENS WALKED	THREW	NOT REACHABLE	JOURNEYS WALKED	WRITES EXERCISED
4	**6**	**5**	**0**	**0**
of 15	to a stack-trace page	need a created run	of 350 in `08-qa/01-e2e/journeys/`	no backend

No journey was walked end to end, and none could be. The repository keeps 350 journey files and they are the handoff contracts, not illustrations — `J2-read-to-cash`, `J12-nsf-life`, `J19-month-end-close`. Every one of them needs a backend to reach step 2. There is no Django and no PostgreSQL in this environment, by the user's decision, so **the six-field journey record this section is supposed to hold is empty**, and I am not going to fill it with described clicks.

What follows is a **screen walk**, not a journey walk: I opened each address in Chromium at 1440 and recorded what rendered. That catches door landings, the four states, the window fit and anything that throws on load. It cannot catch a write that fails, a panel blank on real data, or a swallowed error — and those are the findings I most wanted.

Pass 1 — nothing stubbed. Not one billing screen could be reached.

JOURNEY	screen walk · every billing address
STEP	1 — open the address
SCREEN	all ten routes, one after another
DID	`GET http://127.0.0.1:5199/demoutility/billing…` in Chromium 1194, viewport 1440×1000
EXPECTED	the shell, and each screen's failed-read state
GOT	HTTP 200 on all ten, and **the same thing every time**: the sentence *"We couldn't load your regional settings."* and a *Retry* button. `navCount = 0` **— the left rail does not render either.** The only thing that identified the screen was its id badge in the corner: `BIL-01`, `BIL-01.one-time`, `BIL-02`, `BIL-06`, `BIL-07`, `BIL-05`, `BIL-05.document`.
SEVERITY	BY DESIGN, AND WORTH KNOWING

The cause is documented in full at `ShellWithSession.tsx:344-381`: one boot read, `GET /shell/display` with `retry:false`, gates the entire shell, and on error it returns a bare state-frame *instead of* `<TenantRegionalProvider><AppShell>`. The reasoning is written out — *"carrying on would print timestamps in a zone nobody chose, which is the defect"* — and it is hard-won law 3 applied deliberately.

The history beside it is the part worth carrying forward. That same frame rendered over *every screen in the product* from 2026-08-21 to 2026-09-06, because the two reads it replaced were admin-gated and a Billing Clerk has no cell for them. `DEF-SHL-4272`. The file's own account: *"a clerk signed in successfully, reached `/dashboard`, and the block below rendered `settings.regional.failedTitle` over every screen in the product, with a Retry that could never succeed. **Nothing saw it.** Every frontend test passes formatting as a prop or seeds the query cache; every backend test drives services or an admin fixture; and **every walkthrough in this repo signs in as a system admin.**"*

> **Which is a testability finding, not a defect.** A billing screen cannot be seen at all without a working `/shell/display`. That is the reason a sixteen-day total lockout went unnoticed, and it is the reason my own first pass produced ten identical results. The one observation I will add: **the failure frame carries no navigation.** Retry is the only control. For a transient 500 that is the

documented intent; it is recorded here as the shape of the state, not as a new defect.

Pass 2 — three boot reads stubbed. Four render, six throw.

My own instrument was wrong twice, and I caught both by reading output rather than exit codes. First, the route glob `**/api/**` also matched the app's own `/src/api/*` modules, so vite served JSON where JavaScript was expected and every screen came back blank — the walk would have reported ten failures, all mine. Second, I wrapped the stub payloads as `{data: …}`; `ApiResponse.data` *is* the parsed body, so `requireData` handed the screens my wrapper. Both are the shape this repository calls a check that cannot fail, and both are recorded here because the next person will hit them too.

#	SCREEN	GOT	SEVERITY	UNDERTAKEN BY
01	All bills	**Rendered.** Title measured **40px/800**; 13-item rail, *Billing* on a solid `#ec3013` fill; six tabs with a 2px accent underline; search, filter, download glyph, column chooser and pager all present; `scrollWidth − clientWidth = 0`. Money formatted to 2dp (`$342,180.00`, `$84.00`). Five disabled controls: the download glyph, two sort arrows, two pager arrows.	HEALTHY	— healthy
02	One-time bill	**Rendered.** Both step-1 buttons correctly disabled with no account chosen. Rollup read *"Not counted yet"* ×3 with dotted underlines — the stated-absence style, correct for an empty rollup.	HEALTHY	— healthy
03	Budget Bills	**Rendered**, empty list.	HEALTHY	— healthy
05	Past billing runs	**Rendered** — and this is the confirmation the audit was worth running for. A full-width grey panel reading *"Past billing runs list every run with what it produced and who signed it off. **It arrives with the billing run work.**"* over a module whose run wizard is live and whose run list is already mounted on step 1 of that wizard.	THE JOURNEY COMPLETES, WRONGLY	BIL-S12 New build · BIL-S1 Exists behind
04	Fix a bill	`TypeError: Cannot read properties of undefined (reading 'map')` → *"Unexpected Application Error!"* and a JavaScript stack trace, no left rail.	BLOCKS THE JOURNEY	BIL-S9 New build
06	Run bills wizard	`(servicesQuery.data ?? []).filter is not a function`. **This one is my fault** — my generic fixture returned an object where an array is served. Worth one observation anyway: `?? []` defends against null, not against a wrong type.	MY FIXTURE	— my instrument, not the product
07	Pending queue	`TypeError: … (reading 'open')` → stack-trace page.	BLOCKS THE JOURNEY	BIL-S9 New build
08	Print bills	`TypeError: … (reading 'join')` → stack-trace page.	BLOCKS THE JOURNEY	BIL-S9 New build
09	Bill detail	`TypeError: … (reading 'previous_balance')`, then `(reading 'length')` → stack-trace page.	BLOCKS THE JOURNEY	BIL-S9 New build
10	Bill document	`TypeError: … (reading 'startsWith') at tenantPathFrom` → stack-trace page.	BLOCKS THE JOURNEY	BIL-S9 New build

The honest scope of that. The *trigger* for all six was my minimal payload, and a screen that throws on a fixture missing an optional member is not proof the server can emit one. What is *not* fixture-dependent, and is a source fact:

There is no error boundary and no `errorElement` anywhere in the route tree. A grep across the whole non-test front end for `componentDidCatch`, `getDerivedStateFromError` and `errorElement` returns **exactly one file**: `components/MoneyCell.tsx:326,330`. *Money rendering has a boundary and nothing else does.* `App.tsx` declares no `errorElement` on any of its routes.

So any render-time throw on any billing screen — from a key that moved, a null where an array was expected, a renderer reading a field the server renamed — presents a clerk with a JavaScript stack trace instead of A18 §4's failure state, *and takes the navigation with it.* Two prior instances are already filed and neither names the cause: `DEF-176` (*"THE WHOLE FINANCE SCREEN CRASHES on the seeded demo tenant"*) and `DEF-RBS-4352` (*"the runbook's report blocks do not match the renderer's payload contract, so a clerk cannot read the plan"*). **That second one is the same defect as the Fix-a-bill finding on screen 13** — which makes it a shape, not two bugs.

Not walked, and why

Every data-shape behaviour, because no real row reached a screen. The five wizard steps after step 1, each of which needs a created run. Every write in the module — create, preview, approve, execute, abandon, enrol, override, relevel, remove, build print set, issue, release. And the three swallowed-failure paths, which are the findings a walk would have settled beyond argument.
Every flow in this document is graded UNVERIFIED unless its row says| SEEN.

10

Absence register

Every row is built from the product's own user-facing copy — the sentences a person reads when something cannot be done — not from an auditor's impression of what is missing. **Every row gets exactly one ruling and nothing is left unresolved.**

Each ruling is either a commitment in §14 or a stated absence, and the difference is the whole point: a message a commitment will delete is **work**, and a message that stays is a **promise the product is keeping**. **The seven rotting holds are the ones to read first** — each is a refusal sitting over a screen that has since landed.

ABSENCE KEYS	HOLDING A DEAD CONTROL	ROTTING HOLDS	DEAD COPY	SELF-HEALED
78	**14**	**7**	**2**	**9**
61 reachable from 14 source files	visible and unusable	proven against current code	unreachable in 11 catalogues	derived from a 404, not hardcoded

> **The judgement rule, stated once and applied without exception:** a *recorded ruling* that a thing waits makes it **out of scope**, with the ruling quoted. *No ruling* makes it a **specification** naming what it waits on. The borderline case below is `billing.run.bulk.waiveRefused` — a disabled control whose reason is a *rule* (BR-BIL-060: a waiver is one account at a time, with a reason). A disabled control usually reads as a gap; this one is a decision, so the rule puts it out of scope with its argument quoted, and it is not a specification.

Method, and two false negatives I caught on the second pass

A literal-key grep under-reported by sixteen keys. `billing.run.reason.*`, `.absent.*` and `.finding.*` are built as template literals. The first pass called all sixteen dead. They are live.

And the dead-copy list was entirely wrong the first time. Seven keys have *zero* front-end call sites and are still rendered, because the **backend** ships them as `message_key` and `refusalText.ts:69` resolves them: `billing.budget.refused` ← `budget_api.py:243`, `billing.correction.refused` ← `corrections_api.py:297`, `billing.document.refused`, `billing.oneTime.refused`, `billing.templates.refused`, `billing.run.refused`, `billing.run.cycle_has_no_bill_date`. **A register built from front-end greps alone mislabels all seven.**

The seven rotting holds

A refusal with a clock in it: correct the day it is written, silently false the day the thing it points at lands. Each row below is proven against code at this commit.

#	WHAT THE PRODUCT TELLS A CLERK	WHY IT IS NO LONGER TRUE	CONTROL HELD?	UNDERTAKEN BY
R1	**Past runs, actor and money.** *"Who started a run, and what it came to in money, are not kept against the run — so neither can be shown here."* Rendered **unconditionally**, `billRunResume.tsx:416-419`, and stamped `ruleRef="BR-MET-078 — no actor column, and money is not served"`.	`models.py:316-318` declares `run_by = ForeignKey("identity.User")`. It is populated and on the wire: `api.py:1818` `"ran_by": str(existing.run_by_id)`, also `:681`, `:1999`. **The money half is still true** — no total column on `BillRun`. So it is half-rotten, *and the half that landed is denied inside a rule citation*, which is the form a lane is most likely to believe.	No — a panel	`BIL-S12` New build `BIL-S1` Exists behind
R2	**Usage on the account's Bills tab.** *"The design also shows the usage behind each bill on this tab. Usage is summed from meter reads, and that summary is not built — no number is drawn from nothing."* Unconditional, `BillsListPage.tsx:899-903`.	`GET /accounts/{id}/usage` is mounted — `metering/api.py:2965`, view `:2038`, whose docstring reads *"`meter_read_monthly_usage` is metering's table… the figure is Σ over `meter_reads`"*. That is precisely the summary the copy says is not built.	No — a panel	`BIL-S12` New build
R3	**Base charges in the run.** *"The run does not gather base charges or schedule names."* Unconditional **in every row** of step 2's Plans & Rates table, `billRunStep23.tsx:1347-1351`.	`gather.py:699-705` gathers base charges through a three-way precedence ladder (`_base_charge_for` `:221`, `_base_charge_code_for` `:416`). The *true* narrower statement sits in the adjacent comment — *"the findings **payload** carries neither"* — and the user-facing sentence	No — a cell	`BIL-S73` New read

R4	**The bill document.** *"The saved copy exists but cannot be opened yet… What is missing is only the page that opens it, which has not been built yet."* `BillDetailPage.tsx:831-841`.	The page shipped **2026-08-16**. `App.tsx:932-935` routes it and `:928` says so; `documents_api.py:454-456` serves it. *Caveat I can prove but not resolve:* the route **re-renders** rather than streaming frozen bytes, so a narrower sentence would still be true. The sentence as written is not.	No — a panel	`BIL-S12` New build
R5	**Park.** *"A billing run cannot be parked yet: **nothing would list it.** Use Save & exit — this link reopens exactly here."*	`PastRunsPanel` — a table of runs with an actions column and an `onOpen(runId)` door — is defined at `billRunResume.tsx:312-355` and **mounted on step 1 of the same wizard** (`BillRunWizard.tsx:1808`), fed by `GET /billing/runs`. **Parking is genuinely unbuilt** — `PARKED_HOME_PATHS` has no `bill_run` member — so the control is right to refuse. Only its *reason* is false, and it is the reason a clerk reads on six screens.	YES — ALL 6 STEPS	`BIL-S12` New build
R6	**Bills export.** *"Bills cannot be downloaded yet. No export is built for this list."*	**The copy is still true** — `bills` is not in `EXPORT_RESOURCES`. What has rotted is its *evidence*: `BillsListPage.tsx:660-663` asserts in bold *"there is no `apps/billing/exports.py` at all"*, and that file now exists — it registers `bill_run_releases`. **Flag for re-measurement, not deletion.**	YES — THE GLYPH	`BIL-S3` New build
R7	**Bill not found.** *"…or the endpoint that reads a single bill has not been built yet — the two answer the same way, so neither is claimed over the other."* `BillDetailPage.tsx:256`, and unlike its nine siblings it is **not** gated on route-absence: it renders on any 404.	`invoices/<invoice_id>` is mounted at `api.py:2021`. The second disjunct is now false, so the sentence offers a clerk a possibility that cannot occur.	No	`BIL-S12` New build

Dead copy — two keys, eleven catalogues, no defect row

KEY	THE STRING	WHY IT CAN NEVER RENDER	UNDERTAKEN BY
`billing.arrives.run`	*"Running a billing run is a guided screen of its own: pick the cycle, check what was found, review the customers, see the totals, sign off, and send the bills out. It arrives with the billing run work."*	The router matches `/billing/run` to `BillRunWizard` (`App.tsx:951`) **before** `billing/:tab` (`:953`), so tab id `run` never reaches the fallback. And the string is a description of the shipped wizard, written as a promise.	`BIL-S88` New build
`billing.arrives.oneTime`	*"A one-time bill charges a single account outside the cycle. It uses the same run in single-account mode, and arrives with it."*	`active === "one-time"` is caught at `BillsListPage.tsx:1058` and renders `<OneTimeBillTab/>`. The `one-time` ternary inside the fallback at `:1108-1110` is **dead code**.	`BIL-S88` New build

Latent, and worth one line: no `billing.arrives.budget` or `billing.arrives.fix` key exists at all. Both tabs are built, so neither reaches the fallback — but if either ever did, `t()` would render the raw key string on screen. The fallback is not total.

And one lost placeholder, found while proving the dead-copy list

`templates_api.py:150` sends `params={"detail": str(broken)}`, and **no catalogue has a `{{detail}}` placeholder** in `billing.templates.refused` — all eleven checked; the English reads *"That bill design can't be used as asked. Check its version and try again."* The detail is then deleted platform-wide anyway: `api/client.ts:202` `delete rest.detail;` — *"BR-SHL-020 r2 — never rendered, still in the audit row."* **So the server's specific reason never reaches a clerk on any of the six `billing.*.refused` keys.** An e2e spec still asserts the opposite in prose (`CR-007-templates.spec.ts:821`), quoting wording and a placeholder gone from all eleven files.

The nine that healed themselves — the pattern that works

Nine refusals are gated on a live 404 probe (`runRouteAbsent` / `listRouteAbsent`), so each became **unreachable the day its route landed**, with no wording left standing and nothing to edit. The intent is stated outright at `BillsListPage.tsx:460`: *"the first sentence stops being reachable the moment `GET /invoices` is registered."*

`bills.awaits.endpoint` · `bills.failed.noEndpoint` ← `api.py:2020` · `run.step2.gap` ← `/findings` · `run.step3.gap` ← `/exceptions` · `run.shadow.gap` ← `/pricing-shadow` · `run.step4.noEndpoint` ← `/calculate` · `run.release.noEndpoint` ← `/release` + `/issue` · `run.create.noEndpoint` ← `POST /billing/runs`.

This is the contrast against R1–R5, and it is the whole lesson of the section: derive the absence from the response, never from a hardcoded sentence. Where the module did that, nine holds expired silently and correctly. Where it hardcoded a sentence, five are still lying.

The ruling on every absence

WHAT IS ABSENT	SCREENS	DECISION	REASON, AND WHO RULED IT
Nothing dispatches a bill — no dispatcher, no provider adapter	09, 02	OUT OF SCOPE	**Ruling found, and it holds.** `BR-BIL-016` is **CONFIRMED** (PO A-7, 2026-08-08): delivery is queue-based and e-mail ships later. Verified still true — `delivery_seam.py:308-310`, and US-NOT-002/003 do not exist. ⚠ One clause of a neighbouring rule *is* still owed and is not this row: `BR-BIL-036`'s *"the print batch goes to the vendor"* is unbuilt, which §08 covers.
A bill cannot be cancelled	02	`BIL-S86` New build	**RE-DECIDED — the reason this row gave is false.** It read *"US-BIL-027 and OQ-012, both open"*. `OQ-012` is **CLOSED**, ruled into `BR-PAY-011` and `BR-BIL-009` (*"void flows show the user a plain-language consequence message — reversing entries, balance effect, approvals — before confirm"*, CONFIRMED). A closed question is a ruling, and this one rules that it gets built. This is `CLAUDE.md`'s own trap: the status column said CLOSED while the body went on describing the open world.
Bulk waive	06	OUT OF SCOPE	**Ruling found, and it holds.** `BR-BIL-060` is **CONFIRMED** (PO 2026-08-11) and names this exact carve-out: bulk APPLY of a late fee is allowed, bulk WAIVE is not, because a first-time waiver is a per-case staff decision with a reason (`BR-BIL-003`).
The three draft powers (leave out · re-work · add a charge)	07	`BIL-S4` Exists behind	**No ruling that they wait** — BR-BIL-023 is a *confirmed* rule, and two of the three services are complete and routed. A specification, not a deferral.

The unusually-large-bill list	07	`BIL-S5` New read	BR-BIL-054 requires it; it has never existed (DEF-2855). Specification.
Past billing runs, as a tab	10	`BIL-S1` Exists behind	No ruling that it waits, and the data and the component both exist. Specification.
Bills CSV export and column memory	01	`BIL-S3` New build	The capability row declares both. Specification — and the stated blocker for the column half is already gone.
Cycle · run · date-range filters on the bills list	01	`BIL-S13` Exists behind	**CORRECTED — this was filed with a reason that is two-thirds out of date, and I repeated it.** ~~WAS: already filed as DEF-1062, open; out of this document's scope to re-specify.~~ **NOW:** the recorded blocker says all three *"need something that does not exist: a cycle lookup readable by a Billing Clerk, a runs list, and a date-range control the shared FilterBar does not have."* `selectors.py:905-920` **already filters by** `f.cycle_id`, `f.bill_run_id`, `f.issued_from` **and** `f.issued_to` **in SQL** — the last with the *to* day inclusive on purpose, because *"a chip reading 'to Aug 8' that dropped every bill issued during Aug 8 would be read as a data gap."* So the reads exist and what is missing is two front-end controls. A specification, and the cheapest real win on the screen.
Budget-billing eligibility	12	`BIL-S74` New read `BIL-S75` Blocked	**RE-DECIDED, and split.** This row said *"nothing publishes it"* — the READ half is served and has been since the rating lane shipped (`budget_billing` in `PLAN_FIELDS`, `rating/selectors.py:420`); `OQ-237`'s own row records that this sentence was stale for five days and that screen copy quoted it. **Specification** for the seam callable; **blocked** only for bulk enrolment, which needs an account to record which plan it is on.
Sending a one-time bill	11	`BIL-S2` Exists behind	**The ruling went the other way.** `OQ-241` is CLOSED — PO, 2026-08-23, `CR-029 D8③`: *"it will be capture at the time of bulk billing - not here"* — and the register records *"a cycle-less bill now issues."* So this is not a deferral; it is a screen still refusing something that was ruled buildable. Specification.
A page designer / own wording on the bill	—	OUT OF SCOPE	**Ruling found — but not the one this row cited.** It rested on *copy on the layout tab*, which is a design note and commits nobody. The actual ruling is in the decision log, 2026-08-12, PO verbatim: *"out platform will porvide standard 3 to 5 variouation of printed bill - user can use those only in V1"*. Right answer, wrong evidence — and the evidence is the half that has to survive this document.
Editing a charge line by hand	11, 13	OUT OF SCOPE	**RULED 2026-09-13, and the refusal now has law behind it.** Put to the Product Owner because no ruling existed either way. PO verbatim: *"you cannot change it here - it should be thorugh process only"*. So a charge line is **not** editable on the screen, and the only way a line changes is the correction process in §06 — cancel and regenerate, or a credit note, each leaving a document trail. The docstring argument was right; it simply was not a ruling until now.

Contrast worth stating: the wireframes in `05-prototype/` state *zero* absences. The product's own copy states **78**, **fourteen** of them holding a control a clerk can see and cannot use.

11

Rule compliance

One row per violation, never per screen. The last column is the point of the table: in a repository where most UI rules are ratified and unenforced, a green suite is evidence of nothing, and a reader needs to see which rows a machine would ever have caught. **The last column says which redraw closes each one**, so a violation is either **scheduled** or it is a description. **It**

deliberately carries no state chip: the worst state across a redraw family would read as though the fix itself were blocked, when what is blocked is something else in that family. **Readiness is §12's column**, per screen.

A violation with no commitment against it in §14 is not scheduled. It is a description, and it will still be true after everything in §12 is built.

SCREEN	RULE, AND WHAT IT SAYS	WHAT THE SCREEN DOES	WHERE	ENFORCED?	FIXED BY
12 Budget Bills	**B14 31** — *"no database id is ever the visible value of a field"* (PO 2026-08-31: *"no Databse ID on screen - that is a rule"*)	The *Account* field is a text box holding a raw `accounts.id` UUID, placeholder *"Paste the account's id from its detail page"*. No picker and no search exist on the tab, so the UUID is the only way in.	`BudgetBillsTab.tsx:199-208`	RATIFIED ONLY	§05 screen 25–25.4 · BIL-S10 Exists behi
13 Fix a bill	**B14 31**, same rule	The *Bill* field shows the raw `invoices.id` UUID — and is **prefilled with it by the intended door** (`fixABillPath(detail.id)` from the bill detail).	`FixABillTab.tsx:530-539 · billingOpsShared.tsx:435`	RATIFIED ONLY	§06 screen 26–26.8 · BIL-S80 New build
13 Fix a bill	**Hard rule 2** — money displays at 2 decimals; every amount goes through the shared formatter	Seven figures coerced with `String(x ?? "")`. An absent key yields `""`, which `MoneyCell`'s verbatim branch prints as `" USD"` — **and the coercion defeats `MoneyCell`'s own boundary**, which exists to throw on an absence rather than render one.	`FixABillTab.tsx:150,158,168,182,215,233,236`	RATIFIED ONLY	§06 screen 26–26.8 · BIL-S80 New build
13 Fix a bill	**Hard rule 2**	The line-by-line list is a raw template literal with **no currency at all**, reading three keys the server does not send — so it renders `" · → "`, twice, on the approval path.	`FixABillTab.tsx:367 vs corrections.py:303-311`	NOTHING CHECKS THIS	§06 screen 26–26.8 · BIL-S80 New build
12 Budget Bills	**Hard rule 2**	The *Level* — money — is interpolated into a sentence raw, at served precision: *"set by staff; worked out at {{computed}}"*.	`BudgetBillsTab.tsx:105 · en.json:932`	RATIFIED ONLY	§05 screen 25–25.4 · BIL-S80 New build

12 Budget Bills	**Hard rule 2**	A toast reads *"They are still {{amount}} {{currency}} ahead or behind"* — literally the `${amount} ${currency}` shape. **The exact defect a comment in the neighbouring file warns about** (*"a lane shipped `-147.500000` into a toast with no currency under 21 passing unit tests"*).	`BudgetBillsTab.tsx:475-480 · en.json:941`	RATIFIED ONLY	§05 screen 25–25.4 · BIL-S80 New build
11 One-time bill	**B14 39** — *label → input → helper*, ratified 2026-09-11, ratio ≈1:1:4	All four `help` strings render **between the label and the input**. Not this screen's doing — the shared staff kit does it in **all five field variants across 28 files** (`DEF-SHL-4921`). This is the only billing tab that passes `help` at all, which is why it is the only one visibly breaching it.	`FormModal.tsx:239-244, :310-314, :362-366, :622-628, :785-789`	NO GATE READS COMPUTED SPACING	§04 screen 21–24 · BIL-S82 New build
12 Budget Bills	**B14 16** — pagination on every list table	No pager, no sortable headers, and the client sends no `page` / `per` / `sort` — **while the server pages and orders in SQL correctly**. Page 1 only, for ever.	`billingOpsEndpoints.ts:380-389 vs budget_api.py:286-292`	RATIFIED ONLY	§05 screen 25–25.4 · BIL-S81 New build
13 Fix a bill	**B14 16**	Same shape: `getCorrectionCases(client)` takes **no parameters at all** against a server that paginates.	`billingOpsEndpoints.ts:182-188 vs corrections_api.py:379-383`	RATIFIED ONLY	§06 screen 26–26.8 · BIL-S81 New build
14 Pending queue	**B14 12 · 16 · 17** — a toolbar, a pager, an empty state on every list table	Rendered with `PlainTable`, which is a bare `<table>` — so there is no toolbar to put any of them in. *Arguably exempt* as an operational queue rather than a list table; I log it because the NFR-3 breach beneath it is not arguable.	`PendingQueuePage.tsx:161-229`	RATIFIED ONLY	§03 screen 20–20.3 · BIL-S6 Exists behi
12 Budget Bills	**B14 12** — filter chips	The two status chips use `.seg-opt.active`, which **is not declared in this stylesheet**, so the selected	`BudgetBillsTab.tsx:317-329 · AccountUsageTab.tsx:670`	A CLASS NOTHING DECLARES RENDERS AS NOTHING	§05 screen 25–25.4 · BIL-S82 New build

		chip has no visual selected state. Named as a live bug in a sibling file.			
02 Bill detail	**B14 4** — a button that navigates carries the arrow	Two of the four header buttons navigate and neither carries it. `DEF-1151, OPEN`	`BillDetailPage.tsx:413-434`	`RATIFIED ONLY`	§02 screen 02.1–02.5 · BIL-S20 Exists behi
13 Fix a bill	**B14 32** — no rule id or internal code in clerk-facing copy	The Corrections table prints the machine code where a sentence belongs: `wrong_read_used`. `DEF-2448, OPEN`	`FixABillTab.tsx:684-706`	`RATIFIED ONLY`	§06 screen 26–26.8 · BIL-S83 New build
05 step 2 · 07 step 4	**B14 32**, same rule	Refusal reasons reach a clerk as raw engineering strings (*"premises_jurisdiction..."*), as a printed rule id, and as a catch-all with no status or code. `DEF-518 · DEF-2432 · DEF-2384 · DEF-2895`	`billRunStep23 · BillRunWizard step 4`	`RATIFIED ONLY`	§03 screen · BIL-S83 New build
04 step 1 · 11 One-time	**B14 7** — a date is a `.date-input` with keyed `MM/DD/YYYY` and a Today shortcut, never a bare text input	**Cannot be assessed against the rule as written**, and that is itself the finding: `.date-input` **does not exist in the shipped app** — it is a prototype class. The product uses `DatePickerField`. The rule cites a class the product does not declare.	`UI-RULES.md vs 18 shipped CSS files`	`THE RULE NAMES THE WRONG STYLE WORLD`	§03 screen · BIL-S89 Blocked
01 All bills	**R14** — *"newly created entries should always show on the top"*	~~WAS: default sort is `issue_date desc` and a draft has no issue date, so a newly drafted bill does not sort to the top in any predictable position.~~ **NOW: ANSWERED — I WAS WRONG, and the code says so in as many words.** `_apply_order` applies `nulls_first=True` on descending, deliberately: *"NULLS FIRST on desc is not an accident: a DRAFT has no issue date and a draft run is the newest thing on the*	`selectors.py:925-935`	`NO VIOLATION`	§02 screen

		screen (the S30 default view).” **R14 is satisfied here.** Struck rather than deleted, because the next reader will have the same suspicion.			
10 Past billing runs	**Hard-won law 2's mirror** — when you write a hold, write the test that fires when the hold expires	A refusal over a shipped feature, with no test asserting the destination by address and screen id. Two sibling strings are unreachable dead copy in eleven catalogues.	`BillsListPage.tsx:1107-1110`	`NOTHING ASKS WHETHER A REFUSAL IS STILL TRUE`	§07 screen · BIL-S12 New build

Rules these screens are exempt from — said out loud, so nobody re-files them

B14 16, IN A WIZARD	Summary tables inside a wizard are exempt, so steps 4 and 5 are not logged for the absent pager.
B14 21 (TAILWIND)	**No violation anywhere in billing.** All fifteen `.tsx` files are on `route-b-migrated.txt` and both billing stylesheets are deleted, so utilities compile and render. The `DEF-974` trap does not apply here.
B14 15 (ROW ACTIONS)	**Satisfied** on the bills list — exactly two icons, and the obvious third (*Fix this bill*) is deliberately omitted because it is a write, RBAC-gated, and no overflow component exists in this product (`DEF-2453`).
B14 19 (PLACEHOLDER)	**Satisfied, and better than the prototype.** r10 draws *“Search bills or cycles…”*; the predicate cannot match a cycle, so the shipped placeholder drops that half rather than promising it (`DEF-1001`).

12

Where to start — the build index

Every screen there is work on, what it is, the commitments that cover it, and the strongest state among them. **Built from this document’s own register rather than written out, so it cannot drift from it.**

The nine that cost the most, in order

Ordered by **what their absence costs a person** — not by list order, not by effort, and not by severity as filed. A field whose absence makes a clerk decide wrongly outranks ten cosmetic ones. **The first one is not a billing defect in the ordinary sense:** it is a constant that silently excludes the top of the market this product was re-scoped to serve on 2026-09-06.

COMMITMENT	STATE	WHAT A PERSON CANNOT DO TODAY
BIL-S90	New build	A utility above 25,000 accounts **cannot run a cycle at all**. Not a slow run — no run. The ICP moved to 40,000 on 2026-09-06 and the constant did not move with it.

BIL-S56	Blocked	A leaving customer's deposit is not settled, so the final bill is wrong in the direction nobody re-reads. $153.50 applies before any path is chosen.
BIL-S91	New build	A clerk approves rows that show **no amount**. The decision looks made; the money was never computed.
BIL-S92	New build	An account whose meter simply was not read is **refused from the run**, so it never bills and nothing says so.
BIL-S93	New build	Every bill this product prints reaches a customer as **Courier with space-aligned columns**. It is the only artefact the customer ever sees.
BIL-S9	New build	Any render-time throw shows a clerk a **JavaScript stack trace and takes the navigation with it**. One boundary exists in the whole front end, and it is inside `MoneyCell`.
BIL-S1	Exists behind	Past billing runs shows a grey panel over a read that serves **twelve fields**. A clerk cannot answer *what did we bill last month, and who signed it off*.
BIL-S61	Blocked	An operator picks a correction reason and the system cannot say what it will do, because the reason table is not built.
BIL-S102	New build	Every utility inherits the same $25 late-fee floor, written into the code. A tenant cannot set its own, and nobody had ratified the number until 2026-09-13.

Every build component, and who undertook it

A build component is **anything buildable**, not just a screen. Each class below either names the commitments that cover it or says plainly that nothing does. **Two classes are the finding**: five of the thirteen guarantees are enforced by nothing at all, and twelve of the twenty-four rulings exist in no file.

COMPONENT CLASS	COUNT	WHAT IS TRUE OF IT	UNDERTAKEN BY
Screens	77	every one carries a derived chip; 6 further anchors are layers of screen 17 or an evidence table	BIL-S1 … all 101
Drawings	91	70 REDESIGN · 9 AS-BUILT measured · 12 AS-BUILT from code — each says so in its own caption	— a caption is a statement, not work
Billing tabs	7	the six in the path plus Print bills, added by the 2026-09-12 ruling	BIL-S70 Exists behind
Bill-detail tabs	5	Document · Charges · Corrections · Audit · **Sent**, the last one new	BIL-S19 New read
Pickers	194 from 23 sources	every source named once in §01; three of them do not exist yet	BIL-S31 New build
Table controls	search · filter · columns · pager, per list	the pager is the one that breaches a standing NFR	BIL-S81 New build BIL-S13 Exists behind
Write surfaces	12 named writes	**none was exercised** — there was no backend	BIL-S98 New build
Endpoints	36 across 8 url modules	the count is the masthead's; this document names 37 method+path strings, which is not the same measurement and is not a finding	—

Seams	11 files, plus one *no seam* row	`seam_batch` exists and the prepare pass does not use it	BIL-S46 New read BIL-S96 New build
Guarantees	13	**5 of the 13 are enforced by nothing** — the last reads “NOTHING, and it is why §10 exists”	BIL-S9 New build BIL-S12 New build BIL-S80 New build BIL-S81 New build BIL-S10 Exists behind
Catalogue strings	15,424 keys × 11 locales	2 can never render; 1 mistranslates *billing cycle* as *bicycle*	BIL-S88 New build BIL-S79 New build
Empty states & refusals	7 rotted holds · 14 dead controls	a refusal is true when written and rots silently; nothing checks	BIL-S12 New build
Tests	2 named as missing	the hold-expiry test, and a seeded draft bill so four refusal states can be rendered at all	BIL-S12 New build BIL-S98 New build
Hardcoded caps	2	`MAX_ACCOUNTS_PER_RUN_DEFAULT = 25_000` and `MAX_BILLS_PER_PRINT_SET_DEFAULT = 25_000` against a 40,000 ICP	BIL-S90 New build
Stale comments	2 measured	`late_fee.py:369` says `assess_late_fee` has *no production caller* — it has exactly one, `assess_late_fee_for_plan`, which itself has none. True in effect, wrong as written. And `DEF-3153` is recorded FIXED and is half fixed.	— corrected here; no product work
Migration-list entries	15 of 15	**checked, and the document was right**: all 15 non-test billing `.tsx` files are on `route-b-migrated.txt`, so a utility class emits	— nothing to do
Rulings	27	**15 exist nowhere but here** — and that is now an accepted risk with a ruling behind it, not an open gap	ruled 2026-09-13 OUT OF SCOPE
Open defects relied on	51 cited, 45 open	9 had no commitment until this pass	BIL-S90 BIL-S91 BIL-S92 BIL-S93 BIL-S94 BIL-S95 BIL-S96 BIL-S97 BIL-S98

This section used to be wrong, and it was the worst thing in the back half. It said the drawings to build had moved into §02 and listed nine screens — *01 · 02.1–02.5 · 10 · 13 · 14*. That was written before §03–§08 existed. It **named three screens that are now as-built evidence in §17** and **omitted every screen from 16 to 28.1** — about sixty drawings, including all of §04 to §08. A reader opening the section that says where the work is got a list that was mostly stale.

The state shown is the strongest of the commitments on that screen, worst-first: a

screen with four Exists behind cards and one Blocked reads as blocked, because that is what decides when it can ship. **Rows are sorted by screen number, not by the order you scroll past them. The thirteen screens in §17 are not here** — they are drawn as they are today, and five of them are superseded by a redraw in this list.

The build index — every screen, its commitments, and the hardest thing about it

Generated from the register, in document order. **Strongest state** is the worst state across the commitments naming that screen, so a row that says *build this* also says whether anything is in the way.

SCREEN	WHAT IT IS	COMMITMENTS	STRONGEST STATE
§02 · THE SCREENS — 7 SCREENS			
01	All bills	BIL-S1 BIL-S3 BIL-S9 BIL-S13 BIL-S14 BIL-S15 BIL-S16 BIL-S19 BIL-S24 BIL-S25 BIL-S44 BIL-S79 BIL-S94 BIL-S97 BIL-S99	New build
02	Bill detail	BIL-S9 BIL-S12 BIL-S17 BIL-S20 BIL-S22 BIL-S27 BIL-S28 BIL-S71 BIL-S86 BIL-S98	Blocked
02.1	Bill detail · Document	BIL-S17 BIL-S18 BIL-S19 BIL-S20 BIL-S22 BIL-S98	Blocked
02.2	Bill detail · Charges	BIL-S17 BIL-S20 BIL-S22 BIL-S23	Blocked
02.3	Bill detail · Corrections	BIL-S17 BIL-S20 BIL-S21 BIL-S22	Blocked
02.4	Bill detail · Audit	BIL-S17 BIL-S20 BIL-S22	Blocked
02.5	Bill detail · Sent	BIL-S22	Blocked
§03 · THE BULK BILL RUN — 31 SCREENS			
16	Scope	BIL-S24 BIL-S25 BIL-S26 BIL-S34 BIL-S38 BIL-S46 BIL-S89 BIL-S90 BIL-S95	Blocked
17.0	Checking	BIL-S40 BIL-S41 BIL-S45 BIL-S96	New build
17	Check	BIL-S26 BIL-S27 BIL-S28 BIL-S32 BIL-S40 BIL-S73 BIL-S83 BIL-S90 BIL-S91 BIL-S92 BIL-S96	New build
17.1	No read	BIL-S32 BIL-S33 BIL-S47 BIL-S49 BIL-S91 BIL-S92	New build
17.2	Unusually large	BIL-S32 BIL-S33 BIL-S47	Exists behind
17.3	Unusually small	BIL-S32 BIL-S33 BIL-S47	Exists behind
17.4	Zero use	BIL-S32 BIL-S33 BIL-S47	Exists behind
17.5	Unusual days	BIL-S32 BIL-S33 BIL-S47	Exists behind
17.6	Meter changed	BIL-S32 BIL-S33 BIL-S47	Exists behind
17.7	Multiplier	BIL-S32 BIL-S33 BIL-S47	Exists behind

17.8	Estimated	BIL-S32 BIL-S33 BIL-S47	Exists behind
17.9	Base only	BIL-S32 BIL-S33 BIL-S47	Exists behind
17.10	Held now	BIL-S32 BIL-S47	Exists behind
17.11	Account fix	BIL-S32 BIL-S34 BIL-S39 BIL-S47	New build
17.12	Read fix	BIL-S32 BIL-S47	Exists behind
17.13	Rates fix	BIL-S32 BIL-S47	Exists behind
17.14	Ready	BIL-S32 BIL-S34 BIL-S47	Exists behind
17.15	Masters	BIL-S32 BIL-S34 BIL-S39 BIL-S47	New build
17.16	Estimate all	BIL-S42 BIL-S47 BIL-S48 BIL-S49	New build
17.17	Hold these	BIL-S42 BIL-S47 BIL-S48	New build
17.18	Accept	BIL-S42 BIL-S47 BIL-S48	New build
17.19	Confirm factor	BIL-S42 BIL-S47 BIL-S48	New build
17.20	Waiver	BIL-S42 BIL-S47 BIL-S48 BIL-S77 BIL-S102	New build
18	Sign off	BIL-S29 BIL-S40 BIL-S43 BIL-S50	New build
19	Send	BIL-S30 BIL-S50	New build
17.21	In arrears	BIL-S44 BIL-S47	New build
17.22	Apply all	BIL-S42 BIL-S47 BIL-S48 BIL-S49	New build
20	Held queue	BIL-S35 BIL-S36 BIL-S39 BIL-S44 BIL-S100	New build
20.1	Catch up	BIL-S35	Exists behind
20.2	Hold one	BIL-S37	Blocked
20.3	Release	BIL-S37	Blocked
§04 · SINGLE BILLS — 9 SCREENS			
21	Who and when	BIL-S51 BIL-S54 BIL-S82	New build
22.1	Interim	BIL-S51 BIL-S54 BIL-S55 BIL-S82	New build
22.2	First bill	BIL-S51 BIL-S54 BIL-S55	Exists behind
22.3	Final bill	BIL-S51 BIL-S54 BIL-S55 BIL-S56 BIL-S57	Blocked
22.4	Back bill	BIL-S51 BIL-S52 BIL-S54 BIL-S55	Exists behind

22.4	Back-bill	BIL-S51 BIL-S52 BIL-S54 BIL-S55	Exists behind
22.5	Penalty	BIL-S51 BIL-S52 BIL-S54 BIL-S77 BIL-S102	New build
22.6	Fee	BIL-S51 BIL-S54	Exists behind
23	Confirm and send	BIL-S51 BIL-S56 BIL-S57	Blocked
24	Periods	BIL-S52 BIL-S55	Exists behind
§05 · BUDGET BILLS — 5 SCREENS			
25	Budget bills	BIL-S58 BIL-S59 BIL-S67 BIL-S74 BIL-S75 BIL-S80 BIL-S81 BIL-S82	Blocked
25.1	Enrol	BIL-S58 BIL-S59 BIL-S67 BIL-S74 BIL-S75 BIL-S80 BIL-S82	Blocked
25.2	Settle-up forecast	BIL-S60 BIL-S80	New build
25.3	Asks	BIL-S66 BIL-S67	Exists behind
25.4	Portal · Level pay	BIL-S66	Exists behind
§06 · FIX A BILL — 9 SCREENS			
26	What and why	BIL-S61 BIL-S62 BIL-S80 BIL-S81 BIL-S83	Blocked
26.1	One bill	BIL-S61 BIL-S80 BIL-S83	Blocked
26.2	Several months	BIL-S61 BIL-S69 BIL-S80	Blocked
26.3	A batch	BIL-S61	Blocked
26.4	Goodwill	BIL-S62	Exists behind
26.5	The page only	BIL-S63	Exists behind
26.6	What it changes	BIL-S68	Exists behind
26.7	What it changes · many	BIL-S68 BIL-S69	Exists behind
26.8	Generate and send	BIL-S85	New build
§07 · PAST BILLING RUNS — 1 SCREENS			
27	Past runs	BIL-S1 BIL-S76	New build
§08 · BILL PRINTING — 2 SCREENS			
28	Print bills	BIL-S64 BIL-S65 BIL-S70 BIL-S93	New build
28.1	Held back	BIL-S65 BIL-S70	Exists behind
§17 · THE REST, AS BUILT — 13 SCREENS			
03	Bill document	BIL-S9 BIL-S29 BIL-S79 BIL-S93	New build

04	Run bills · step 1 of 6 — pick the cycle	BIL-S1 BIL-S9 BIL-S12 BIL-S30 BIL-S84 BIL-S88 BIL-S89	Blocked
05	Run bills · step 2 of 6 — what was found	BIL-S9 BIL-S12 BIL-S73 BIL-S84 BIL-S91	New build
06	Run bills · step 3 of 6 — the customers	BIL-S9 BIL-S84	New build
07	Run bills · step 4 of 6 — the totals	BIL-S4 BIL-S5 BIL-S9 BIL-S84	New build
08	Run bills · step 5 of 6 — sign off	BIL-S9 BIL-S84	New build
09	Run bills · step 6 of 6 — send	BIL-S9 BIL-S71 BIL-S72 BIL-S84	New build
10	Past billing runs	BIL-S1 BIL-S9 BIL-S12 BIL-S76	New build
11	One-time bill	BIL-S2 BIL-S9 BIL-S12 BIL-S82 BIL-S86 BIL-S88	New build
12	Budget Bills	BIL-S8 BIL-S9 BIL-S10 BIL-S11 BIL-S80 BIL-S81 BIL-S82	New build
13	Fix a bill	BIL-S7 BIL-S8 BIL-S9 BIL-S10 BIL-S11 BIL-S16 BIL-S80 BIL-S81 BIL-S83	New build
14	Pending queue	BIL-S6 BIL-S9 BIL-S25 BIL-S100	New build
15	Print bills	BIL-S9 BIL-S30	New build

13

Cross-module

Run once, from above, because these are the failures that belong to nobody. **Both modules are named in every row** — a finding filed against one is a finding the other module's owner never reads. Built from the ⇄ rows in §02's field tables, not from the import graph, so it is the boundary the screens actually cross. **The last column says who has undertaken each one. Eight rows carried** No commitment until this pass — and that was the finding this section exists for: **not one of their defect ids was named by any commitment card**, which is what happens to a fault belonging to two modules and therefore to neither. **All eight are undertaken now**, by BIL-S71 – BIL-S79, minted against the very defect ids these rows already carried.

1 · Does the handoff land?

FROM	TO	THE FACT	WHAT IS WRONG	WHAT BREAKS	UNDERTAKEN BY

billing	notifications	One delivery row per issued bill	The rows land and nothing reads them onward: no dispatcher, no provider adapter. **Declared, not hidden** — the screen says *“Nothing leaves the building.”*	Nothing, today. It is the honest state of a pre-release product.	`BIL-S15` New build
billing	notifications	The bill-ready e-mail's contents	`DEF-BIL-4448` **Every value on it is an em-dash** — bills are queued with no render context.	The first e-mail this product ever sends is blank where the figures go.	`BIL-S71` New build
billing	notifications	Which template a queued row uses	`DEF-BIL-4452` Rows are pinned to a *billing* template on **every** channel, and the notifications renderer cannot use one.	The handoff is type-incompatible at the boundary, in both modules' terms.	`BIL-S72` New build
billing	metering	Nine seam reads per account, in the prepare pass	`DEF-2704` The pass asks nine cross-module seams **once per account** and no owning module publishes a batch read. Measured: **66,053 SQL statements at 1,000 accounts**, paid again on every refresh.	At T5 the prepare step is unusable, and it is the step a clerk refreshes while reading exceptions.	`BIL-S46` New read
billing	rating	Base charge and schedule name	**Gathered and then dropped.** `gather.py:699-705` resolves the base charge through a three-way ladder; the findings payload carries neither, and the screen tells the clerk the run does not gather them. See §05 R3.	A lane reading the screen goes looking for a gatherer that already exists.	`BIL-S73` New read
billing	rating	Does this plan offer budget billing?	**Half of this went stale and this document repeated it.** The READ *is* served: `budget_billing` sits in `PLAN_FIELDS` and is answered at `rating/selectors.py:420`, as `DEF-476` said. What is genuinely missing is narrower — `billing/rating_seam.py` names no callable, so `eligible` is **null by construction**, and no account records which plan it is on, which is what a bulk enrolment would iterate. `OQ-237` is open for that second half only.	The Enrol button cannot enable itself; a clerk must know the answer already.	`BIL-S74` New read `BIL-S75` Blocked — healthy
billing	customers	`customer_name` on a run-case row	Null on **every** row — a stated B1/G6 seam limit (`selectors.py:3206-3210`).	Step 2 and step 3 identify accounts by number only, on the screens where a clerk decides what to do about a person.	`BIL-S17` New read
billing	ledger	A correction's bucket deltas	Money arrives correctly and is **rendered** raw at six decimals with the currency as just another key (§02 screen 13).	The approver reads the ledger effect at a precision the rest of the product never shows.	`BIL-S7` Exists behind
billing	payments · ledger	`past_due` on a bill	Three-term derivation and billing may read none of it: *unpaid* is the payments/ledger position, *today* is the tenant clock. Served `None` on every row, honestly, with an explainer on screen.	The status column can never say past due, and the segment strip offers a filter the data cannot answer.	`BIL-S44` New build

2 · Does any module write another's tables?

No. I looked for the real test — an import that reaches a manager and calls `.create()`, `.update()` or `.delete()` — across all six cross-boundary imports, and found none.

What does exist is `DEF-2845`, open, partially fixed: *"the six-ring rule says billing never imports another module and seven imports in five files do."* On this read I count **six** `from apps.*` imports across **five** files — `documents.py` (`apps.rating.pricing.wording`), `documents_api.py`, `frozen_bill.py`, `print_set.py` and `exports.py` (all `apps.tenancy`: `document_storage`, `document_store`, the export kernel). The difference from seven is probably an import form my pattern missed, and I would rather say so than round.

All six are reads or platform services, and one is the sanctioned shape: owner modules are *supposed* to call `register_export_resource` from `AppConfig.ready`, which is what `exports.py:236` does. So the finding is a boundary bypassed, not a boundary violated — which is the right severity, and is why DEF-2845 is *low*.

3 · Does one fact have two homes?

THE FACT	HOME 1	HOME 2	VERDICT	UNDERTAKEN BY
Who ran a bill run	`bill_runs.run_by` — a real FK to `identity.User`, populated, served as `ran_by`	*a sentence on the screen saying the column does not exist*	`FORKED` Not two copies of the data — one copy and one contradicting claim about it, which is worse, because the claim is what a lane reads.	`BIL-S1` Exists behind
A bill's name and address	`premises`, reached by walking (hard rule 8)	the **frozen payload** on the bill	`HEALTHY` This is the one case where a second copy is *correct*: BR-BIL-030 freezes the document at issue precisely so a reprint shows July's address. A snapshot is a different fact from the live value, and the product is explicit about it.	Nothing to do — this row is healthy
A bill's status vs its run's status	`invoices.status` — advances correctly	`bill_runs.status` — *written once as `draft` and never advanced by anything in the module*	`DEAD COLUMN` `DEF-171` and `DEF-602`, both high, both open. `api.py:2092` states the consequence: *"a Status column rendered from this field alone would title every historical run 'Draft'."* The as-built drawing on screen 04 shows exactly that.	`BIL-S76` New build
The late-fee floor	billing's own constant	collections' constant	`DEF-205` open — *"two separate copies of one ruled number."* The classic second-copy-free-to-drift, on a money figure.	`BIL-S77` New build
BR-BIL-030 itself	one file	another file, *as a different rule*	`DEF-052` open — *"defined twice, in two files, as two different rules — and both definitions are live."* Not a data fork; a *rule* fork, which the same reasoning condemns.	`BIL-S78` New build

4 · Does the same thing have two names?

CONCEPT	WHAT BILLING CALLS IT	VERDICT	UNDERTAKEN BY

The thing a customer owes	**Bill** on every screen, in every catalogue — while the table, the endpoint, the type and the serializer all say `invoice`	`HEALTHY, DELIBERATELY.` One word for the clerk, another for the schema, and the boundary is consistent. Nothing user-facing says *invoice*.	Nothing to do — this row is healthy
Held here / in the books	**Recorded** and **In the books**	`HEALTHY` B14 rule 34's one pair of words, in all eleven catalogues.	Nothing to do — this row is healthy
Mark for later	the rule says *"or mark for later"*; the route is `/hold`; the screen says *"Leave this account out of this run"*; the queue calls it **held**	`THREE NAMES FOR ONE ACT.` The clerk-facing pair (*leave out* → *held*) is coherent; the route name and the rule's wording are a third and fourth. Low cost today, and exactly the drift rule 34 exists to stop.	`BIL-S4` Exists behind
Billing cycle	**"Bicycle"** in Japanese and French	`DEF-BIL-4759` high, open. A Japanese bill line reads *"bicycle charge"*. This is a bill a real customer reads, and it is the one vocabulary finding here with a customer on the other end of it.	`BIL-S79` New build

Seam health

Ten seam files, 94 named functions, and not one returns a raw queryset. I tested for the thing that matters — what a seam *hands back*, not what its docstring says — and found no `return …objects.`, no `-> QuerySet`, no `return qs` in any of them. That is the boundary drawn properly.

SEAM	FUNCTIONS	TO	GRADE	UNDERTAKEN BY
`rating_seam.py`	20	billing → rating	`HEALTHY`	
`posting_seam.py`	13	billing → ledger	`HEALTHY`	— healthy
`delivery_seam.py`	12	billing → notifications	`HEALTHY` — and it is the seam that says `nothing_was_dispatched` out loud	— healthy
`payments_seam.py`	11	billing → payments	`HEALTHY`	— healthy
`seam_batch.py`	8	batching across seams	`EXISTS, AND DEF-2704 SAYS THE PREPARE PASS DOES NOT USE IT PER-ACCOUNT`	`BIL-S46` New read
`settings_seam.py`	8	billing → tenancy	`HEALTHY`	— healthy
`metering_seam.py`	7	billing → metering	`HEALTHY`	— healthy
`customers_seam.py`	6	billing → customers	`HEALTHY`	— healthy
`tenancy_seam.py`	6	billing → tenancy	`HEALTHY`	— healthy
`run_alert_seam.py`	3	billing → notifications	`HEALTHY`	— healthy
no seam	6	billing → tenancy (document storage, export kernel) · billing → rating (`pricing.wording`)	`LEAKY BY RULE` — reads only, one of them the sanctioned registration shape. DEF-2845.	`BIL-S96` New build

One seam finding I could not settle and am not going to guess at. `DEF-079`, high, open:

"Five documented seams still SUBSTITUTE a fact over tables that now exist." That is the §05 rotting-hold shape at the seam layer, and proving it needs reading five seam implementations against five schemas — more than this pass did. It is named here so it is not lost, and **it is the single highest-value thing a next pass should pick up.**

14

Specification register

One row per commitment, no exceptions. **If a thing is not here, nobody has undertaken to build it** — everything else in this document is description. **The index is an index:** it named the work and then repeated **3,158 words of the cards below it verbatim**, so a seventy-row index ran ten pages. What each commitment *waits on*, and what done looks like, is in its card.

A new ID series, on purpose. These are `BIL-S1` … `BIL-S30`, minted for this document. **They are not allocated into the repository's live series** and nothing has been reserved — this pass wrote nothing to the repo. The live series are `US-BIL-###` for stories, `OQ-…` for questions, `DEF-…` for defects and `CR-###-BRC##` for change requests, and borrowing from any of them would make this look scheduled when it is not. **Every row carries one of four state chips, and only four** — Exists behind New read New build Blocked — and the chip says **what the work waits on**, never how confident anyone is. Of the **100** here: **50 exist behind a screen that does not ask for them**, 6 need a read published, 38 are new build, and 6 are blocked. **32 were minted across the three audit passes and two were withdrawn** — each withdrawn by a ruling that made the absence deliberate rather than by anyone deciding it did not matter.

COMMITMENT	WHAT A PERSON COULD THEN DO	WAITS ON	SCREENS	STATE
`BIL-S1`	A clerk can see what was billed last month, and who signed it off	Nothing.	01, 27, 04, 10	Exists behind
`BIL-S2`	A clerk can issue a one-time bill, not just draft one	Nothing — and this is the one card where the ruling went the other way.	11	Exists behind
`BIL-S3`	A clerk can download the bills list, and keep the columns they chose	A bills resource in the export registry.	01	New build
`BIL-S4`	A clerk can leave a draft out, re-work it after a fix, or add a charge from the utility's own list	Nothing for two of the three.	07	Exists behind
`BIL-S5`	A clerk can see which bills came out unusually large before sending them	BR-BIL-054's list, which has never existed (DEF-2855).	07	New read
`BIL-S6`	A T5 utility can open the pending queue	Nothing — the pattern is in the same module three times over.	14	Exists behind

BIL-S7	An approver can read what a correction moves, line by line	Nothing.	13	Exists behind
BIL-S8	A clerk can find a correction case, and a budget-billing row, past the first fifty	Nothing on the server — both endpoints paginate and order in SQL already.	12, 13	Exists behind
BIL-S9	A clerk sees the product's own failure state instead of a JavaScript stack trace	Nothing.	01, 02, 03, 04, 05, 06, 07, 08, 09, 10, 11, 12, 13, 14, 15	New build
BIL-S10	A clerk can identify a bill and an account by their number, not their database id	Nothing.	12, 13	Exists behind
BIL-S11	A clerk is told when a budget-billing action fails	Nothing.	12, 13	New build
BIL-S12	A refusal stops being true and the suite goes red	Nothing but the decision to do it.	02, 04, 05, 10, 11	New build
BIL-S13	A clerk can narrow the bills list to one cycle, one run or a date range	Nothing on the server.	01	Exists behind
BIL-S14	The Billing header tells a clerk something true, and takes them somewhere	Nothing for two of the three.	01	Exists behind
BIL-S15	A clerk can see whether a bill has actually been printed	The outbound queue has no table (BR-BIL-016 / r10 #8).	01	New build
BIL-S16	A clerk can start a correction from the row the bill is on	An overflow-menu component, which does not exist anywhere in this product (DEF-2453) — which is…	01, 13	New build
BIL-S17	A clerk can tell whose bill they are looking at, and how much, without scrolling	One read.	02, 02.1, 02.2, 02.3, 02.4	New read
BIL-S18	The bill overview reads as seven questions, each with a one-word label	Nothing for five of the seven.	02.1	Exists behind
BIL-S19	A clerk can see whether the bill reached the customer, without hunting for it	Nothing to move it.	01, 02.1	New read
BIL-S20	Small things stop costing a clerk a click or a squint	Nothing, except the Audit tab — and that one is not billing's to fix: it is metering's S61 viewe…	02, 02.1, 02.2, 02.3, 02.4	Exists behind
BIL-S21	A clerk can read what a correction did, in order, and what the customer owes because of it	Nothing for the sequence — GET /invoices/{id}/chain and the correction case are both served, and…	02.3	Exists behind
BIL-S22	An audit row cannot be misread	Not billing's to fix.	02, 02.1, 02.2, 02.3, 02.4, 02.5	Blocked
BIL-S23	The Charges tab says how much water, not just how much money	Nothing.	02.2	Exists behind
BIL-S24	A clerk settles the routine decisions once, before the run starts	Nothing.	01, 16	Exists behind
BIL-S25	A clerk can bill part of a cycle, sweep up who was missed, and take over an open run	Scope needs a filter on the	01, 16, 14	Exists behind

		run's account set.		
BIL-S26	Pricing runs in the background, with progress, and money is on screen before any decision	Nothing.	16, 17	Exists behind
BIL-S27	One review screen shows every account, searchable, filterable and paged	Search and filters over the run's account set;	02, 17	New build
BIL-S28	No account can fall out of a run silently	Nothing — it is one subtraction over counts already served.	02, 17	Exists behind
BIL-S29	The sign-off says what signing does, in three groups	Nothing.	18, 03	Exists behind
BIL-S30	The run can print the bills it just issued	Nothing.	19, 04, 15	Exists behind
BIL-S31	Every input on a billing screen uses the right control	The multi-select half.	—	New build
BIL-S32	Step 2's chips are generated from what the run raised, in three bands	Nothing.	17, 17.1, 17.2, 17.3, 17.4, 17.5, 17.6, 17.7, 17.8, 17.9, 17.10, 17.11, 17.12, 17.13, 17.14, 17.15	Exists behind
BIL-S33	Every option states whether it bills or holds the account	Nothing, and it is the highest-value label in the section: two prorate choices are recorded and…	17.1, 17.2, 17.3, 17.4, 17.5, 17.6, 17.7, 17.8, 17.9	Exists behind
BIL-S34	A master account's bill says how many units it covers	Nothing.	16, 17.11, 17.14, 17.15	Exists behind
BIL-S35	A clerk can bill a held account from the queue	Nothing on the server —	20, 20.1	Exists behind
BIL-S36	The held-accounts queue is dated, filtered and paged	Nothing.	20	Exists behind
BIL-S37	An account can be held out of billing until a review date	A ruling.	20.2, 20.3	Blocked
BIL-S38	A run's service scope is part of its idempotency key	Nothing but the change itself.	16	New build
BIL-S39	A master bill prices each connection on its own plan and its own tier ladder	Nothing.	17.11, 17.15, 20	New build
BIL-S40	A pre-bill check runs stages 1–5 and writes nothing; bills are created at sign-off	Nothing architectural.	17.0, 17, 18	Exists behind
BIL-S41	The check runs in a worker and the screen fills in while it does	Nothing on the server —	17.0	Exists behind
BIL-S42	A bulk action can apply to every row in a chip, not just the visible page	Nothing but the UI and one server call.	17.16, 17.17, 17.18, 17.19, 17.20, 17.22	New build
BIL-S43	Sign-off states what is NOT being billed, and the money foots on screen	One new axis and one new aggregate.	18	New build
BIL-S44	Arrears are visible while the clerk is still deciding	One thing, and it is the same thing three screens need: the	01, 17.21, 20	New build

		business day has to reach the past-d…		
BIL-S45	The served note claiming no run has ever been timed is corrected	Nothing.	17.0	Exists behind
BIL-S46	Metering publishes one aggregate count of a cycle's metering inventory	One new published read, owned by metering.	16	New read
BIL-S47	A clerk can select rows and act on the selection	Nothing on the server.	17.1, 17.2, 17.3, 17.4, 17.5, 17.6, 17.7, 17.8, 17.9, 17.10, 17.11, 17.12, 17.13, 17.14, 17.15, 17.16, 17.17, 17.18, 17.19, 17.20, 17.21, 17.22	Exists behind
BIL-S48	A bulk action acts on the selection the clerk made, and reports what moved	Nothing new on the server —	17.16, 17.17, 17.18, 17.19, 17.20, 17.22	Exists behind
BIL-S49	The estimate disposition can actually be submitted	A fix, not a feature.	17.1, 17.16, 17.22	Exists behind
BIL-S50	A clerk can open a real bill before anything is sent, and a specimen before anything is created	Step 4's three bills are a link to a served endpoint.	18, 19	New build
BIL-S51	A clerk can raise all six one-time kinds through one three-step wizard	Nothing on the server.	21, 22.1, 22.2, 22.3, 22.4, 22.5, 22.6, 23	Exists behind
BIL-S52	The back-bill's period list shows what the grouping costs the customer	Nothing but the panel.	22.4, 22.5, 24	Exists behind
BIL-S53	Mid-period split billing gets a caller	A wiring fix, not a feature.	—	Exists behind
BIL-S54	Step 2 shows what a one-time bill will price on, before it prices it	Nothing new on the server.	21, 22.1, 22.2, 22.3, 22.4, 22.5, 22.6	Exists behind
BIL-S55	A clerk can enter a missing reading on the bill screen, and it lands in Meter Reads	Nothing on either server.	22.1, 22.2, 22.3, 22.4, 24	Exists behind
BIL-S56	A final bill applies the deposit first, then offers exactly one settlement path	The arithmetic and the master path are both built.	22.3, 23	Blocked
BIL-S57	Step 3 shows what the bill will do to the books — including when it cannot	Both deposit entries are written and signed;	22.3, 23	Exists behind
BIL-S58	A utility can put one customer on a flat monthly amount	Nothing on the server.	25, 25.1	Exists behind
BIL-S59	The three level figures are never collapsed into one	Nothing —	25, 25.1	Exists behind
BIL-S60	The settle-up is already posted by the cycle run, and a clerk can see it coming	Nothing on the server, and nothing to build there either.	25.2	Exists behind
BIL-S61	An operator picks a reason and the system says what it will do	The taxonomy is written;	26, 26.1, 26.2	Blocked

BIL-S61	An operator picks a reason and the system says what it will do	The taxonomy is written,	26, 26.1, 26.2, 26.3	Blocked
BIL-S62	A clerk can cancel and regenerate one account's bills over a window	Nothing new —	26, 26.4	Exists behind
BIL-S63	A clerk can cancel and regenerate a batch, and cannot undo it	All six scope selectors and the aggregate are built, including the tie-out (Σ credit –	26.5	Exists behind
BIL-S64	A print set can be scoped to a period, not only to a run	A screen change, not a contract change.	28	Exists behind
BIL-S65	Every bill that will not print says why, and whose job it is	Nothing on the server —	28, 28.1	Exists behind
BIL-S66	A customer can ask for level pay from the portal, and the button works	One write, and it is a write the portal is already allowed to make.	25.3, 25.4	Exists behind
BIL-S67	A clerk can see what a customer was promised beside what is correct now	Nothing new on the server —	25, 25.1, 25.3	Exists behind
BIL-S68	A clerk can see what happens to money already paid against a corrected bill	Nothing on the server —	26.6, 26.7	Exists behind
BIL-S69	A six-month correction is settled per month, and the two big numbers are separated	Nothing new —	26.2, 26.7	Exists behind
BIL-S70	Print bills is a Billing tab with a searchable table of the period	A screen change, not a contract change, twice over.	28, 28.1	Exists behind
BIL-S71	A clerk can send a bill-ready e-mail that has the customer's figures on it, not em-dashes	—	02, 09	New build
BIL-S72	A queued delivery row names a template the dispatcher can actually render	—	09	New build
BIL-S73	A clerk can see the base charge and the schedule name the run already gathered	—	17, 05	New read
BIL-S74	The Enrol button can enable itself, because the plan's budget-billing flag is answered through the seam	—	25, 25.1	New read
BIL-S75	A clerk can enrol every eligible account at once	OQ-237 —	25, 25.1	Blocked
BIL-S76	A clerk can trust a run's status, because the column advances or it is gone	—	27, 10	New build
BIL-S77	The late-fee floor is one number, read from one place	—	17.20, 22.5	New build
BIL-S78	BR-BIL-030 means one thing, because it is defined once	—	—	New build
BIL-S79	A Japanese or French bill line says billing cycle, not bicycle	—	01, 03	New build
BIL-S80	A clerk reads money at two decimals on every billing screen	—	25, 25.1, 25.2, 26, 26.1, 26.2, 12, 13	New build
BIL-S81	A clerk can page, sort and filter every billing list, at any size	—	25, 26, 12, 13	New build
BIL-S82	A billing form reads label, then input, then help — and its chips render	—	21, 22.1, 25,	New build

			25.1, 11, 12	
BIL-S83	A clerk never reads a rule id or an internal code on screen	—	17, 26, 26.1, 13	New build
BIL-S84	Nobody can reach the old six-step run wizard, because it is gone	§03 landing first	04, 05, 06, 07, 08, 09	New build
BIL-S85	A clerk can generate and send the corrected documents at the end of Fix a bill	—	26.8	New build
BIL-S86	A clerk can cancel a bill, and is told what the cancellation moves before confirming	—	02, 11	New build
BIL-S88	A clerk never meets copy describing a screen that cannot be reached	—	04, 11	New build
BIL-S89	B14 rule 7 can be complied with, because it names the style world the product uses	a B14 amendment	16, 04	Blocked
BIL-S90	A utility with more than 25,000 accounts can run a cycle at all	—	16, 17	New build
BIL-S91	A clerk never decides on a row that shows no amount	—	17, 17.1, 05	New build
BIL-S92	An account whose meter simply was not read still reaches the run	—	17, 17.1	New build
BIL-S93	A customer receives a bill that looks like a bill	—	28, 03	New build
BIL-S94	The bills list answers at 40,000 accounts, because the search stops building every matching row	—	01	New build
BIL-S95	A lost race refuses cleanly instead of failing the whole transaction	—	16	New build
BIL-S96	Five seams read the tables that now exist instead of substituting a fact	—	17.0, 17	New build
BIL-S97	A clerk can find a bill by its cycle, which the search box already offers to do	—	01	New build
BIL-S98	The four refusal states on the bill header can be seen before they ship	—	02, 02.1	New build
BIL-S99	A clerk reads the tax and the service period on the bills list, without opening the bill	—	01	Exists behind
BIL-S100	A clerk can see when a hold started and whether it was ever caught up	—	20, 14	Exists behind
BIL-S102	A utility sets its own late-fee balance floor, instead of inheriting a number written into the code	—	17.20, 22.5	New build

The cards in full

BIL-S1

A clerk can see what was billed last month, and who signed it off

STATUS Exists behind

WAITS ON Nothing. `GET /billing/runs` is served, `PastRunsPanel` is written, and `bill_runs.run_by` is populated and on the wire. The blocker named on screen — *"nothing would list it"* / *"no actor column"* — is false in both halves that matter.

SCREENS 27, 10, 04, 01

DONE LOOKS LIKE Opening *Billing → Past billing runs* shows a table whose first row reads `BR-2608-0042 · 08-A · 01 Aug – 31 Aug 2026 · 1,263 bills · $338,904.00 ·` and the panel that currently says it arrives later is gone.

`Dana Reyes · Issued`, and the panel that currently says it arrives later is gone.

BIL-S2

A clerk can issue a one-time bill, not just draft one

STATUS	Exists behind
WAITS ON	Nothing — and this is the one card where the ruling went the other way. `OQ-241` was **closed** by the PO on 2026-08-23 (`CR-029 D8③`) and the server's refusal was removed on 2026-09-02. Only the screen's own hardcoded sentence, in eleven catalogues, still refuses.
SCREENS	11
DONE LOOKS LIKE	The One-time bill tab no longer shows *"Sending the bill"*, and a drafted one-time bill can be issued from the screen that drafted it.

BIL-S3

A clerk can download the bills list, and keep the columns they chose

STATUS	New build
WAITS ON	A `bills` resource in the export registry. The machinery is in place — `apps/billing/exports.py` exists and registers `bill_run_releases` — and the preference service the column half was waiting for is wired and already used by this screen for its page size.
SCREENS	01
DONE LOOKS LIKE	The download glyph on *All bills* is enabled, produces a CSV that mirrors the list's own `q` / `sort` / `f.*` state, and a column you hide is still hidden when you come back tomorrow.

BIL-S4

A clerk can leave a draft out, re-work it after a fix, or add a charge from the utility's own list

STATUS	Exists behind
WAITS ON	Nothing for two of the three. `runs.mark_for_later` and `runs.retry_account` are complete and routed as `/hold` and `/retry`; the screen offers neither. BR-BIL-023 is a *confirmed* rule. The third (add a charge) needs a route.
SCREENS	07
DONE LOOKS LIKE	Step 4's *"What you cannot yet do to a draft here"* panel is replaced by three working row actions, and pressing *Leave this account out* puts that account in the pending queue with a reason.

BIL-S5

A clerk can see which bills came out unusually large before sending them

STATUS	New read
WAITS ON	BR-BIL-054's list, which has never existed (`DEF-2855`). It needs a worked-out bill and the account's own trailing average served to one screen; the run computes both.
SCREENS	07
DONE LOOKS LIKE	Step 4 lists the flagged accounts with this period against their average, and the three actions above them are buttons that work rather than text.

BIL-S6

A T5 utility can open the pending queue

STATUS	Exists behind
WAITS ON	Nothing — the pattern is in the same module three times over. `StandardPagination` already serves `/invoices`, `/corrections` and `/budget-billing`.
SCREENS	14
DONE LOOKS LIKE	*Pending queue* shows `Showing 1–50 of 1,204 · Rows per page 50 · Page 1 of 25`, the age column sorts in SQL, and the KPI number no longer re-runs the list query.

BIL-S7

An approver can read what a correction moves, line by line

STATUS	Exists behind
WAITS ON	Nothing. Every figure is already in the served payload under the names the server sends — `label`, `original`, `corrected`, `delta`. The screen reads three keys that do not exist.
SCREENS	13
DONE LOOKS LIKE	The *Line by line* block on a correction reads `Water — 8 kgal @ 7.5000 · $60.00 → $48.00 · −$12.00` instead of `" · → "`, and the Difference tile reads `−$12.00` instead of `" USD"`.

BIL-S8

A clerk can find a correction case, and a budget-billing row, past the first fifty

STATUS	Exists behind
WAITS ON	Nothing on the server — both endpoints paginate and order in SQL already. The clients send no `page`, `per` or `sort` and render no pager.
SCREENS	12, 13
DONE LOOKS LIKE	Both tables carry the standard pager and sortable headers, and *Corrections so far* has a search box that reaches the server.

BIL-S9

A clerk sees the product's own failure state instead of a JavaScript stack trace

STATUS	New build
WAITS ON	Nothing. There is no error boundary and no `errorElement` anywhere in the route tree; the only one in the whole front end is inside `MoneyCell`.
SCREENS	all 15, and every other screen in the product
DONE LOOKS LIKE	A screen whose read returns an unexpected shape shows A18 §4's framed message *with the left rail still there*, and *“Unexpected Application Error!”* never reaches a clerk.

BIL-S10

A clerk can identify a bill and an account by their number, not their database id

STATUS	Exists behind
WAITS ON	Nothing. Both fields need a searchable select over reads that already exist (`GET /invoices?q=`, `GET /accounts?q=` — the One-time tab already uses the second one).
SCREENS	12, 13
DONE LOOKS LIKE	The *Bill* field reads `BL-2609-018320 — R. Alvarez · 09-A · $84.00` and the *Account* field reads `100244 — R. Alvarez`. No UUID is visible on either screen.

BIL-S11

A clerk is told when a budget-billing action fails

STATUS	New build
WAITS ON	Nothing. Two of the three row actions are `async` handlers with no `try` / `catch`, invoked through an `onClick` typed `() => void`, so the rejection is never awaited.
SCREENS	12, 13
DONE LOOKS LIKE	Pressing *Take off budget billing* on a row the server refuses shows the server's sentence in a toast, and the row does not change silently.

BIL-S12

A refusal stops being true and the suite goes red

STATUS	New build
WAITS ON	Nothing but the decision to do it. Hard-won law 2's mirror: when you write a hold, write the test that fires when the hold expires — asserting the destination by address *and* by the screen id it opens.
SCREENS	10, 04, 05, 02, 11
DONE LOOKS LIKE	Seven tests exist, one per rotting hold in §05, each naming the route and screen id that would make its refusal false. Mounting `PastRunsPanel` in the history tab turns one of them red and names it.

BIL-S13

A clerk can narrow the bills list to one cycle, one run or a date range

STATUS	Exists behind
WAITS ON	**Nothing on the server.** `f.cycle_id`, `f.bill_run_id`, `f.issued_from` and `f.issued_to` are all filtered in SQL today (`selectors.py:905-920`) and all four are already in the client's own filter list. What is missing is a cycle lookup a Billing Clerk may read, and a date-range control in the shared FilterBar, which renders option lists only.
SCREENS	01 · R4 ②
DONE LOOKS LIKE	Choosing *Cycle 09-A* narrows the list to that cycle and the chip reads `Cycle 09-A ×`; a range chip reads `Issued 09/01–09/30 ×` and includes bills issued on the 30th.

BIL-S14

The Billing header tells a clerk something true, and takes them somewhere

STATUS	Exists behind
WAITS ON	Nothing for two of the three. The unresolved count is already computed and the screen it belongs to already exists. *Out this cycle* is hardcoded `None` (`selectors.py:1101-1102`) and is either a sum over the cycle or a deletion — a decision, not a blocker.
SCREENS	01 · R4 ①③④
DONE LOOKS LIKE	The header reads `Bills lifetime 7240 · Out this cycle $342,180.00 · Unresolved from the last run 3 →`, the last is a link that lands on the pending queue, and the past-due sentence is a closed disclosure below the table rather than two lines above it.

BIL-S15

A clerk can see whether a bill has actually been printed

STATUS	New build
WAITS ON	**The outbound queue has no table** (BR-BIL-016 / r10 #8). The largest of the eight, and it crosses into notifications, so it wants a change request of its own.
SCREENS	01 · R4 ⑤
DONE LOOKS LIKE	The bills list carries a *Printed* column reading a date for a printed bill, *Not yet* for an issued one that has not been, and — for a draft. Printing is the only channel that reaches a customer, and this is the only place a clerk could check it.

BIL-S16

A clerk can start a correction from the row the bill is on

STATUS	New build
WAITS ON	An overflow-menu component, which **does not exist anywhere in this product** (`DEF-2453`) — which is why the row stops at two icons under UI-RULES 15.
SCREENS	01 · R4 ⑦ · 13
DONE LOOKS LIKE	A row's `···` opens *Fix this bill* and lands on the correction screen with the bill already chosen and shown by its number, not an empty box asking for a pasted id.

BIL-S17

A clerk can tell whose bill they are looking at, and how much, without scrolling

STATUS	New read
WAITS ON	**One read.** The customer's name and the service address — the address by walking to `premises` plus the unit label, which hard rule 8 already requires of every other entity. Everything else in the new heading (the amount, the due date, the bill date, the cycle name) is already on the payload.
SCREENS	02, 02.1–02.4 · R5.1
DONE LOOKS LIKE	The heading reads `Clara Castillo · 105 Cedar Ridge Way, Unit 1, Silver Creek TX 78611 · AC-2026-00003` above `$251.45 due 08/22/2026 · billed 08/02/2026 for 07/01 – 07/31 · Silver Creek – monthly`, and a clerk answers *who and how much* without touching the page.

BIL-S18

The bill overview reads as seven questions, each with a one-word label

STATUS	Exists behind
WAITS ON	Nothing for five of the seven. **Amount**, **Dates**, **Origin** and **Record** are today's panels regrouped, and **Breakdown** summarises a table that is already served. **Usage** needs `GET /accounts/{id}/usage`, which **is mounted** — only the trailing average is a new read. *Owed today* needs the live account balance.
SCREENS	02.1 · R5.1
DONE LOOKS LIKE	The Document tab reads **Amount** → **Breakdown** → **Usage** → **Dates**, then the bill itself, then **Origin** and **Record** closed. *Owed today* sits under the four as-at-issue figures with a link to the account, so *what do they owe now* has an answer on the screen.

BIL-S19

A clerk can see whether the bill reached the customer, without hunting for it

STATUS	New read
WAITS ON	Nothing to move it. The print-set membership is the same read that fills screen 01's *Printed* column (`BIL-S15`); the paperless election and contact details need a customers read.
SCREENS	02.1 → new Sent tab · R5.3 · 01
DONE LOOKS LIKE	`Sent 2` is a tab of its own, its rows read **Print** and **Customer portal** rather than `pdf_print` and `portal`, and the print row names the set it went out in.

BIL-S20

Small things stop costing a clerk a click or a squint

STATUS	Exists behind
WAITS ON	Nothing, except the Audit tab — and that one is **not billing's to fix**: it is metering's S61 viewer used by every detail screen in the product, so changing it changes them all. **It needs its own change request.**
SCREENS	02, 02.1–02.4 · R5.1–R5.5
DONE LOOKS LIKE	Tab labels carry counts. The heading and tab strip stay put while a long Charges tab scrolls. The bill number can be copied in one click. The Audit log is newest-first, its timestamp reads `08/02/2026 04:00` rather than `2026-08-02T09:00:00+00:00`, and `7 more fields` expands in place.

BIL-S21

A clerk can read what a correction did, in order, and what the customer owes because of it

STATUS	Exists behind
WAITS ON	Nothing for the sequence — `GET /invoices/{id}/chain` and the correction case are both served, and the *Replaced* / *Open the bill that counts now* strings already exist. The **net effect** figure is arithmetic over rows already on the payload.
SCREENS	02.3 · R5.4b
DONE LOOKS LIKE	A corrected bill's Corrections tab reads `Replaced → Open BL-2609-000051, the bill that counts now`, then five dated rows from *issued* to *replacement bill*, ending `Net effect on the customer −$53.15`.

BIL-S22

An audit row cannot be misread

STATUS	Blocked
WAITS ON	**Not billing's to fix.** The tab is metering's S61 viewer, used by every detail screen in the product on purpose — *"one audit surface keeps one set of masking and window rules"*. Changing the alignment here changes it everywhere, which is right and needs **its own change request**.
SCREENS	02.4 · R5.5b · every detail screen
DONE LOOKS LIKE	One table row per changed field — *Field · From · To* — with the date and author spanning them; newest first; `08/02/2026 04:00` instead of `2026-08-02T09:00:00+00:00`; and `▸ Show the other 6 fields that changed` expanding in place.

BIL-S23

The Charges tab says how much water, not just how much money

STATUS	Exists behind
WAITS ON	Nothing. Every line already carries its quantity and unit; a *Usage* column on *What each service cost* is a sum of rows on the payload. The six alignment faults fold out of moving two text columns under the description.

SCREENS 02.2 · R5.2

DONE LOOKS LIKE *What each service cost* reads `Water · 19 kgal · 4 lines · $132.02`; the three numeric columns are all right-aligned and tabular; *Total* is emphasised; and no column wraps a date across two lines.

BIL-S24

A clerk settles the routine decisions once, before the run starts

STATUS Exists behind

WAITS ON Nothing. The three defaults — missing read, unusually large, part periods — are choices the engine already makes per account; making them once at run level is a parameter on `POST /billing/runs`.

SCREENS 16 · step 1

DONE LOOKS LIKE Step 1 has a **Defaults** group, and a run over 1,284 accounts arrives at step 2 with **12 accounts needing a decision instead of hundreds**, because the recurring cases were settled once.

BIL-S25

A clerk can bill part of a cycle, sweep up who was missed, and take over an open run

STATUS Exists behind

WAITS ON Scope needs a filter on the run's account set. The held-account sweep is the pending queue's own data. Taking over an open run needs only `ran_by`, **which is already served**.

SCREENS 16 · step 1 · 14

DONE LOOKS LIKE Step 1 can scope to one route; *"the 21 accounts held from RUN-0002"* is a checkbox; and when somebody else has a run open the screen **names them, dates it, and offers to take it over** rather than refusing with no way forward.

BIL-S26

Pricing runs in the background, with progress, and money is on screen before any decision

STATUS Exists behind

WAITS ON **Nothing.** `run_calculate` runs the prepare pass itself, takes no decisions as input, is idempotent by construction, and its `background: true` mode answers 202 with a poll address the framework already serves. **It has gone uncalled since 11 August.**

SCREENS 16 → 17

DONE LOOKS LIKE Pressing *Price this run* returns immediately with a progress figure; the clerk may close the tab; and step 2 opens with an **Amount** on every row — where today `amount`, `currency`, `quantity` and `unit` are null on every one.

BIL-S27

One review screen shows every account, searchable, filterable and paged

STATUS New build

WAITS ON Search and filters over the run's account set; pagination on `/findings`, `/exceptions` and `/breaks`, three of which are unpaginated by declared design.

SCREENS 17 · step 2

DONE LOOKS LIKE Step 2 has a search box, filter chips **carrying counts**, a column chooser and a pager, and a clerk can go straight to *the 3 unusually large ones* without reading 1,284 rows.

BIL-S28

No account can fall out of a run silently

STATUS	Exists behind
WAITS ON	Nothing — it is one subtraction over counts already served.
SCREENS	17 · step 2
DONE LOOKS LIKE	Step 2 states `1,251 ready + 12 need you + 21 held + 0 broke = 1,284 in the cycle` and **refuses sign-off if it does not reconcile**. Today an account whose customer type has no base charge produces no bill and *appears in no list* (`DEF-2381`, high).

BIL-S29

The sign-off says what signing does, in three groups

STATUS	Exists behind
WAITS ON	Nothing. The gate's five facts are unchanged; only the prose around them shrinks.
SCREENS	18 · step 3
DONE LOOKS LIKE	The step reads in three groups — **What you are signing · Your sign-off · What happens when you sign** — and states plainly that *the bills are still drafts until you press Issue*. It carried nine paragraphs and two absence panels at ratification.

BIL-S30

The run can print the bills it just issued

STATUS	Exists behind
WAITS ON	Nothing. `POST /billing/print-sets` is served and takes a `bill_run_id`.
SCREENS	19 · step 4 · 15
DONE LOOKS LIKE	Step 4 carries *Print bills →* scoped to this run. Today the step tells a clerk printing is the only channel that reaches a customer and offers no way to do it — they must leave the run and find a button in the Billing list header.

Ordered by what they cost, not by severity. Sixteen of the thirty — `BIL-S1`, `BIL-S7`, `BIL-S8`, `BIL-S10` `BIL-S13` and `BIL-S18`'s five regrouped panels — need **no new back-end read at all**. Everything they draw is already served, under names the server already sends. `BIL-S2` is smaller still: it is deleting a sentence the PO ruled against nineteen days ago.

BIL-S31

Every input on a billing screen uses the right control

STATUS	New build
WAITS ON	The multi-select half. Dates, dropdowns and chips are already law and the kit components exist; **a filter holding two values in one dimension is not** — it moves a signed URL grammar, the server's filter application and roughly fifteen shared list screens. Ruled 2026-09-11; needs a commit of its own.
SCREENS	all
DONE LOOKS LIKE	Opening any billing list, ticking *Water* then *Gas* shows both, the footer counts them together, two chips are removed independently, and the same click behaves identically on the payments list.

BIL-S32

Step 2's chips are generated from what the run raised, in three bands

STATUS	Exists behind
WAITS ON	Nothing. The counts are served. This removes the three tabs that exist only to say a feature is not built, and the four that can never populate.
SCREENS	17, 17.1–17.15
DONE LOOKS LIKE	A run that raised no *days threshold* case shows **no days-threshold chip**, and every chip present reads `<name> <count>` with its money beneath.

BIL-S33

Every option states whether it bills or holds the account

STATUS	Exists behind
WAITS ON	Nothing, and it is the highest-value label in the section: **two *prorate* choices are recorded and unpriced**, so a clerk choosing the most reasonable option today gets no bill and no warning.
SCREENS	17.1–17.9
DONE LOOKS LIKE	The *days threshold* dropdown reads `Prorate to the cycle's days — holds the bill, pricing not built`, and a clerk choosing it sees the account arrive in Held with that reason.

BIL-S34

A master account's bill says how many units it covers

STATUS	Exists behind
WAITS ON	Nothing. The covered-unit list is what the run already walks to build the bill.
SCREENS	17.11, 17.14, 17.15, 16
DONE LOOKS LIKE	Account 100302 appears in step 2 and in Ready as `Hillcrest Apartments · covers 12 units`, and its $69.60 is never presented as a single-unit bill.

BIL-S35

A clerk can bill a held account from the queue

STATUS	Exists behind
WAITS ON	Nothing on the server — the catch-up endpoint is served and keyed. The screen has no control because the list would have had to choose a parent run; it shows the choice instead.
SCREENS	20, 20.1
DONE LOOKS LIKE	Selecting two July holds and pressing *Bill these now* produces **July** bills through a catch-up run, the queue rows resolve, and the September run is untouched.

BIL-S36

The held-accounts queue is dated, filtered and paged

STATUS	Exists behind
WAITS ON	Nothing. The *as of* day is already a required parameter the screen fixes to today; the filters are new UI over rows already served. It also closes a

WAITS ON — pagination breach with no defect row.

SCREENS: 20

DONE LOOKS LIKE: Setting the date to 08/31/2026 re-states every age against that day; filtering to *escalated* narrows to 9; 400 rows page 50 at a time and sort on the server.

BIL-S37

An account can be held out of billing until a review date

STATUS: Blocked

WAITS ON: **A ruling.** No status means *billable, but hold the bill* — a vacation hold still bills the standing charges. Whether a standing hold may exist, and whether the review date is mandatory, are yours to answer.

SCREENS: 20.2, 20.3

DONE LOOKS LIKE: Holding 100294 with reason *billing dispute* and a review date of 12/11/2026 keeps it out of the next three runs, shows it in every run's blocked band, and escalates it when the review date passes.

BIL-S38

A run's service scope is part of its idempotency key

STATUS: New build

WAITS ON: Nothing but the change itself. Today a service-scoped run consumes the whole cycle's key, so the rest of that cycle can never be billed and never enters the held queue. **No defect row and no open question exists for it.** Until it lands, screen 1 shows scope and does not choose it.

SCREENS: 16

DONE LOOKS LIKE: Running *Water only* for September, then running *Sewer* for September, produces two runs and two sets of bills instead of a duplicate refusal.

BIL-S39

A master bill prices each connection on its own plan and its own tier ladder

STATUS: New build

WAITS ON: Nothing. Both halves ruled 2026-09-11 — *per connection*. An engine change: a line set, a tariff resolution and a plan snapshot **per connection**, and a tier ladder that resets per connection. Two gather refusals are then deleted rather than handled. Reverses the bill-level tax ruling of 2026-08-11 **for master accounts**; the supersession is written down first.

SCREENS: 17.11, 17.15, 20

DONE LOOKS LIKE: Elm Court HOA, with a hallway meter and a stairwell meter both electric, bills `$64.00` on four lines — two base charges of $12.00 and two usage ladders of $20.00. Hillcrest Apartments, spanning *Common Electric* and *Common Water*, bills `$70.17`: $69.60 of charges plus a 2.00% city water fee of `$0.57` levied on the water connection alone, not $1.39 on the whole bill. Both leave the blocked band. Bayside Holdings, whose coverage cannot be read, is still refused.

BIL-S40

A pre-bill check runs stages 1–5 and writes nothing; bills are created at sign-off

STATUS: Exists behind

WAITS ON: Nothing architectural. Stages 1–5 are already pure, a draft is already *freely discardable*, the one-time preview already does exactly this, and the signed rule *the preview and the real run are the same code* forbids a second implementation. The change is **where the orchestrator stops**. It also stops re-pricing burning bill numbers.

SCREENS	17.0, 17, 18
DONE LOOKS LIKE	Opening Check on a 1,284-account run shows every figure and **creates no invoice row and mints no bill number**; changing a decision three times still leaves the bill-number sequence untouched; pressing *Sign off and create 1,260 bills* is the first write.

BIL-S41

The check runs in a worker and the screen fills in while it does

STATUS	Exists behind
WAITS ON	Nothing on the server — the 202, the job id, the poll and `progress {done, total}` are all served, and the wizard already reads them. What is missing is the **screen**: Check opens immediately, populated as the pass proceeds, and says the clerk may leave. Plus two engine calls: **refuse rather than run inline** above the bar, and pass the resume checkpoint on every path.
SCREENS	17.0
DONE LOOKS LIKE	A 25,000-account run opens Check within a second, shows `9,412 of 25,000` from the job's own progress, lets a clerk decide on arrived rows, survives closing the tab, and a worker killed at 9,000 does not re-walk the first 9,000.

BIL-S42

A bulk action can apply to every row in a chip, not just the visible page

STATUS	New build
WAITS ON	Nothing but the UI and one server call. It must name what will change, what cannot and why, the money delta, and ask for the count above 50 rows — the kit already has that control.
SCREENS	17.22, 17.16–17.20
DONE LOOKS LIKE	On a 25,000-account run, selecting the *Missing read* chip selects all 231 rows; the confirm reads `198 will change · 33 cannot · +$11,842.60`; typing 198 commits; the 33 with no history stay on *Needs you* and are never reported as done.

BIL-S43

Sign-off states what is NOT being billed, and the money foots on screen

STATUS	New build
WAITS ON	One new axis and one new aggregate. The **regular/master split** is derivable — the account's kind is on the row the run already reads — unlike the three axes that genuinely cannot be. The **run-level sum of payments applied** is an aggregate over a figure already computed per account for the bill's own arrears block. Held and blocked counts the run already holds.
SCREENS	18 — *this commitment names a sign-off variant **18.1** that this document never drew. It is recorded against screen 18, the screen that changes.*
DONE LOOKS LIKE	Step 3 reads `1,273 regular · 11 master covering 229 areas`, consumption per service with its unit, `charges − credits = net` as a visible subtraction, `$128,402.55 − $96,318.20 = $32,084.35` carried in, and **24 accounts, about $6,240, not invoiced** with why and where each fix is.

BIL-S44

Arrears are visible while the clerk is still deciding

STATUS	New build
WAITS ON	One thing, and it is the same thing three screens need: **the business day has to reach the past-due filter**. It refuses today for exactly one reason — the tenant's clock does not reach it. The balance itself is already computed. **Billing sends no notice**: payments owns a four-rung ladder, and the unmapped rung renders as an absence rather than being rounded.

SCREENS	17.21, 01, 20
DONE LOOKS LIKE	Check shows *In arrears 312 · $84,204.60 owed*, each row with its rung and the one account on the final 48-hour rung flagged; the *send a notice* control is disabled with its reason; setting the as-at date to 08/31 restates every age against that day.

BIL-S45

The served note claiming no run has ever been timed is corrected

STATUS	Exists behind
WAITS ON	Nothing. It is a stale sentence on a live response: *"there is no timing figure for a bill run of any size anywhere in this product yet"*, against a measured **137 ms per account** quoted four files away. Every consumer of that note — including this document, one message ago — concluded the measurement was owed. **No defect row exists.**
SCREENS	17.0, and every caller
DONE LOOKS LIKE	The note names the measured figure and the bar it implies; the background threshold is set from it rather than left unset; and a defect row records why a served absence outlived its own fix.

BIL-S46

Metering publishes one aggregate count of a cycle's metering inventory

STATUS	New read
WAITS ON	One new published read, owned by **metering**. Billing's existing metering reads are all per unit — service points, reads, sizes, disputes — so a cycle-wide count through them is one call per unit. The aggregate itself is one query.
SCREENS	16
DONE LOOKS LIKE	Opening *Scope* on cycle 09-A shows `1,598 meters · 96 sub-meters · 1,526 points with no meter` within a second, on a 25,000-account tenant, from a single call — and the figures move when the cycle changes.

BIL-S47

A clerk can select rows and act on the selection

STATUS	Exists behind
WAITS ON	Nothing on the server. A 38px selector column, a tri-state header that selects **the page**, and a separate explicit *select all in this chip* — never the header tick. Changing chip clears it; paging keeps it; shift-click selects a range; a row decided by hand is never overwritten. No selector where nothing can be actioned.
SCREENS	17.1–17.22
DONE LOOKS LIKE	On the *Missing read* chip a clerk ticks three rows and the bar appears offering only the actions all three accept; ticking the header selects the 50 on the page and says so; *Select all 231* is a second control; switching to *Unusually large* clears the selection with a stated count.

BIL-S48

A bulk action acts on the selection the clerk made, and reports what moved

STATUS	Exists behind
WAITS ON	Nothing new on the server — `POST /billing/runs/{id}/exceptions/{case}/resolve` already takes a list of case ids and mints one shared action id. What is owed is the **reply**: the count actually actioned, and any row whose facts changed between selection and commit. **Ruled 2026-09-12** — the selection is a snapshot and the snapshot wins, so the client holds the ids rather than the server re-resolving the set. *Select all* refuses above 10,000 rows

in one chip.

SCREENS 17.16–17.20, 17.22

DONE LOOKS LIKE Selecting 231 and confirming records 231 decisions under one action id; the reply reads `231 actioned · 2 have since been read and will bill on the reading`; and no account is estimated over a reading that now exists.

BIL-S49

The estimate disposition can actually be submitted

STATUS Exists behind

WAITS ON **A fix, not a feature.** The endpoint accepts a quantity and a unit and **no screen has ever sent them**, so choosing *Estimate from history* refuses on the explicit-units rule — the run's most common decision cannot be recorded today, and there is no defect row. Per the PO ruling of 2026-09-12 the screen orchestrates and displays, with a progress state; **the estimate rule stays on the server** so two paths cannot produce two figures.

SCREENS 17.1, 17.16, 17.22

DONE LOOKS LIKE Choosing *Estimate from history* on 100263 records 8.1 kgal and bills $51.30. In bulk over 231 the screen shows `Working out 231 estimates… 140 done`, then writes 231 rows; the 33 with no history are actioned and held, with the reason named, rather than given an invented quantity.

BIL-S50

A clerk can open a real bill before anything is sent, and a specimen before anything is created

STATUS New build

WAITS ON Step 4's three bills are a link to a served endpoint. The sign-off specimen is new: today the renderer only draws from a sample invoice, not from the run's own priced data before the drafts exist.

SCREENS 19, 18

DONE LOOKS LIKE Step 4 opens with three named bills — the master, an ordinary one and an estimated one — each opening the real issued PDF, and the send button cannot be pressed until one has been opened. At sign-off, *See a specimen* renders account 100302's bill from the priced data, writes nothing, and mints no bill number.

BIL-S51

A clerk can raise all six one-time kinds through one three-step wizard

STATUS Exists behind

WAITS ON Nothing on the server. `POST /billing/one-time/preview` and `POST /billing/one-time` are served, the six kinds are published, and the request object already refuses a hand-typed amount by having no field for one. What is missing is **a screen per kind**: today one tab asks for everything and explains nothing.

SCREENS 21, 22.1–22.6, 23

DONE LOOKS LIKE Choosing *First* shows the move-in date as read-only from the account and refuses if it is outside the window; choosing *Final* asks four things and applies the deposit first; choosing *Back-bill* demands a period list; and no screen anywhere offers a box to type an amount into.

BIL-S52

The back-bill's period list shows what the grouping costs the customer

STATUS Exists behind

WAITS ON Nothing but the panel. Both figures come from the same preview pass, which writes nothing. **18 kgal as one period prices the usage at $104.50;**

six monthly groups price it at $79.50. The screen shows both and names the cause, because after the bill exists the difference is invisible — both foot.

SCREENS 24, 22.4, 22.5

DONE LOOKS LIKE A six-month back-bill shows `$79.50` against `$104.50` with *13 kgal into the top tier* named as the difference; *suggest from the cycle calendar* fills the table and is editable; nothing is grouped without the clerk.

BIL-S53

Mid-period split billing gets a caller

STATUS Exists behind

WAITS ON **A wiring fix, not a feature.** A plan or rate change mid-cycle should split the period into segments, price each on the version in force, run one tax pass and produce one draft. It is written, it is tested, and **nothing calls it anywhere in the product** — so today a mid-cycle rate change prices the whole period on one version. Hard-won law 2, on a money path.

SCREENS §03 — the bulk run

DONE LOOKS LIKE A cycle containing a rate change dated mid-period produces bills whose lines carry two date ranges and two version ids, and the run's own totals tie to them.

BIL-S54

Step 2 shows what a one-time bill will price on, before it prices it

STATUS Exists behind

WAITS ON **Nothing new on the server.** The gather pass already resolves premises, units, service points, assets, the plan, the rate version in force at the bill date, taxes and discounts, and it writes nothing. **What is missing is that a clerk never sees any of it:** today a one-time bill either prices or refuses, and they find out at the end.

SCREENS 22.1–22.6, 21

DONE LOOKS LIKE Choosing *Final* for account 100377 and continuing shows **12 Foundry Ln**, its one unit, its two service points with their assets and sizes, **COM-W-2026 v4, in force 01 Jul 2026**, and a block naming the three things that would still refuse — all before any figure is computed.

BIL-S55

A clerk can enter a missing reading on the bill screen, and it lands in Meter Reads

STATUS Exists behind

WAITS ON **Nothing on either server.** Metering already serves `POST /reads`, `reads/estimate` and `reads/quick-context`. Billing may not write `meter_reads` and does not — all nineteen of its metering calls are reads — so the field posts to **metering's own endpoint** and the bill re-prices once the reading exists there. **One fact, one owner.** The alternative, billing keeping its own copy for one-time bills, is two homes for one fact.

SCREENS 22.1, 22.2, 22.3, 22.4, 24

DONE LOOKS LIKE Typing **47,290** as the final read on 22.3 and continuing creates a read row in Meter Reads against `M-88213` dated 09 Sep 2026, visible on that meter's own history, and step 3 prices **6.1 kgal** from it.

BIL-S56

A final bill applies the deposit first, then offers exactly one settlement path

STATUS Blocked

WAITS ON	**The arithmetic and the master path are both built.** A $150.00 deposit held 14 months at 2% earns $3.50, so $153.50 applies before any path is chosen; the accrual engine landed 2026-09-03. The transfer target is **resolved from the data, never supplied** — your ruling of 2026-08-24, *"master account gets debt - if sub accoun tis unable to pay it ; even aftr deposite settlement"*. **Two things are genuinely missing:** there is **no settlements table**, so the chosen path has nowhere to live and one-path-per-bill is enforced inside a call and not across calls; and the owner rung needs `premises.owner_customer_id`, measured absent.
SCREENS	22.3, 23
DONE LOOKS LIKE	A commercial move-out with $289.40 of charges and a $150.00 deposit shows **–$150.00 and –$3.50 as separate lines**, **$135.90 still due**, and three paths of which *Transfer to master* names **100299 Alder Holdings** — or is greyed with the reason no master resolved.

BIL-S57

Step 3 shows what the bill will do to the books — including when it cannot

STATUS	Exists behind
WAITS ON	**Both deposit entries are written and signed;** the bill's own issue entry already posts. All three are distinct events on one invoice, so a resumed run replays all three and writes none twice. **The refusal is the part that must reach the screen:** where the deposit is worth more than the closing position, applying it would put **–$11.20 into accounts receivable**, and the signed refund row pays from *customer credit balances* where the customer holds **$0.00** — so the posting refuses by name rather than choosing where the excess lives. Nothing signs that transfer yet.
SCREENS	23, 22.3
DONE LOOKS LIKE	Step 3 for the collect case lists **6950/2300 $3.50**, **2300/1200 $153.50** and the issue entry **1200/revenue $289.40**; step 3 for a refund case shows the same panel **refusing**, naming $153.50 against $142.30, **before** the clerk presses the button.

BIL-S58

A utility can put one customer on a flat monthly amount

STATUS	Exists behind
WAITS ON	**Nothing on the server.** `budget.py` and `budget_service.py` are 60KB of finance-signed arithmetic with an append-only event ledger, and seven routes are mounted. `enroll` **had no caller outside tests** — hard-won law 2. The form is the whole gap, and the nine facts its first group shows are all served by the account read today.
SCREENS	25.1, 25
DONE LOOKS LIKE	Enrolling account **100612** shows nine facts in three bands — including **$312.40 past due** and **autopay on** — then 13 months of history and the plan passing, a computed level of **$78.00** from $936.00, an override band of **$54.60 – $101.40**, and writes an enrolment plus one event row.

BIL-S59

The three level figures are never collapsed into one

STATUS	Exists behind
WAITS ON	**Nothing — all three are served.** The risk is a screen, not a contract: the ±30% bound is measured against the **computed** level, so a table or form showing only *what they pay* lets an override be typed blind, and **three audited 30% steps walk a level to twice what the trailing twelve months say**.
SCREENS	25, 25.1
DONE LOOKS LIKE	The table carries **Computed · Override · They pay** as three columns, and the form states the band as a money range computed from the computed level, not from the current one.

BIL-S60

The settle-up is already posted by the cycle run, and a clerk can see it coming

STATUS	Exists behind
WAITS ON	**Nothing on the server, and nothing to build there either.** The run posts the settle-up on the account's own bill, in its own settle-up month, **idempotently per invoice**. ⚠ So what is needed is a **read**: an earlier draft of this card asked for a bulk settle action, which would have been a **second writer for money that already has one**. The settle-up month is copied onto each account at enrolment, so the forecast spans nine months rather than one.
SCREENS	25.2
DONE LOOKS LIKE	Settling 37 accounts writes **24 collect** and **13 credit** lines onto next bills, posts through the budget-variance account, and adds **$0.00** of tax.

BIL-S61

An operator picks a reason and the system says what it will do

STATUS	Blocked
WAITS ON	**The taxonomy is written; two things are not.** Thirteen reasons in three classes, the strongest-class rule, the refusal of a document-class case with a money delta, and the five routed-away reasons all exist in code. ⚠ **The reason→class map is PROPOSED, not ratified**, and the tenant-extensible reason table **has no class column and is not built** — so a tenant's own reason is refused rather than defaulted.
SCREENS	26, 26.1, 26.2, 26.3
DONE LOOKS LIKE	Ticking *Goodwill* renders *"issues a goodwill credit on the account, excluding tax: the bill is not touched…"* in the operator's words, and **no mechanism dropdown exists anywhere in the workflow**.

BIL-S62

A clerk can cancel and regenerate one account's bills over a window

STATUS	Exists behind
WAITS ON	**Nothing new — a class-R restatement already is cancel-and-regenerate** and the date-range scope is one of six selectors. The credit note is the cancellation, the rebill is the regeneration, and they execute together per bill. **What is missing is the screen that shows the window biting:** the 12-month cap applies to a correction that ADDS money (PO 2026-08-11, *the cap applies*), so a six-month redo can be partly refused.
SCREENS	26.4, 26
DONE LOOKS LIKE	A March–August redo on account 100377 lists six eligible bills, an aggregate of **+$159.80** with its ↑/↓ split, and a seventh bill from Aug 2025 **disabled with its age and reason printed** rather than silently dropped.

BIL-S63

A clerk can cancel and regenerate a batch, and cannot undo it

STATUS	Exists behind
WAITS ON	**All six scope selectors and the aggregate are built**, including the tie-out (Σ credit − Σ rebill = Σ net) and the mandatory ↑/↓ split, whose absence was proved RED against a build showing only the net. **There is no rollback and none may be added:** a posted entry cannot be un-posted and a delivered bill cannot be un-sent, so the sentence goes where the button would have been.
SCREENS	26.5
DONE LOOKS LIKE	A run+service scope resolves **412 of 1,263** bills *once, at preview*, shows **389 owing more** and **23 owing less** beside a net of **+$4,593.80**, names **+$349.90** of tax by authority, and offers **no undo control at all**.

BIL-S64

A print set can be scoped to a period, not only to a run

STATUS	Exists behind
WAITS ON	**A screen change, not a contract change.** The server already accepts a cycle filter; the screen only ever offered a bill run or every issued bill. The engine is measured (**0.012 s and 6 pages per bill**, ceiling **25,000**) and was **proved in a real browser** — archive downloaded, PDFs counted, manifest rows counted, one row checked against the bill it names. ⚠ **Exactly one manifest format is registered.**
SCREENS	28
DONE LOOKS LIKE	Choosing cycle **09-A** and 01–30 Sep shows **404 will print · 8 held · 412 in the period**, *about 5 seconds* and *~2,424 pages*, all before the button.

BIL-S65

Every bill that will not print says why, and whose job it is

STATUS	Exists behind
WAITS ON	**Nothing on the server** — the eight held-back reasons are written and ship in `held-back.csv`. What is missing is the screen: today a clerk gets a CSV and has to work out that three of the eight are **billing defects rather than their own work**.
SCREENS	28.1, 28
DONE LOOKS LIKE	Eight bills, each with exactly one of the eight reasons, a *what to do* column that sends address problems to the premises and numbering problems to us, and **page count unknown shown as still printing**.

BIL-S66

A customer can ask for level pay from the portal, and the button works

STATUS	Exists behind
WAITS ON	**One write, and it is a write the portal is already allowed to make.** Both reads exist and neither recomputes anything; the join control is **held** because *"enrolling is a write to* `account_budget_status` *and the portal may write rows of exactly six kinds; budget enrolment is none of them."* ⚠ Today the standing read **names where to join** and the button there does nothing — a dead control on a money screen. The fix is not a seventh write point: the ask raises an **ordinary service request**, exactly as the hardship screen's own `level_pay` route already does.
SCREENS	25.4, 25.3
DONE LOOKS LIKE	Pressing *Ask to join level pay* on account ACC-10519-C returns a reference, states **nothing has changed yet** and **a clerk decides** as fields on the response rather than as screen copy, and the ask appears on screen 25.3 within the minute.

BIL-S67

A clerk can see what a customer was promised beside what is correct now

STATUS	Exists behind
WAITS ON	**Nothing new on the server** — the level is the same frozen function and the service-request queue already exists. What is missing is the pairing: **the level shown at the ask** has to be carried on the request, because a month can pass and a bill can land before a clerk opens it.
SCREENS	25.3, 25.1, 25
DONE LOOKS LIKE	The queue shows six asks; **SR-4468 reads $112.00 shown against $119.00 now**, the two ineligible rows carry their reason in words (*7 months only*, *plan does not offer it*) with a *Tell them why* action, and *Enrol* → opens screen 25.1 pre-filled.

BIL-S68

A clerk can see what happens to money already paid against a corrected bill

STATUS	Exists behind
WAITS ON	**Nothing on the server — the whole settlement is computed there already**, in a fixed order whose position is the rule: credit note, then the payment is released from the old bill, then the replacement draws it down. Four figures are served (*released · credit on account · takes · returns to credit · still owed*) precisely so a screen does not subtract them itself — *"a frontend that subtracted the corrected total from a payment would be a second definition of settled."* **What is missing is that no screen shows any of it.**
SCREENS	26.6, 26.7
DONE LOOKS LIKE	A $289.40 bill paid in full and corrected to $190.16 reads left to right as **they paid $289.40 → comes off the old bill $289.40 → meets the new bill $190.16 → $99.24 to their credit**, with **"this is not a refund"** stated, the account's other credit named and signed, and the replacement's **original due date** on screen.

BIL-S69

A six-month correction is settled per month, and the two big numbers are separated

STATUS	Exists behind
WAITS ON	**Nothing new** — the release-and-re-apply already runs per bill. The screen is the gap: a batch total cannot answer the call that comes in, which is about one month. And the two figures a clerk will be asked about are **different things**: what the correction changed, and what is still owed including bills that were never paid.
SCREENS	26.7, 26.2
DONE LOOKS LIKE	Six months on one account show a row each, part-paid June reading **$180.00 moves across and $86.20 still to pay**, and the panel separating **+$159.80 (the correction)** from **$709.40 (still to pay, mostly July and August)**.

BIL-S70

Print bills is a Billing tab with a searchable table of the period

STATUS	Exists behind
WAITS ON	**A screen change, not a contract change, twice over.** The server already accepts a cycle filter, and the bills are already listable. Today the screen is reached from the Billing header and offers a form, counts and a button with **no list at all** — so the commonest question here, *is this customer's bill in the set*, can only be answered by generating the archive and looking inside it.
SCREENS	28, 28.1
DONE LOOKS LIKE	The seventh tab opens on cycle **09-A** showing **404 will print · 8 held · 412 in the period**, a searchable table of all 412 with the held reason in the row, and *about 5 seconds · ~2,424 pages* before the button.

Minted in this pass — nineteen

Every one of these was **already described somewhere in this document** and undertaken by nobody: a defect named in the cross-module section, a rule violation in §11, an absence whose "out of scope" rested on an open question rather than a ruling, or a screen that no card reached. **A description commits nobody** — so each is now a commitment with what it waits on, the screens it changes, and what done looks like.

BIL-S71

A clerk can send a bill-ready e-mail that has the customer's figures on it, not em-dashes

STATUS	New build
WAITS ON	—
SCREENS	09, 02
DONE LOOKS LIKE	The queued e-mail preview shows amount, due date and account, each a real value.
MINTED FROM	§13 — DEF-BIL-4448

BIL-S72

A queued delivery row names a template the dispatcher can actually render

STATUS	New build
WAITS ON	—
SCREENS	09
DONE LOOKS LIKE	A queued row shows its template name and the dispatcher accepts it without a type error.
MINTED FROM	§13 — DEF-BIL-4452

BIL-S73

A clerk can see the base charge and the schedule name the run already gathered

STATUS	New read
WAITS ON	—
SCREENS	17, 05
DONE LOOKS LIKE	Step 2's row shows the base charge and its schedule name instead of dropping both.
MINTED FROM	§10 R3 · §13 — gather.py:699-705

BIL-S74

The Enrol button can enable itself, because the plan's budget-billing flag is answered through the seam

STATUS	New read
WAITS ON	—
SCREENS	25.1, 25
DONE LOOKS LIKE	Eligibility reads yes or no with its reason, instead of a tick-box a clerk must assert.
MINTED FROM	§10 — OQ-237, READ half already served at rating/selectors.py:420

BIL-S75

A clerk can enrol every eligible account at once

STATUS	Blocked
WAITS ON	OQ-237 — and on no account recording which plan it is on
SCREENS	25, 25.1
DONE LOOKS LIKE	A bulk enrol states the count and the level range before the click.

MINTED FROM	§10 — OQ-237, the half that is genuinely open

BIL-S76

A clerk can trust a run's status, because the column advances or it is gone

STATUS	New build
WAITS ON	—
SCREENS	27, 10
DONE LOOKS LIKE	Past runs shows a status that changes as the run moves; no row reads draft forever.
MINTED FROM	§13 — DEF-171, DEF-602

BIL-S77

The late-fee floor is one number, read from one place

STATUS	New build
WAITS ON	—
SCREENS	17.20, 22.5
DONE LOOKS LIKE	Both screens quote the same floor, and a test fails if the two constants diverge.
MINTED FROM	§13 — DEF-205

BIL-S78

BR-BIL-030 means one thing, because it is defined once

STATUS	New build
WAITS ON	—
SCREENS	—
DONE LOOKS LIKE	No screen changes. The two definitions are reconciled and the losing one is deleted.
MINTED FROM	§13 — DEF-052

BIL-S79

A Japanese or French bill line says billing cycle, not bicycle

STATUS	New build
WAITS ON	—
SCREENS	03, 01
DONE LOOKS LIKE	The cycle label reads correctly in all eleven catalogues.
MINTED FROM	§13 — DEF-BIL-4759

BIL-S80

A clerk reads money at two decimals on every billing screen

STATUS	New build
WAITS ON	—
SCREENS	25, 25.1, 25.2, 26, 26.1, 26.2, 12, 13
DONE LOOKS LIKE	No figure renders at six decimals or through String(); every one goes through the money cell.
MINTED FROM	§11 — hard rule 2, four violations

BIL-S81

A clerk can page, sort and filter every billing list, at any size

STATUS	New build
WAITS ON	—
SCREENS	25, 26, 12, 13
DONE LOOKS LIKE	Each list has a pager and sortable headers, and the sort reaches the server.
MINTED FROM	§11 — B14 16, NFR-3

BIL-S82

A billing form reads label, then input, then help — and its chips render

STATUS	New build
WAITS ON	—
SCREENS	21, 22.1, 25, 25.1, 11, 12
DONE LOOKS LIKE	Help sits under its input, and no filter chip uses a class no stylesheet declares.
MINTED FROM	§11 — B14 39, B14 12, B14 21

BIL-S83

A clerk never reads a rule id or an internal code on screen

STATUS	New build
WAITS ON	—
SCREENS	26, 26.1, 17, 13
DONE LOOKS LIKE	Refusals and correction reasons appear in the clerk's words; no BR- or DEF- id is visible.
MINTED FROM	§11 — B14 32, two violations

BIL-S84

Nobody can reach the old six-step run wizard, because it is gone

STATUS	New build
WAITS ON	§03 landing first
SCREENS	04, 05, 06, 07, 08, 09
DONE LOOKS LIKE	The old routes 404 or redirect to the four-step run; no door anywhere still points at them.
MINTED FROM	§17 — four as-built steps no commitment named

BIL-S85

A clerk can generate and send the corrected documents at the end of Fix a bill

STATUS	New build
WAITS ON	—
SCREENS	26.8
DONE LOOKS LIKE	Step 4 posts the corrections, shows what was written, and cannot be run twice.
MINTED FROM	§06 — the step that moves the books, named by no commitment

BIL-S86

A clerk can cancel a bill, and is told what the cancellation moves before confirming

STATUS	New build
WAITS ON	—
SCREENS	02, 11
DONE LOOKS LIKE	A cancel control states the reversing entries and the balance effect, then confirms.
MINTED FROM	§10 — OQ-012 is CLOSED → BR-BIL-009, BR-PAY-011

BIL-S88

A clerk never meets copy describing a screen that cannot be reached

STATUS	New build
WAITS ON	—
SCREENS	04, 11
DONE LOOKS LIKE	The two unreachable strings are gone from all eleven catalogues.
MINTED FROM	§10 — billing.arrives.run, billing.arrives.oneTime

BIL-S89

B14 rule 7 can be complied with, because it names the style world the product uses

STATUS	Blocked
WAITS ON	a B14 amendment
SCREENS	16, 04
DONE LOOKS LIKE	The date rule cites the kit the screens are built from, and a screen can be graded against it.
MINTED FROM	§11 — the rule names the wrong style world

Minted after the ruling — one

The audit found the $25 late-fee floor was a hard-coded default that nobody had ratified. It was put to the Product Owner in plain English with its worked figures; the answer made it **tenant configuration**, which is work rather than a description.

BIL-S102

A utility sets its own late-fee balance floor, instead of inheriting a number written into the code

STATUS	New build
WAITS ON	— **ruled 2026-09-13.**
SCREENS	17.20, 22.5
DONE LOOKS LIKE	The floor is read from tenant settings beside the late percentage, the minimum and the grace window; no default argument decides it.
MINTED FROM	A Product Owner ruling of 2026-09-13, verbatim: *"Yes, if it is limit set in setting then - it is not hard coded; it should be based on setting"*. **Measured:** the floor is `LATE_FEE_BALANCE_FLOOR`, a *default argument* of `assess_late_fee` (`late_fee.py:561`, `:748`), while `BR-BIL-021` already makes the late percentage, the minimum and the grace window tenant config. That file's own exception (7) says the question *"was never put"* and stamps itself `VERIFIED-UNRATIFIED`. It has been put, and answered.

Minted in the commitment pass — twelve

Nine of these close an OPEN defect this document already cited and nobody had undertaken. A defect row records that somebody noticed; it undertakes nothing. Two more are fields the endpoint *already serves* and no screen shows — found by counting served against shown rather than by reading a drawing. The last one is this document's own single point of failure. **Every figure below was measured against the tree at `a1d37a68`, not quoted from a register.**

BIL-S90

A utility with more than 25,000 accounts can run a cycle at all

STATUS	New build
WAITS ON	— nothing. `NFR-5` already forbids a per-tenant cap and `CLAUDE.md` names it load-bearing
SCREENS	16, 17
DONE LOOKS LIKE	Scope accepts a 40,000-account cycle and the run completes; no constant caps it.
MINTED FROM	§13 — `DEF-BIL-4224`. **Measured:** `run_jobs.py:227` `MAX_ACCOUNTS_PER_RUN_DEFAULT = 25_000` against a T5 ceiling of 40,000. **Blast radius: one constant, one module — and every tenant above 25,000.**

BIL-S91

A clerk never decides on a row that shows no amount

STATUS	New build
WAITS ON	—
SCREENS	17, 17.1, 05
DONE LOOKS LIKE	Every reviewable row carries an amount and a currency, or says in words why it cannot.
MINTED FROM	§13 — `DEF-2596`. **Measured:** `read_analysis.py:328-329` returns `amount: None, currency: None`.

BIL-S92

An account whose meter simply was not read still reaches the run

STATUS New build

WAITS ON —

SCREENS 17, 17.1

DONE LOOKS LIKE An unread meter appears as a decidable row, not a refusal, and a customer type with no base charge still bills.

MINTED FROM §13 — `DEF-3221`, high and open

BIL-S93

A customer receives a bill that looks like a bill

STATUS New build

WAITS ON —

SCREENS 28, 03

DONE LOOKS LIKE The printed page uses the product's own layout, not Courier with space-aligned columns.

MINTED FROM §08 — `DEF-BIL-4524`. **Measured:** `documents.py:2017` still reasons from *“Courier’s whole appeal here was that a column position is integer arithmetic”*. **Blast radius: every bill this product prints.**

BIL-S94

The bills list answers at 40,000 accounts, because the search stops building every matching row

STATUS New build

WAITS ON —

SCREENS 01

DONE LOOKS LIKE Search and filter resolve in SQL; the response time does not grow with the account count.

MINTED FROM §02 — `DEF-BIL-4713` · `NFR-3`

BIL-S95

A lost race refuses cleanly instead of failing the whole transaction

STATUS New build

WAITS ON —

SCREENS 16

DONE LOOKS LIKE Two clerks starting one cycle: the second is refused by name, and the first run is unharmed.

MINTED FROM §16 — `DEF-2868`: the re-read runs inside the transaction the failed insert already broke. This is hard-won law 14 — a SAVEPOINT is the only fix, and the suite runs on SQLite where the bug cannot appear.

BIL-S96

Five seams read the tables that now exist instead of substituting a fact

STATUS New build

WAITS ON —

SCREENS 17, 17.0

DONE LOOKS LIKE	Each of the five returns the real row, no seam answers from a literal.
MINTED FROM	§13 — `DEF-079`, high and open: *“Five documented seams still SUBSTITUTE a fact over tables that now exist.”*

BIL-S97

A clerk can find a bill by its cycle, which the search box already offers to do

STATUS	New build
WAITS ON	—
SCREENS	01
DONE LOOKS LIKE	Typing a cycle code returns that cycle's bills; the placeholder stops promising what the predicate cannot do.
MINTED FROM	§02 — `DEF-1001`

BIL-S98

The four refusal states on the bill header can be seen before they ship

STATUS	New build
WAITS ON	—
SCREENS	02, 02.1
DONE LOOKS LIKE	A seeded draft bill exists, and a test renders each of the four disabled reasons.
MINTED FROM	§02 — `DEF-2901`: no database in this repo has a draft bill, so three of those four states have never been rendered by anybody.

BIL-S99

A clerk reads the tax and the service period on the bills list, without opening the bill

STATUS	Exists behind
WAITS ON	— the read already serves all five
SCREENS	01
DONE LOOKS LIKE	The list offers subtotal, tax and the service period as columns; none needs a new read.
MINTED FROM	§02 — **measured**: `invoice_row` serves **20** fields, **12** reach screen 01, and `subtotal`, `tax_total`, `service_date`, `period_start`, `period_end` are served and shown nowhere.

BIL-S100

A clerk can see when a hold started and whether it was ever caught up

STATUS	Exists behind
WAITS ON	— the read already serves all five
SCREENS	20, 14
DONE LOOKS LIKE	The queue shows the held-on date, how it was held, and the run that later billed it.
MINTED FROM	§17 — **measured**: `pending_row_dict` serves **16** fields, **7** reach screen 14, and `held_on`, `origin`, `detail`, `resolved_by_run_id`, `resolved_on` are served and shown nowhere. **A clerk cannot tell a caught-up hold from a live one.**

15

What this pass changed, line by line

A changelog of the analysis itself, so a reader of a later version knows what moved and — more importantly — what deliberately did not.

Corrections applied to my own reading, and what each pass added

This was a running log of every pass. **It ran to 3,058 words about the analysis rather than about the product**, so it is a table now: what moved, and the correction worth knowing. The full narrative is in the session it came from, not here.

PASS	WHAT IT ADDED, AND THE CORRECTION IT CARRIED
The first reading	Four corrections to my own instrument, kept rather than tidied: a route glob that matched the app's own API modules, a stub envelope one level too deep, a dead-copy list built from front-end greps that **mislabelled all seven** keys (they render from the backend), and a literal-key grep that **under-reported by sixteen**. Each would have produced a confident report of something untrue.
§03 — the bulk run	Six steps became four. **The obstacle was that money does not exist until step 4**, so the decision step showed nulls and a clerk decided blind — and the fix was already built and uncalled since 11 August. Twelve engine/screen disagreements tabled together for the first time.
§04 — single bills	Built as a three-step wizard. The boundary that shaped it: **billing may not store a reading, but the screen may still take one** — the field posts to Meter Reads.
§05–§08	Budget bills, fix a bill, past runs and printing. **An alignment defect the Product Owner found that no check here could see:** a CSS rule naming only `td` left **143 of 143 numeric headings** left-aligned over right-aligned values, in every wireframe since screen 01.
Budget bills, confirmed	They run inside the cycle, never separately — which **invalidated one of my own drawings**: a bulk settle-up button that would have been a second writer for money the run already posts. And the portal's *Join level pay* was found to be a dead control on a money screen.
§06 as a wizard	Four steps, and the treatments rewritten in a clerk's words after the PO said the engine vocabulary was unreadable. **Two large numbers that mean different things** now sit apart on the batch screen.
The tag audit	The four state chips were applied to **fifteen places out of eighty-five**; all seventy commitments carried a word that is not one of them. The distribution is the finding: **48 of 70 already exist behind a screen that does not ask for them**.
The duplication pass	**§14's index repeated 3,158 words of the cards beneath it verbatim** — 67 of 70 waits-on texts byte-identical. And §12, the section that says where the work is, **listed nine screens and omitted sixty**.

The chip audit — and three contradictions it found in my own work

Product Owner, 2026-09-13: *"inspect entire document — screens, rules, any aspect that we need to pass to claude code for build — everything should be tagged with chip."* The four state chips reached **485 other chips** that are a different axis — *SERVED*, *UNCHANGED*, *HEALTHY*, *MOVED* — and converting those would have destroyed information, so they stand. What was genuinely untagged, and what was wrong, is below.

WHAT THE AUDIT FOUND	WHAT IT WAS, AND WHAT IT IS NOW

⚠ Three screens were told to build twice, differently	Screens **10, 13 and 14** each carried a *BUILD THIS* band **and** a *REDRAWN AS* stamp pointing somewhere else — two drawings to build for one screen, which is exactly the confusion the PO warned about when the redraws moved. The bands read `SUPERSEDED` now, naming the destination, and the drawings stay as the earlier reading rather than as work.
⚠ Seven as-built screens said "No redraw" and were redrawn	My own stamp pass covered only screens 10–15, so **04, 05, 06, 07, 08, 09 and 14** all read *No redraw. There is nothing to build from it.* §03 replaces every one of them — six run steps into four screens. All seven now name their destination, and the three that **collapse into one screen** say so, because that collapse is §03's central change.
⚠ Three false `No commitment` **chips**	Screens 06, 08 and 09 carried one on my instruction that an absence is a finding. It was not an absence — §03's cards cover all three. Removed. **One screen in the whole document now carries it**, and it is a carry-over table rather than a screen.
Commitment ids were drawn as defects	§10's ruling table rendered each `BIL-S` id as a **red chip** — the colour this document uses for something broken — when a commitment is the opposite. Six rows now carry the id and **its state**.
A violation was scheduled or it was not, and you could not tell	§11 gains a **Fixed by** column: all **17** violations name the redraw that closes them and what that work waits on.
Eleven BUILD THIS bands said nothing about readiness	Each now carries its screen's **strongest state**, so a band that says *build this* also says whether anything is in the way.
The chip meant two different things	**485 of the 757 chips on this page were descriptions** — *SERVED*, *MOVED*, *HEALTHY*, *STUBBED*, bare defect ids — drawn as the same coloured pill as a commitment state, in the same five colours. Green meant *a read is served* in one table and *the work is already built* in the next. Descriptions now take a rule-left tag; **the chip is the five commitment states and nothing else**.
The register could not be derived from, so it was typed from	Its *Screens* column is prose — `01 · R4 ® · 13`, `§03 — the bulk run`, *all 15*. Every card now carries `data-screens` resolved to canonical anchors, and **§12, §14 and all 77 screen bands are generated from it**. **This is the root cause of four false-chip findings in one session**, each previously fixed one at a time.
61 of 83 screens carried no state chip at all	Including **every screen of §04–§08** — the 26 built in this session. The chip pass predated them and nothing re-ran it. All 77 now derive their chips from the commitments that name them, and each chip links to its register entry.
Two commitments named a screen that was never drawn	`BIL-S40` and `BIL-S43` both named **18.1**, a sign-off variant this document does not contain. Recorded against screen 18, with the undrawn variant stated on the card rather than left pointing at nothing.
Eight cross-module faults were undertaken by nobody	That was §13's finding and it stayed a finding. `BIL-S71` – `BIL-S79` now name them, against the defect ids the rows already carried.
Out of scope rested on things that are not rulings	Re-tested against the repository, not against this document's memory of it. **Two were wrong.** *A bill cannot be cancelled* cited `OQ-012` as open — it is **CLOSED**, ruled into `BR-BIL-009`, which says build it. *Budget-billing eligibility* repeated *"nothing publishes the flag"*, which that question's own row records as stale for five days. **Two were right for the wrong reason**, and one — hand-editing a charge line — has no ruling anywhere and is now a specification that waits on one.
§09 cited no commitment on any journey failure	Seventeen findings, six of them blocking, and not one said who would close it. All ten walked screens now cite theirs; the six stack traces close on `BIL-S9`, the missing error boundary that is the one source fact behind them.
Not one drawing said what it was	**91 figures, zero captions** — the `.pw-cap` class was defined in the stylesheet and never used once. A builder had to read the band above a figure, and in §03–§08 there was no band. Every drawing now opens with **AS-BUILT** (with the date and commit it was measured at), **AS-BUILT**

what it was	figure, and in §03–§08 there was no band. Every drawing now opens with **AS-BUILT** (with the date and commit it was measured at), **AS-BUILT, FROM CODE** (with the reason it was never rendered) or **REDESIGN** (with what the product ships instead, and NEVER BUILT). **70 of the 91 are redesigns that have never been built**, which no single figure previously admitted.
Twelve rulings existed nowhere but in this file	The appendix had two ruling tables and neither asked the question. Grepped against the tree: **12 of the 24 rulings this document quotes exist in no file** — including the whole shape of §03, §06 and §08, the *per connection* rule that decides what a master account is charged, and the confirmation that approval moves the books. **The constraint that keeps the tree clean is the same one that puts them at risk.** Now one table — and on 2026-09-13 the Product Owner ruled the risk accepted rather than landed, so the commitment that stood here is withdrawn and the row is out of scope with a ruling.
A defect row was being read as a plan	51 defect ids are cited here; **9 open ones had no commitment at all**. A defect records that somebody noticed — it undertakes nothing. `BIL-S90` – `BIL-S98` now name that work.
Counting rows drawn instead of fields served	A count of drawn rows is not checkable. Counted from the row builders instead: **screen 10 serves twelve fields and shows none of them**; screens 12 and 13 print a raw UUID while the read beside them already serves `account_no` and `case_no`. So B14 31's two breaches need **no new read**, which the violation table did not say.
Three numbers this document repeated were wrong	**Measured, not quoted.** (1) *"89 open billing defects"* — the register holds 228 billing rows and **86** are open. (2) *"39 rules"* for B14 — it holds rule rows 1–21 and 31–39, so **30 rules** with 39 the highest number; `CLAUDE.md` says 38 and B14's own line says thirty-one, so three sources gave three numbers and none was the count. (3) A code comment says `assess_late_fee` has *no production caller* — it has exactly one, and that one has none, so the claim is true in effect and wrong as written. **One claim was checked and upheld:** all 15 non-test billing `.tsx` files really are on `route-b-migrated.txt`.
Five guarantees were enforced by nothing and owned by nobody	The guarantee table graded them and stopped there. All 13 now cite a commitment or say *enforced*; so do all 11 seam rows. The seam table's *Grades, not work* label was true until two of its rows turned out to be work.
Nothing said what any of it costs a person	101 commitments in register order tell a reader nothing about where to start. The nine that cost the most is ordered by **what a person cannot do today**, and the top entry is not a billing defect in the ordinary sense: a constant of `25_000` that silently excludes the top of the market this product was re-scoped to serve on 2026-09-06.
Three rulings arrived, and two commitments stopped being work	**Put to the Product Owner, answered the same day, recorded here.** (1) *"We ahve to write everything in HTML artifact only Billing, as built - noting in repo"* — so the twelve un-landed rulings are an **accepted risk with a ruling behind it**, and `BIL-S101` is withdrawn. (2) *"you cannot change it here - it should be thorugh process only"* — a charge line is never editable on screen, so `BIL-S87` is withdrawn and that absence is out of scope **with law behind it at last**. (3) *"it is not hard coded; it should be based on setting"* — the late-fee floor becomes tenant configuration, which is the one ruling that **created** work: `BIL-S102`.
The artifact became the system of record, so it grew the two registers it was missing	If nothing goes in the repo, the decision log and the open-questions register have to live here or nowhere. Decisions taken here records the two judgments that were **ours** — the arrears threshold, and the pre-bill check running in a worker (137 ms × 25,000 = 57 minutes against a 60-second wall, so the arithmetic decided it, not a preference). Still open holds the one question only the Product Owner can close.
A question was answered in half, and only the answered half was recorded	The floor ruling settles that it is a **setting**. It does not settle **what the setting is measured against** — the part of the bill that is actually ageing, or the whole bill. On a $60 bill with $40 disputed those two readings differ by the **entire fee**. Taking the ruling as covering both halves would have been the cheapest mistake available, so the second half is written up as still open rather than assumed.
My own generator overwrote a table and left a stale copy of another	The §12 regenerator targeted *the first table in the section*. Two tables were added above the build index in the previous pass, so it overwrote the cost ranking and left a **stale duplicate build index still naming two withdrawn commitments**. Found by the dangling-link check, not by reading. The build index now has its own anchor `#bidx` and the generator asserts on it, and a new check fails if a second copy ever appears — the fix is the anchor, not the repair.

What the chips are deliberately NOT applied to: §13's seam grades and §02's field tables. `SERVED`, `HEALTHY` and `FORKED` answer *what is true today*; the five answer *what the work waits on*. A row can be `NOT SERVED` and Exists behind at the same time without contradiction, and collapsing the two axes would lose that. **Which is why they no longer share a shape** — a description is set as a rule-left tag and a commitment as a filled chip, so the two axes cannot be confused at a glance. Before this pass they were the same pill in the same five colours, **485 of them**, and green meant two different things on one page.

Findings I expected and did not get

- **Every one of the seven routes has at least one audited inbound door.** I expected unreachable screens — that is usually this audit's biggest catch — and there are none. The door finding here is thinner and different: **the pending queue and Print bills each hang off exactly one button on one screen**, and neither is in `SIDEBAR_DESTINATIONS`, so no reachability test covers the two screens that carry the only working delivery channel and the only view of unbilled accounts.
- **No dead links.** Every constructed billing path resolves against the route table.
- **No cross-module write.** I looked for the real test and found none.
- **No seam returns a raw queryset.** Ten seam files, 94 functions, zero.

Structural faults reported and deliberately not repaired

- **CLAUDE.md says B14 holds thirty-eight rules; it holds thirty-nine.** Rule 39 was ratified 2026-09-11 — today. This is the *sixth* time that pointer has lagged its target, and CLAUDE.md's own text predicts it: *"ratifying a rule and updating the pointer to it are two acts and only the first one feels like the work."* **Not repaired, because this pass writes nothing to the repo.**
- `DEF-3153` **is recorded as FIXED and is half fixed.** The backend comment was corrected 2026-09-02; the front-end catalogue string still refuses, in eleven locales. Reported, not reopened.
- **Screen numbering has no gap and the wizard steps are numbered 04–09 inside a 01–15 sequence.** If a later pass adds a screen, it appends — nothing is renumbered, for the same reason `OQ-291` is not `OQ-00291`.

What was deliberately left out

- **Line numbers are cited and will move.** Values are pinned as literals; paths are stable; line numbers are a convenience and are stale the moment someone edits above them.
- **The source catalogues** — 15,424 keys × 11 locales. Quoted where a string is the finding.
- **Bill detail's Lines, Chain and Audit tabs are described, not drawn.** Three tab panels of one screen; the Document tab carries the finding.
- **The 350 journey files.** None was walked, because none can be without a backend.
- `DEF-079` **'s five substituting seams** — the highest-value thing a next pass should pick up, and named in §08 rather than guessed at.
- **The 86 open billing defects** are drawn on where they corroborate a finding; this document is not a re-issue of the register. *(This read 89 until the commitment pass counted the register: 228 billing rows, 86 of them open.)*

Nothing was deleted, and nothing was written. No row was struck from any register, because no register was touched. **The findings in this document are recorded nowhere in the repository** — not as a defect, not as a decision-log row, not as an amended open question. Under the repo's own step W that is a gap, and it is a deliberate one: this pass was asked to analyse and not to write. **Eighty-nine specifications** — nineteen of them minted in this pass — seven rotting holds, two dead-copy keys, one missing error boundary and one un-filed NFR-3 breach live only here. **Two of the corrections above are corrections to the repository's**

breach live only here. **Two of the corrections above are corrections to the repository's own registers**, not to this document: a question recorded CLOSED whose body still describes the open world, and a business-rule sentence that went stale and was quoted onward. Neither is filed, for the same reason.

16

Appendix

Eleven tables, each exhaustive over *this document*, built from a section-by-section walk rather than from memory.

Every ruling this document quotes, and whether it exists anywhere else

A ruling that lives only in this document is a ruling at risk. It cannot be grepped, nothing can count what depends on it, and if this file is lost the decision is lost with it. The *Where else it lives* column is the result of a grep against the tree at `a1d37a68`, run for this pass — not an assumption about what was probably written down.

Fifteen of the twenty-seven exist nowhere but here, and they are not minor: they include the entire shape of §03, §06 and §08, the *per connection* rule that decides what a master account is charged, and the confirmation that approval moves the books. **Every one was given in this chat, and this chat is under a standing instruction not to write to the repository.** That was put to the Product Owner on **2026-09-13** as a choice between landing them in the repo, producing a patch for somebody else to apply, or accepting the risk. **They chose the third, in these words: *"We ahve to write everything in HTML artifact only Billing, as built - noting in repo"*.** So this is no longer a gap awaiting work — it is an accepted risk with a recorded ruling behind it, and the commitment that used to sit here (`BIL-S101`) is **withdrawn**. **What the ruling costs is still true and worth saying once:** these twelve cannot be grepped, nothing can count what depends on them, and if this file is lost the decisions go with it. **The ruling is itself an entry in this table** — the instruction that nothing goes in the repo exists only in the artifact, which is the shape of the whole risk in one line.

RULING, VERBATIM	DATE	WHERE ELSE IT LIVES	WHAT IT DECIDES
"we dont what full view of the document - you give a link to download but full view of document is not requires - it does not add any value to user"	2026-09-11	**NOWHERE** — this document is the only copy, and that is ruled	The bill detail's embedded PDF viewer is removed; the document becomes a download link.
"our idea is to show all the data to billing clark before submitting the bill run - it kind of allow him to do all decision before hand, see the data correct the data and then submit it"	2026-09-11	**NOWHERE** — this document is the only copy, and that is ruled	The whole shape of §03: every decision is made before the run is submitted, not after.
"between step 1 and step 2 - we are bascially running pre bill check"	2026-09-11	**NOWHERE** — this document is the only copy,	A pre-bill check runs stages 1–5 and writes nothing; screen 17.0 exists because of this.

		and that is ruled	
“per conenction”	2026-09-11	**NOWHERE** — this document is the only copy, and that is ruled	A master account’s base charge is per connection, not per account.
“as per connection”	2026-09-11	**NOWHERE** — this document is the only copy, and that is ruled	Each connection is governed by its own plan — its own taxes, fees and discounts. Worked: fee $0.57, bill $70.17.
“all dates should be date picker, all drop down should be drop down, all chips should be multi select”	2026-09-11	**NOWHERE** — this document is the only copy, and that is ruled	Dates, dropdowns and chips across every billing screen; multi-select on chips is the new part.
“you can calculate it on screen - in such case we will have to show work in progress to clark”	2026-09-12	**NOWHERE** — this document is the only copy, and that is ruled	The screen may compute and must then show a work-in-progress state; the rule stays on the server.
“we will ahve to make it wizard - 4 steps”	2026-09-12	**NOWHERE** — this document is the only copy, and that is ruled	Fix a bill becomes a four-step wizard — the whole of §06.
“so im confirming books wil be updated”	2026-09-12	**NOWHERE** — this document is the only copy, and that is ruled	Approval signs and execute posts, inside one transaction per bill.
“We will need Print bills as tab at Billing level”	2026-09-12	**NOWHERE** — this document is the only copy, and that is ruled	Print bills becomes the seventh Billing tab — the whole of §08.
“Budget bills run inside the regular cycle and must not run separately”	2026-09-12	**NOWHERE** — this document is the only copy, and that is ruled	§05’s shape, and why the bulk settle-up button was removed as a second writer.
“everything should be tagged with chip”	2026-09-13	**NOWHERE** — this document is the only copy, and that is ruled	The five-chip vocabulary and the commitment law this document is built on.
“delivery is QUEUE-based … email ships W6”	2026-08-08	`BR-BIL-016` CONFIRMED (PO A-7)	No dispatcher yet, and that absence is ruled rather than missing.
“B An export only”	2026-08-11	`BR-BIL-065` · `OQ-112`	The release authorisation record gets no screen.
“Give a running number always even for Draft”	2026-08-11	`BR-BIL-063`	A bill number is minted when the draft is created; an unused one is lost.
“option A: the cap APPLIES”	2026-08-11	`OQ-104` CLOSED · `corrections.py:974`	The 12-month back-bill window caps an UPWARD correction too.
“bulk APPLY of a late fee is allowed, bulk WAIVE is not”	2026-08-11	`BR-BIL-060` CONFIRMED	Bulk actions everywhere except a first-time late-fee waiver.
“out platform will porvide standard 3 to 5 variouation of printed bill - user can use those only in V1”	2026-08-12	decision log, 2026-08-12	No custom bill design in V1. This is what makes “no page designer” out of scope.

"Money 4 decimal, percentage 6 decimal, display both 2 decimal	2026-08-22	hard rule 2	Every money figure on every screen.
"it will be capture at the time of bulk billing - not here"	2026-08-23	`CR-029` D8③ · `OQ-241` CLOSED	A cycle-less one-time bill takes its due date from the run.
"you ahve to remove - Filters that arrive later … Export and column choices"	2026-08-27	decision log · `CR-043` finding 10	An absence is stated where the control would be, not in an empty state.
"no Databse ID on screen - that is a rule"	2026-08-31	`B14` rule 31	No database id is ever the visible value of a field.
"Treat every number drawn in a wireframe as an illustration, never a fact to build to"	2026-09-10	`B14` rule 37	Every figure in every drawing here is an illustration.
"in the forms for the fields. It starts with the field name. then the input field, and then it should show the helper text below that input field."	2026-09-11	`B14` rule 39	One spacing scale; a field is label → input → helper.
"We ahve to write everything in HTML artifact only Billing, as built - noting in repo"	2026-09-13	**NOWHERE** — this document is the only copy, and that is ruled	Nothing is written to the repository. This artifact is the system of record for all of it — **including this ruling**.
"Yes, if it is limit set in setting then - it is not hard coded; it should be based on setting"	2026-09-13	**NOWHERE** — this document is the only copy, and that is ruled	The late-fee balance floor is tenant configuration, not a constant in the code. `BIL-S102` **New build**
"you cannot change it here - it should be thorugh process only"	2026-09-13	**NOWHERE** — this document is the only copy, and that is ruled	A charge line is never editable on the screen; a line changes only through the correction process. This ratifies a refusal that until now rested on a docstring.

Decisions taken here, because they were ours to take

A judgment recorded with its figure is the deliverable; a question is the failure to make one. These two were on the open list and **neither needed the Product Owner** — one is a display threshold that moves no money, the other is arithmetic against a measured number. Both are revisable, and both are written down here rather than left as questions.

DECISION	TAKEN	THE FIGURE IT RESTS ON, AND WHY IT WAS OURS
An account is shown as in arrears at 60 days past due, or above twice its trailing average owed	2026-09-13 `LANE JUDGMENT`	Screen 17.21 needed a threshold to draw and had none. **Why ours:** it decides when arrears are *surfaced* to a clerk who is still deciding — no money moves, no signed rule is reversed, and nothing is charged differently. Drawn as 60 days / 2× average because both are already how the dunning ladder talks about age. **Revisable:** if a utility wants 30 or 90 it is a setting, not a redraw.
The pre-bill check runs in a worker with a progress state, and never inline	2026-09-13 `ARITHMETIC`	**Measured: 137 ms per account on real PostgreSQL.** 25,000 accounts is **57 minutes**; the request wall is about **60 seconds**. So inline is not a preference that lost, it is impossible above roughly 440 accounts. **Why ours:** the figure decides it, not a policy. This is what screen 17.0 already draws and what `BIL-S41` already carries. **It does not refuse above the bar** — it runs asynchronously and the screen fills in, because a refusal would leave a T5 utility unable to check at all.

Still open, and only the Product Owner can close it

One question, narrowed. It is here rather than in an open-questions register because nothing is written to the repository — so **this table is the register**.

Which balance does the late-fee floor test — the part that is actually ageing, or the whole bill?

A disputed amount does not age (`BR-MET-133` clause 4, ruled 2026-08-26). So on a **$1,000** bill with **$400** disputed, only **$600** ages, and the fee is **$9.00** rather than $15.00. That part is settled. What is not settled is which figure the floor is compared against:

On the ageing part: a **$60** bill with **$40** disputed leaves **$20** ageing, which is under the floor — **no fee at all**. **On the whole bill:** $60 is over the floor, so a fee is charged on the $20. **The two readings differ by the entire fee on every small part-disputed bill.**

Why this is yours and not ours: it changes what a customer is charged. Your 2026-09-13 ruling settled that the floor is a *setting* rather than a constant, which is built as `BIL-S102` — but a setting still has to say what it is measured against, and that half was not in the question you answered. **One line closes it:** the ageing part, or the whole bill.

Fields served against fields shown

A count of drawn rows is not checkable; **this is**. For each screen with a read behind it: how many fields the endpoint returns, how many reach the screen, and which are served and shown nowhere. Read out of the row builders in `apps/billing/selectors.py` at `a1d37a68` and counted against this document's own field tables.

Screen 10 is the finding: twelve fields served, zero shown. The screen is a grey panel saying the feature *"arrives with the billing run work"* over a read that returns run number, period, account count, bills issued, held count and status. That is the same screen §09 walked and graded *the journey completes, wrongly*, and it is why `BIL-S1` is Exists behind rather than new work. **Two more screens print a raw UUID while the read beside it already serves the number** (`account_no` on 12, `case_no` on 13) — so B14 31's two breaches need no new read at all, which is not what a reader would assume from the violation table.

#	SCREEN	THE READ BEHIND IT	SERVES	REACH	SERVED AND SHOWN NOWHERE	UNDERTAKEN BY
01	**All bills**	`invoice_row` · `GET /invoices`	20	12	`subtotal`, `tax_total`, `service_date`, `period_start`, `period_end` — plus three ids correctly withheld under B14 31	`BIL-S99` Exists behind
02	**Bill detail**	`invoice_detail` · `GET /invoices/{id}`	14	—	**Not counted — this screen has no field table in this document.** It is the screen a clerk lives in and it has the richest read in the module.	**a gap in this document**
10	**Past billing runs**	`_bill_run_history_row` · `GET /billing/runs`	12	0	**all twelve** — `run_no`, `cycle_id`, `period_start`, `period_end`, `issued_at`, `accounts`, `bills_issued`, `held`, `status`, `bill_type`, `created_at`, `id`	`BIL-S1` Exists behind
12	**Budget Bills**	`budget_status_row` · `GET /budget-billing`	12	6	`account_no` — **and the screen shows a raw UUID instead** · `level_override`, `level_override_reason`, `enrolled_on`, `last_leveled_at`	`BIL-S10` Exists behind

13	**Fix a bill**	`correction_case_row` · `GET /billing/corrections`	18	4	`case_no` — **and the screen shows a raw invoice UUID instead** · the whole approval trail: `initiated_by`, `approved_by`, `approved_on`, `executed_on`, `closed_on`, `posting_period`	`BIL-S10` `BIL-S7` Exists behind
14	**Pending queue**	`pending_row_dict` · `GET /billing/pending`	16	7	`held_on`, `origin`, `detail`, `resolved_by_run_id`, `resolved_on` — plus four ids correctly withheld	`BIL-S100` Exists behind

Not counted, and that is a gap in this document rather than in the product: screen 02 and its five tabs, and all **26** redesign screens in §03–§08, carry no field table, so no served-against-shown count can be taken for them. Screen 02 is the worst of those to be missing.

A · Every rule this document states, and where it lives

The **home** column is the point. `INHERITED` — it exists elsewhere and this document follows it. `EXTENDS` — it adapts an existing rule to this context. `NO HOME` — stated only here, which means it is about to be broken.

RULE AS STATED HERE	WHERE	HOME
A route existing is not a door; an answer not read out of the source is not an answer	§02 every *How you get here*	INHERITED — the module-analysis method
Absences come from the product's own copy, never from an auditor's impression	§05	INHERITED
A recorded ruling makes an absence out of scope; no ruling makes it a specification	§05	INHERITED
A screen's health is the *worst* of its fields, never an average	§02, App. J	INHERITED
A seam is graded on what it *hands back*, not on what its docstring says	§08	INHERITED
Wireframes are one 1440 window scaled as a unit, never reflowed; desktop only	every drawing	INHERITED — R7, R10
Tokens from the shipped app, layout from the prototype, and never a prototype class presented as the product's	§01	INHERITED — R9
A remedy is additive; REMOVED needs a quoted ruling and nothing else will do	§07	INHERITED
The shipped page title is 40px, not the 32px `tokens.css` calls the app default	§01	**NO HOME** — measured here; `tokens.css:203` still says otherwise
A coercion that turns an absence into a malformed value defeats a boundary built to catch the absence	§02 screen 13, §06	**NO HOME** — the general form of DEF-180, stated only here
A renderer whose payload contract does not match its server's builder is a screen-killing class, not two bugs	§02 screen 13, §04	**NO HOME** — `DEF-RBS-4352` and the Fix-a-bill finding are instances; nothing names the class
A refusal derived from the response self-heals; a hardcoded one rots. Nine did the first, five the second.	§05	EXTENDS hard-won law 2's mirror — the law says *write the test*; this says *derive the sentence*
A server that paginates and a client that cannot ask for page 2 is an NFR-3 breach on the client side	§06, App. J	**NO HOME** — NFR-3 is written as a server obligation

Four NO HOMEs. Each is a rule stated only in a document nobody has to read.

B · Every worked figure, verbatim and unrounded

Not one figure below is a signed figure, and every drawing in this document is an illustration. B14 rule 37, PO 2026-09-10: *"Treat every number drawn in a wireframe as an illustration, never a fact to build to."* These are test cases. Do not assert one in a test and do not seed a fixture from one. The rules that govern them are hard rule 2 (stored to 4 decimals, displayed at 2) and B4's rounding, which is ❄ frozen and finance-signed.

FIGURE	WHERE, AND THE SENTENCE AROUND IT	STATUS
`$84.00`	Screens 01, 02, 03, 13 · the worked bill: `$18.00` base + `$60.00` usage + `$6.00` tax. Carried through every screen so one customer can be followed.	illustration
`$60.00`	Screen 01 row 2 · the Northside Warehouse fire line. **Chosen deliberately:** CLAUDE.md records that this figure was `$110.00` until the PO removed private hydrants on 2026-09-03, making it `$60.00` — *"–$50.00 a month on a real customer."*	illustration, and a real supersession
`8 kgal @ 7.5000`	Screens 03, 05, 13 · four-decimal rate, two-decimal display, per hard rule 2	illustration
`$342,180.00` · `7240` · `3`	Screen 01 rollup · taken from the prototype's own Billing head	illustration
`1,284` · `1,263` · `21` · `34`	Screens 04–07 · accounts in the cycle, bills produced, held, needing a decision. `1,284 − 21 = 1,263` reconciles on purpose.	illustration
`62.500000`	Screen 12 · the budget *Level*, drawn at **six decimals on purpose** — that is the defect (hard rule 2 says display 2)	illustration of a defect
`" USD"` and `" · → "`	Screen 13 · **not illustrations.** These are what the screen renders today, and they are the finding.	**observed behaviour**
`−$12.00`	Screen 13 remedy · the correction: `$84.00 → $72.00`, from `$60.00 → $48.00` on one line	illustration
`27 days` · `11 days`	Screen 14 · ages measured against `as_of = 09/11/2026`, which is printed beside them because BR-TEN-043 requires it	illustration
`1,204` / `25` pages · `~1,200` rows at T5	§02 screen 14, §07 R2 · the scale argument. Derived: 3% of 40,000.	arithmetic, stated
`66,053` SQL statements · `~2 minutes`	§02 screen 05, §08 · **not mine.** Measured and recorded in `selectors.py:2987-2989`, quoted.	**the repo's own measurement**
`25,000` vs `40,000`	§02 screen 14 · the bill-run ceiling against the ICP ceiling. From `DEF-BIL-4224` and CLAUDE.md's T5 definition.	quoted
`40px/800` · `13` nav items · `0px` radius	§01 · **measured in the running app**, not read off a stylesheet	**measured**

C · Every screen, with its exact contents

#	ROUTE	TABS	TABLE CONTROLS	WRITE SURFACES	HOW YOU GET HERE
01	`/billing`	6, in the path	search ✓ server · filter ✓ server · columns ✓ (page size only) · sort+page ✓ SQL	detail ✓ · add — · edit — · update —	nav item 7 + 3 named
02	`/billing/bills/:id`	**4, as** `?tab=` — Document · Charges · Corrections · Audit (drawn 02.1–02.4)	none — not a list	detail ✓ · add — · edit — · update *via a new case*	7 named
03	`/billing/bills/:id/document`	none	none	detail ✓ · add — · edit — · update —	3 named
04	`/billing/run` step 1	none (carries S30's strip)	past-runs: **NOTHING**; endpoint SQL-bounded, ruled	detail ✓ · add ✓ · edit — · update —	5 named
05	step 2	**6 sub-tabs**	**NOTHING**; unpaginated by declared design	detail ✓ · add — · edit — · update —	step 1
06	step 3	case chips	**NOTHING**	detail ✓ · add — · edit ✓ *per row* · update ✓	step 2
07	step 4	none	**NOTHING**; `/breaks` unbounded	detail ✓ · add — · edit — · update ✓ *retry/hold only*	step 3
08	step 5	none	none — a form	detail — · add — · edit — · update ✓	step 4
09	step 6	none	none	detail — · add — · edit — · update ✓ *idempotent*	step 5
10	`/billing/history`	is a tab	none — no table exists	**NOTHING ×4**	tab strip only
11	`/billing/one-time`	is a tab	no record list; one server `?q=` into a select	detail **NOTHING** · add ✓ · edit — *by design* · update —	tab strip only
12	`/billing/budget`	is a tab	search **NOTHING** · filter ✓ server · columns **NOTHING** · **page 1 only**	detail **NOTHING** · add ✓ · edit ✓ *unprefilled, below the table* · update ✓ *2 of 3 swallow failure*	tab strip only
13	`/billing/fix`	is a tab	**NOTHING ×4**; no params sent at all	detail ✓ *in-page, no deep link* · add ✓ · edit — *by design* · update ✓ *1 of 3 swallows failure*	tab strip + 2 cross-module
14	`/billing/pending`	none	**NOTHING ×4**; unbounded + Python sort	detail ✓ · add — · edit — · update —	**exactly 1 door**
15	`/billing/print`	none	no table; one scope select	**NOTHING ×4**	**exactly 1 door**

NOT NAMED count: 0. Every route's doors were audited and every one has at least one.

D · Decisions recorded here and nowhere else

D1 — rules that live only in this document

The four `NO_HOME` rows of table A. Each is an observation this pass made and did not write back

D3 — rulings taken in this session, recorded *only here*. These need a commit.

Three rulings landed while this document was being written. **This pass wrote nothing to the repository**, so each exists in this file and in the chat transcript and nowhere else — and a transcript is not the repo. Each names the surfaces it has to reach.

Two of these three close open questions and one reverses a signed rule, which is the most consequential half-hour in this document — and none of it is in the repo. Closing a question is a fan-out, not a status edit: the row's status, the row's body, the decision log, the module's business rules, the change request, the epic, the execution plan and any story frontmatter all have to move in the same commit, or the next lane reads a closed question's body and builds the world it still describes.

D2 — rulings already recorded elsewhere. *Do not re-ask.*

One search technique that gave a false negative, recorded because the next person will hit it: `grep 'billing.arrives'` over the front end misses sixteen sibling keys built as template literals, and misses seven more supplied by the *backend* as `message_key`. Search the backend too, and reconstruct literal prefixes.

E · Every guarantee, and what enforces it

Only a constraint refuses. A convention is a habit with a good reason, and a habit does not stop a caller. Sorting them this way is uncomfortable and always useful.

GUARANTEE	WHAT ENFORCES IT	LEVEL	UNDERTAKEN BY
No tenant table is touched without `app.tenant_id`	RLS `FORCE` in PostgreSQL	`DATABASE CONSTRAINT — REFUSES`	— enforced
A bill run is idempotent	`bill_runs.idempotency_key` unique index	`DATABASE CONSTRAINT — REFUSES`	— enforced
An issued bill takes no UPDATE or DELETE	a database trigger, narrowed for exactly three `payments` columns	`DATABASE CONSTRAINT — REFUSES`	— enforced
A one-time bill's amount is never typed by hand	**the type**: `OneTimeBody` carries no amount member	`DATABASE CONSTRAINT-EQUIVALENT — THE REQUEST CANNOT BE FORMED`	— enforced
A bill run cannot be started twice for one cycle	BR-BIL-005's lost-race check at the service boundary	`SERVICE RULE — REFUSES` `DEF-2868: THE RE-READ RUNS INSIDE THE TRANSACTION THE FAILED INSERT JUST BROKE`	BIL-S95 New build
Approving a correction is System Admin only	the endpoint's own permission class	`SERVICE RULE — REFUSES` *and the button does not, so a clerk meets the refusal as a failure*	BIL-S61 **Blocked**
An age never renders without the day it was measured at	`as_of` is a **required** query parameter	`SERVICE RULE — REFUSES`	— enforced

Every money action writes `audit_log`	the posting service	`SERVICE RULE`	— enforced
Money displays at two decimals	`formatMoney` / `MoneyCell` — **and four places in billing bypass it**	`CONVENTION — NOTHING REFUSES`	BIL-S80 New build
No database id is the visible value of a field	nothing. Two billing fields breach it.	`CONVENTION — NOTHING REFUSES`	BIL-S10 Exists behind
Every list paginates server-side	nothing for billing. `/pending` and `/breaks` breach it; two clients cannot reach page 2.	`CONVENTION — NOTHING REFUSES`	BIL-S81 New build BIL-S94 New build
A screen shows the product's failure state, not a stack trace	nothing. One boundary exists in the whole front end, inside `MoneyCell`.	`CONVENTION — AND NOT EVEN THAT`	BIL-S9 New build
A refusal is still true	**nothing whatsoever.** No gate, no test, no review step asks it.	`NOTHING — AND IT IS WHY §05 EXISTS`	BIL-S12 New build

F · What this document assumes and never says

Ordered by cost, most expensive last.

ASSUMPTION	WHERE IT HIDES	WHAT BREAKS IF IT IS WRONG
Line numbers are roughly current	every citation	A reader looks in the wrong place. Cheap — paths and values are pinned.
The `en.json` string is what every locale says	§05, quoted in English	Ten catalogues are `machine_unverified`, and one of them translates *billing cycle* as *bicycle*. A copy finding proved in English may be a worse finding elsewhere.
What a screen renders on a minimal payload resembles what it renders on a real one	§04's six crashes	Six of my ten results would be mine, not the product's. I say so in the section; the *missing error boundary* is what survives either way.
The branch I was given is the product	the masthead — **and it says "could not check"**	Everything. `origin/HEAD` does not resolve here.
An open defect row means the defect is still live	every `DEF-…` chip in this document	**The most expensive assumption here.** I verified perhaps a dozen of the 86 open billing defects against current code, and three of those turned out half-fixed with the row unchanged (`DEF-3153`, `DEF-2421`, `DEF-2845`). The register is a claim about the past, and this document treats some of it as a claim about the present.

G · Every UI rule violation

Seventeen rows, in §06 with the *enforced / ratified* column that makes them countable.
Summary: **2 breaches of B14 31** · **4 of hard rule 2** · **1 of B14 39** (inherited from the shared kit, 28 files) · **3 of B14 16** · **4 of B14 32/4/12** · **1 unassessable** because the rule names a class the shipped app does not declare · **1 R14 flag**. **Fifteen of the seventeen are ratified and unenforced** — no machine would ever have caught them.

H · Every cross-module finding

In §08, both modules named in every row, split the four ways Step 10 asks the questions. **Question 2 — does any module write another's tables — found nothing, and that is stated rather than left to be inferred from an absent heading.** Questions 1, 3 and 4 found nine, five and four.

17

The rest of the screens, as built

Thirteen screens, audited in full, moved here on 2026-09-12 so §02 holds only the two a clerk lives in and §03 comes up. **Nothing was cut — every drawing, legend, spec table, data flow and field table is exactly as it was, and every anchor still resolves**, so a link from anywhere in this document still lands on the screen it named.

WHAT IS HERE	WHY IT IS AT THE BACK RATHER THAN IN §02
03 Bill document	A reading screen with no redraw in this pass. The bill detail's Document tab (02.1) is where a clerk actually goes.
04–09 the six as-built run steps	**These are the evidence §03 is measured against.** The *six steps become four* table and its element arithmetic both count against these six drawings, so they are kept whole and linked from there.
10 Past billing runs CARRIES A DRAWING	Still has a drawing to build (`BIL-S1`): the panel saying the list arrives later is false in both halves that matter. At the back because it is not part of the run journey.
11 One-time bill · 12 Budget Bills	Billing outside the cycle. Untouched by the bulk-run redraw.
13 Fix a bill CARRIES A DRAWING	Has its own remedy drawing. Reached from a bill, never from a run.
14 Pending queue SUPERSEDED	⚠ **Its redraw is now §03's screen 20, *Held queue*** — the one with the missing handle, the date picker and the filters. **Build 20, not this.** This entry is the as-built record it was drawn from.
15 Print bills	The best-behaved screen in the module. Its one weakness is the door.

---

03 Bill document

New build S9 S79 S93 Exists behind S29

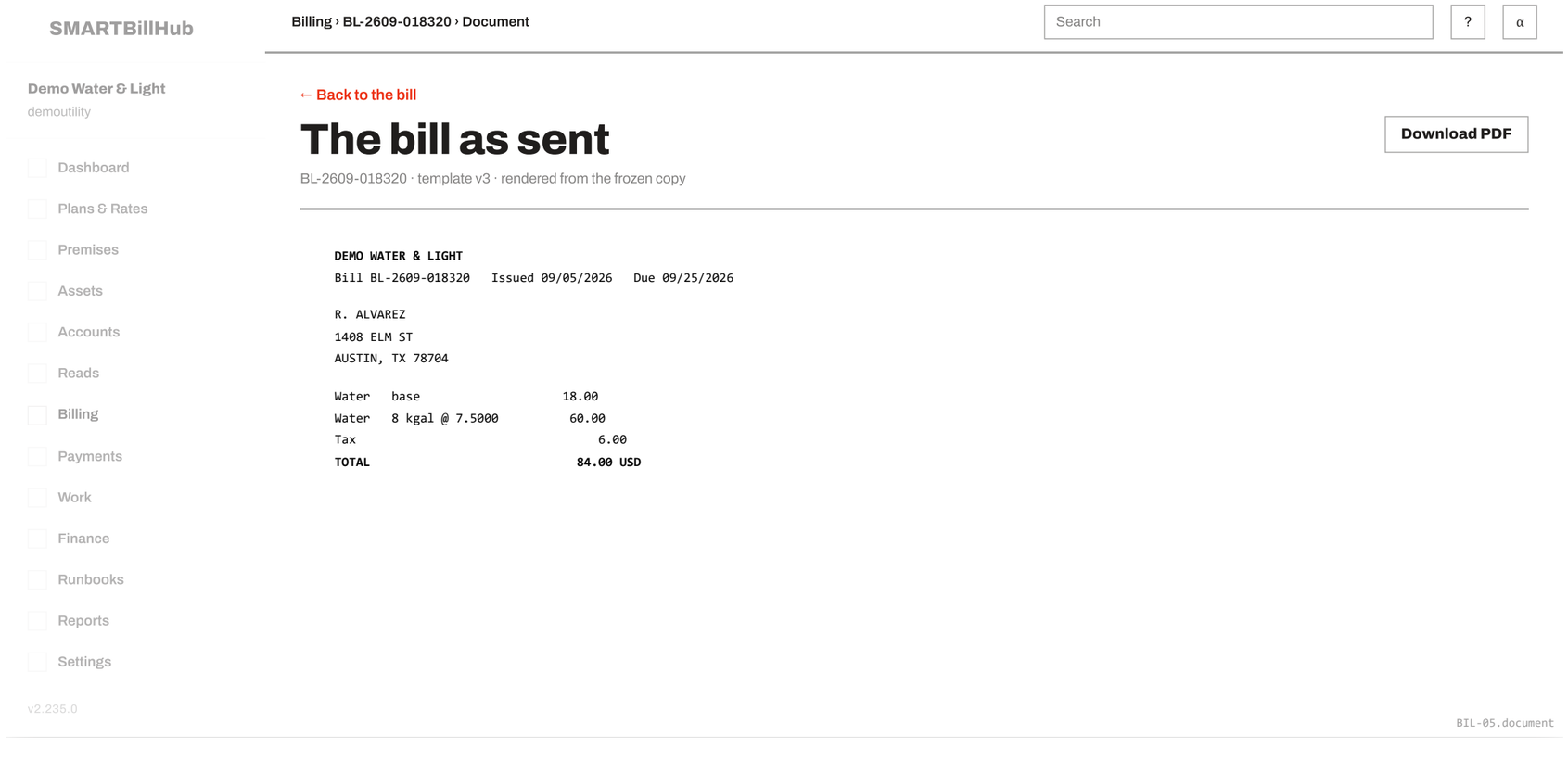
No redraw. This is the current state and there is nothing to build from it.

`/:tenant/billing/bills/:invoiceId/document · BIL-05.document · no A1 id`

"Show me exactly what landed in their letterbox."

> **AS-BUILT, FROM CODE** composed from source at commit `a1d37a68` and **never rendered**, because it threw on load — `(reading 'startsWith')` at `tenantPathFrom`. Treat the shape as

read, not as seen.



For	Opening the issued bill as the customer received it — the page a clerk reads back over the phone.
How you get here	Three doors: *View the bill as sent* in the detail header (`:434` , rendered only when `document_url` is non-null), *Open the bill as the customer received it* in the Document panel (`:863`), and the account rail's printer glyph (`accountRailReadiness.ts:226-230` , disabled when no bill has been issued).
Primary action	*Download PDF* — `GET /invoices/{id}/document.pdf` , `documents_api.py:454` .
Tabs	*None.*
Table controls	*None* — it is a document, not a list.
Write surfaces	**Detail** — this is the document itself. **Add · Edit · Update** — **NOTHING**, all three, correctly: a reprint re-renders the issued instance and never recomputes (BR-CUS-040).
Walked	**Not walked.** Threw at `tenantPathFrom … reading 'startsWith'` on a minimal payload.
Cognitive load	It removes the whole class of *"which version of their address was on it"* — the render is from the frozen instance and the pinned template version.
States	**Loading** — skeleton. **Empty** — cannot occur; a bill with no document does not link here. **Failed** — the renderer's own sentence, 422 for a bill with no pinned template. **Domain** — draft, which has no document at all and no door to here.
Words	*The bill as sent.* Not *preview*, which would suggest it could still change.

Components	`PageHeader` · an embedded frame · `DownloadButton`
Rules	BR-BIL-030 · BR-BIL-026 · BR-CUS-040
Audience	Staff.
Health	**SERVED**, with one caveat carried below.

#	FIELD	COMES FROM	OWNED BY	HEALTH
①	The rendered bill	`GET /invoices/{id}/document` → HTML + `pdf_url`	billing	SERVED
②	The PDF	`GET /invoices/{id}/document.pdf`	billing	SERVED
③	Name and address on it	the **frozen payload**, not a live join — so it is the name and address as at issue	billing (snapshot of customers ⇄)	SERVED

Two things about this page that matter more than they look. First, it **re-renders on every request** rather than streaming stored bytes — *"there is no document table and no blob store"*. The frozen *payload* is the record; the *rendering* is recomputed, so a change to the renderer changes what a reprint looks like. Second, `DEF-BIL-4524` **every bill this product prints is unstyled** — the page is Courier with space-aligned columns, which is what the drawing above shows, because that is what it is. The drawing is not a placeholder.

Group B · Producing bills

The run is six screens, and they are numbered here as six. `BILL_RUN_STEPS` (`billRunShared.tsx:206-213`) declares them with a *phase* each: `cycle` · `found` · `customers` are *prepare*, `totals` is *breaks*, `confirm` is *summary*, `send` is *release*. They share one route, `/:tenant/billing/run`, which is why an inventory built from the router alone would report one screen where a clerk meets six. **None of the six was walked** — each needs a created run, and there is no backend here.

04 Run bills · step 1 of 6 — pick the cycle

Blocked S89 · New build S9 S12 S84 S88 · Exists behind S1 S30

REDRAWN AS §03 screen 16 Scope. Build that one. This drawing is what exists today, kept because the findings in §09–§11 point at it.

`/:tenant/billing/run` · S31 · BIL-02 · step from `BILL_RUN_STEPS[0]`

"It's the end of the month. Bill everybody."

> **AS-BUILT, FROM CODE** composed from source at commit `a1d37a68` and **never rendered**, because the run wizard threw on my fixture, and steps 2–6 each need a created run. Treat the shape as read, not as seen.

Run bills

Save & exit | Park

Step 1 of 6 · nothing has been created yet

1 Pick the cycle | 2 What was found | 3 The customers | 4 The totals | 5 Sign off | 6 Send

Billing cycle

09-A · 01 Sep – 30 Sep 2026 ▾

The cycle decides the period, the accounts and the due date.

Bill date

09/30/2026

MM/DD/YYYY · the tenant's clock, never the browser's

What this step no longer asks you

The design draws a statement-template picker and a typed due date here. Neither is asked for any more: a bill's template follows the account's category, and the due date is worked out from the bill date and the cycle's days to pay.

Past runs

RUN	PERIOD	ISSUED	ACCOUNTS	STATUS	ACTIONS
—	01 Aug – 31 Aug 2026	09/02/2026	1,284	Draft	👁
—	01 Jul – 31 Jul 2026	08/02/2026	1,279	Draft	👁

Who started a run, and what it came to in money, are not kept against the run — so neither can be shown here.

r10 PAST BILLING RUNS · BR-MET-078 — NO ACTOR COLUMN, AND MONEY IS NOT SERVED

For	A clerk starting the month's billing, and the only place in the product that can reach the run engine.
How you get here	Five audited doors — the Dashboard's *Run billing* primary (`DashboardPage.tsx:1205` , **hidden** rather than disabled without the permission); the cycle detail's *Open in Billing* → (`CycleDetailPage.tsx:877`); S30's own primary (`BillsListPage.tsx:1026`); the account Bills tab's empty-state CTA (`:894`); and the bill detail's run link (`BillDetailPage.tsx:1011`). The *Run bills* tab also lands here, because the router ranks this static segment above `billing/:tab` .
Primary action	Advancing the step. *Save & exit* is real; ***Park* is disabled on all six steps** — see the box.
Tabs	*None on the step.* The wizard carries S30's six-tab strip in its own header (`BillRunWizard.tsx:1394-1397`), so a clerk can leave sideways.
Table controls	The Past-runs table has **none** — no search, no filter, no columns, no pager. The endpoint behind it (`GET /billing/runs`) is **SQL-bounded, not paginated**, clamped 1–200 with a written ruling: *"a run history grows at the TOP, so page 2 of it is not a stable read… that is a keyset cursor, not `page` bolted onto this."* `A STATED, REASONED DEVIATION`
Write surfaces	**Detail** — a past run's row opens it (`billRunResume.tsx` `onOpen(runId)`). **Add** — the step itself is the creator; `POST /billing/runs` (`api.py:2028`). **Edit · Update — NOTHING**, and correctly: a run is advanced, not edited.
Walked	**Not walked.** Threw on my fixture (an object where an array is served — *my* error, stated in §04).
Cognitive load	It removes the date arithmetic: pick the cycle and the period, the account set and the due date all follow.
States	**Loading** — skeleton. **Empty** — no past runs, which is a real first-month state. **Failed** — derived from the response, so it self-heals when a

States	**Loading** — skeleton. **Empty** — no past runs, which is a real first-month state. **Failed** — derived from the response, so it self-heals when a route lands. **Domain** — a run already open for this cycle, refused with BR-BIL-005's lost-race message. `DEF-106: THAT REFUSAL SHOWS NO WHO, NO WHEN AND NO LINK`
Words	*Run bills*, never *batch*. *Cycle*, matching Reads and Settings.
Components	`WizardFrame` (26px square badges, 2px rules top and bottom) · `PastRunsPanel` · `DatePickerField` · `SelectField`
Rules	B14 39 (field anatomy — **breached by the kit**) · B14 7 (a date is a `.date-input` with a Today shortcut) · BR-BIL-005 · BR-TEN-043
Audience	Staff. `reads_billing_runs` Full; the wizard refuses View outright.
Health	**NOT SERVED** — two of the Past-runs columns cannot be filled.

Data flow

```
bill_runs.run_by ──→ models.py:316-318  ForeignKey("identity.User")   ** EXISTS **
                  └→ api.py:1818  "ran_by": str(existing.run_by_id)  ** SERVED **
                  └→ ...and the panel prints "no actor column"       FORKED

bill_runs.status ──→ runs.create_run writes "draft"
                  └→ NOTHING IN THE MODULE EVER ADVANCES IT (DEF-171, DEF-602)
                  └→ every historical run reads Draft                STUBBED

bill_runs.run_no ──→ api.py:2092 exc(1) "IS SERVED NULL AND NOT DERIVED"  STUBBED
```

#	FIELD	COMES FROM	OWNED BY	HEALTH
①	Billing cycle	`cycles` via a lookup read	billing → metering ⇄	SERVED
②	Bill date	typed; interpreted in the **tenant's** zone, not the browser's (BR-TEN-043)	billing	SERVED
③	Run #	*served* `null` *on every row*	—	STUBBED
④	Period	`bill_runs.period_start/end`	billing	SERVED
⑤	Accounts	counted server-side, batched to 3 queries	billing	SERVED
⑥	Status	`bill_runs.status` — *a dead column: written once as* `draft`*, never advanced*	billing	STUBBED
⑦	Who ran it	`bill_runs.run_by` — **exists and is served**, and the panel says it does not	billing → identity ⇄	SERVED
⑧	What it came to	*no total column on* `BillRun` *— genuinely absent*	—	NOT SERVED

The strongest rotting hold in the module is on this screen, and it makes a false claim in its own rule reference.

The panel under the Past-runs table says *"Who started a run, and what it came to in money, are not kept against the run — so neither can be shown here,"* and stamps itself `BR-MET-078 — no actor column, and money is not served`. **The actor column exists.** `models.py:316-318` declares `run_by = ForeignKey("identity.User")`, it is populated, and `api.py:1818` puts it on the

wire as `ran_by`. The money half is still true. So the sentence is half-rotten — and the half that landed is denied in a rule citation, which is the form a lane is most likely to believe.

And *Park* is disabled on all six steps with a reason that is false. It reads *"A billing run cannot be parked yet: nothing would list it."* The list is `PastRunsPanel`, drawn above, mounted on **this very step** (`BillRunWizard.tsx:1808`). Parking genuinely is unbuilt — `PARKED_HOME_PATHS` has no `bill_run` member — so the control is right to refuse. Only its reason is wrong, and it is the reason a clerk reads six times.

---

05 Run bills · step 2 of 6 — what was found

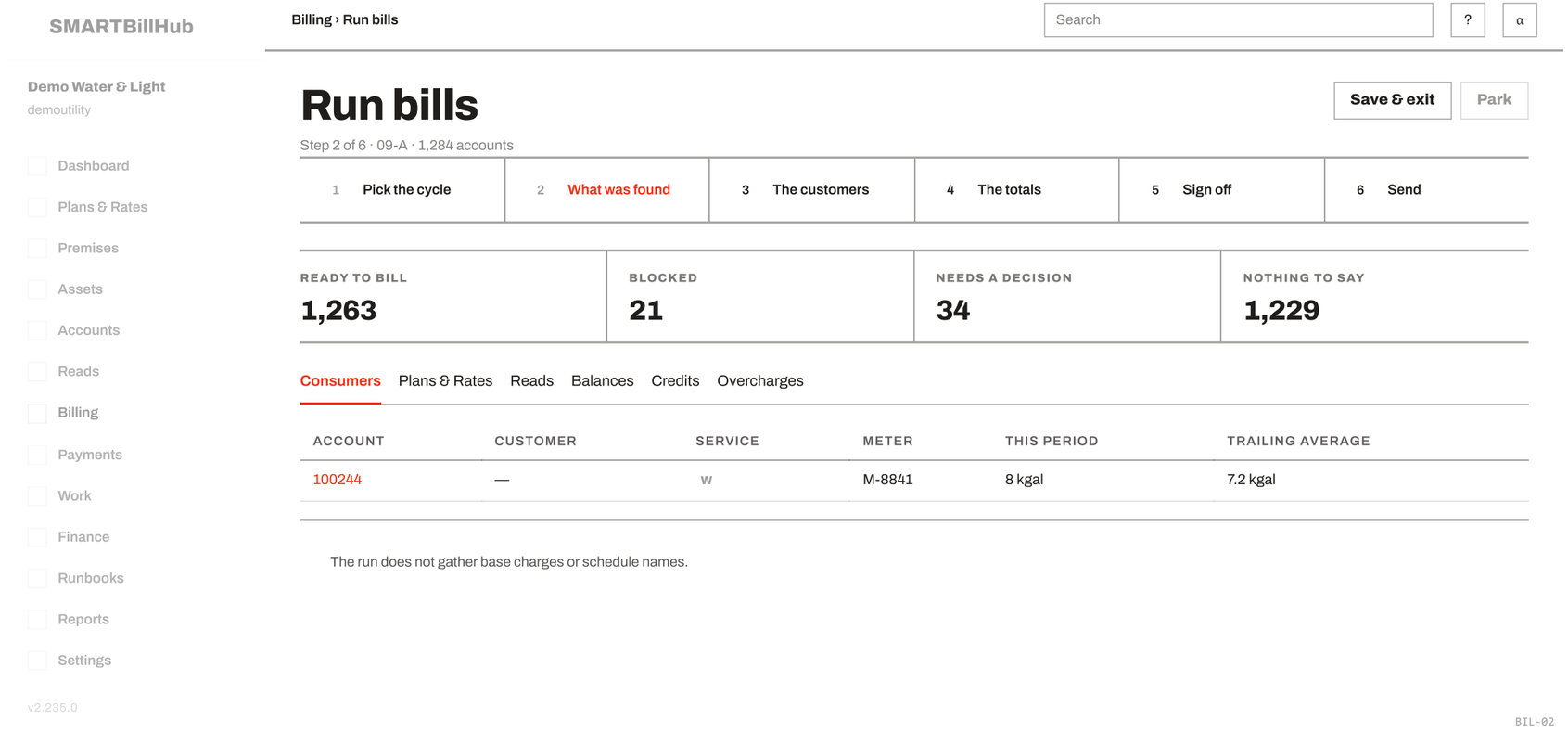
New build S9 S12 S84 S91 New read S73

REDRAWN AS §03 screen 17 Check. Three of the six steps collapse into this one. Build that one. This drawing is what exists today, kept because the findings in §09–§11 point at it.

`/:tenant/billing/run` · `BILL_RUN_STEPS[1]`, phase *prepare* · six sub-tabs

"Before you bill 1,284 people — is anything wrong?"

> **AS-BUILT, FROM CODE** composed from source at commit `a1d37a68` and **never rendered**, because it needs a created run; no backend existed to create one. Treat the shape as read, not as seen.



For Reading the prepare pass: what the run found when it looked at every account in the cycle.

How you get here	Step 1's *Continue*. No other door, and none needed.
Primary action	Advancing to step 3, once the clerk has read the exceptions.
Tabs	**Six**, and they are tabs of a step, with no address of their own: *Consumers · Plans & Rates · Reads · Balances · Credits · Overcharges* (`billRunShared.tsx:459-464`). `DEF-1081: THREE OF THE SIX EXIST ONLY TO EXPLAIN THAT THEIR FEATURE IS NOT BUILT` `DEF-1082: THE TABS PROMISE DETAILS AND SHOW ONLY EXCEPTIONS, SO A CLEAN TAB IS AN EMPTY SCREEN`
Table controls	**None on any of the six.** And the endpoint is **unpaginated by declared design** — `selectors.py:2978-2984`: *"NOT PAGINATED, deliberately, where every list endpoint in this API is...* `PreparedRun` *is not a queryset: it is one in-memory pass over the run's accounts."* **The cost is measured in the same comment:** *"At 1,000 accounts the pass is 66,053 SQL statements... a clerk pays for the whole pass on every refresh, and at that size waits about two minutes."* `OQ-843` was closed on *recompute*, so it is not cached.
Write surfaces	**Detail** — the account cell links out to the account. **Add** — **NOTHING**. **Edit** — **NOTHING**. **Update** — decisions are recorded on step 3, not here.
Walked	**Not walked.**
Cognitive load	It is supposed to remove *"which of these 1,284 will go wrong"*. `DEF-1084: IT TAKES A LONG TIME AND SHOWS EIGHT GREY BARS WHILE IT DOES, AND THE PRODUCT NEVER SAYS WHAT IT IS DOING`
States	**Loading** — eight skeleton bars, for up to two minutes. **Empty** — a clean tab renders nothing to report, which DEF-1082 says reads as broken. **Failed** — derived from the response. **Domain** — a blocked account, whose reason is the whole point of the screen. `DEF-2384 · DEF-518 · DEF-2432: REASONS REACH A CLERK AS RAW ENGINEERING STRINGS, RULE IDS AND A CATCH-ALL WITH NO STATUS OR CODE`
Words	*What was found*, not *validation results*.
Components	`WizardFrame` · `KPIStrip` · `AwaitsCell` · `PlainTable` · `RunGap`
Rules	NFR-3 (deviation stated) · BR-BIL-015 · B14 32 (description budget)
Audience	Staff.
Health	**NOT SERVED** — the whole Plans & Rates tab denies a gather the run performs.

Data flow

```
meter_reads ──⇄→ gather.py  one in-memory pass per account
               └→ GET /billing/runs/{id}/findings   api.py:2037 → :717
               └→ 4 unbounded arrays: ready · blocked · decisions · silent
               └→ ** 66,053 SQL statements at 1,000 accounts, per refresh **

rate_plans.base_charge ──→ gather.py:699-705 _base_charge_for(...)   ** GATHERED **
                         └→ findings PAYLOAD carries neither         NOT ON THE WIRE
                         └→ every row of the Plans tab prints
                            "The run does not gather base charges"   FORKED
```

#	FIELD	COMES FROM	OWNED BY	HEALTH
①	Account #	`accounts.account_no`	billing → customers ⇄	SERVED

②	Customer	*null on every run-case row — a B1/G6 seam limit (`selectors.py:3206-3210`)*	—	NOT SERVED
③	Service · meter	read through metering	billing → metering ⇄	SERVED
④	This period · trailing average	the prepare pass's own computation	billing	SERVED
⑤	Base charge · schedule name	**gathered by** `gather.py` and absent from the findings payload; the cell denies the gather	billing → rating ⇄	NOT SERVED
⑥	Money on any case row	`amount`, `currency`, `quantity`, `unit` *null on **every** row and* `totals` *always* `{}` *— phase 1 computes no money at all*	—	STUBBED

A cell on every row of this screen denies something the code demonstrably does. *"The run does not gather base charges or schedule names."* `gather.py:699-705` gathers base charges through a three-way precedence ladder. The *true* narrower statement is sitting in the adjacent comment — *"The findings **payload** carries neither"* — and the user-facing sentence generalised it into a claim about the run itself. A lane reading the screen would go looking for a gatherer that is already there.

---

06 Run bills · step 3 of 6 — the customers

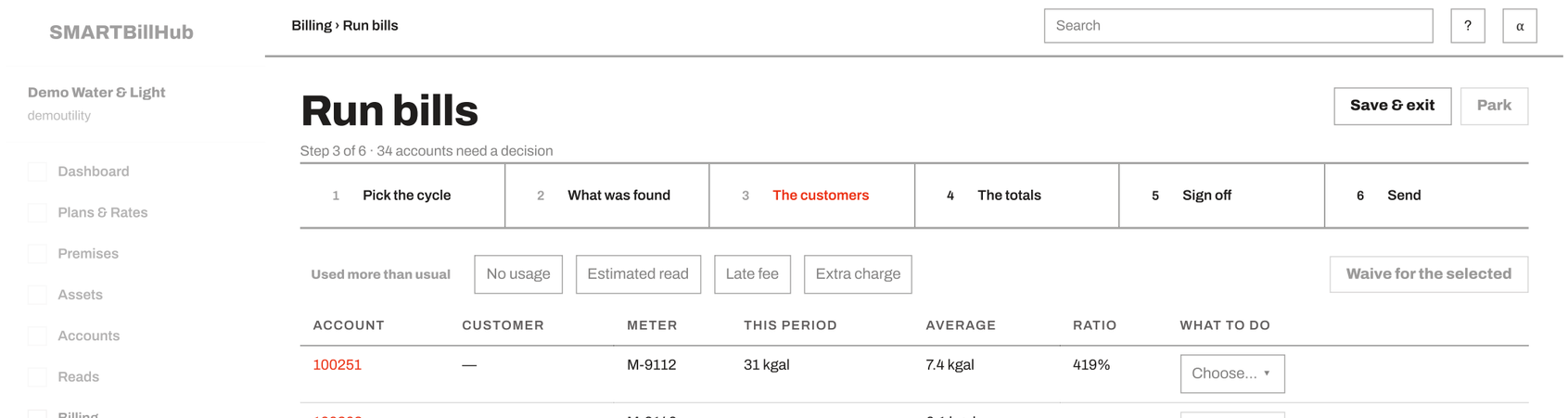
New build S9 S84

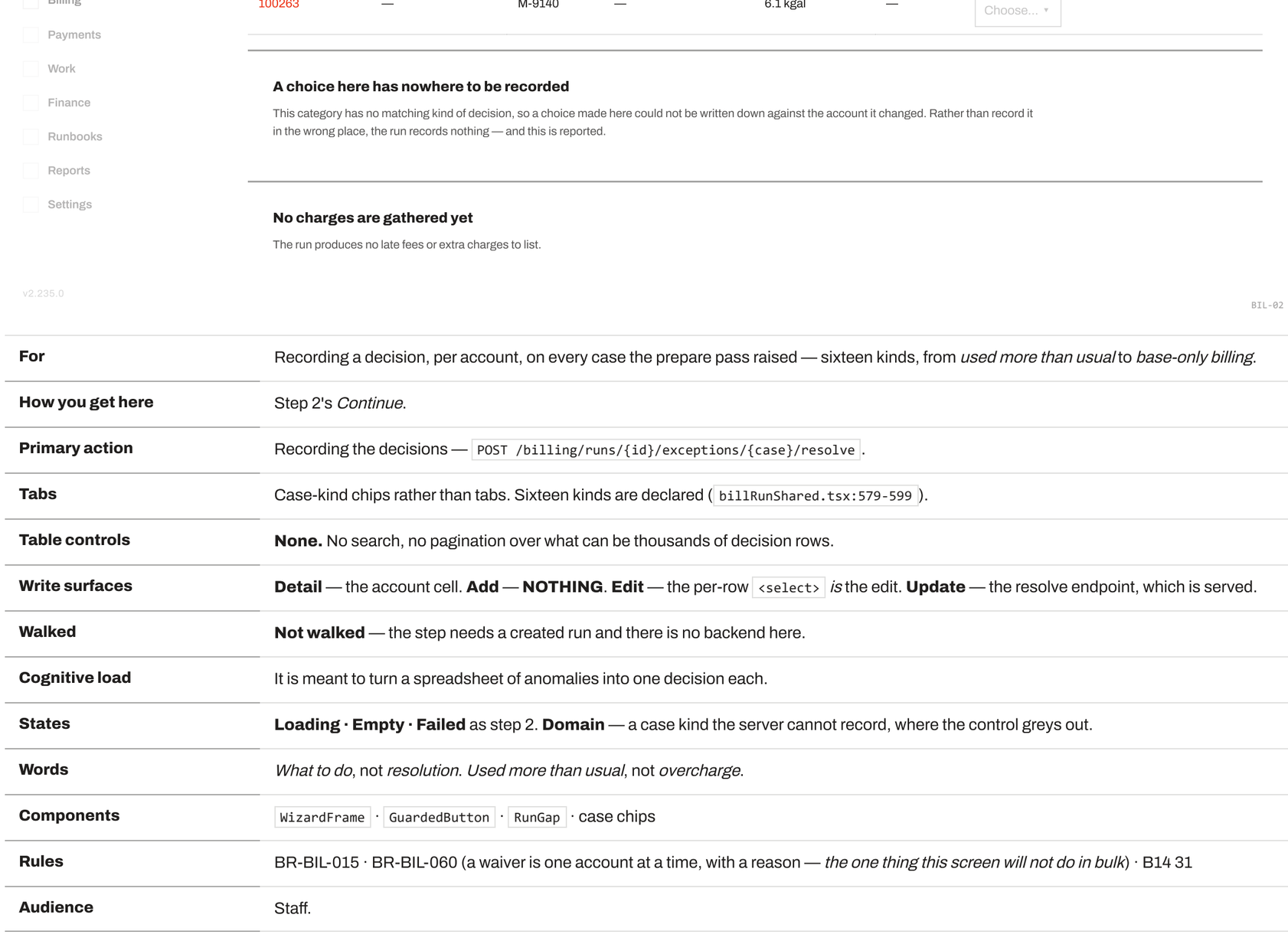
REDRAWN AS §03 screen 17 Check. Three of the six steps collapse into this one. Build that one. This drawing is what exists today, kept because the findings in §09–§11 point at it.

`/:tenant/billing/run` · `BILL_RUN_STEPS[2]` · sixteen case kinds

"Thirty-four of these look odd. Tell me what to do about each one."

> **AS-BUILT, FROM CODE** composed from source at commit `a1d37a68` and **never rendered**, because it needs a created run; no backend existed to create one. Treat the shape as read, not as seen.





For	Recording a decision, per account, on every case the prepare pass raised — sixteen kinds, from *used more than usual* to *base-only billing*.
How you get here	Step 2's *Continue*.
Primary action	Recording the decisions — `POST /billing/runs/{id}/exceptions/{case}/resolve`.
Tabs	Case-kind chips rather than tabs. Sixteen kinds are declared (`billRunShared.tsx:579-599`).
Table controls	**None.** No search, no pagination over what can be thousands of decision rows.
Write surfaces	**Detail** — the account cell. **Add** — **NOTHING**. **Edit** — the per-row `<select>` *is* the edit. **Update** — the resolve endpoint, which is served.
Walked	**Not walked** — the step needs a created run and there is no backend here.
Cognitive load	It is meant to turn a spreadsheet of anomalies into one decision each.
States	**Loading · Empty · Failed** as step 2. **Domain** — a case kind the server cannot record, where the control greys out.
Words	*What to do*, not *resolution*. *Used more than usual*, not *overcharge*.
Components	`WizardFrame` · `GuardedButton` · `RunGap` · case chips
Rules	BR-BIL-015 · BR-BIL-060 (a waiver is one account at a time, with a reason — *the one thing this screen will not do in bulk*) · B14 31
Audience	Staff.
Health	**STUBBED** — no money on any row, by phase-1 design.

#	FIELD	COMES FROM	OWNED BY	HEALTH
①	Case kind	the prepare pass's classification, 16 kinds	billing	SERVED
②	Customer	*null on every row (B1/G6 seam limit)*	—	NOT SERVED
③	Usage · average · ratio	the prepare pass	billing → metering ⇄	SERVED
④	The decision control	`bill_run_decisions.case`	billing	SERVED
⑤	Late fees · extra	*the run emits nine case kinds and **never produces a late fee or an extra charge** — DEF-2596*	—	NOT SERVED

	charges	*high, open*

Two refusals here are right, and they are worth naming as the good kind. The bulk *Waive* is disabled by **rule**, not by a gap: *"Waiving a late fee is decided one account at a time, with a reason. It is the one thing this screen will not do in bulk."* And the per-row control greys out on an `extra_charge` row because *the server records no choice for that kind at all* — the screen refuses rather than writing a decision into the wrong column. Both say what they are and why.

---

07 Run bills · step 4 of 6 — the totals

New build S9 S84 · New read S5 · Exists behind S4

REDRAWN AS §03 screen 17 Check. Three of the six steps collapse into this one. Build that one. This drawing is what exists today, kept because the findings in §09–§11 point at it.

`/:tenant/billing/run` · `BILL_RUN_STEPS[3]`, phase *breaks*

"It's worked out. Does the number look right, and what broke?"

> **AS-BUILT, FROM CODE** composed from source at commit `a1d37a68` and **never rendered**, because it needs a created run; no backend existed to create one. Treat the shape as read, not as seen.

SMARTBillHub

Billing › Run bills

Search ? α

Demo Water & Light
demoutility

- Dashboard
- Plans & Rates
- Premises
- Assets
- Accounts
- Reads
- Billing
- Payments
- Work
- Finance
- Runbooks
- Reports
- Settings

v2.235.0

Run bills

Save & exit | Park

Step 4 of 6 · worked out · 1,263 bills

1 Pick the cycle | 2 What was found | 3 The customers | 4 The totals | 5 Sign off | 6 Send

BILLS	TOTAL	LEFT OUT	BROKE
1,263	$342,180.00	21	0

Unusually large bills

The flagged bills cannot be listed yet

The comparison needs both a worked-out bill and the account's own recent average, and nothing serves either to a screen.

- Send it knowingly
- Leave this account out of this run
- Send it back for a reading check

What you cannot yet do to a draft here

A draft can be left out with a reason, re-worked after a fix, or given a charge from the utility's own list — and a worked-out amount is never typed over. None of those three has anywhere to send its request, so none is offered.

For	Reading what the calculate pass produced, and dealing with anything that broke before sign-off.
How you get here	Step 3's *Continue*, or a resumed run: re-POSTing `calculate` reads the checkpoint (`run_resume.py:132-189`). **There is no `/resume` route** — resume is a side effect.
Primary action	Advancing to sign-off.
Tabs	*None.*
Table controls	**None**, and `GET /billing/runs/{id}/breaks` is **unbounded** — `selectors.py:2782-2785` `list(rows.order_by(...).values())`, no slice. Bounded only by accounts-per-run, itself capped at 25,000. NFR-3 BREACH
Write surfaces	**Detail** — a break row. **Add** — **NOTHING**. **Edit** — **NOTHING**, correctly: an amount is never typed over. **Update** — per-account *retry* and *hold* are served (`api.py:2047` , `:2048`); **the three draft powers are not offered at all**, see the box.
Walked	**Not walked** — the step needs a created run and there is no backend here.
Cognitive load	One total, one exclusion count, one break count — the three numbers a clerk signs against.
States	**Loading** — the calculate pass. **Empty** — nothing broke, which is the good case. **Failed** — per-account break reasons. **Domain** — an unusually large bill, which is the case this screen cannot show.
Words	*Broke*, not *errored*. *Left out*, not *excluded*.
Components	`WizardFrame` · `KPIStrip` · `RunGap` · `billRunResume` 's break panel
Rules	BR-BIL-023 (the three draft powers) · BR-BIL-029 / BR-BIL-054 (unusually large) · BR-BIL-056 (the panel recalculates)
Audience	Staff.
Health	**NOT SERVED**.

#	FIELD	COMES FROM	OWNED BY	HEALTH
①	Bills · total · left out · broke	the calculate pass, recomputed live (BR-BIL-056)	billing	SERVED
②	Unusually large bills	*BR-BIL-054's list **has never existed** — DEF-2855, high, open*	—	NOT SERVED
③	The three overcharge actions	*rendered as inert `<span>` s — no button, no handler*	—	NOT SERVED
④	Leave out · re-work · add a charge	*BR-BIL-023's three draft powers: `runs.mark_for_later` and `runs.retry_account` are **complete** and the screen offers neither — DEF-2841, DEF-2856*	—	NOT SERVED

Three actions on this screen are text, not controls. *Send it knowingly*, *Leave this account out of this run* and *Send it back for a reading check* are each a bare `<span>` inside an `<li>` (`BillRunWizard.tsx:2577-2583`) — **no button, no handler** — sitting directly above a panel that says the list they act on cannot be served. The comment says why: *"named even while the list cannot be served, because the PO asked for both."* That is a defensible call, and it is worth

knowing that the words look clickable and are not.

And the draft powers below them are a confirmed rule drawn on the screen and switched off (DEF-2841) while two of the three services — `runs.mark_for_later` and `runs.retry_account` — are complete and routed as `/hold` and `/retry` (DEF-2856). The panel says *"None of those three has anywhere to send its request."* Two of them do.

---

08 Run bills · step 5 of 6 — sign off

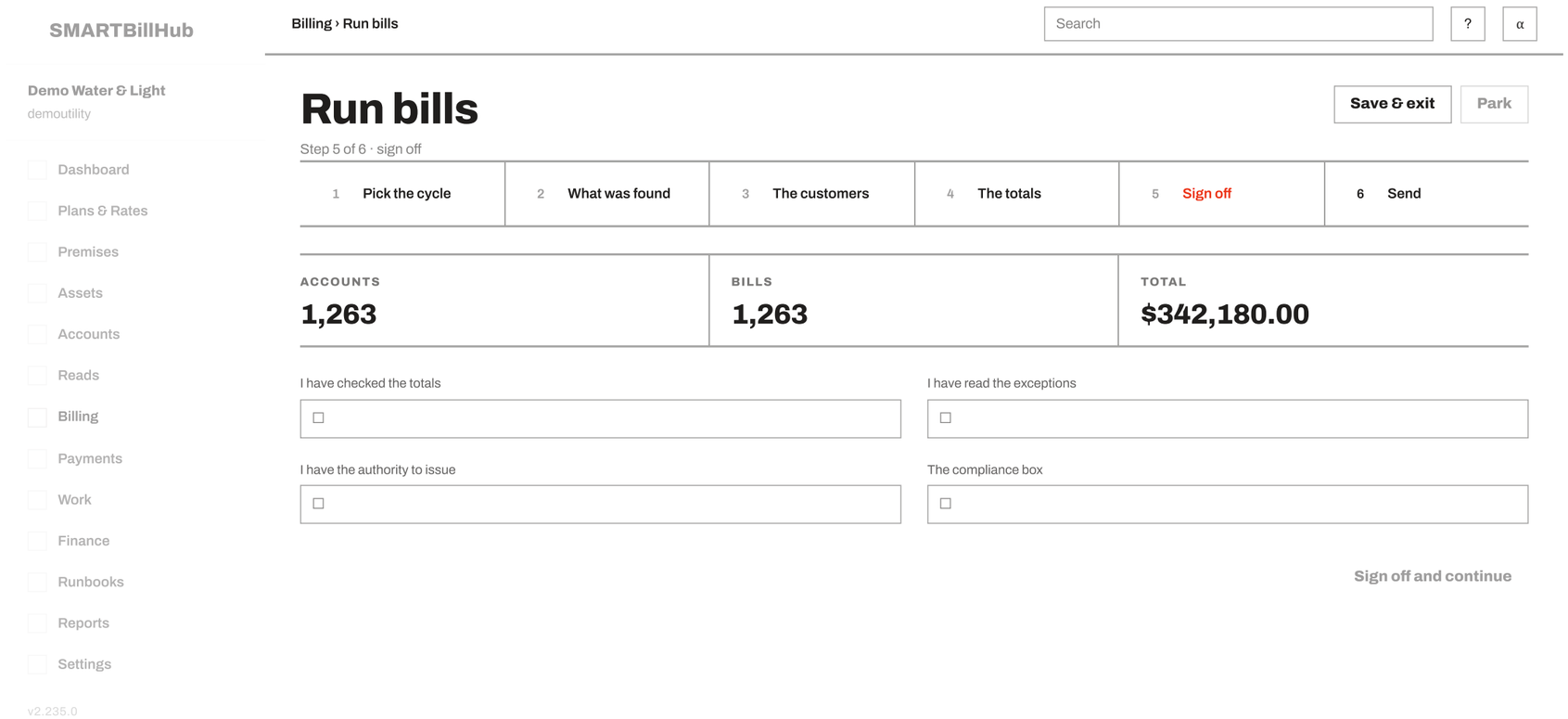
New build S9 S84

REDRAWN AS §03 screen 18 Sign off. Build that one. This drawing is what exists today, kept because the findings in §09–§11 point at it.

`/:tenant/billing/run` · `BILL_RUN_STEPS[4]`, phase *summary*

"I am about to send 1,263 bills. Record that it was me."

> **AS-BUILT, FROM CODE** composed from source at commit `a1d37a68` and **never rendered**, because it needs a created run; no backend existed to create one. Treat the shape as read, not as seen.



For	The authorisation record: five separately recorded facts (BR-BIL-058) that make *"what did I approve on 1 July?"* answerable.
How you get here	Step 4's *Continue*.
Primary action	*Sign off and continue* — `POST /billing/runs/{id}/release` (`api.py:2071`)

Primary action	*Sign off and continue* — `POST /billing/runs/{id}/release` (`api.py:2071`).
Tabs	*None.*
Table controls	*None* — it is a form, not a list.
Write surfaces	**Detail · Add · Edit** — **NOTHING**, all three. **Update** — the release endpoint, once.
Walked	**Not walked** — the step needs a created run and there is no backend here.
Cognitive load	It makes the sign-off explicit rather than implied by pressing *Send*.
States	**Loading · Failed** as elsewhere. **Empty** — cannot occur. **Domain** — an already-released run, which refuses.
Words	*Sign off*, not *approve* — the PO's own word.
Components	`WizardFrame` · the staff form kit
Rules	BR-BIL-058 (five recorded facts) · BR-BIL-056 (this panel recalculates; the stored row does not) · **BR-BIL-065** · B14 32 `COUNTED AT RATIFICATION: THIS STEP CARRIED NINE PARAGRAPHS AND TWO ABSENCE PANELS`
Audience	Staff, and the only step where the *authority* to issue is asserted.
Health	**SERVED**.

#	FIELD	COMES FROM	OWNED BY	HEALTH
①	Accounts · bills · total	recomputed live — deliberately *not* the stored figures	billing	SERVED
②	The five gate facts	`bill_runs` release columns	billing	SERVED
③	The authorisation record itself	**an export, not a screen** — `bill_run_releases`, `exports.py:191`	billing	SERVED

The one place in this module where a ruling was honoured in full, and it is worth reading as the good case. The PO was shown two options for the release authorisation record and chose in two words — *"B An export only"* — declining a screen. For a while the repo recorded *not doing A* and nothing recorded that *B was still missing*; `exports.py`'s own header names the shape: **"honouring the declined half of a ruling reads exactly like compliance."** B is built now. The stored figures are *not* recomputed, which is the whole point: a bill voided next week leaves the live panel above and does not leave that export.

09 Run bills · step 6 of 6 — send

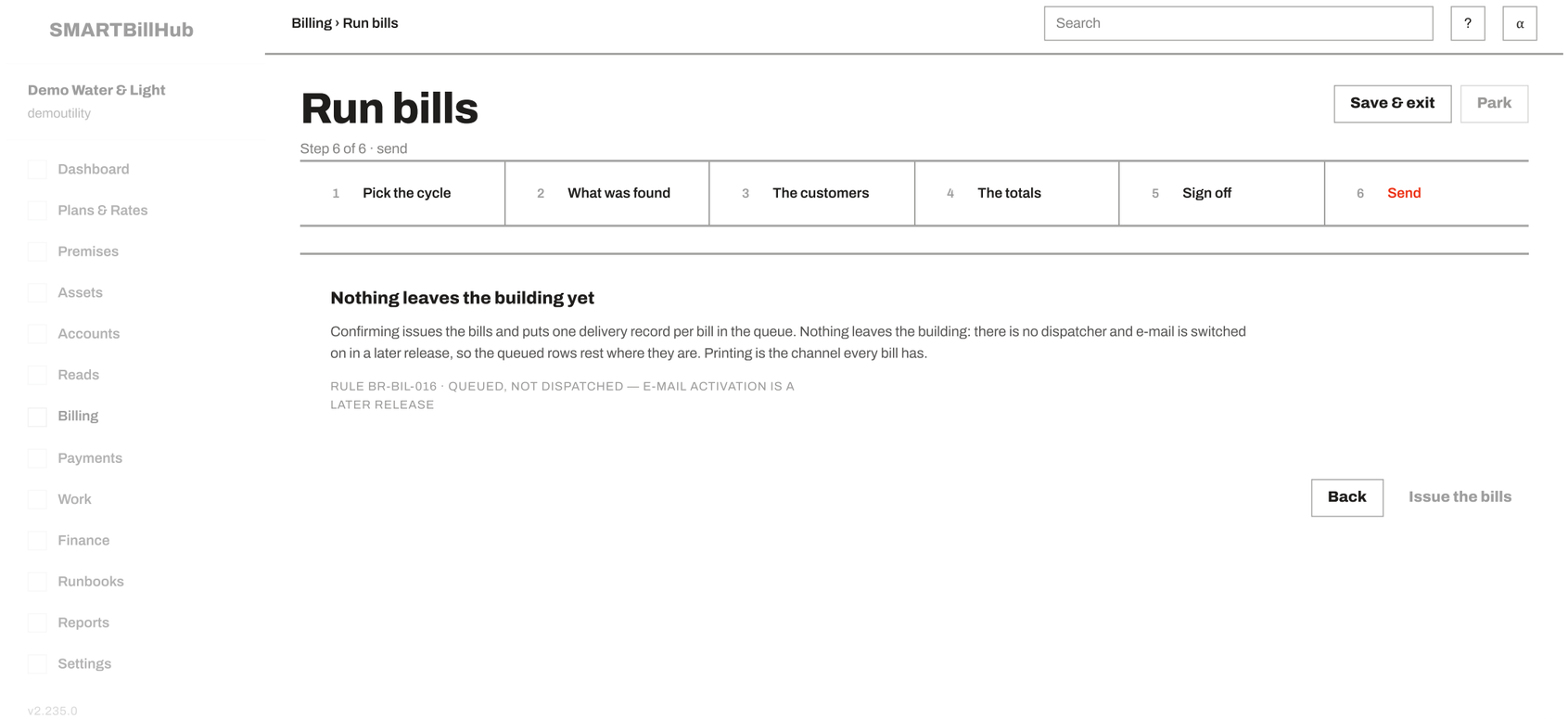
New build S9 S71 S72 S84

REDRAWN AS §03 screen 19 Send. Build that one. This drawing is what exists today, kept because the findings in §09–§11 point at it.

`/:tenant/billing/run` · `BILL_RUN_STEPS[5]`, phase *release*

"Send them. And tell me honestly whether anything actually went out."

> **AS-BUILT, FROM CODE** composed from source at commit `a1d37a68` and **never rendered**, because it needs a created run; no backend existed to create one. Treat the shape as read, not as seen.



BIL-02

For	Issuing the bills and queueing one delivery record each — and saying plainly that queueing is as far as it goes.
How you get here	Step 5's *Sign off and continue*.
Primary action	*Issue the bills* — `POST /billing/runs/{id}/issue` (`api.py:2072`). This is the write that mints bill numbers, freezes each document payload and pins its template version.
Tabs	*None.*
Table controls	*None.*
Write surfaces	**Detail · Add · Edit** — **NOTHING**. **Update** — the issue endpoint, once, idempotently (hard rule 6).
Walked	**Not walked** — the step needs a created run and there is no backend here.
Cognitive load	It removes the false confidence of a *Send* button: the screen tells you the bills are queued, not delivered.
States	**Loading** — the issue pass. **Empty** — cannot occur. **Failed** — a partial issue, which idempotency makes safe to retry. **Domain** — already issued.
Words	*Nothing leaves the building* — the best sentence in the module, and it is a stated absence doing exactly its job.
Components	`WizardFrame` · `RunGap`

Rules	BR-BIL-016 · hard rule 6 (idempotency) · hard rule 7 (audit) · BR-BIL-030 (locked at issue) · BR-BIL-063 (a number from the moment it is drafted)
Audience	Staff.
Health	**SERVED** — and the absence on it is **verified still true**: `delivery_seam.py:308-310` `nothing_was_dispatched`, and US-NOT-002/003 do not exist.

#	FIELD	COMES FROM	OWNED BY	HEALTH
①	The issue write	`POST /billing/runs/{id}/issue`	billing	SERVED
②	One delivery row per bill	queued	billing → notifications ⇄	SERVED
③	Anything actually sent	*nothing — no dispatcher, no provider adapter. The screen says so.*	—	NOT SERVED

What the run produces and the customer then receives is where this module's worst open defects are, and none of them is on this screen. | `DEF-BIL-4448` every value on the bill-ready e-mail is an em-dash — bills are queued with no render context. | `DEF-BIL-4452` a bill's queue rows are pinned to a *billing* template on every channel, which the notifications renderer cannot use. | `DEF-BIL-4759` *billing cycle* is translated as **bicycle** in Japanese and French, so a Japanese bill line reads *"bicycle charge"*. That last one is a bill a real customer reads.

10 Past billing runs

New build S9 S12 S76 Exists behind S1

REDRAWN AS §07 screen 27 — Past billing runs. **Build that one.** This drawing is what exists today, kept because the findings in §09–§11 point at it.

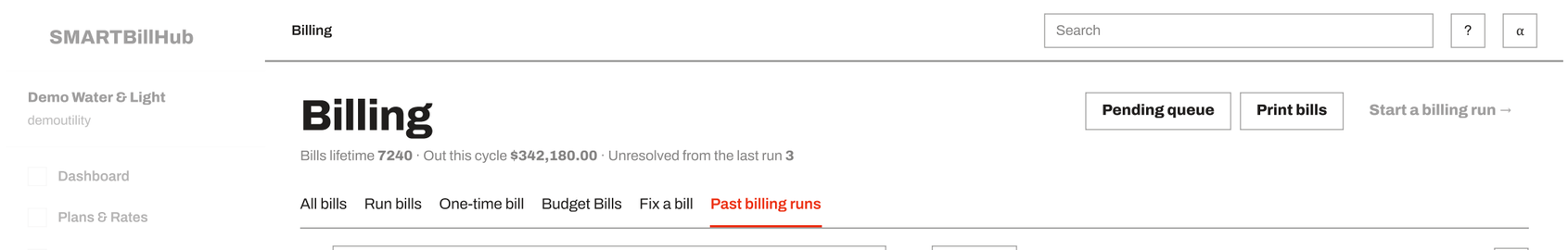
`/:tenant/billing/history` · BIL-01.history · S30

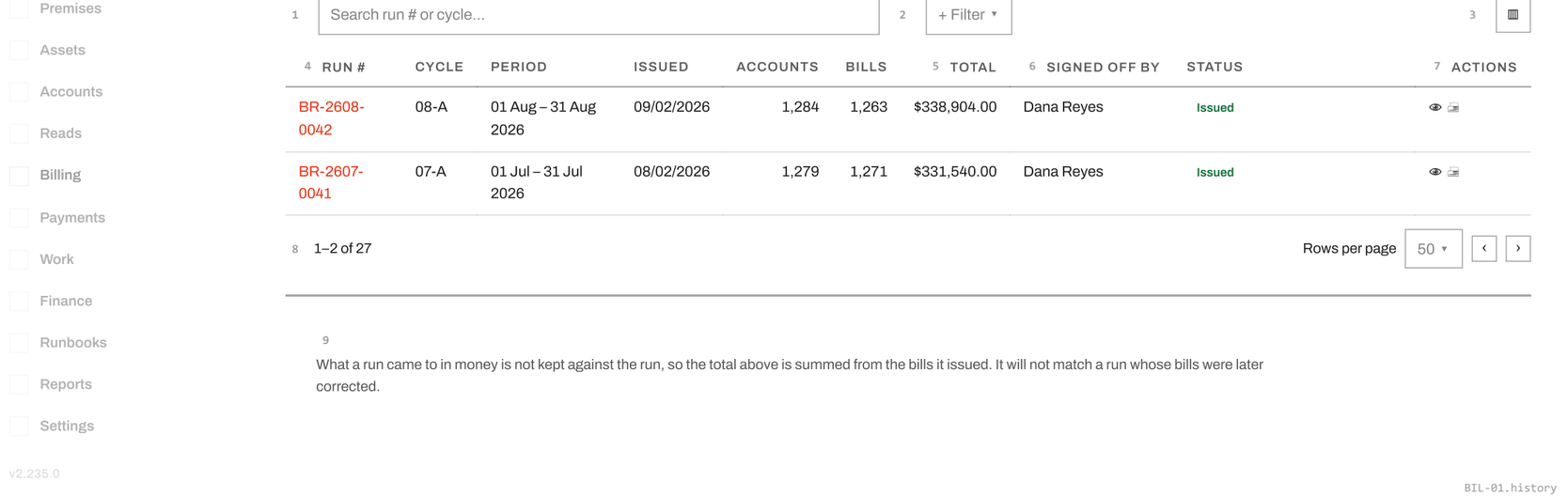
"What did we bill last month, and who signed it off?"

SUPERSEDED This proposal is replaced by §07 screen 27. Kept as the earlier reading, not as work.

Past billing runs — fill the tab with the panel that already exists | CLOSES BIL-S1 · DEF-2434

> **AS-BUILT** measured in Chromium 1194 at 1440×1100 on **2026-09-11**, at commit `a1d37a68`.
> This is what the product renders today.





BIL-01.history

1 **Search** · posts `?q=` to `GET /billing/runs` · UI-RULES 12 · **NEW READ** — the endpoint takes no `q` today

2 **Filter chips** · `f.cycle_id=`, `f.status=` · UI-RULES 12 · **NEW READ**

3 **Column chooser** · client preference via `/user-preferences/table.S30h.columns` · UI-RULES 14 · **NEW**

4 **Run #** · `bill_runs.run_no` · B14 31 (a number, never the uuid) · **NEW READ** — served `null` today

5 **Total** · `SUM(invoices.total)` over the run · **NEW READ** — there is no total column on `BillRun`

6 **Signed off by** · `bill_runs.run_by` → `identity.User` · **UNCHANGED DATA, NEW COLUMN** — the column exists and is already on the wire

7 **Row actions** · open the run · print its set · UI-RULES 15 (two + overflow) · **NEW**

8 **Pagination** · the existing SQL `LIMIT` becomes a keyset cursor on `started_at`, per the ruling already written at `api.py:590-598` · NFR-3 · UI-RULES 16 · **NEW**

9 **The honest caveat** · replaces the current false sentence · **RELABELLED**

Carry-over

AS-BUILT FIELD	DISPOSITION	WHY
The refusal panel (title + body)	RELABELLED	It stops claiming the run list is unbuilt and starts stating the one thing that genuinely is — that a run keeps no money total, so the figure is summed from its bills. Old wording quoted in §05 R5; new wording at ⑨.
The six-tab strip	UNCHANGED	Same tabs, same order, same words.
The page header, rollup and three actions	UNCHANGED	Untouched.

```
As-built fields     1   (the refusal panel is the whole of the tab today)
  unchanged         0
  moved             0
  relabelled        1   -> callout 9
  removed           0
Remedy fields      10   = 1 - 0 removed + 9 new      RECONCILES
```

Why this one first: the component is already written and already mounted. `PastRunsPanel` (`billRunResume.tsx:312-355`) renders on step 1 of the wizard. Three of the nine new elements need no back-end work at all (⑥⑦⑨), and ⑥ is the column the current copy denies. **The smallest honest version of this remedy is: mount the existing panel in the tab and rewrite the sentence.** Everything else here is separable.

▶ **As built today — kept because the findings below point at it. *Do not build this.***

For	Nothing, today. It is the one tab of six that renders a refusal.
How you get here	The six-tab strip on S30, and the same strip on the wizard. **No other door** — nothing else in the codebase links to `/billing/history`.
Primary action	*None.* The header's primary belongs to the screen, not the tab.
Tabs	It *is* a tab. Its panel is the fallback branch of `BillsListPage.tsx:1107-1110` — the only tab id that still reaches it.
Table controls	*None* — there is no table.
Write surfaces	**NOTHING**, all four.
Walked	SEEN Rendered at 1440 with the 13-item rail, the six-tab strip and the header actions. `scrollWidth − clientWidth = 0`.
Cognitive load	None. It adds a promise.
States	One. There is no loading, empty, failed or domain state because nothing is read.
Words	The copy is good prose describing work that has landed.
Components	`ArrivesWith`
Rules	hard-won law 2's mirror — a refusal with a clock in it, and nothing checks whether it has expired
Audience	Staff.
Health	**NOT SERVED** — one field, and the data for it exists.

#	FIELD	COMES FROM	OWNED BY	HEALTH
①	The run list	*the panel says it is not built.* `GET /billing/runs` *is served (*`api.py:2028`*) and* `PastRunsPanel` *already renders it on step 1 of the wizard*	billing	NOT SERVED

This is the module's headline rotting hold, and I saw it in a browser. The panel says *"It arrives with the billing run work."* The billing run work arrived: BIL-02 is live, `GET /billing/runs` is served, and `PastRunsPanel` — a table of runs with a period, an issued date, an account count, a status and an actions column — is **mounted on step 1 of that wizard** (`billRunResume.tsx:312-355` → `BillRunWizard.tsx:1808`). `CycleDetailPage.tsx:896` lists real runs too.

runs too.

Already filed as | DEF-2434 *"S30's 'Past billing runs' tab still renders an arrives-later placeholder while the run wizard now lists every run"* — open. This audit confirms it in a browser and adds the mechanism: the component that would fill it already exists and is already mounted elsewhere.

And two of its sibling strings are dead copy. `billing.arrives.run` and `billing.arrives.oneTime` exist in all eleven catalogues and can never render: the router matches `/billing/run` to the wizard before `billing/:tab`, and `one-time` is caught by its own branch twelve lines earlier. The `one-time` ternary inside the fallback is dead code. Neither key has a defect row.

Group C · Billing outside the cycle

---

11 One-time bill

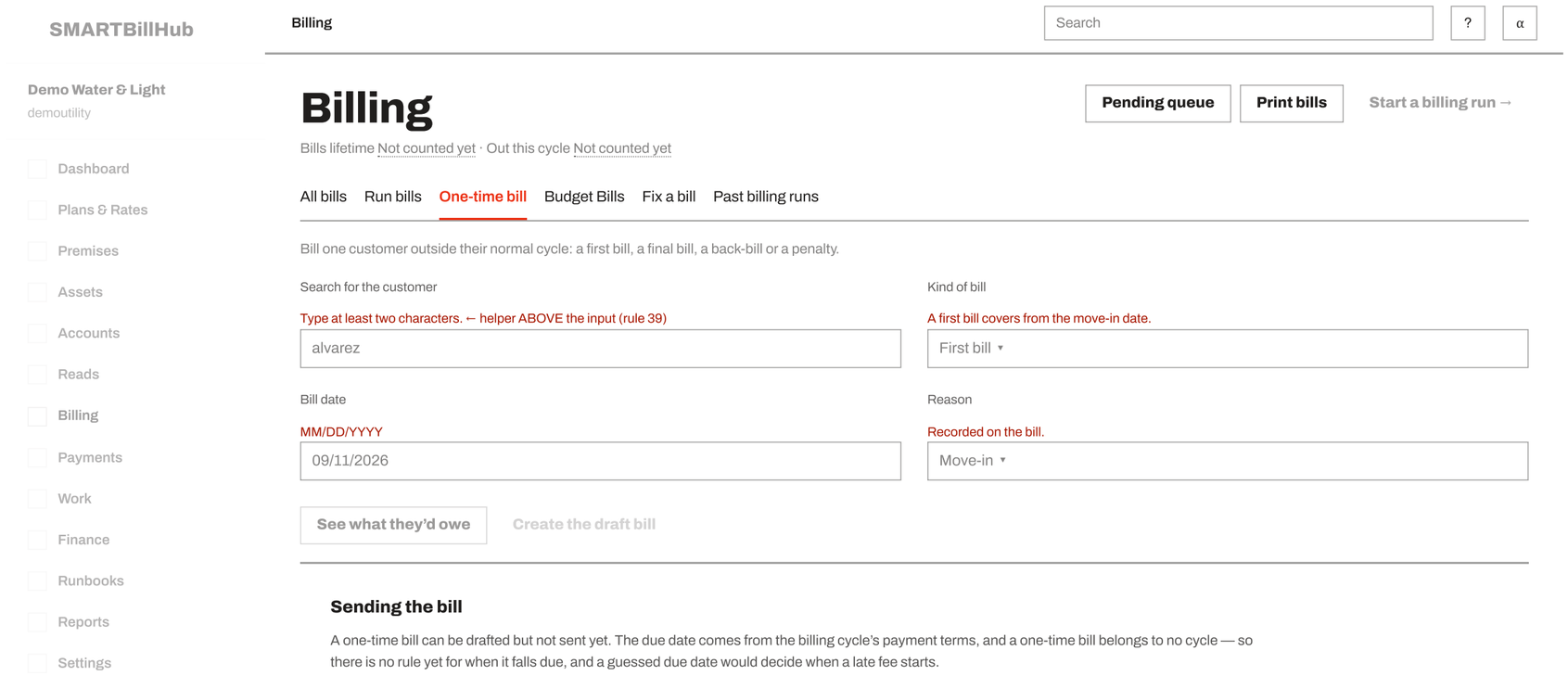
New build S9 S12 S82 S86 S88 Exists behind S2

REDRAWN AS §04 screens 21–24 — the one-time wizard. **Build that one.** This drawing is what exists today, kept because the findings in §09–§11 point at it.

`/:tenant/billing/one-time` · `BIL-01.one-time` · S30

"This one customer needs a bill now, outside the cycle."

> **AS-BUILT** measured in Chromium 1194 at 1440×1100 on **2026-09-11**, at commit `a1d37a68`.
> This is what the product renders today.



For	A first bill, a final bill, an ad-hoc bill, a back-bill or a penalty bill — five engines that had no door at all until this tab landed.
How you get here	The six-tab strip, on S30 and on the wizard. **No other door.**
Primary action	*Create the draft bill* — `POST /billing/one-time` (`api.py:2026`), enabled only after a preview.
Tabs	It is a tab. Its panel is fully built.
Table controls	No record list. Two read-only output tables inside the preview panel, in server order. There *is* one server-side search — `?q=` over `GET /accounts`, at two characters — but it feeds a `<select>`, not a table.
Write surfaces	**Detail** — the draft shows in-panel and **does not link to the bill it just created**; the number is plain text. **Add** — the form is the creator. **Edit** — **NOTHING, by design**: the editable line table and *+ Add line* were removed, and `OneTimeBody` carries no amount member, so it is structural rather than a convention. **Update** — **NOTHING**; failure *is* surfaced, in a `role="alert"` banner.
Walked	SEEN Rendered at 1440 with the 13-item rail, the six-tab strip and the header actions. `scrollWidth − clientWidth = 0`.
Cognitive load	It removes *"which of five one-off billing paths is this"* by asking one question: what kind of bill.
States	**Loading** — the preview. **Empty** — no account chosen, which is the resting state drawn above. **Failed** — the alert banner. **Domain** — a cycle-less bill, which is what the panel at the bottom is about.
Words	*See what they'd owe* rather than *Preview* — good.
Components	the staff form kit (`TextField` · `SelectField` · `DatePickerField`) · `MoneyCell` · `ArrivesWith` · `GapNotes`
Rules	B14 39 — **breached in four places on this screen** · B14 8 (a select of five or more is searchable) · hard rule 2 (money — **clean here**, every figure through `MoneyCell`)
Audience	Staff.
Health	**SERVED** for every field drawn. This screen's problems are its copy and its form construction, not its data.

#	FIELD	COMES FROM	OWNED BY	HEALTH
①	Customer	`GET /accounts?q=`, server-side, ≥2 chars	billing → customers ⇄	SERVED
②	Kind of bill	*a literal list in the component* — five kinds, and the five engines behind them	—	HARDCODED
③	Bill date	typed, tenant zone	billing	SERVED
④	Reason	`GET /corrections/reasons`-style catalogue	billing	SERVED
⑤	The preview figures	`POST /billing/one-time/preview`	billing → rating ⇄	SERVED
⑥	The created draft's number	`invoices.invoice_no`, minted at draft (BR-BIL-063)	billing	SERVED

The panel at the foot of this screen cites a question the Product Owner closed nineteen days ago.

It says a one-time bill *“can be drafted but not sent yet”* because a cycle-less bill has no due-date rule. It cites `OQ-241` in its own source three times. **`OQ-241` is CLOSED** — ruled by the PO on 2026-08-23 at `CR-029 D8③`, verbatim: *“what due date a cycle-less one-time bill gets - it will be capture at the time of bulk billing - not here”*, and the register records that *“a cycle-less bill now issues.”*

The backend half was fixed and the screen was not. `api.py:1857-1867` now says *“AND IT IS NO LONGER REFUSED THERE — THIS COMMENT WAS STALE FOR TEN DAYS (DEF-3153, corrected 2026-09-02)”*, and the gap was deleted from the server’s list. But this tab states it **from its own hardcoded catalogue string**, independent of the server, in all eleven locales. `DEF-3153` is recorded as FIXED. It is half fixed.

And the four form fields put their helper text above the input. That is not this screen’s fault — it is the shared staff kit doing it in all five variants across 28 files (`FormModal.tsx:239-244` and four siblings, `DEF-SHL-4921`). B14 rule 39 ratified *label → input → helper* today. This is the only one of the four tabs that passes `help` at all, which is why it is the only one visibly breaching it.

---

12 Budget Bills

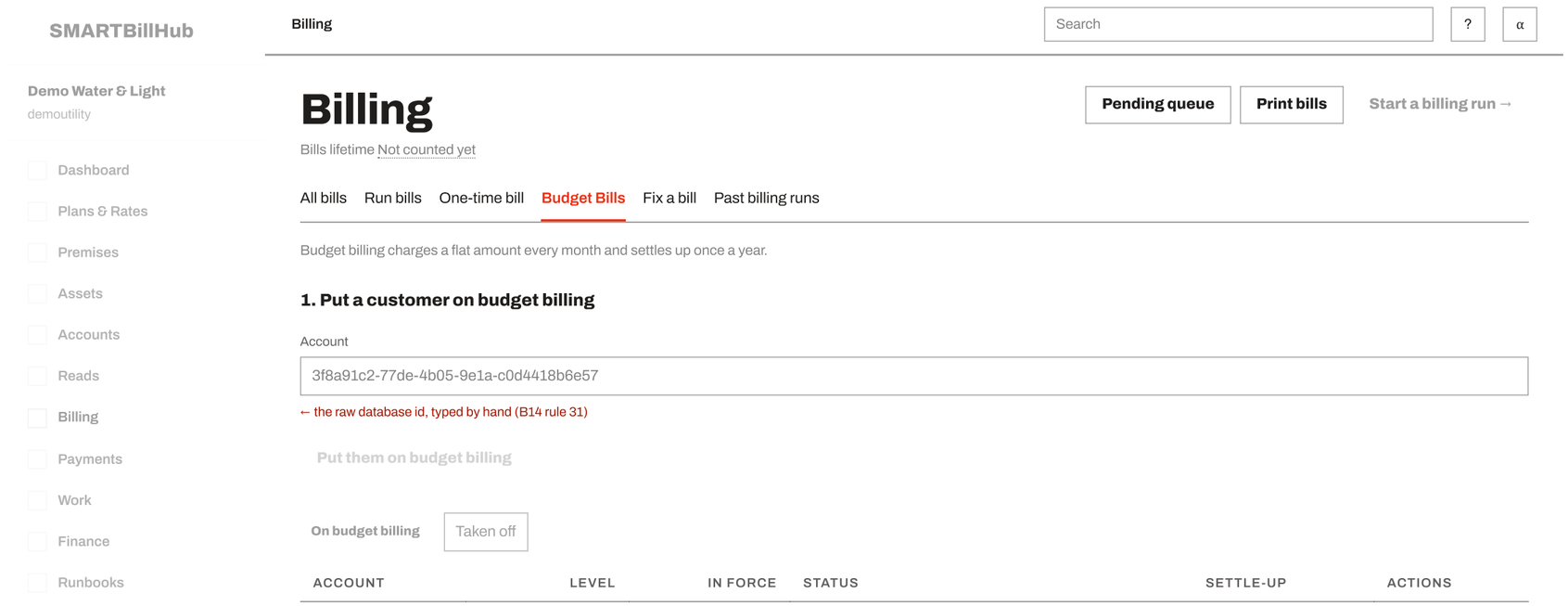
New build `S9` `S11` `S80` `S81` `S82` Exists behind `S8` `S10`

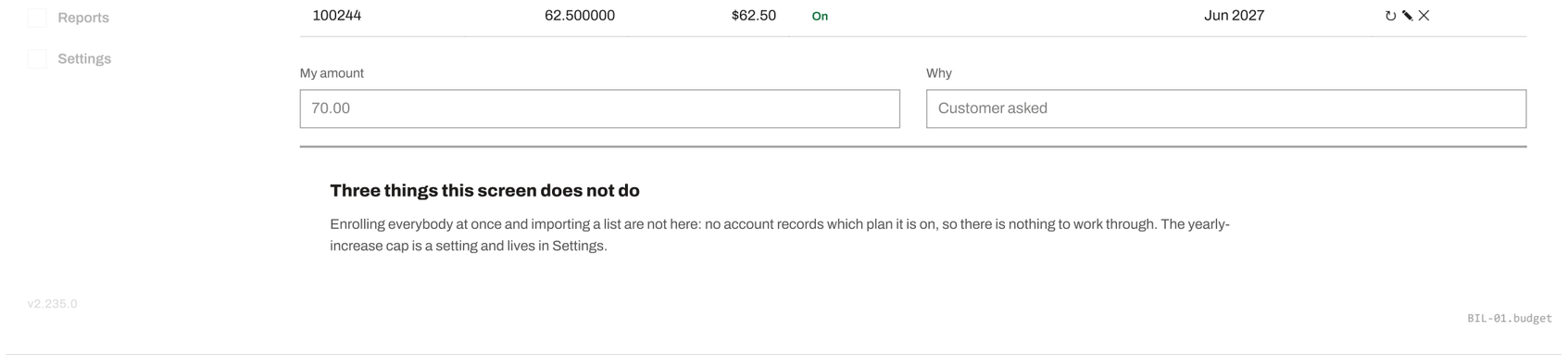
REDRAWN AS §05 screens 25–25.4 — budget bills. **Build that one.** This drawing is what exists today, kept because the findings in §09–§11 point at it.

`/:tenant/billing/budget` · `BIL-01.budget` · `S30`

“Put this customer on a flat monthly amount, and let me change it when they ask.”

> **`AS-BUILT`** measured in Chromium 1194 at 1440×1100 on **2026-09-11**, at commit `a1d37a68`.
> This is what the product renders today.





For	Enrolling an account on budget billing, re-levelling it, overriding the amount and taking it off. `budget_service.enroll` had no caller outside tests until this tab.
How you get here	The six-tab strip only.
Primary action	*Put them on budget billing* — `POST /budget-billing/enrollments`.
Tabs	It is a tab.
Table controls	**Search — NOTHING. Filter — two segmented chips** that *do* send `f.status=`, applied in SQL. `BUT .SEG-OPT.ACTIVE IS NOT DECLARED IN THIS STYLESHEET, SO THE SELECTED CHIP HAS NO VISUAL SELECTED STATE` **Columns — NOTHING**, six hardcoded headers. **Sort and pagination — the server is compliant and the client cannot reach it:** `budget_api.py:286-292` orders in SQL and paginates, and the client sends no `page`, `per`, `sort` or `dir` and renders no pager. **Page 1 only, for ever.**
Write surfaces	**Detail — NOTHING**, the account cell is plain text, so the table is a dead end. **Add** — the step-1 panel. **Edit** — partial, and **it does not load current values**: the override form is blank on mount, never prefilled, and sits *below the whole table* rather than in the row, so editing row 3 means typing into a form that names no row. Two of seven fields are editable. **Update** — three writes, and **two of the three swallow failure entirely**.
Walked	`SEEN` Rendered at 1440 with the 13-item rail, the six-tab strip and the header actions. `scrollWidth - clientWidth = 0`.
Cognitive load	It is meant to remove the annual arithmetic. It adds the job of finding a UUID.
States	**Loading** — skeleton. **Empty** — drawn above, no rows. **Failed** — only two of six reads have a failure branch. **Domain** — eligibility, which the product cannot determine.
Words	*Level* for the flat amount, which is the domain's word; *Settle-up* rather than *reconciliation*.
Components	hand-rolled `label` + `input` (not the shared kit) · `PlainTable` · `RowActions` · `ArrivesWith`
Rules	**B14 rule 31 — breached** · hard rule 2 — **breached twice** · NFR-3 · B14 16 (pagination on every list table) — **breached**
Audience	Staff.
Health	**NOT SERVED** — eligibility is null by construction.

#	FIELD	COMES FROM	OWNED BY	HEALTH
①	Account (the input)	*a raw* `accounts.id` *UUID typed into a text box* — placeholder: *“Paste the account's id from its detail page”*	billing → customers ⇄	`SERVED`

②	Eligibility	`eligible` *is* ***null by construction****, never true or false: BR-RAT-043 puts the flag on the rate plan and nothing publishes it —* `OQ-237`	billing → rating ⇄	NOT SERVED
③	Level	`budget_enrollments.level` — **money, rendered raw at six decimals with no currency** through a translation placeholder	billing	SERVED
④	In force	the effective level, through `MoneyCell`	billing	SERVED
⑤	Status · settle-up month	`budget_enrollments`	billing	SERVED
⑥	Variance carried	**money, interpolated raw into a toast** as `{{amount}} {{currency}}`	billing	SERVED

The account field asks a clerk to paste a database UUID, and B14 rule 31 is *“no database id is ever the visible value of a field”*. | `DEF-1181`, still true. There is no account picker and no search on this tab — the UUID is the *only* way in. The irony is local: the table’s own account cell was fixed for this very rule (DEF-3158) while the form one field away still asks for the id.

Two money figures reach the screen outside the formatter — hard rule 2 says every amount displays at two decimals. The *Level* column interpolates `row.level` into a sentence (*“set by staff; worked out at {{computed}}”*) raw, at whatever precision the server sent. And removing someone from budget billing raises a toast reading *“They are still {{amount}} {{currency}} ahead or behind”* — **literally the** `${amount} ${currency}` **shape**. That is the exact defect a comment in the *neighbouring file* warns about: *“A lane shipped* `-147.500000` *into a toast with no currency under 21 passing unit tests.”* Reproduced one file over.

And two of the three row actions cannot report a failure. *Recheck the amount* and *Take off budget billing* are `async` handlers with no `try` / `catch` and no `isError` branch, invoked through `RowAction.onClick`, which is typed `() => void` — so the rejection is never awaited and nothing appears on screen. A clerk presses *Take off budget billing*, the server refuses, and the row does not change. Only *Use my amount* surfaces its error.

Group D · Putting a bill right

---

13 Fix a bill

New build S9 S11 S16 S80 S81 S83 Exists behind S7 S8 S10

REDRAWN AS §06 screens 26–26.8 — the fix-a-bill wizard. **Build that one.** This drawing is what exists today, kept because the findings in §09–§11 point at it.

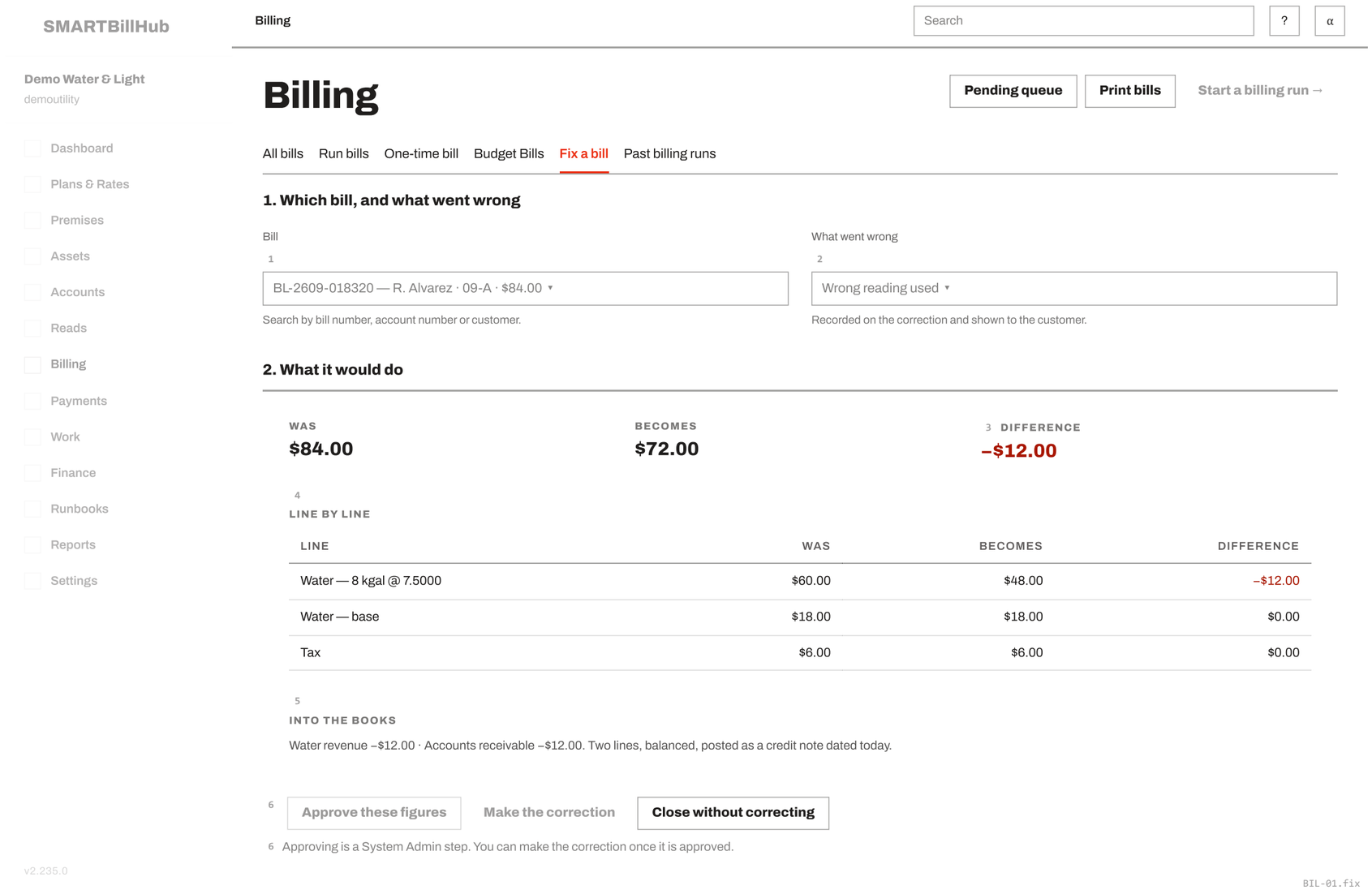
`/:tenant/billing/fix` · also `/billing/fix?bill=<uuid>` · BIL-01.fix · S30

“We billed them on the wrong reading. Put it right, and show me what that does before I commit.”

SUPERSEDED This proposal is replaced by §06 screens 26–26.8. Kept as the earlier reading, not as work. **Fix a bill — make the approval path readable**

CLOSES BIL-S7 · DEF-180 · DEF-BIL-4633

AS-BUILT, FROM CODE composed from source at commit `a1d37a68` and **never rendered**, because it threw on load — `(reading 'map')` to a stack trace. Treat the shape as read, not as seen.



1 **Bill** · a searchable select over `GET /invoices?q=`, showing the **bill number** and enough to recognise it · **B14 31** (never the uuid) · UI-RULES 8 (5+ options are type-to-filter) · **RELABELLED + NEW READ**

2 **What went wrong** · `GET /corrections/reasons`, showing the **sentence** not the code · B14 32 · **UNCHANGED DATA**

3 **Difference** · `section.delta` through `MoneyCell` with no `String()` coercion, so an absence *throws to the boundary* and states itself instead of printing `" USD"` · hard rule 2 · **CHANGED**

4 **Line by line, as a table** · reads the keys the server actually sends — `label`, `original`, `corrected`, `delta` — every figure through `MoneyCell` · hard rule 2 · **CHANGED**

5	**Into the books** · `net_by_bucket` rendered as a sentence with formatted money, instead of a generic key/value dump at six decimals · hard rule 2 · hard rule 5 · **CHANGED**	
6	**Approve** · gated on the **System Admin** cell that the endpoint actually requires, with the reason stated rather than a button that fails · BR-SHL-025 · **CHANGED**	

Carry-over

AS-BUILT FIELD	DISPOSITION	WHY
Bill (the identifier field)	RELABELLED	Same field, same place, same purpose — it stops showing a UUID and starts showing the bill number. The old label *Bill* is kept; only the *value* changes, which is what B14 31 asks.
What went wrong	UNCHANGED	The value becomes the sentence rather than the code, which is DEF-2448, not a field change.
Was · Becomes	UNCHANGED	Same figures, same place; the coercion behind them changes.
Difference	UNCHANGED	Same field. It starts rendering.
Line by line	MOVED	From a bullet list to a table, in the same position, because four figures per line need columns. **Nothing is dropped** — it currently shows nothing at all.
Into the books	UNCHANGED	Same section, same position; the generic key/value dump becomes a sentence.
The customer's balance	UNCHANGED	Not drawn above for space; it carries over untouched, with the same coercion fix.
The three action buttons	UNCHANGED	Same three, same order, same words. Only the gate on *Approve* changes.
The no-rollback note	UNCHANGED	BR-BIL-047. Load-bearing.
The no-edit note	UNCHANGED	**Load-bearing.** *"There is no way to change a line on an issued bill, and there never will be."*
Corrections-so-far table	UNCHANGED	Untouched by this remedy; its pagination is BIL-S8, separately.

```
As-built fields   11
  unchanged        9
  moved            1   -> line-by-line becomes a table, same position
  relabelled       1   -> the Bill field's VALUE stops being a uuid
  removed          0
Remedy fields     11   = 11 - 0 removed + 0 new      RECONCILES

** NOT ONE NEW FIELD. Every figure drawn above is already in the served
   payload and is already meant to be on this screen. The remedy is a
   rendering fix, which is why it is the cheapest of the three. **
```

▸ **As built today** — kept because the findings below point at it. ***Do not build this.***

For	Opening a correction case, previewing its whole effect, approving it and executing it. The correction engine has been complete and golden-verified since 2026-08-15 and this tab is the first thing in the product that can reach a line of it.

How you get here	Three doors, and two are cross-module: the six-tab strip; *Fix this bill* on the bill detail (`BillDetailPage.tsx:420`); and *Issue credit* on the account's icon rail (`accountRailReadiness.ts:258`).
Primary action	*Make the correction* — `POST /corrections/{id}/execute`.
Tabs	It is a tab; inside it, three numbered steps in one scroll.
Table controls	**All four NOTHING.** `getCorrectionCases(client)` takes **no parameters at all**. The server paginates in SQL and the client cannot reach page 2.
Write surfaces	**Detail** — a case opens *in-page* via a button, with **no deep link**, so a clerk cannot send a colleague a correction case. **Add** — *Show me what it would do* opens the case. **Edit** — **NOTHING, and the screen says so**, which is exactly right. **Update** — three transitions; *Close without correcting* **swallows its failure**.
Walked	**Not walked** — threw on a minimal payload; no error boundary, so React Router's stack-trace page. See §04.
Cognitive load	It is meant to remove the fear: show the whole effect — the bill, the ledger, the customer's balance — before anything is written.
States	**Loading** — the preview. **Empty** — *"No credit note or replacement bill is recorded against this bill"*, which the repo holds up as the model of a good empty state. **Failed** — the preview's error *is* surfaced. **Domain** — a draft, where the door disables with *"A draft is not corrected — it is re-run."*
Words	*What went wrong*, not *reason code*. `DEF-2448: THE CORRECTIONS TAB PRINTS THE MACHINE CODE WHERE THE SENTENCE BELONGS — WRONG_READ_USED , DRAWN ABOVE`
Components	hand-rolled fields · `PlainTable` · `NineSections` · `MoneyCell` · `GuardedButton`
Rules	**hard rule 4** (corrections are new rows — honoured exactly) · **B14 rule 31 — breached** · **hard rule 2 — breached on the approval path** · BR-BIL-047 (no rollback)
Audience	Staff — and the approve step is **System Admin only** while the button is gated on the clerk cell.
Health	**NOT SERVED** — the line-by-line list reads three keys the server does not send.

Data flow

```
corrections.preview ──→ corrections.py:303-311 builds each line as
                        { key, label, kind, original, corrected, delta, moved }
                      ↳ GET /corrections/{id}          corrections_api.py:701
                      ↳ FixABillTab.tsx:365-368 reads
                        line.description · line.before · line.after   ** NONE EXIST **
                      ↳ renders an empty bullet and " · -> "
                                              BREAKS AT THE RENDERER
```

#	FIELD	COMES FROM	OWNED BY	HEALTH
①	Bill (the input)	*the raw* `invoices.id` *UUID,* ***prefilled from the query string by the intended door***	billing	SERVED
②	What went wrong	`GET /corrections/reasons` — a static catalogue	billing	SERVED
③	Was · Becomes	`section.original_total` · `corrected_total`, coerced with `String(x ?? "")`	billing	SERVED

④	Difference	`section.delta`, same coercion — **renders `" USD"` when the key is absent**	billing	SERVED
⑤	Line by line	*reads `description` / `before` / `after`; the server sends `label` / `original` / `corrected`*	—	NOT SERVED
⑥	Ledger hand-off	*bucket deltas are money and print through `String()` at six decimals with the currency as just another key*	billing → ledger ⇄	SERVED
⑦	Approval mode · back-bill window	*served null-with-reason rather than defaulted — "a hardcoded 12 here would be this surface inventing a finance figure"*	billing	NOT SERVED

The list an approver is supposed to check before approving a correction renders as two empty bullets. This is the most consequential new finding in the audit and I do not find a defect row for it.

`FixABillTab.tsx:367` builds each line as `` `  · ${String(line.before ?? "")} → ${String(line.after ?? "")}` `` — a raw template literal, no `MoneyCell`, **no currency at all**. And the server's line dicts (`corrections.py:303-311`) carry `key · label · kind · original · corrected · delta · moved`. There is **no `description`, no `before`, no `after`**. So the label renders empty and the movement renders literally `" · → "`, on every line, on the screen where a System Admin signs off a change to a customer's money.

It is the same defect as `DEF-RBS-4352` one module over — *"the runbook's report blocks do not match the renderer's payload contract, so a clerk cannot read the plan"* — which suggests a shape worth a gate rather than two fixes.

`DEF-180` is still live and its visible string has changed. Seven money figures here go through `String(x ?? "")`. `MoneyCell` is built to *throw* on an absence and let its own boundary state the problem (that is what `DEF-151` added) — but `String(x ?? "")` converts an *absence* into a *malformed amount*, which falls to the verbatim branch and prints silently. `DEF-180` recorded `"undefined USD"`; today it prints `" USD"`, drawn above. **The coercion defeats the guard.**

And a clerk sees an enabled button the server will refuse. *Approve these figures* gates on the clerk-level cell (`:604`) while `POST /corrections/{id}/approve` is **System Admin only** (`corrections_api.py:545-546`).

Group E · Getting bills out, and holding them back

---

14 Pending queue

New build S9 Exists behind S6 S25 S100

REDRAWN AS §03 screens 20–20.3 — the held queue, at this same route. **Buil d that one.** This drawing is what exists today, kept because the findings in §09–§11 point at it.

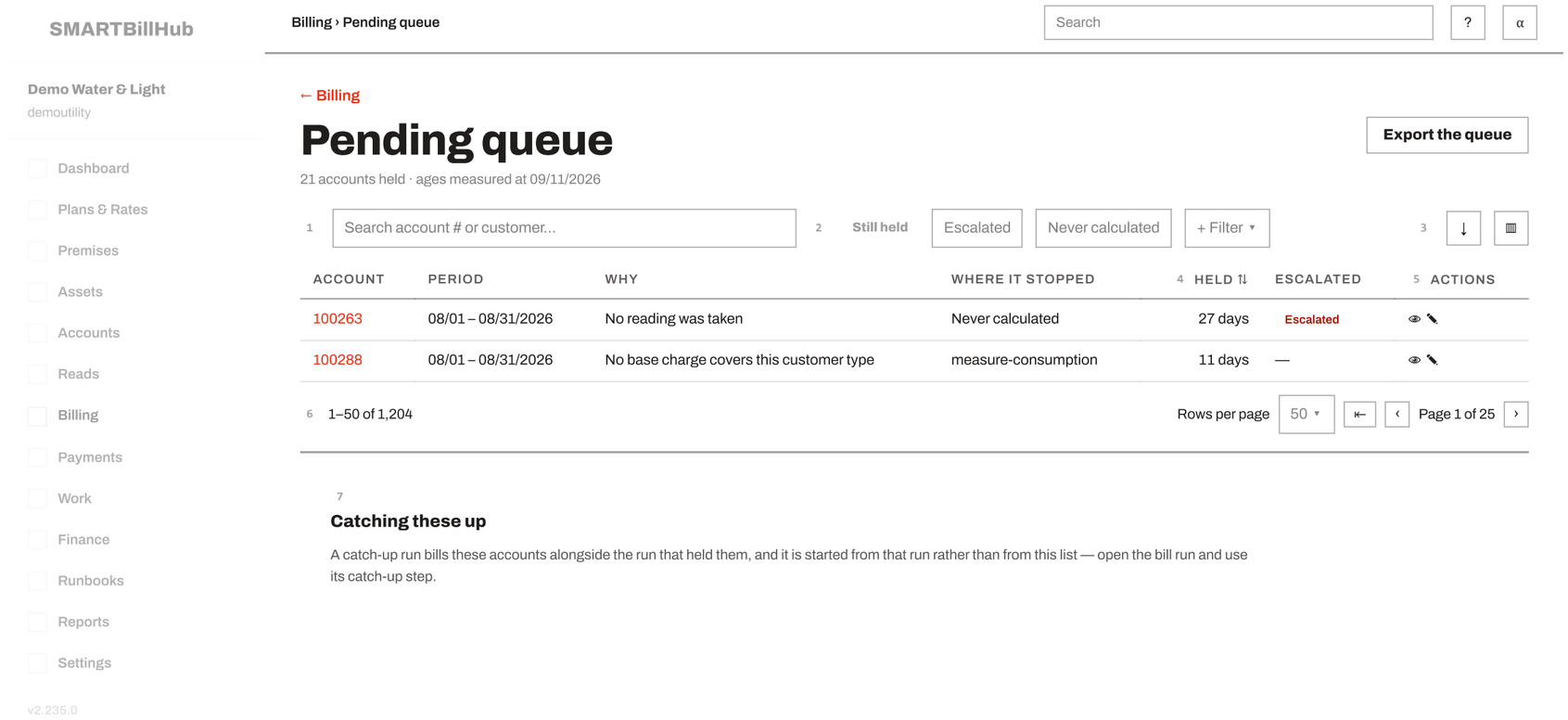
`/:tenant/billing/pending` · BIL-06 · **no A1 id**

"The run said it left twenty-one accounts out. Which ones, and why?"

SUPERSEDED This proposal is replaced by §03 screens 20–20.3. Kept as the earlier reading, not as work.

Pending queue — page it in SQL before a T5 tenant opens it | CLOSES BIL-S6 · NFR-3 · NFR-5

> **AS-BUILT, FROM CODE** composed from source at commit `a1d37a68` and **never rendered**, because it threw on load — `(reading 'open')` to a stack trace. Treat the shape as read, not as seen.



1. **Search** · `?q=` over account # and customer, resolved in SQL · UI-RULES 12 · **NEW READ**
2. **Filter chips** · `f.escalated=`, `f.origin=`, `f.stage=` — the two columns this screen exists for become filters · UI-RULES 12 · **NEW READ**
3. **Export** · a `pending` resource in the export registry beside `bill_run_releases` · BR-SHL-016 · **NEW**
4. **Sort** · `ORDER BY age_days DESC` in SQL, replacing the Python `sorted()` at `pending.py:324` · UI-RULES 13 · **CHANGED**
5. **Row actions** · open the account · enter a reading · UI-RULES 15 · **NEW**
6. **Pagination** · `StandardPagination`, exactly as `/invoices` already does it · NFR-3 · UI-RULES 16 · **NEW**
7. **The catch-up note** · unchanged, and still correct · **UNCHANGED**

Carry-over

AS-BUILT FIELD	DISPOSITION	WHY
Account #	UNCHANGED	Same place, same words

Account #	UNCHANGED	Same place, same words.
Period	UNCHANGED	
Why	UNCHANGED	And it becomes filterable at ②, which is additive.
Where it stopped	UNCHANGED	Likewise.
Held (days)	UNCHANGED	Same column; the *sort* moves from Python to SQL, which a clerk cannot see.
Escalated	UNCHANGED	
The *as-of* anchor in the header	UNCHANGED	**Load-bearing.** BR-TEN-043: an age never renders without the day it is an age at.
The include-resolved toggle	MOVED	It becomes one chip in the filter row at ② rather than a lone control, so it sits with the other three questions of the same shape.
The catch-up note	UNCHANGED	Verified still true.

```
As-built fields    9
  unchanged        8
  moved            1   -> include-resolved becomes a chip
  relabelled       0
  removed          0
Remedy fields     15   = 9 - 0 removed + 6 new      RECONCILES
```

The back-end half is the point, and it is three changes in one file. Give `pending_rows()` a `page`/`per` and an `ORDER BY`; drop the Python `sorted()` at `:324`; and stop `pending_count()` re-running the whole query — it can be a `COUNT(*)` over the same filtered queryset and still tie to the list, which is the property its docstring rightly protects. **None of that is new machinery:** `StandardPagination` is already used by `/invoices`, `/corrections` and `/budget-billing` in this same module.

▶ **As built today — kept because the findings below point at it.** ***Do not build this.***

For	Naming every account a run declined to bill. *Hold ≠ forget.* Until this screen landed, the run said *"21 excluded"* and nothing in the product could name them.
How you get here	**Exactly one door** — *Pending queue* in S30's header (`BillsListPage.tsx:1009`). It is **not** in `SIDEBAR_DESTINATIONS`, so the sidebar-reachability suite does not cover it, and the whole screen hangs off one button on one screen.
Primary action	*None*, deliberately — see below.
Tabs	*None.* One boolean toggle, *including resolved*.
Table controls	**All four NOTHING**, and the endpoint is the module's clearest NFR-3 breach — see the box. Rendered with `PlainTable`, so there is no toolbar to put them in.

Write surfaces	**Detail** — the account cell links out. **Add · Edit · Update** — **NOTHING**, all three, and correctly: a queue row is a record of a decision, not a thing to edit.
Walked	**Not walked** — threw on a minimal payload; no error boundary, so React Router's stack-trace page. See §04.
Cognitive load	It removes the single worst question in billing — *"who did we silently not bill?"* — and it splits the two answers that need different fixes: *we never calculated this* (enter a reading) from *we calculated it and it broke* (fix configuration).
States	**Loading** — skeleton. **Empty** — nothing held, the good case. **Failed** — plain-language. **Domain** — escalated, past the tenant's threshold, which is *reported as not-a-setting-yet rather than invented*.
Words	*Why* and *Where it stopped* — two columns doing genuinely different work, which is the reason this screen exists.
Components	`PageHeader` · `PlainTable` · `StatusPill` · `GapNotes` · `ArrivesWith`
Rules	BR-BIL-015 · BR-BIL-021 · **BR-TEN-043 — honoured exactly** · **NFR-3 — breached**
Audience	Staff.
Health	**SERVED** for every field. This screen's problem is its shape at scale, not its data.

Data flow

```
bill_run_exclusions ─→ pending.py:284-290  queryset, NO LIMIT, NO SLICE
                    └→ :291-323  loops the WHOLE result into a Python list
   └→ :324  tuple(sorted(out, key=-age_days))   ** PYTHON SORT **
   (discarding the SQL order_by at :290)
└→ api.py:1762  pending_count() RE-RUNS pending_rows
   ** THE FULL SET IS MATERIALISED TWICE PER REQUEST **
└→ GET /billing/pending?as_of=  api.py:2027 → :1726
└→ data.items.map(...) in a PlainTable          WHOLE, UNBOUNDED
```

#	FIELD	COMES FROM	OWNED BY	HEALTH
①	Account #	`accounts.account_no` via `select_related` — **no N+1**	billing → customers ⇄	SERVED
②	Period	`bill_runs.period_start/end`	billing	SERVED
③	Why	`reason_code` → a sentence	billing	SERVED
④	Where it stopped	`_stage_of(reason_code)` — the column this screen exists for	billing	SERVED
⑤	Held (days)	computed server-side against a **required** `as_of`; the anchor date is printed beside it	billing	SERVED
⑥	Escalated	`age_days > threshold`, strictly greater	billing	SERVED
⑦	Money	*none drawn and none served — a queue row is an account that has **not** been billed, so a zero would be a claim*	—	SERVED

This is a fifth NFR-3 breach, structurally identical to the four CLAUDE.md already names, and I find no defect row for it.

`GET /billing/pending` takes **no** `page`**, no** `per`**, no** `sort` — the client sends only `as_of` and `include_resolved`. `pending.py` materialises every unresolved exclusion for the tenant, then **sorts in Python at** `:324`, discarding its own SQL ordering. And `pending_count()` — the KPI number — **re-runs the same unbounded query**, so the full set is built *twice per request*.

That is the `GET /outstandings` shape exactly: *"Unpaginated ON PURPOSE… and it sorts in Python, not SQL."* The difference is that `/outstandings` at least has a written rationale naming the old 3,000-connection ceiling. This has none. At T5 — 40,000 accounts, the top of the current ICP — a run excluding 3% leaves about **1,200 unpaginated rows**; and the run that produced them is itself capped at 25,000 accounts (`DEF-BIL-4224`, high, open), which is *13,000 short of the ICP*.

What this screen gets right, for the record: `select_related` means no N+1, and `as_of` is a *required* parameter with the anchor date printed next to the ages it produced — so *"held 27 days"* can never silently change meaning for a reader in another timezone. And the missing catch-up button is **declared**, not hidden: *"named rather than guessed."* That is the right way to leave a gap, and it is the contrast case against the dead controls elsewhere in this module.

---

15 Print bills

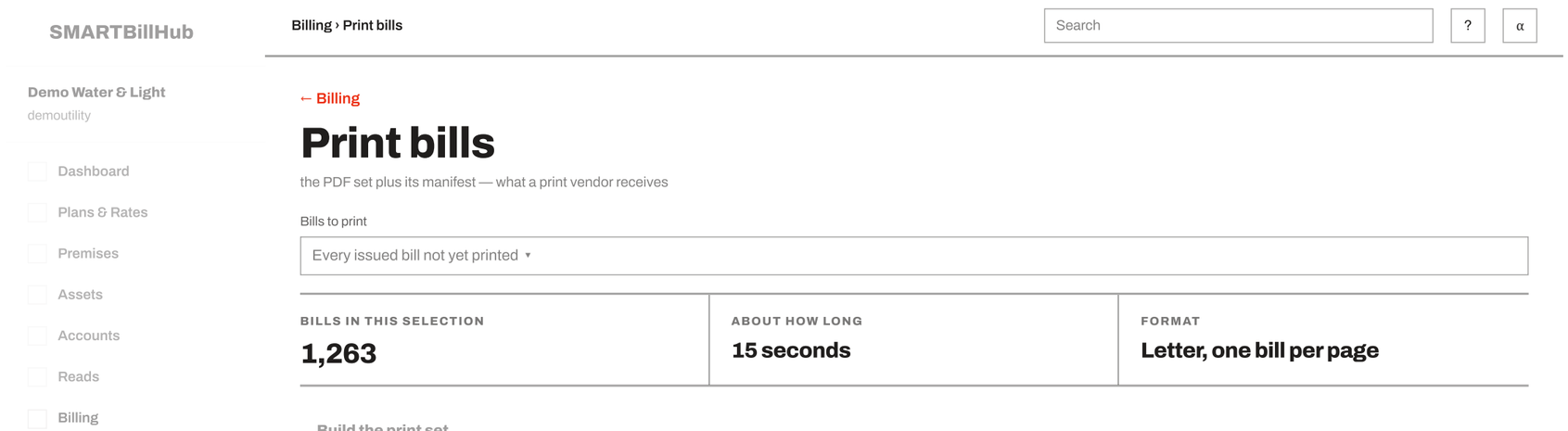
New build `S9` Exists behind `S30`

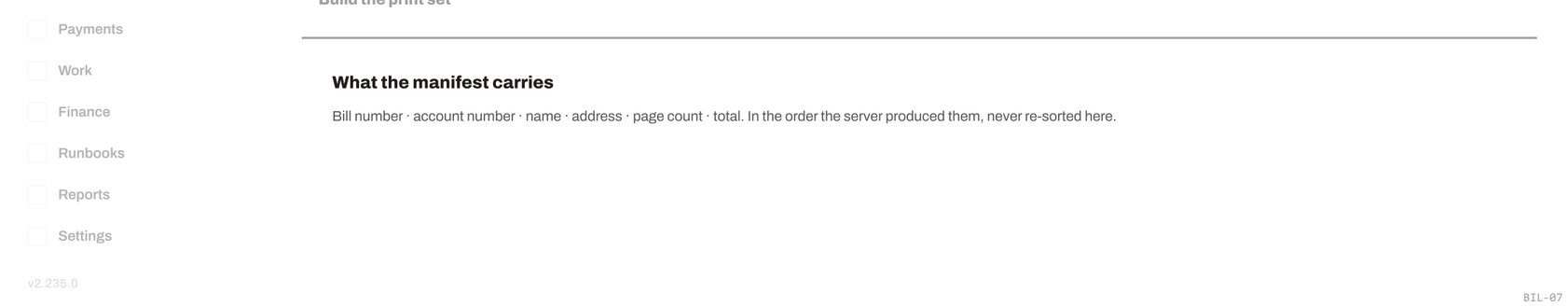
REDRAWN AS §08 screens 28 and 28.1 — bill printing. **Build that one.** This drawing is what exists today, kept because the findings in §09–§11 point at it.

`/:tenant/billing/print` · BIL-07 · **no A1 id**

"Printing is the only channel that works. Give me the file the vendor needs."

> **AS-BUILT, FROM CODE** composed from source at commit `a1d37a68` and **never rendered**, because it threw on load — `(reading 'join')` to a stack trace. Treat the shape as read, not as seen.





For	Producing the PDF set and its manifest — and it matters more than it looks, because *printing is the channel every bill has* and nothing else dispatches.
How you get here	**Exactly one door** — *Print bills* in S30's header (`BillsListPage.tsx:1020`). Like the pending queue, it is not in `SIDEBAR_DESTINATIONS`.
Primary action	*Build the print set* — `POST /billing/print-sets`, then a polled job. A `GuardedButton` with **four** distinct refusals.
Tabs	*None.*
Table controls	**No table of any kind.** One scope `<select>`, which does send `bill_run_id`. The manifest's columns are printed as prose, in server order, *explicitly never re-sorted here* — which is the right call for a vendor file.
Write surfaces	**Detail · Add · Edit · Update** — **NOTHING**, all four. The file says so twice: *"it has no form, no field and no save."* The one write is the build, whose failure *is* surfaced in a `role="alert"` block, and a job that succeeds with no file **says so**.
Walked	**Not walked** — threw on a minimal payload; no error boundary, so React Router's stack-trace page. See §04.
Cognitive load	It answers *"how many, how long, what format"* before committing to a job that takes minutes.
States	**Loading** — the count preview, then the polled job. **Empty** — no issued bills in the selection, which is one of the four refusals. **Failed** — surfaced, and *"finished but no file was reported"* is a distinct fifth state, which is unusually careful. **Domain** — over the ceiling, where the refusal names the number.
Words	*About how long* rather than a promised duration — the estimate comes from a measured constant, not a budget.
Components	`PageHeader` · `GuardedButton` · `KPIStrip` · `PrintSetNotes`
Rules	BR-SHL-016 (server-side export under the caller's tenant context) · hard rule 1
Audience	Staff — *producing the file is a write-shaped act*, so Admin or Clerk.
Health	**SERVED**.

#	FIELD	COMES FROM	OWNED BY	HEALTH
①	Bills to print	`GET /billing/print-sets` — a `COUNT(*)`, no rows serialised	billing	SERVED
②	About how long	`MEASURED_SECONDS_PER_BILL = 0.012` — *a measurement, not a budget*	—	DERIVED
③	Format	*always* `DEFAULT_FORMAT_NAME` — *the single existing shape*	—	HARDCODED
④	The manifest columns	served inside the preview, not a separate endpoint	billing	SERVED

⑤	The archive	downloaded through the platform route `GET /api/exports/{job_id}/file`	billing → tenancy ⇄	SERVED

The best-behaved screen in the module, and worth saying so in an audit that has mostly found the opposite: four distinct refusals instead of one generic one, a fifth state for *"succeeded but produced no file"*, an estimate derived from a measured constant with the measurement named, a manifest deliberately not re-sorted, and an explicit twice-stated declaration that it writes nothing. **Its one weakness is the door:** the channel that every bill depends on is reachable from exactly one button on one screen, and no reachability test covers it.

I · Every field, and where its data comes from

Sorted by source, not by screen — the point is to put every consumer of one column on adjacent rows, because the question this table gets asked after delivery is *"if I change this column, what breaks?"*

FIELDS TRACED	SERVED	NOT SERVED	STUBBED	HARDCODED · DERIVED	NEVER TRACED
71	**46**	**16**	**5**	**4**	**0**
across 15 screens		the screen wants it; no read exists	fixed value regardless of input		an ungraded field is not a healthy one

SOURCE	SCREENS	ENDPOINT	OWNED BY	HEALTH
`invoices.invoice_no`	01, 02, 03, 11, 13	`GET /invoices`, `…/{id}`	billing	SERVED
`invoices.total` · `subtotal` · `tax_total` · `currency`	01, 02, 03, 13	same	billing	SERVED — and the **rendering** breaks on 13
`invoices.status`	01, 02	same	billing	SERVED
`invoices.issued_at` · `due_date`	01, 02, 03	same	billing	SERVED
`invoices.past_due`	01	same	billing ⇄ payments + ledger	STUBBED — `None` on every row, `selectors.py:961-963`
`invoices.template_version`	01, 02, 03	same	billing	SERVED
`invoices.bill_payload`	02, 03	`…/{id}/document`	billing	SERVED — re-rendered, not stored
`invoices.cycle_label`	01	`GET /invoices`	billing ⇄ metering	SERVED
`invoices.bill_run_id`	01, 02	same	billing	SERVED
`bill_runs.run_by`	04, **10**	`GET /billing/runs` as `ran_by`	billing ⇄ identity	SERVED — **and denied on screen**
`bill_runs.status`	04	same	billing	DEAD — written once, never advanced
`bill_runs.run_no`	04, 10	same	billing	STUBBED — null on every row

`bill_runs.period_start/end`	04, 14	same, and `/billing/pending`	billing	SERVED
a run's money total	04, 10	—	—	NOT SERVED — no column exists
`bill_run_exclusions.*`	14	`GET /billing/pending`	billing	SERVED, unbounded
`bill_run_decisions.case`	06	`…/exceptions/{id}/resolve`	billing	SERVED — no row for `extra_charge`
run-case money (`amount`, `quantity`, `unit`, `totals`)	05, 06	`…/findings`, `…/exceptions`	billing	STUBBED — null on every row; phase 1 computes no money
late fees and extra charges	06	—	—	NOT SERVED — DEF-2596
the unusually-large list	07	—	—	NOT SERVED — DEF-2855; `flagged_drafts` raises `OQ-1202` and has zero callers
`accounts.account_no`	01, 05, 06, 12, 14	several	billing ⇄ customers	SERVED
`accounts.id`	**12, 13**	typed by hand into a text box	billing ⇄ customers	B14 31 BREACH
customer name on a run-case row	05, 06	`…/findings`	billing ⇄ customers	NOT SERVED — B1/G6 seam limit
`premises.address_*`	01 *(search only)*, 03 *(frozen)*	`?q=` via metering's predicate	billing ⇄ metering	SERVED — never a column
`meter_reads` → usage	05, **01's account tab**	`GET /accounts/{id}/usage`	billing ⇄ metering	SERVED — **and denied on screen** (§05 R2)
`rate_plans.base_charge`	05	gathered, **not on the wire**	billing ⇄ rating	NOT SERVED — and denied on screen (§05 R3)
BR-RAT-043's budget-billing flag	12	—	billing ⇄ rating	NOT SERVED — `OQ-237`
`budget_enrollments.level` · `effective_level`	12	`GET /budget-billing`	billing	SERVED — rendered raw at 6dp
`corrections.*` preview sections	13	`GET /corrections/{id}`	billing	SERVED
correction line `description` / `before` / `after`	**13**	the server sends `label` / `original` / `corrected`	—	NOT SERVED — the key mismatch
`net_by_bucket`	13	same	billing ⇄ ledger	SERVED — rendered as a key dump
approval mode · *back-bill window*	13	`/corrections/reasons`	billing	NOT SERVED — null-with-reason, deliberately
delivery queue rows	02, 09	served; nothing reads them onward	billing ⇄ notifications	SERVED
anything actually dispatched	09, 02	—	—	NOT SERVED — and the screen says so
`print_set` count · manifest	15	`GET /billing/print-sets`	billing	SERVED

`MEASURED_SECONDS_PER_BILL`	15	a module constant	—	DERIVED
`DEFAULT_FORMAT_NAME`	15	the single existing shape	—	HARDCODED
the five one-time bill kinds	11	a literal list in the component	—	HARDCODED
the export archive	15, 08	`GET /api/exports/{job}/file`	billing ⇄ tenancy	SERVED
a bills CSV	01	—	—	NOT SERVED — no `bills` resource

J · Health, and what the grade rests on

A screen's health is the worst of its fields, never an average. An average turns one broken field among nine into *"90% healthy"*, and the screen still does not work.

J1 · Fields

VERDICT	COUNT	EVERY NON-SERVED FIELD, WITH ITS SCREEN
SERVED	46	—
NOT SERVED	16	01 delivery status + run # · 01 bills CSV · 02 the document · 02 amount-due summary · 02 per-authority tax · 04 run money total · 05 customer name · 05 base charge · 06 customer name · 06 late fees + extra charges · 07 the unusually-large list · 07 the three overcharge actions · 07 the three draft powers · 09 anything dispatched · 12 eligibility · 13 the correction line keys · 13 approval mode + window
STUBBED	5	01 `past_due` · 04 `run_no` · 04+10 `status` · 05 run-case money · 06 run-case money
DEAD	1	04 `bill_runs.status` — written once as `draft`, never advanced
HARDCODED	2	11 the five bill kinds · 15 the print format
DERIVED	1	15 the time estimate — from a measurement, and it says so
NEVER TRACED	0	—

J2 · Flows — one row per screen

SCREEN	VERDICT
01 All bills	WHOLE for the nine served fields, SEEN — and BREAKS AT THE SERIALIZER for `past_due`
02 Bill detail	BREAKS AT `DOCUMENT_URL` — the route exists, the panel refuses on a null. UNVERIFIED beyond that.
03 Bill document	UNVERIFIED — not walked
04 step 1	FORKED — `run_by` is served *and* denied on screen
05 step 2	FORKED — base charge gathered, dropped at the payload, denied on screen
06 step 3	BREAKS AT THE DECISION TABLE — no row shape for `extra_charge`
07 step 4	BREAKS AT THE SERVICE — two of three draft powers are complete and unrouted to the screen

08 step 5	`UNVERIFIED`
09 step 6	`UNVERIFIED` — and the absence is verified true at the seam
10 Past billing runs	`BREAKS AT THE COMPONENT`, `SEEN` — the data and the renderer both exist and are not connected
11 One-time bill	`WHOLE`, `SEEN` — its faults are copy and form construction, not data
12 Budget Bills	`BREAKS AT THE CLIENT`, `SEEN` — the server pages and the client cannot ask
13 Fix a bill	`BREAKS AT THE RENDERER` — three keys that do not exist, on the approval path
14 Pending queue	`WHOLE` and **unbounded** — correct at T7, 13,000 accounts short of T5
15 Print bills	`WHOLE`

4 seen · 11 UNVERIFIED or partly so. No flow is graded WHOLE on a read alone without the word UNVERIFIED beside it.

J3 · Seams

In §08: **10 HEALTHY** (94 functions, none returning a raw queryset) · **1 LEAKY by rule** (six imports in five files bypassing a seam, reads only, one of them the sanctioned registration shape) · **0 WRITING** · **0 ABSENT**. And `DEF-079` **'s five substituting seams are ungraded**, named rather than guessed at.

K · Everything an upgrade would drop, and who ruled it

One row — and it is the good outcome, because it has a ruling. ~~WAS: None. Every field on every upgraded screen carries over.~~ **NOW: the bill detail's embedded document viewer is removed**, ruled by the Product Owner on 2026-09-11. Everything else across nine remedies still carries over — eleven elements move tab or group, four are relabelled, and nothing else goes.

Rows with no quoted ruling: 0. That is the number this table exists to keep at zero, and a long K would mean a document quietly proposing to shrink the product.